



# Lecture 10: 3D Generative Models Cont. & 3D Backbone Networks

Li Yi

Apr 24, 2025

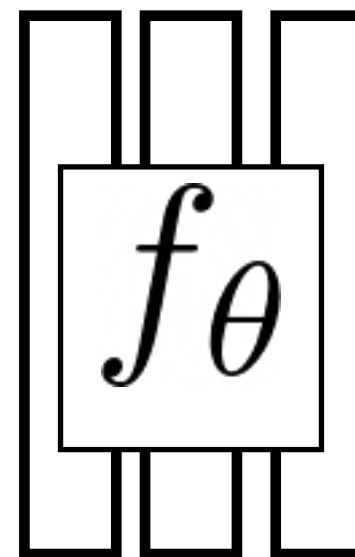
# 3D Generative Models

- 3D GAN
- 3D Autoregressive Models
- 3D Diffusion Models
- 3D Generation without 3D Training Data
- Part-based 3D Generation

# Generating 3D Shapes without 3D training data



Learning



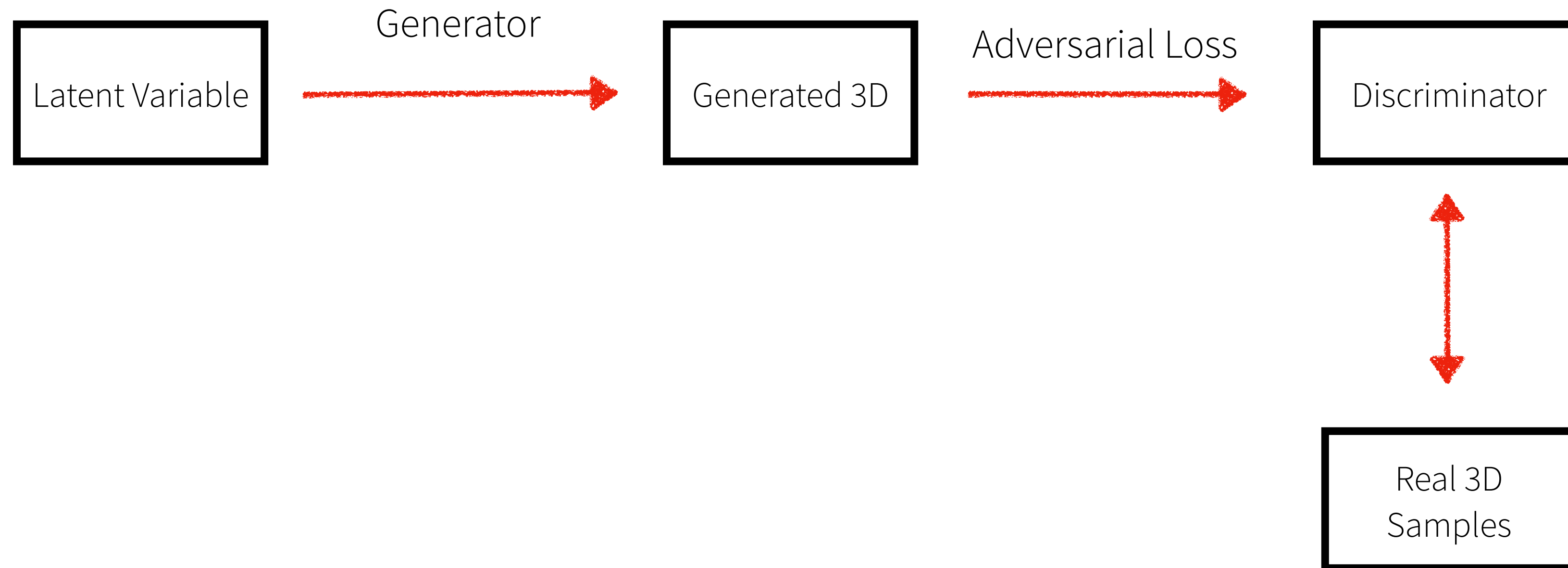
Inference



*Can we learn a generative 3D model from 2D images?*



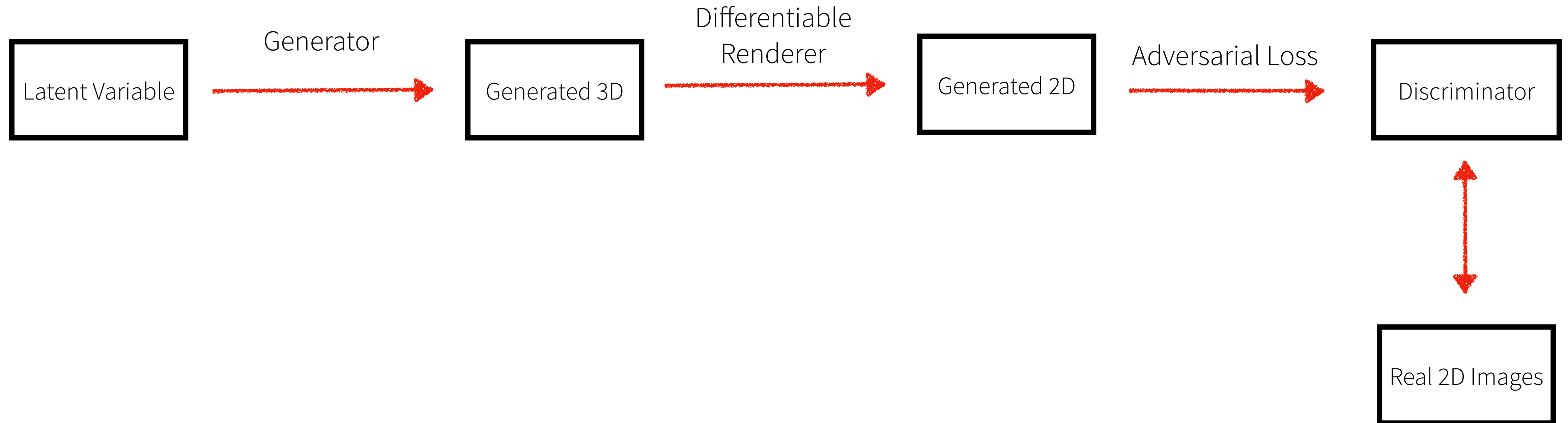
# Generating 3D Shapes without 3D training data



Key Idea: Generate 3D representations such that —  
they are indistinguishable from real samples

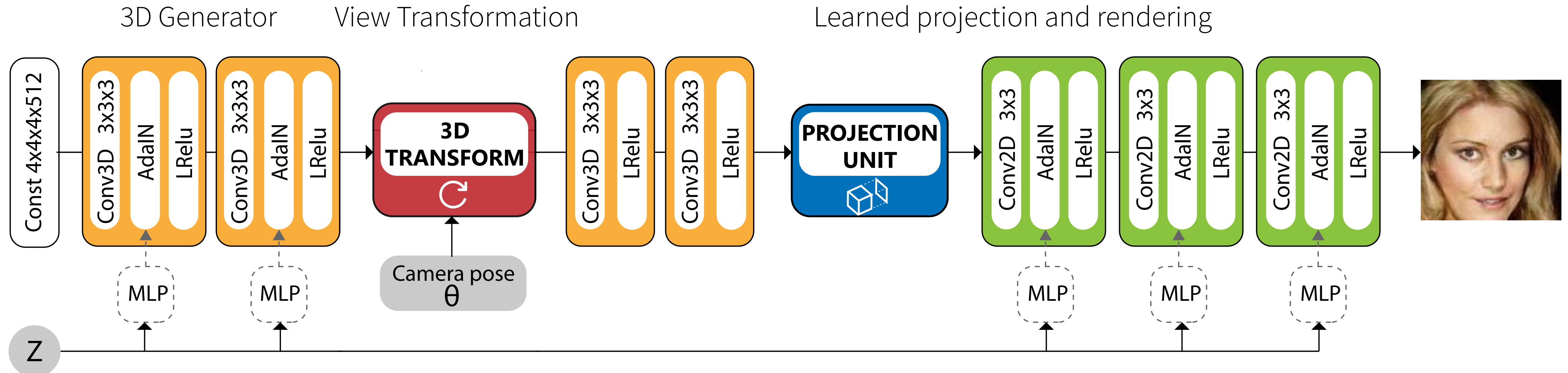


# Generating 3D Shapes without 3D training data



Key Idea: Generate 3D representations such that —  
**their renderings** are indistinguishable from real samples

# Generating 3D using 2D Adversarial Nets



Key Idea: Generate 3D representations such that —  
**their renderings** are indistinguishable from real samples

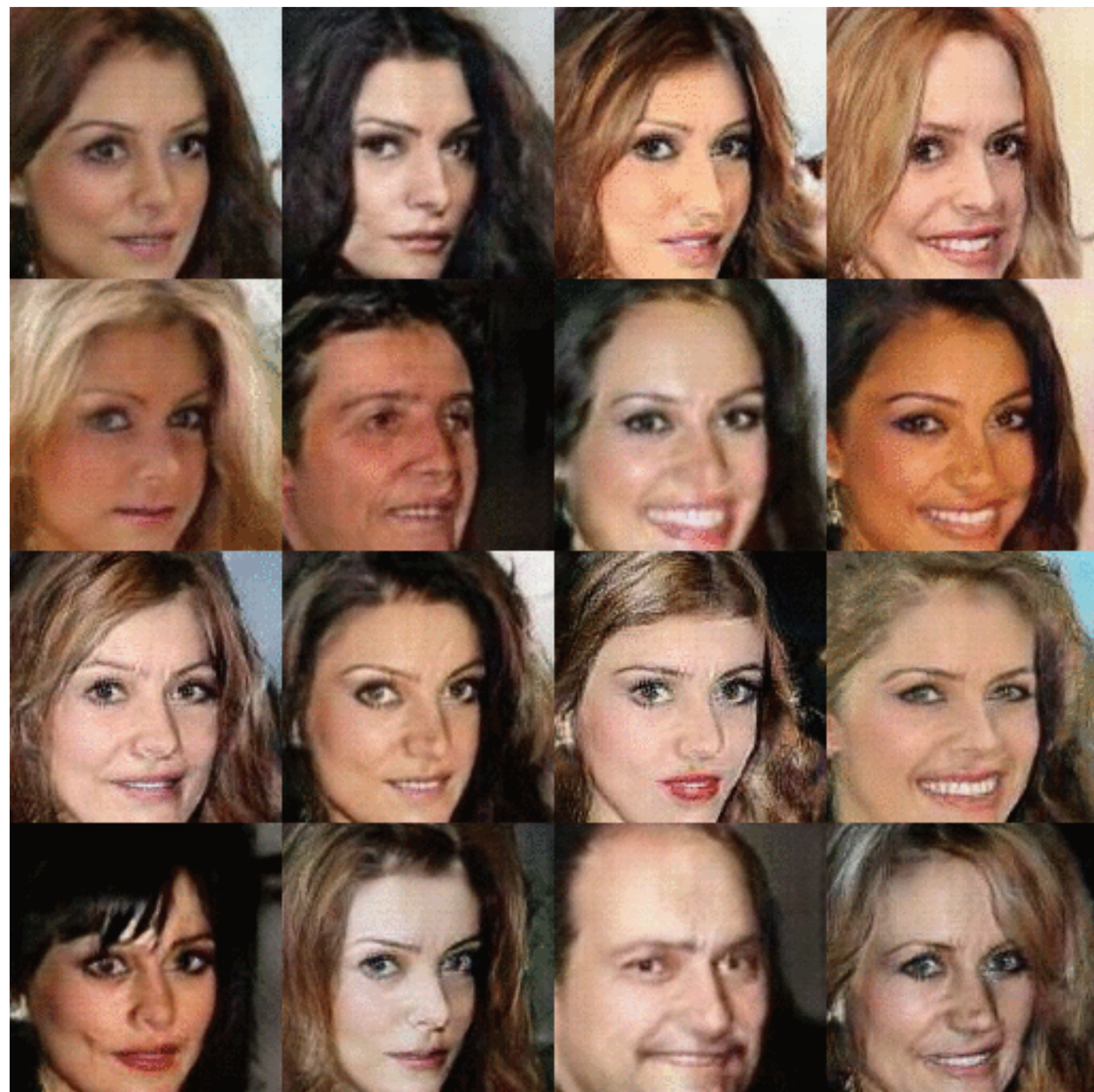


# Generating 3D using 2D Adversarial Nets



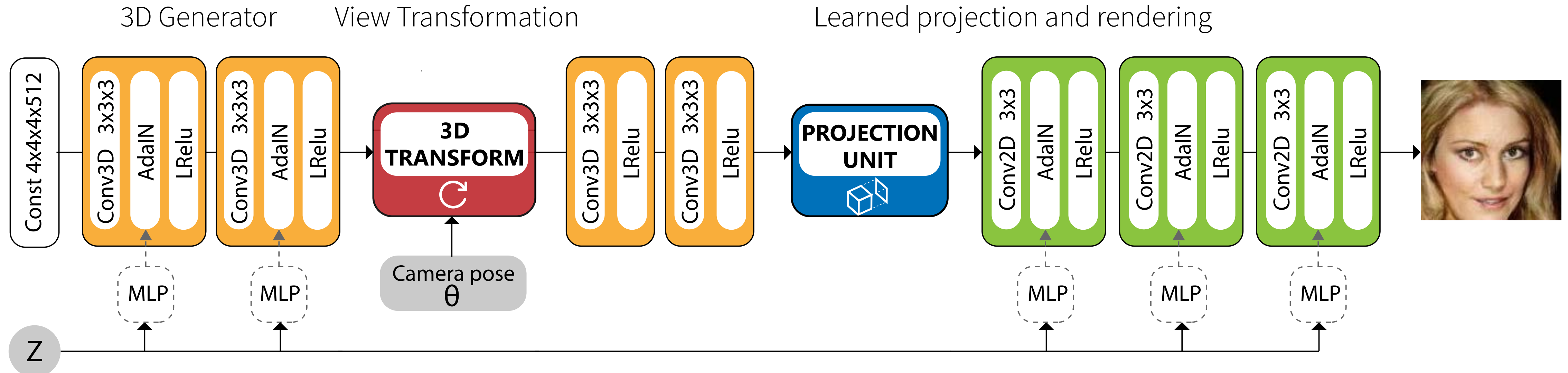


# Generating 3D using 2D Adversarial Nets





# Generating 3D using 2D Adversarial Nets

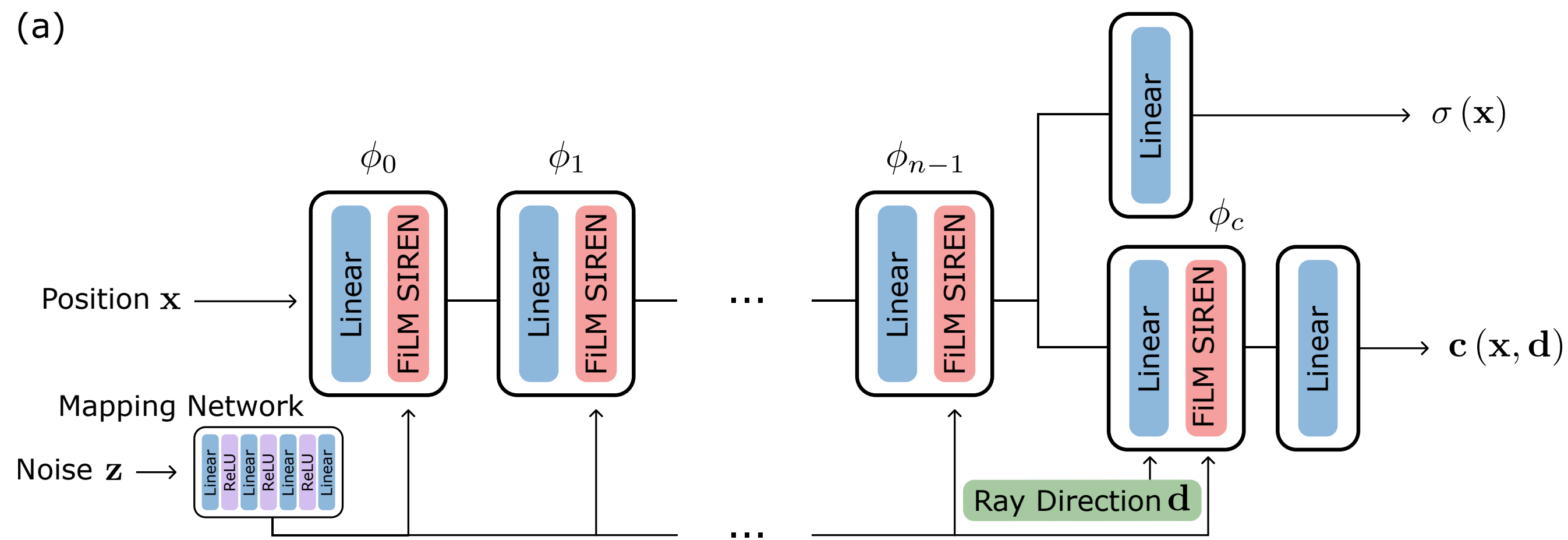


Some inconsistency across views

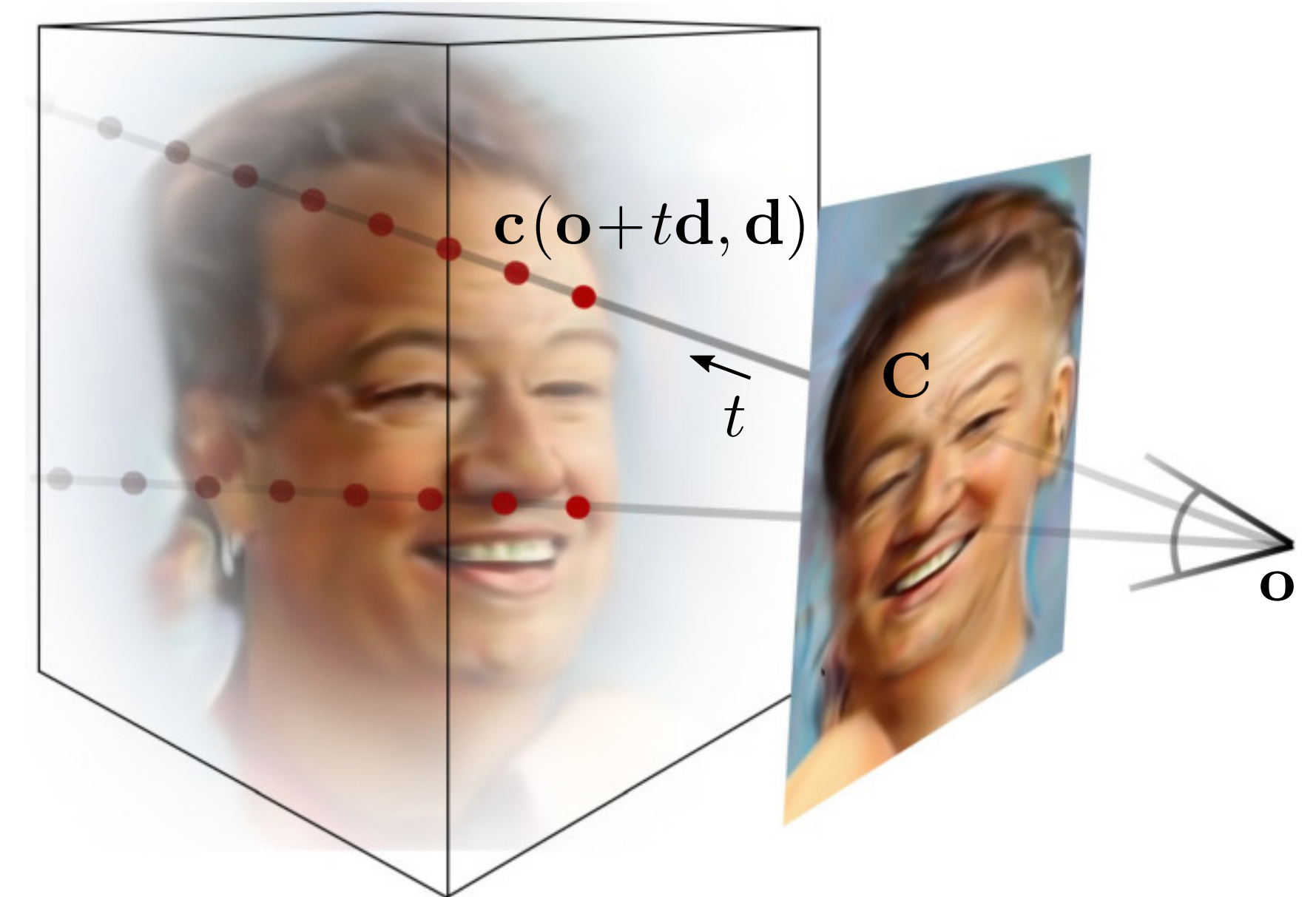
Some 3D inducing biases, but not enough (e.g. learned instead of volumetric rendering)



# Generating 3D using 2D Adversarial Nets



Generator: A conditional NeRF model

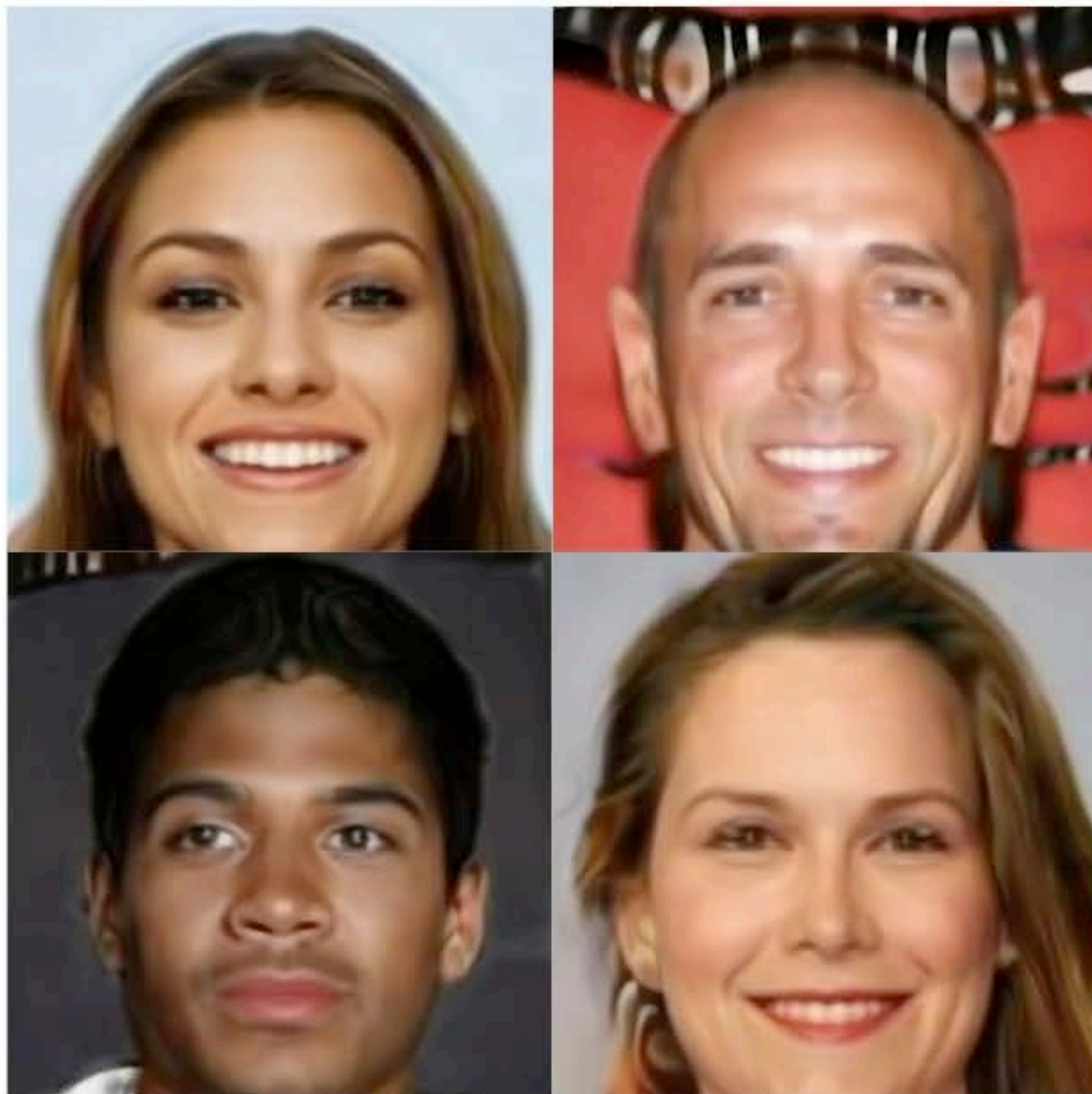


Renderer: Volume rendering



# Generating 3D using 2D Adversarial Nets

CelebA



Cats



CARLA





# Generating 3D using 2D Adversarial Nets



Figure 12: In a failure case reminiscent of the hollow-face illusion, our model sometimes generates objects with inverted sections.

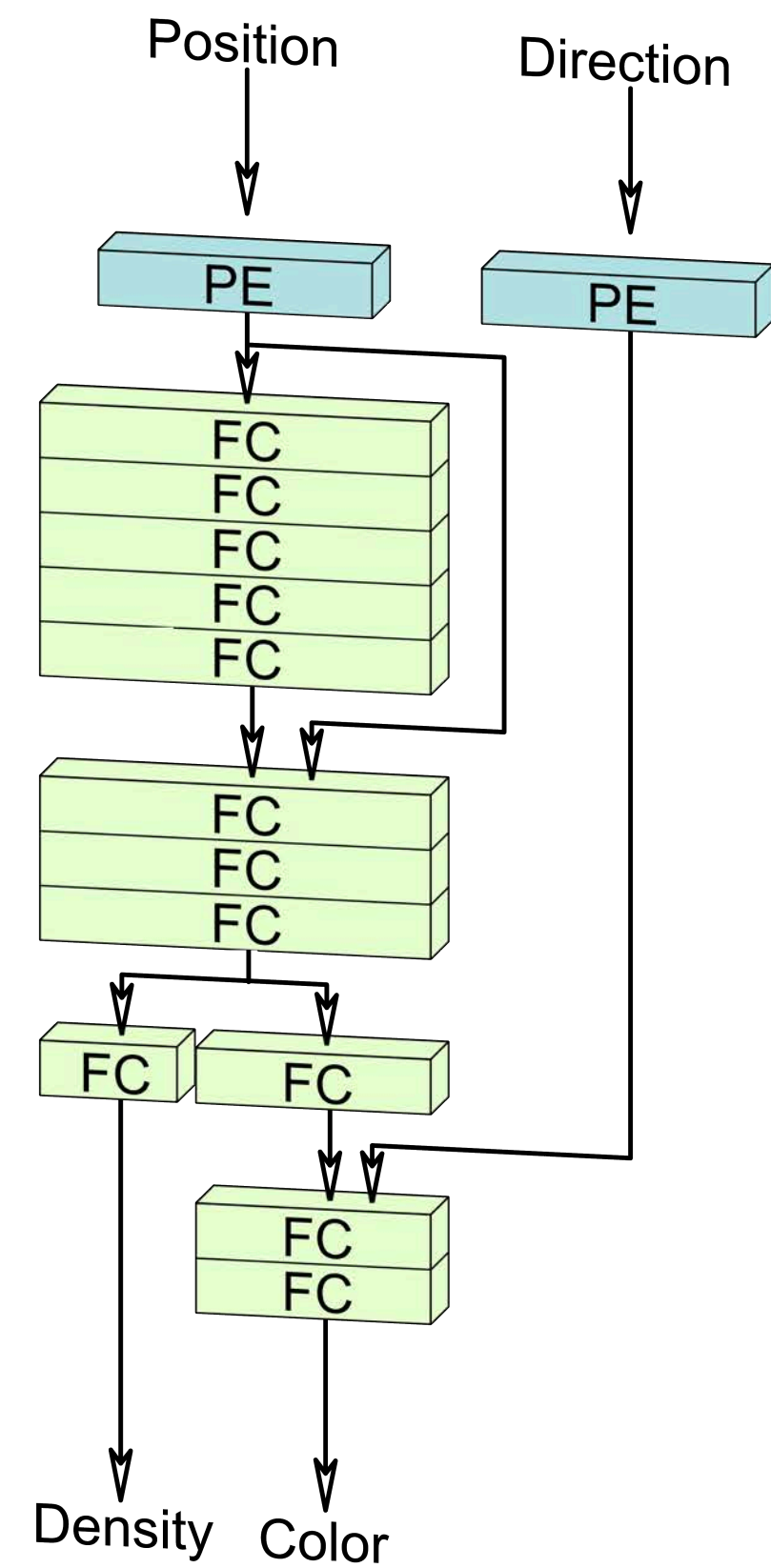
Although not always great



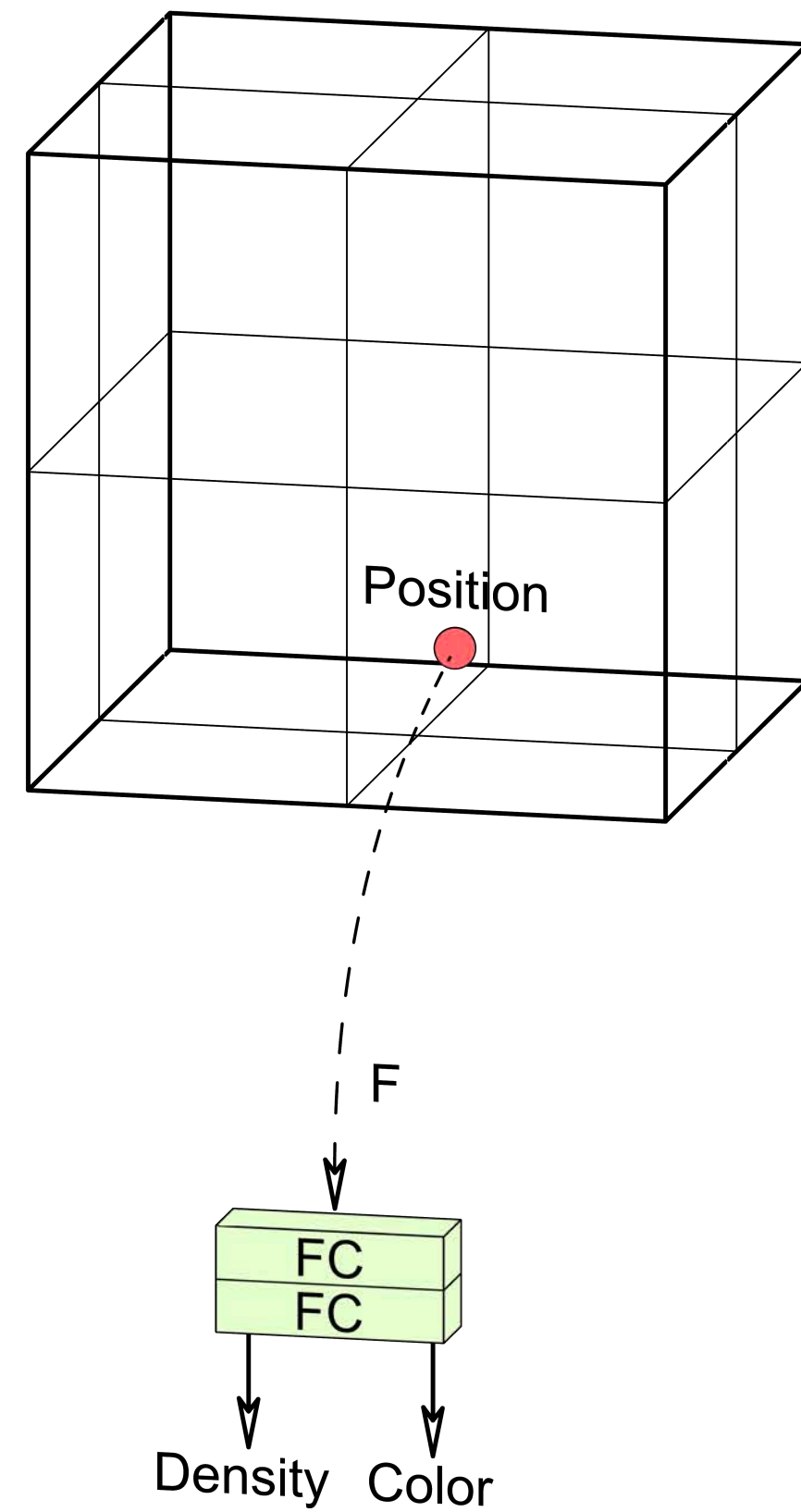
Also possible to extract some geometry



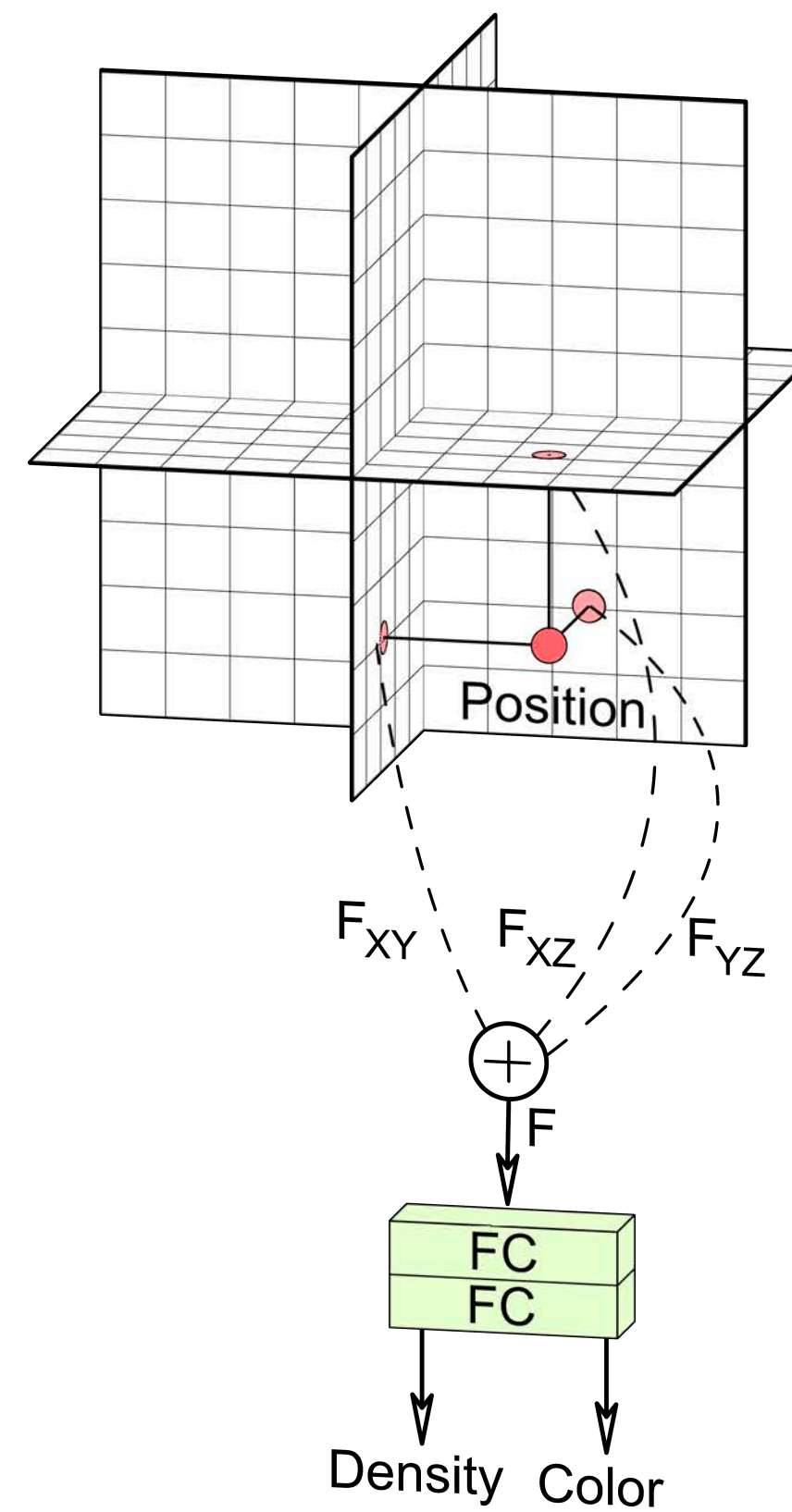
# Generating 3D using 2D Adversarial Nets



(a) NeRF (Implicit)



(b) Voxels (Explicit or Hybrid)



(c) Ours (Hybrid)

Replace NeRFs via hybrid representations

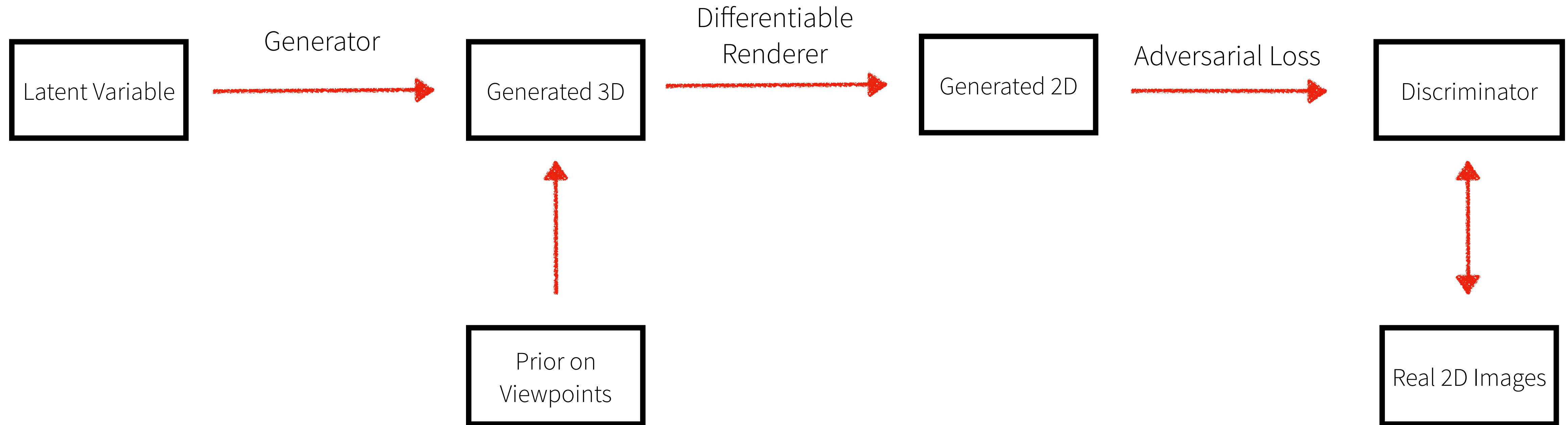


# Generating 3D using 2D Adversarial Nets





# Generating 3D Shapes without 3D training data



Crucial to leverage the 3D structure of the problem

Assumed known view distribution e.g. faces seen frontally

Typical success stories on ‘nice’ categories (lots of clean images, known views...)



# Optimizing Text-conditioned 3D

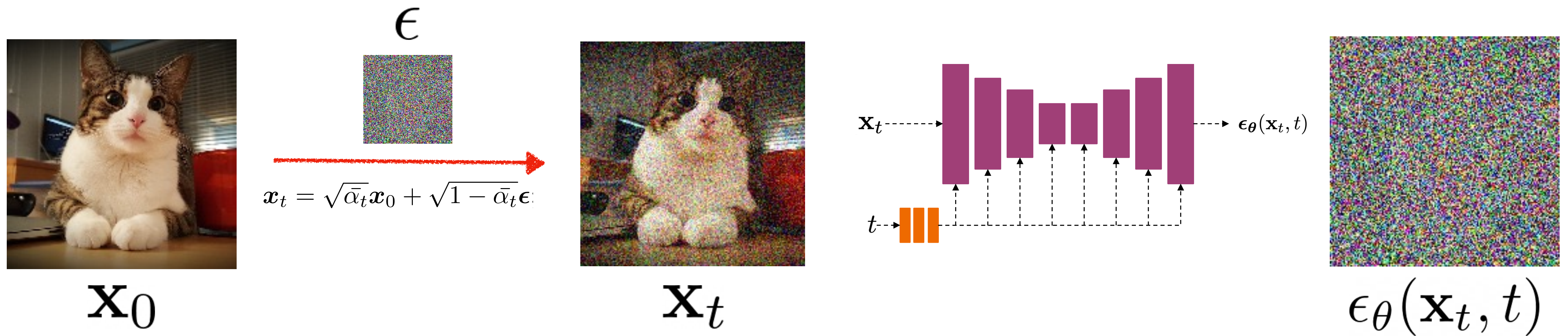


Let's see more combinations:  
<https://dreamfusion3d.github.io/>

“A squirrel wearing a kimono and riding a motorbike”



# Background: Likelihood in Diffusion Models

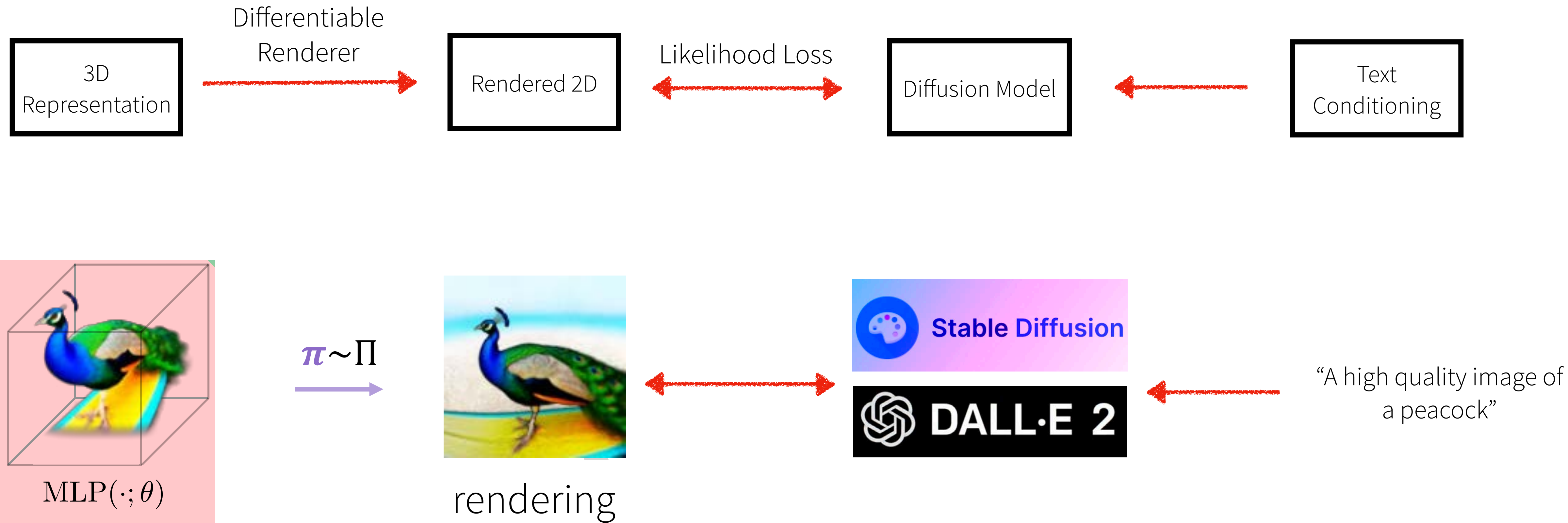


Training Objective: Noise prediction error  $\|\epsilon_\theta(\mathbf{x}_t, t) - \epsilon\|^2$

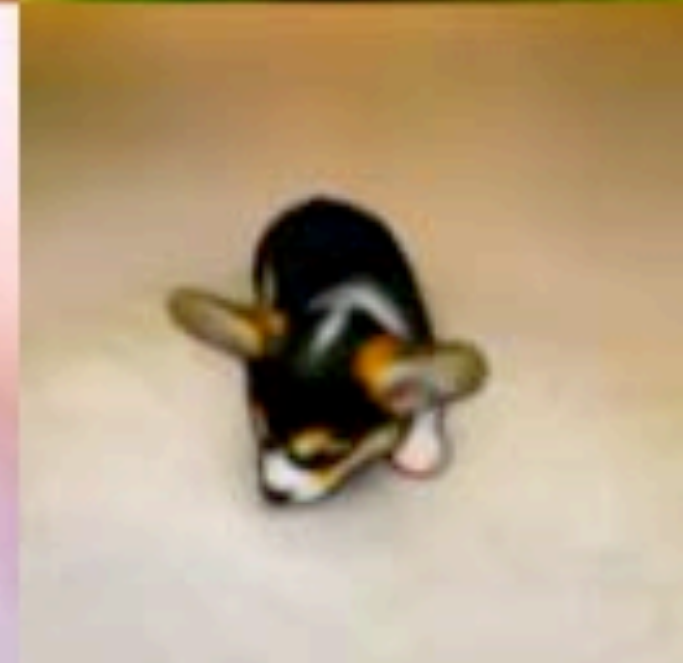
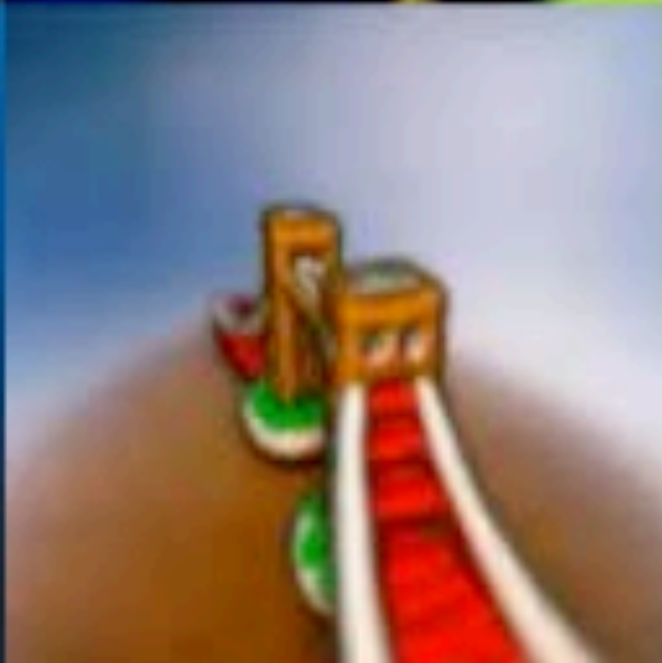
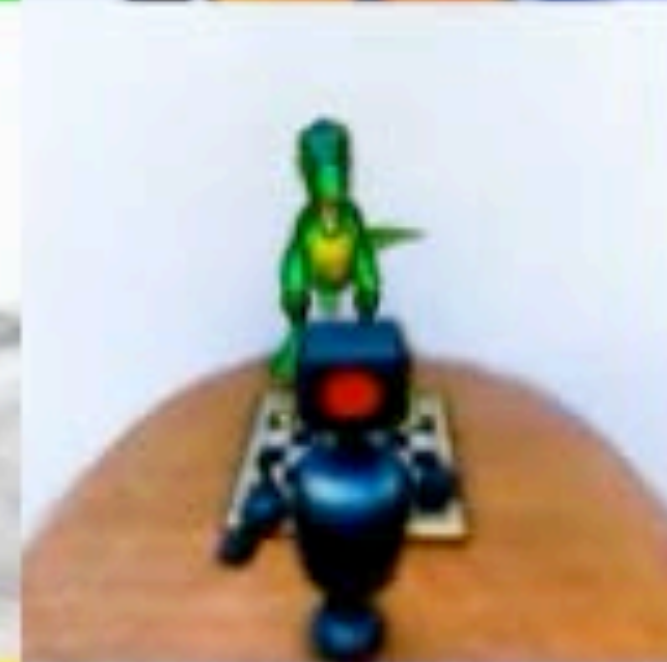
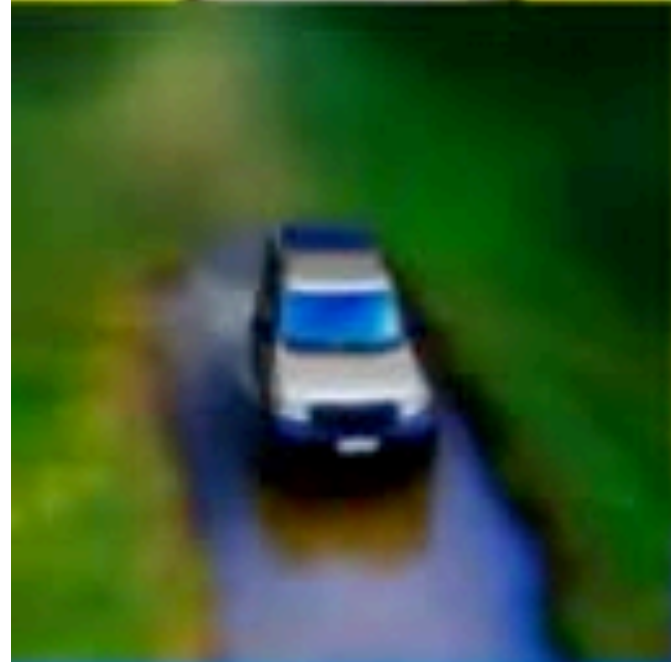
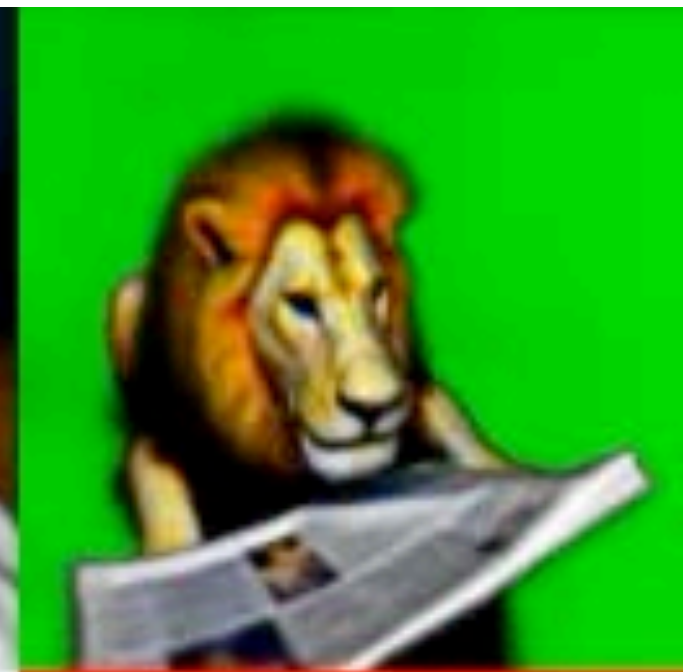
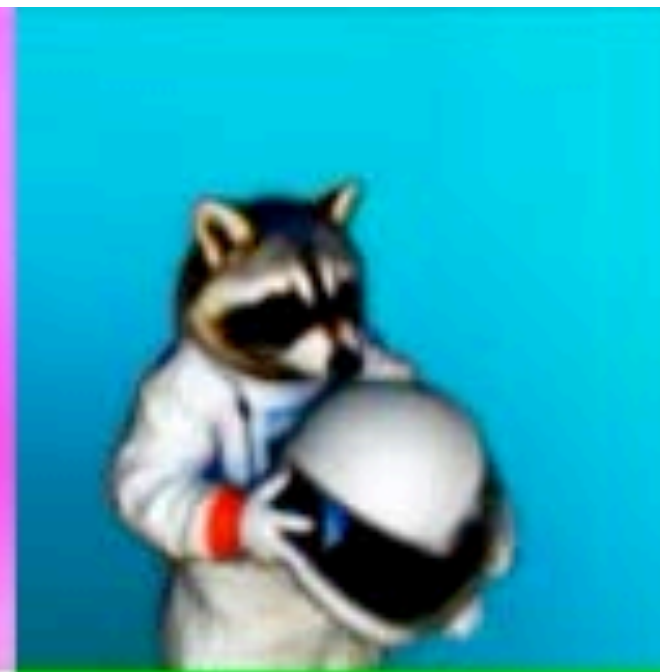
Prediction error also be correlated to  $\mathbf{p}(\mathbf{x})$  (images with high likelihood yield lower error)



# Optimizing Text-conditioned 3D









# Optimizing Text-conditioned 3D



Allows 3D generation for complex structures

Not a ‘generative’ model in a probabilistic sense — no distribution over 3D is inferred

(relatively) compute intensive — per instance optimization



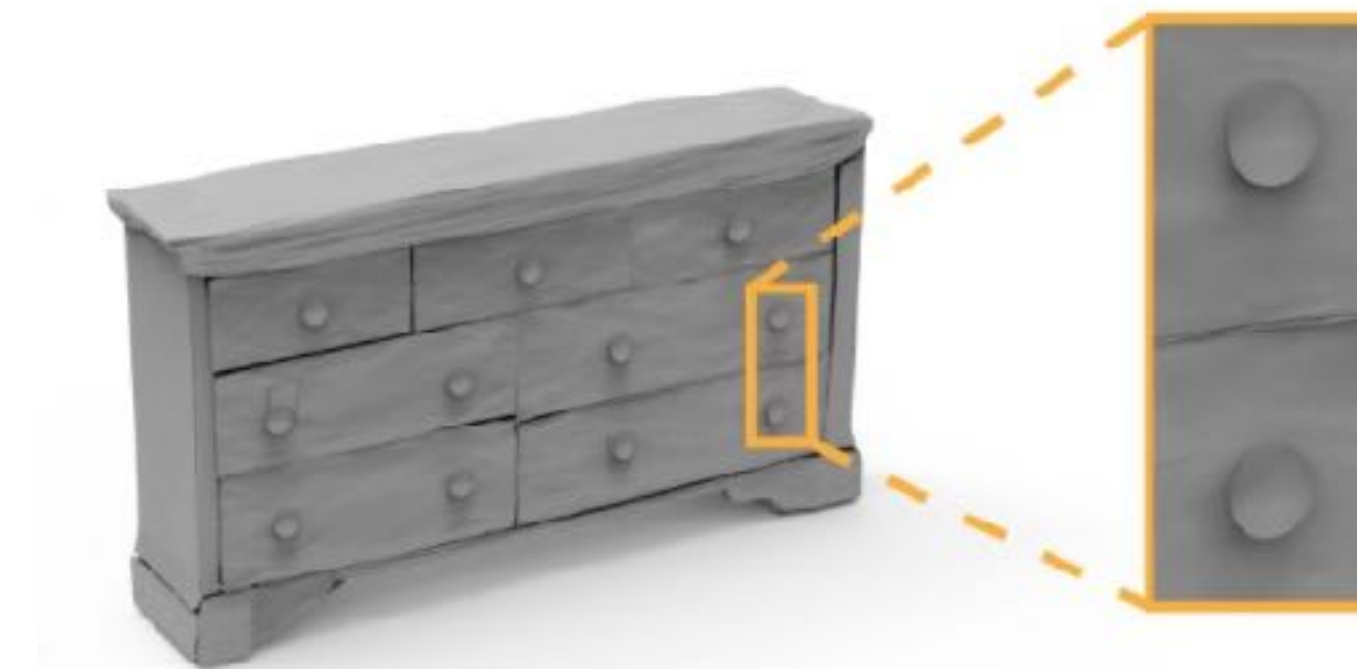
# Outline

- 3D GAN
- 3D Autoregressive Models
- 3D Diffusion Models
- 3D Generation without 3D Training Data
- Part-based 3D Generation



# Part-based 3D Shape Generation

- Detailed Part Geometry
- Clear Shape Structures
- Sharp Part Boundaries
- With Semantic Part Labels

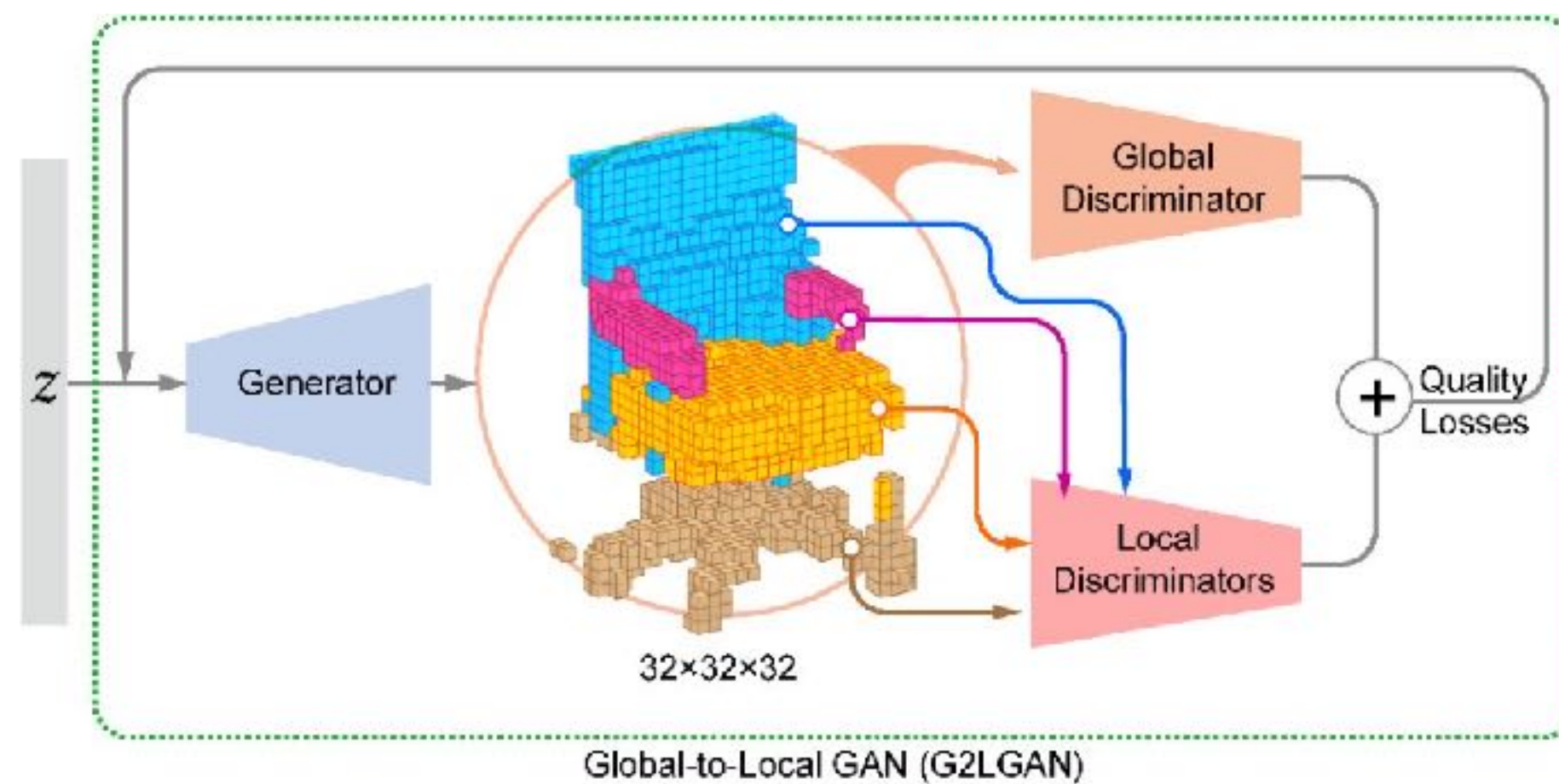


Yang and Mo et al., “**DSG-Net: Learning Disentangled Structure and Geometry for 3D Shape Generation**”, *ACM ToG* 2021

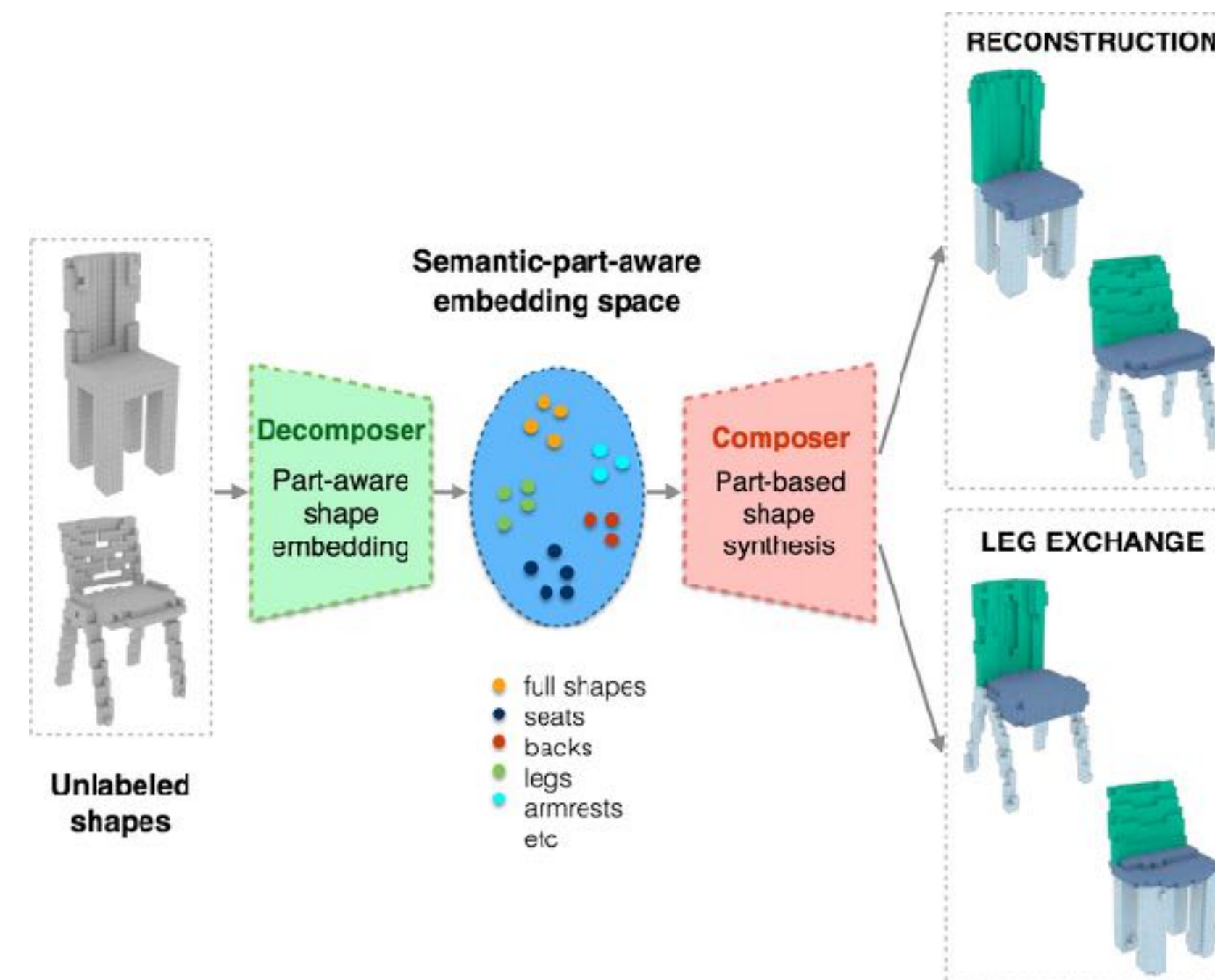


# Semantic-level Synthesis and Assembly

## Per-part Geometry Generation and Part Assembly



Wang and Schor et al., “**Global-to-Local Generative Model for 3D Shapes**”, Siggraph Asia 2018

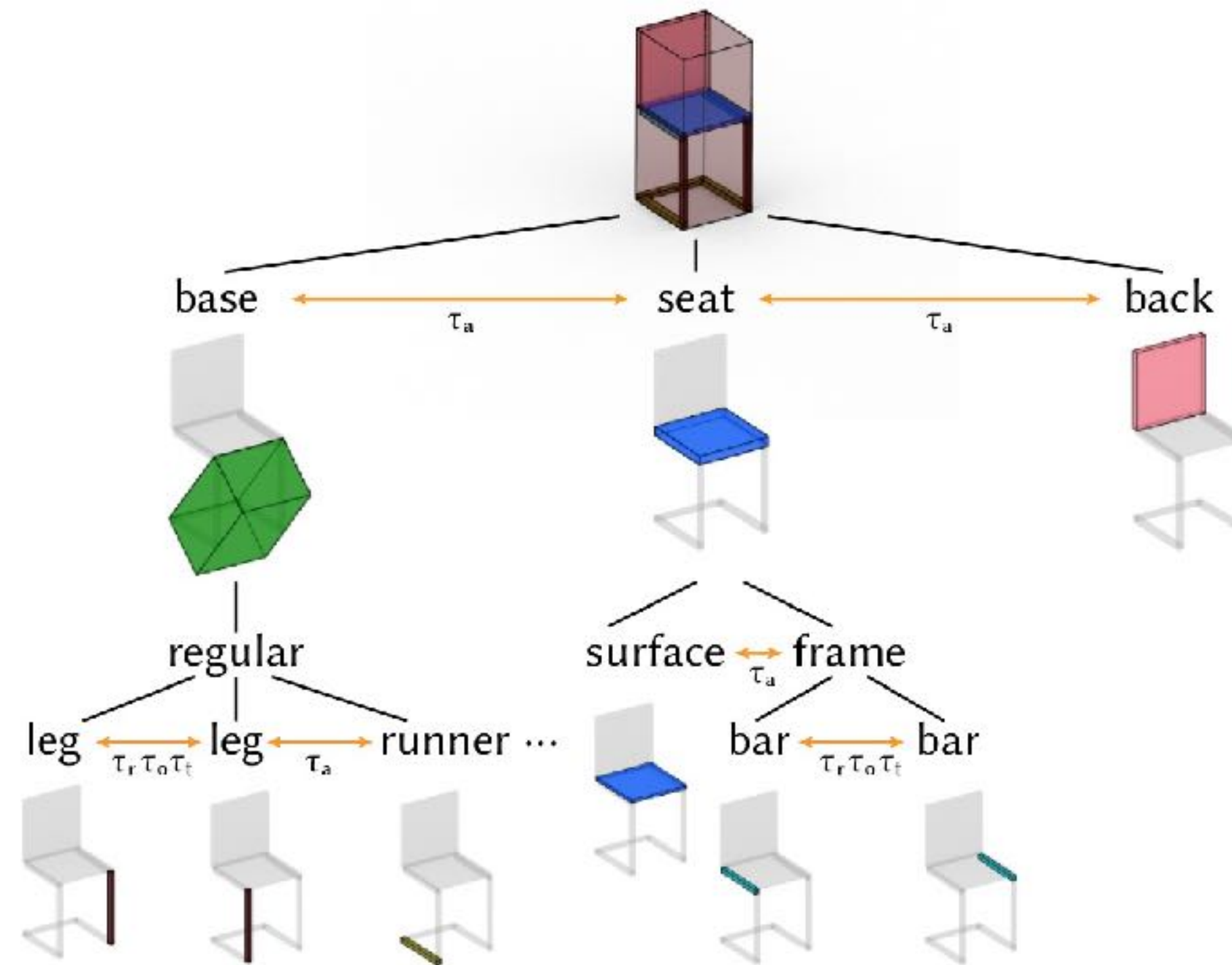


Dubrovina et al., “**Composite Shape Modeling via Latent Space Factorization**”, ICCV 2019



# Hierarchical Generation

For fine-grained parts, we need coarse-to-fine generation

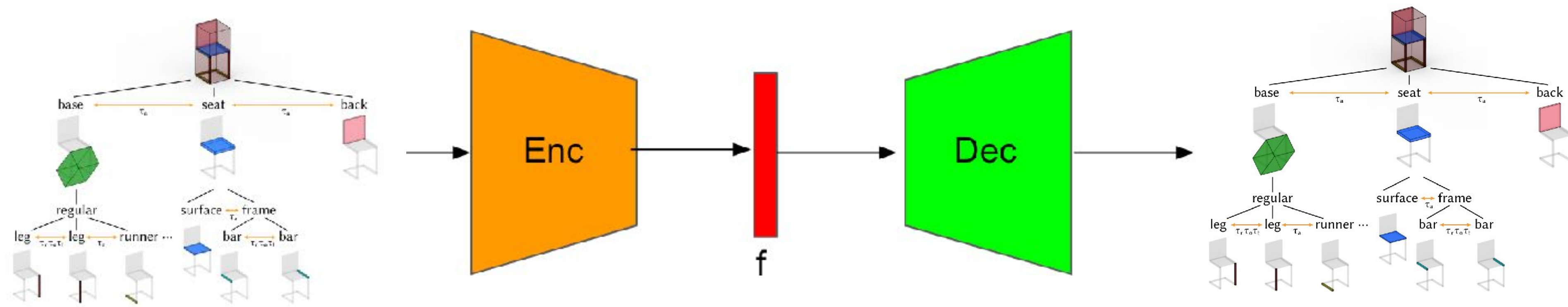


Mo et al., "StructureNet: Hierarchical Graph Networks for 3D Shape Generation", *Siggraph Asia 2019*



# Hierarchical Generation

We use Variational Autoencoder (VAE)

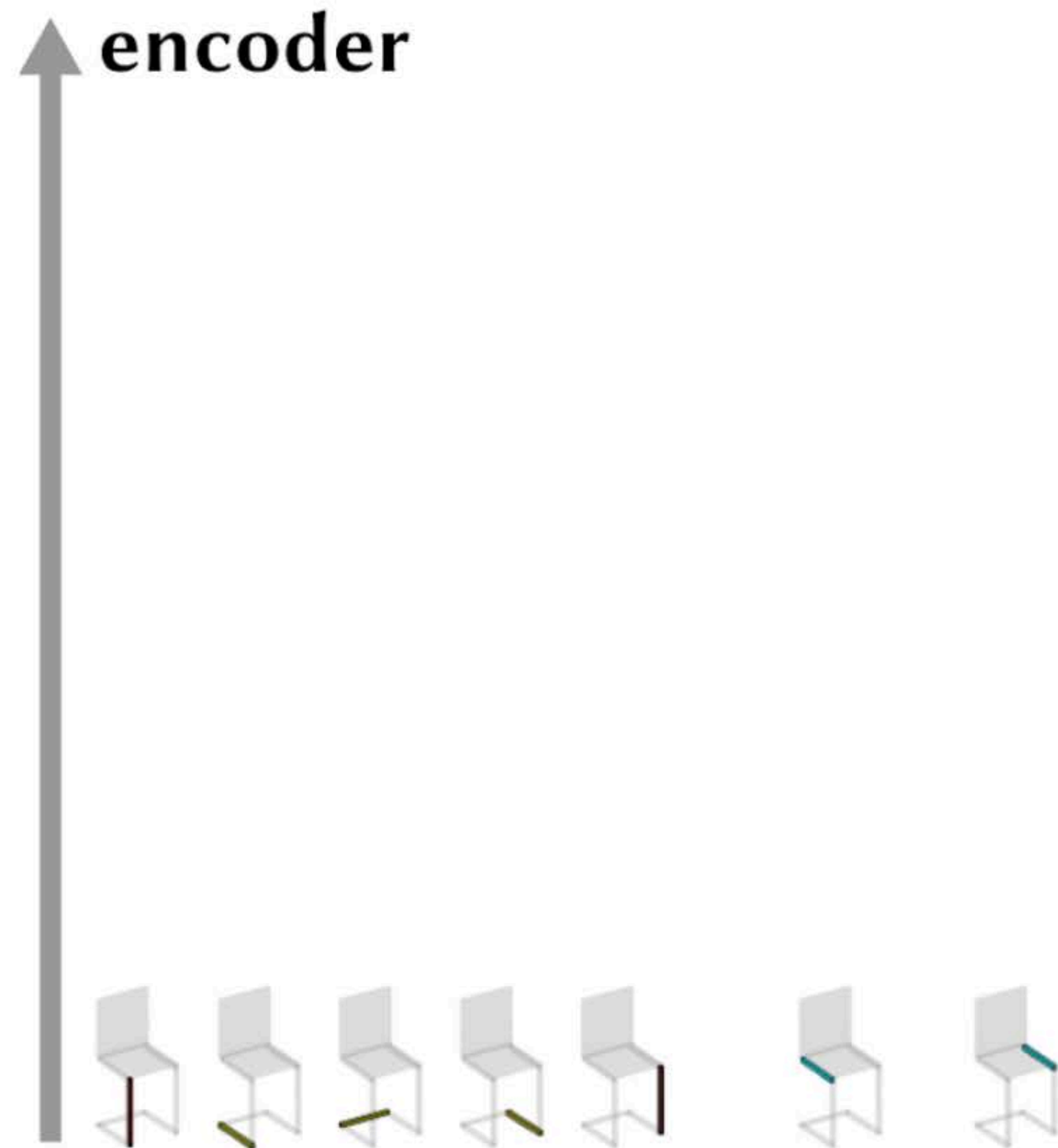


$$f \sim N(0, I)$$

Mo et al., “**StructureNet: Hierarchical Graph Networks for 3D Shape Generation**”, *Siggraph Asia 2019*



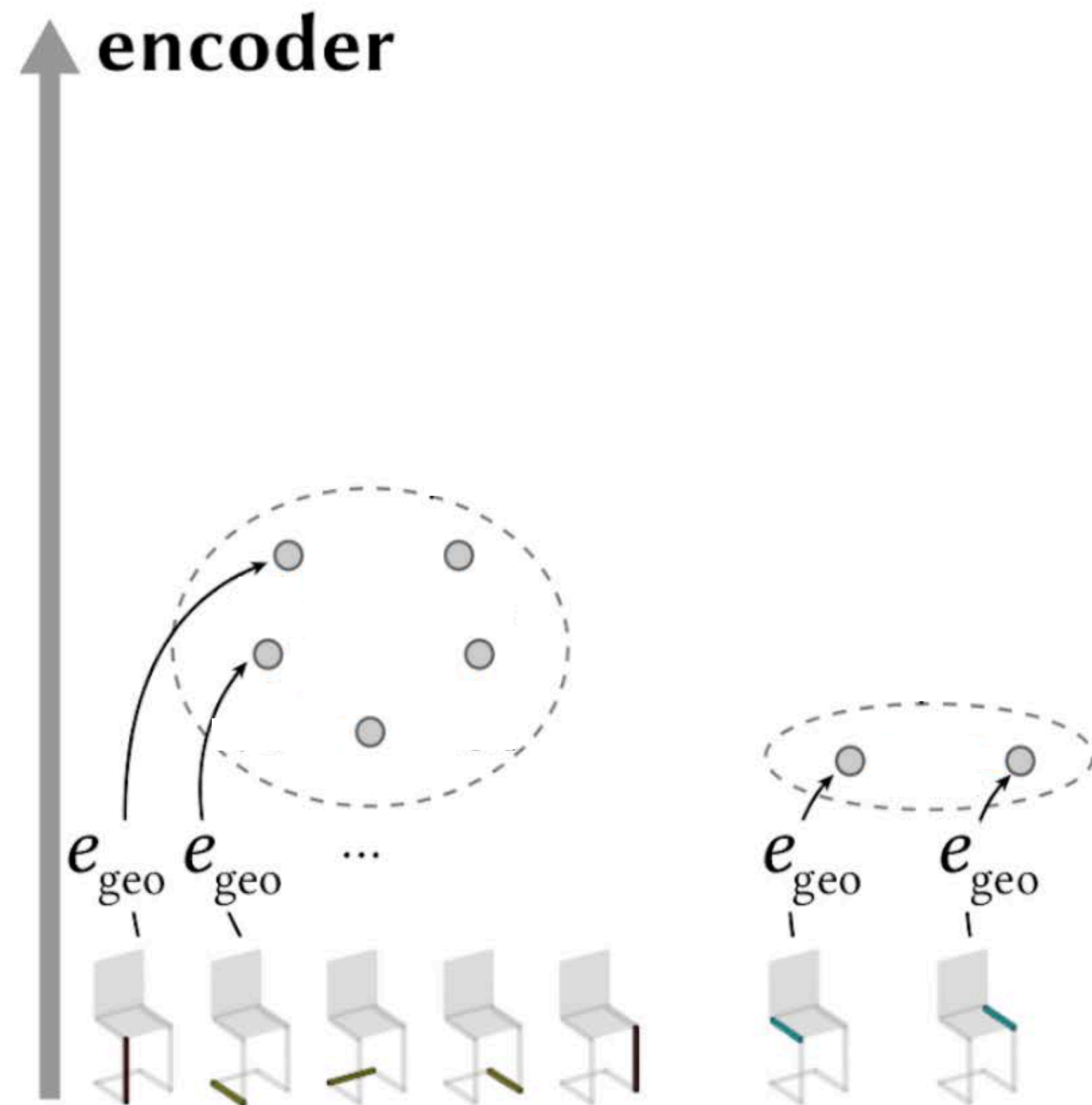
# Hierarchical Generation



Mo et al., “**StructureNet: Hierarchical Graph Networks for 3D Shape Generation**”, *Siggraph Asia 2019*



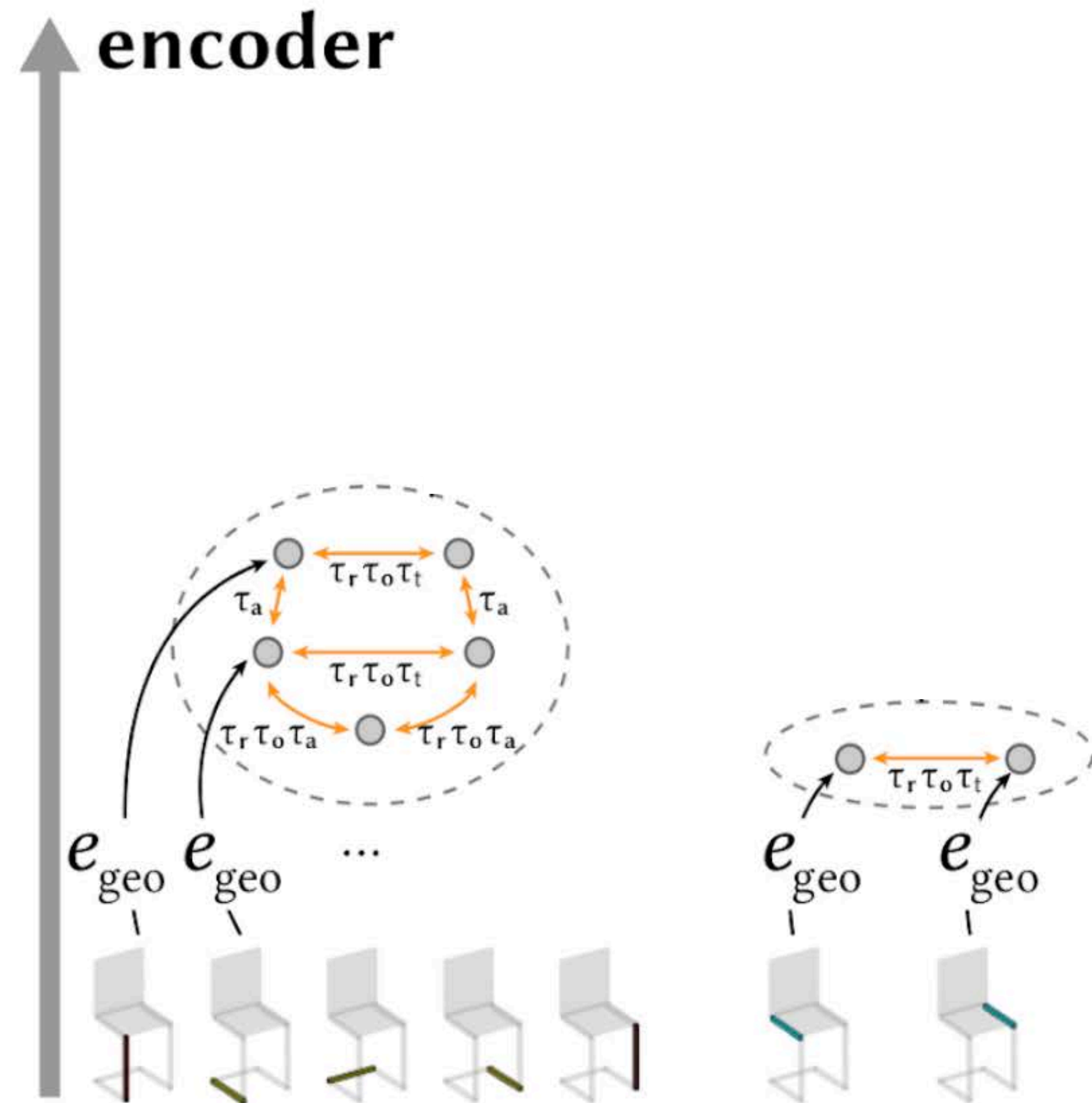
# Hierarchical Generation



Mo et al., "StructureNet: Hierarchical Graph Networks for 3D Shape Generation", *Siggraph Asia 2019*



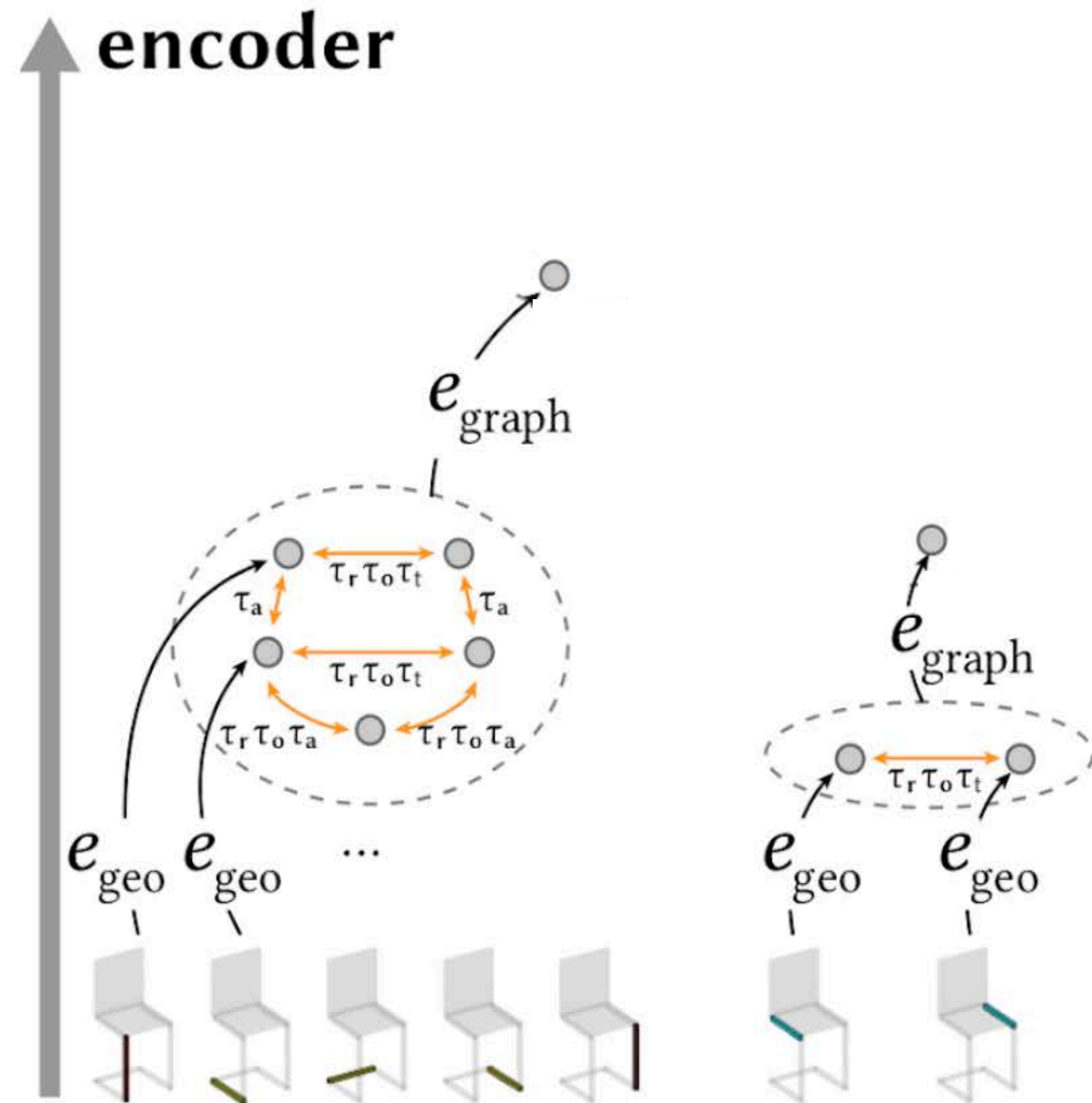
# Hierarchical Generation



Mo et al., "StructureNet: Hierarchical Graph Networks for 3D Shape Generation", *Siggraph Asia 2019*



# Hierarchical Generation

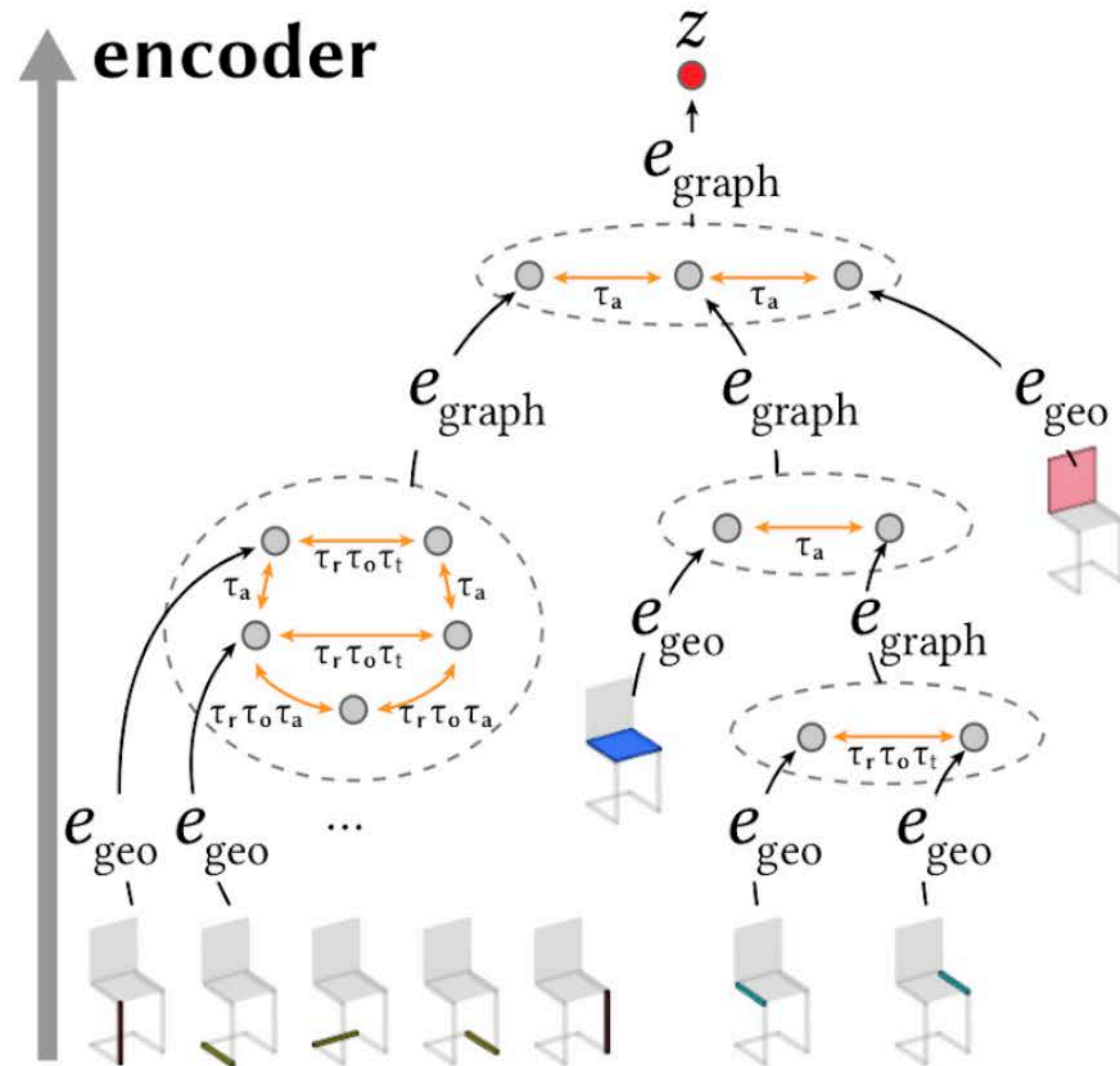


## Graph Neural Network

Mo et al., "StructureNet: Hierarchical Graph Networks for 3D Shape Generation", *Siggraph Asia 2019*



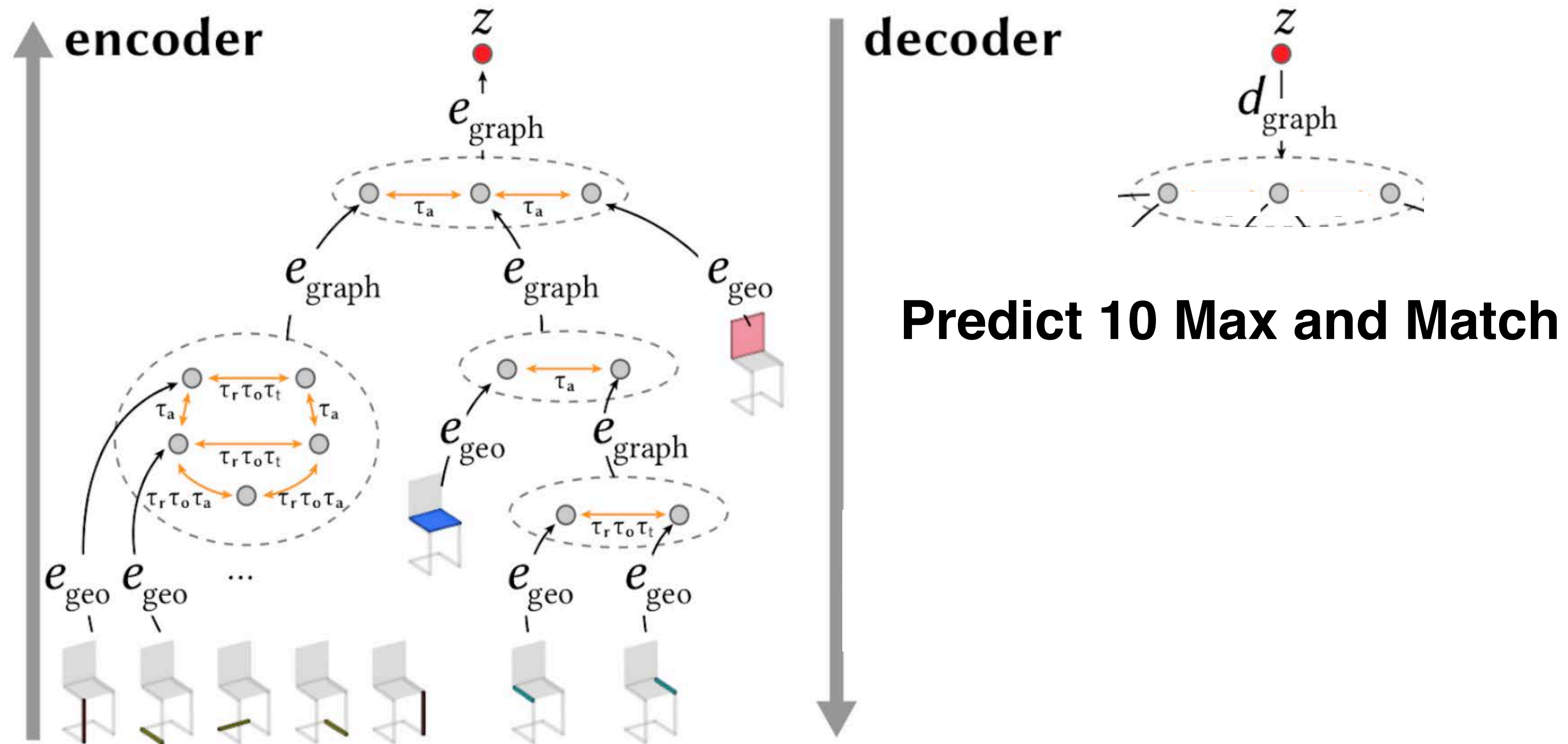
# Hierarchical Generation



Mo et al., "StructureNet: Hierarchical Graph Networks for 3D Shape Generation", *Siggraph Asia 2019*



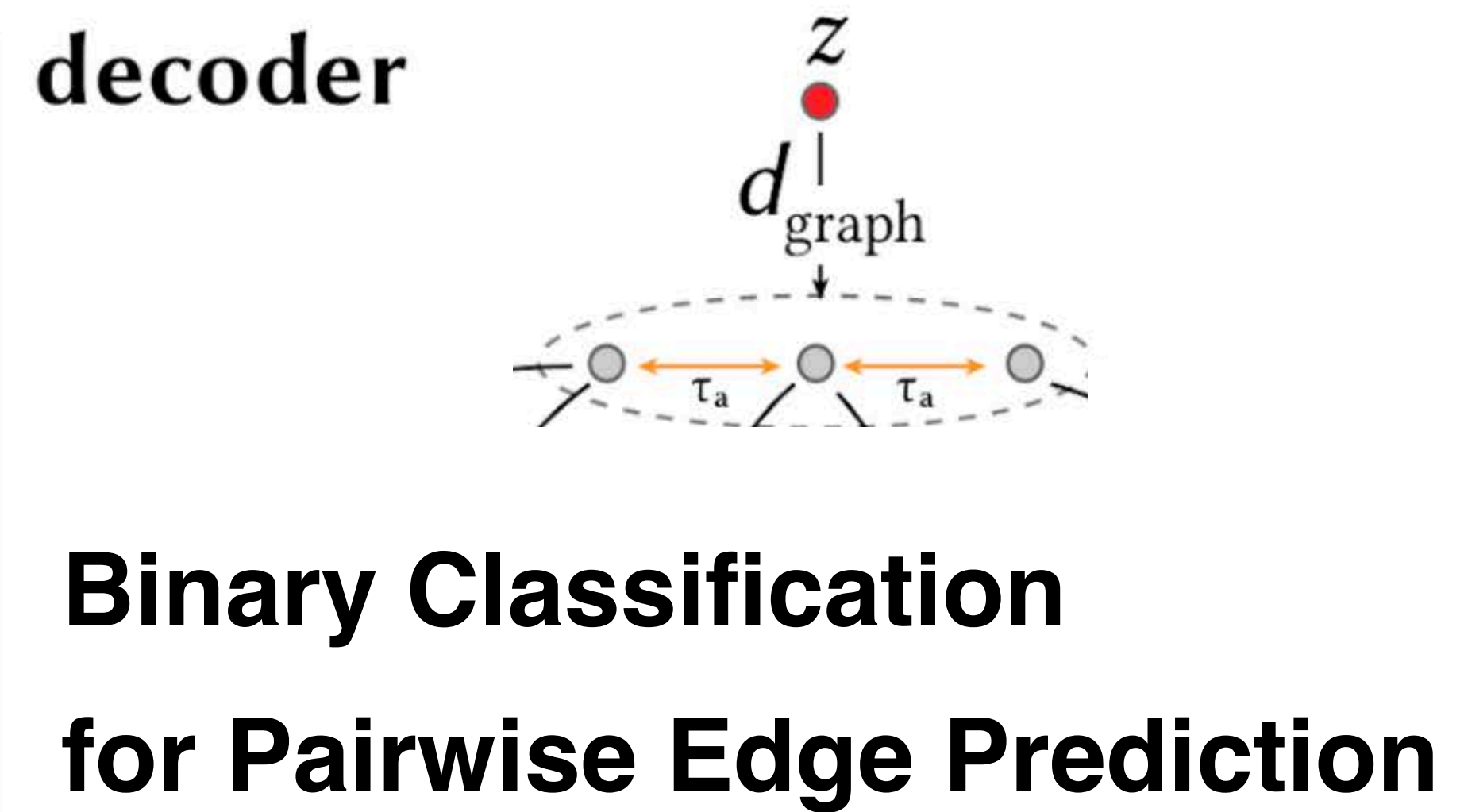
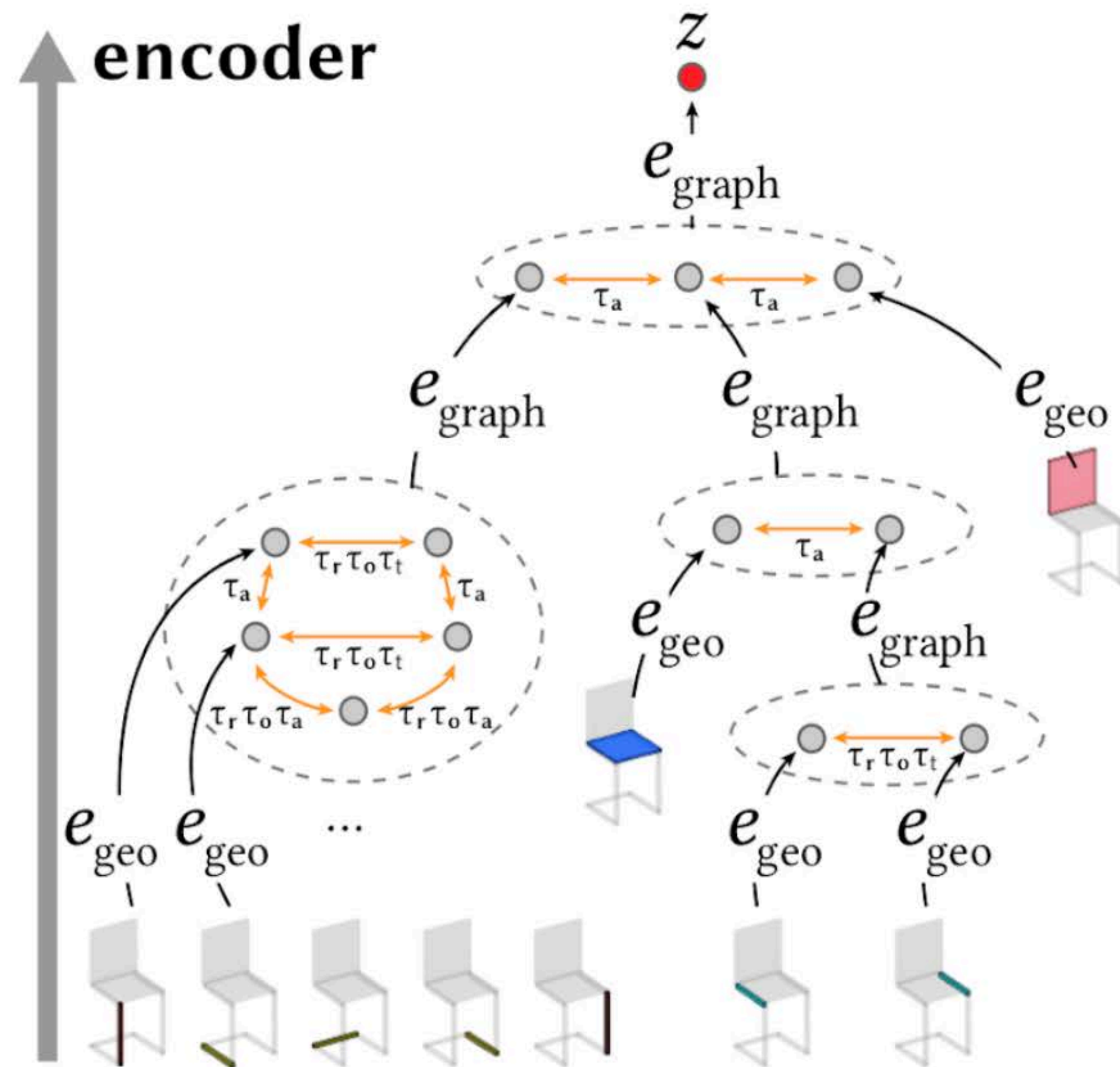
# Hierarchical Generation



Mo et al., "StructureNet: Hierarchical Graph Networks for 3D Shape Generation", *Siggraph Asia 2019*



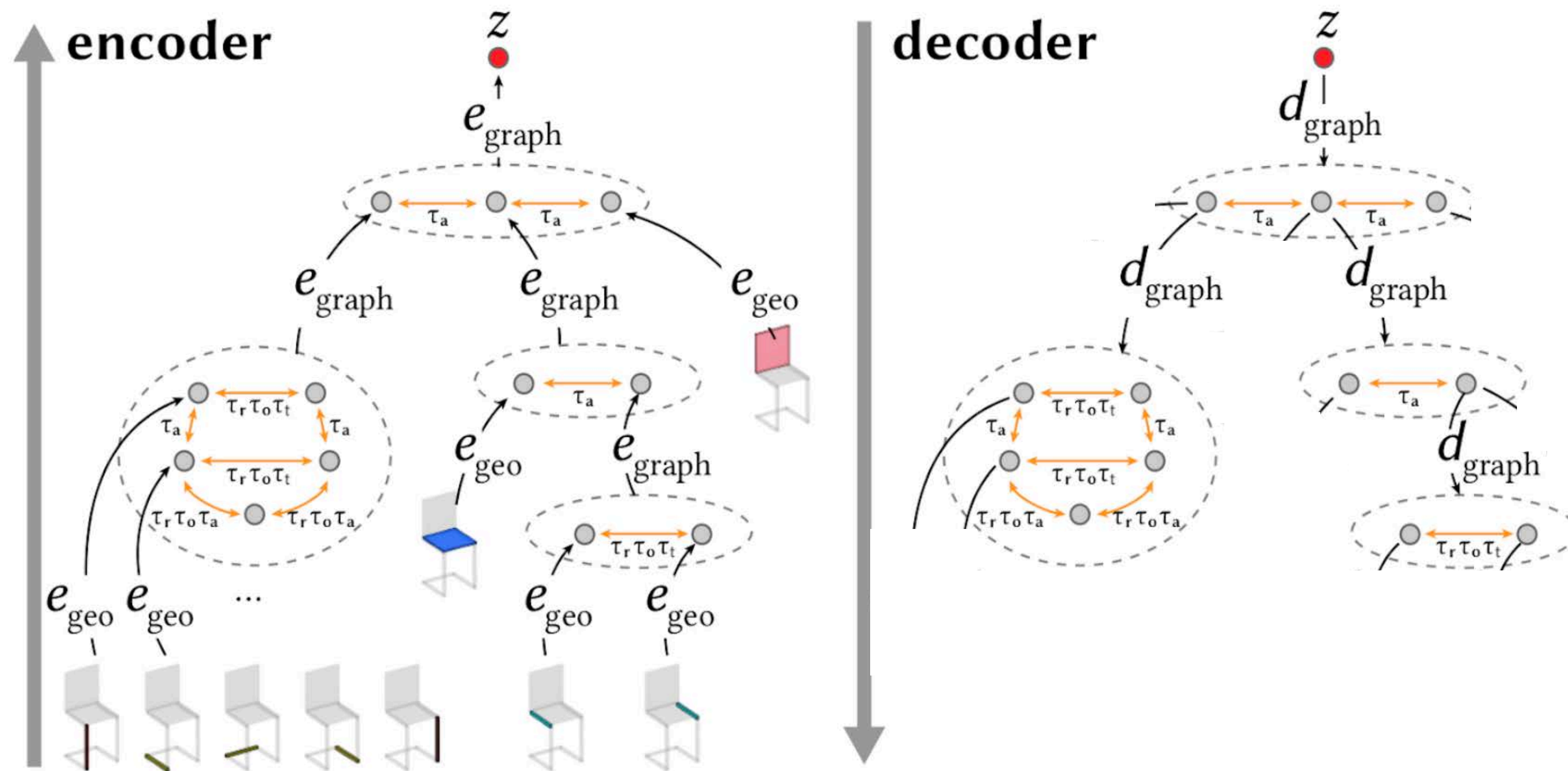
# Hierarchical Generation



Mo et al., "StructureNet: Hierarchical Graph Networks for 3D Shape Generation", *Siggraph Asia 2019*



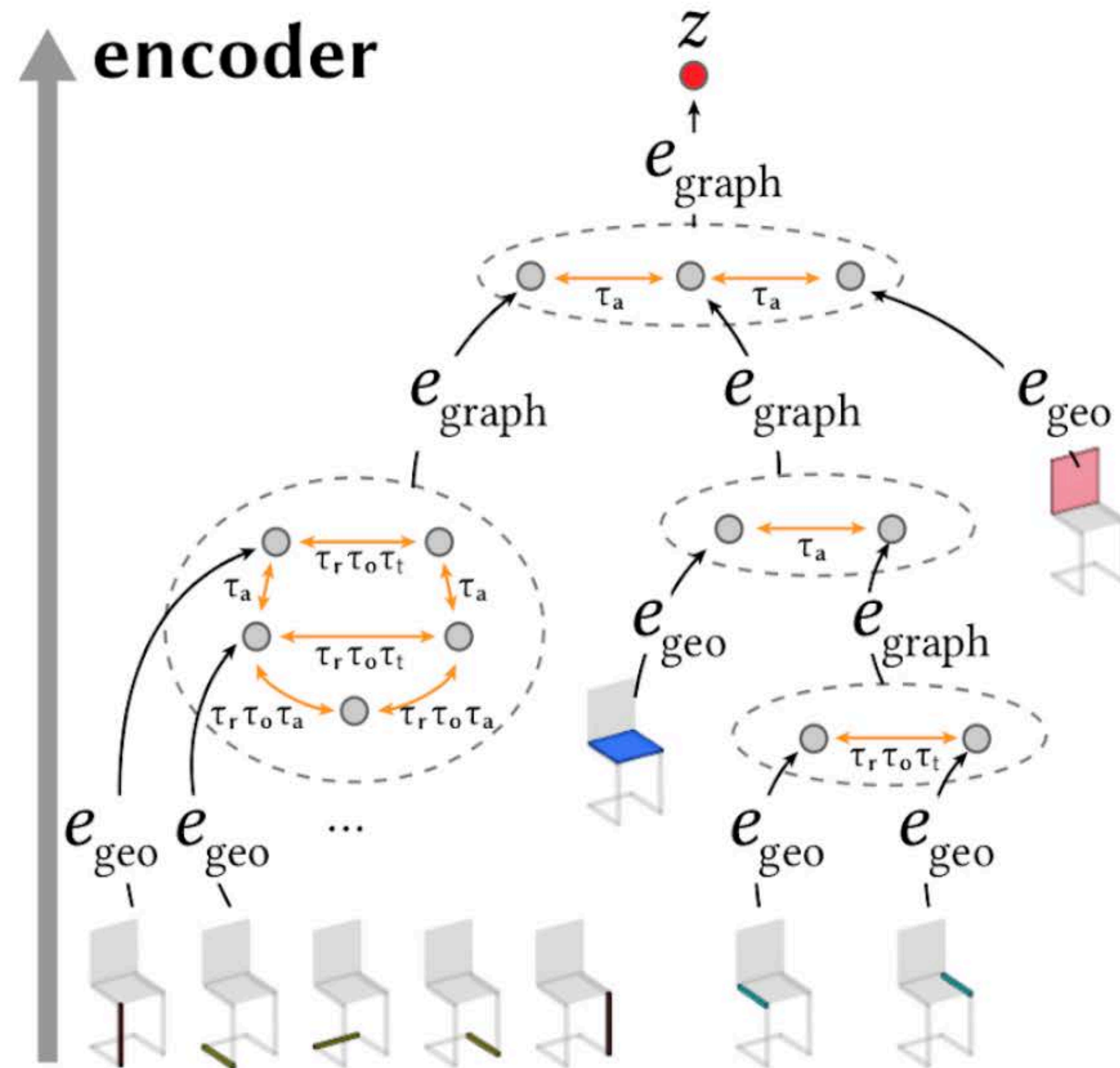
# Hierarchical Generation



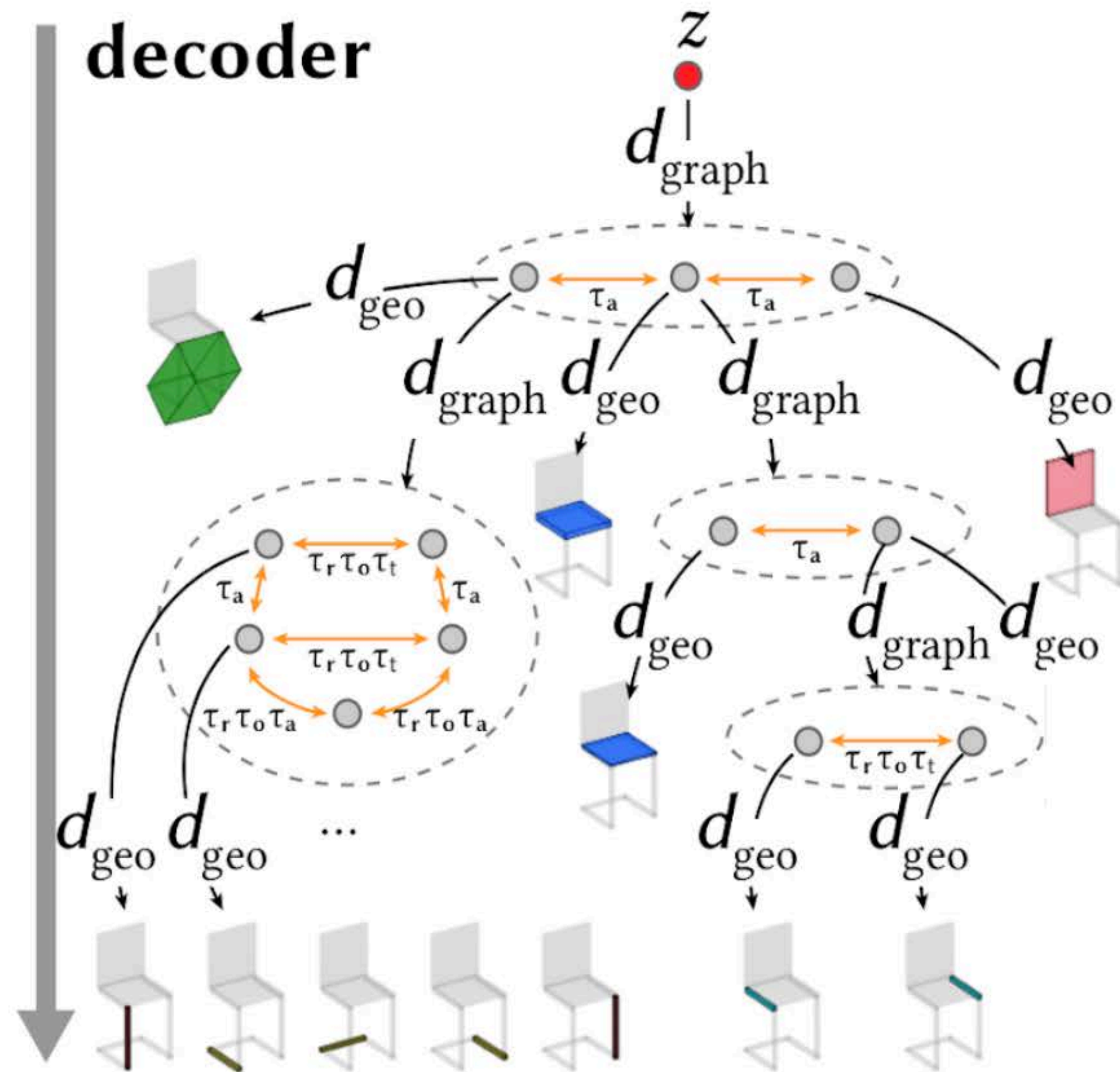
Mo et al., "StructureNet: Hierarchical Graph Networks for 3D Shape Generation", *Siggraph Asia 2019*



# Hierarchical Generation



**Recursive Neural Network**



Mo et al., "StructureNet: Hierarchical Graph Networks for 3D Shape Generation", *Siggraph Asia 2019*



# Hierarchical Generation: Results



Mo et al., "StructureNet: Hierarchical Graph Networks for 3D Shape Generation", *Siggraph Asia 2019*



# Hierarchical Generation: Results

Use mesh generation for leaf nodes

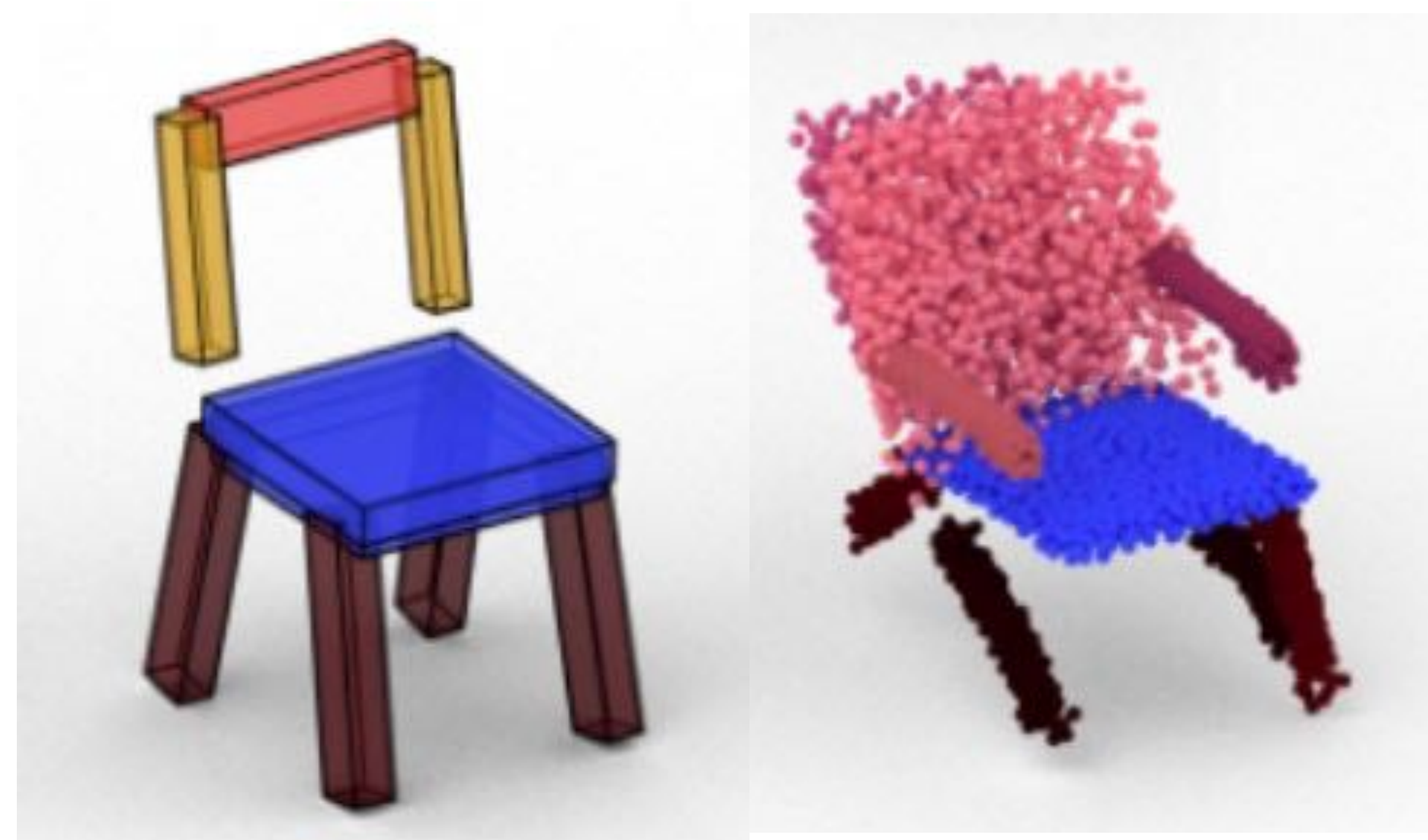


Yang and Mo et al., “**DSG-Net: Learning Disentangled Structure and Geometry for 3D Shape Generation**”, *ACM ToG* 2021



# Part Connection Issue

There could be floating parts



Mo et al., “**StructureNet: Hierarchical Graph Networks for 3D Shape Generation**”, *Siggraph Asia 2019*



# Part Connection Issue

## Post-processing



*Optimize part poses  
using the predicted part  
adjacency relationships*

Yang and Mo et al., “**DSG-Net: Learning Disentangled Structure and Geometry for 3D Shape Generation**”, *ACM ToG* 2021

## Joint Re-synthesis

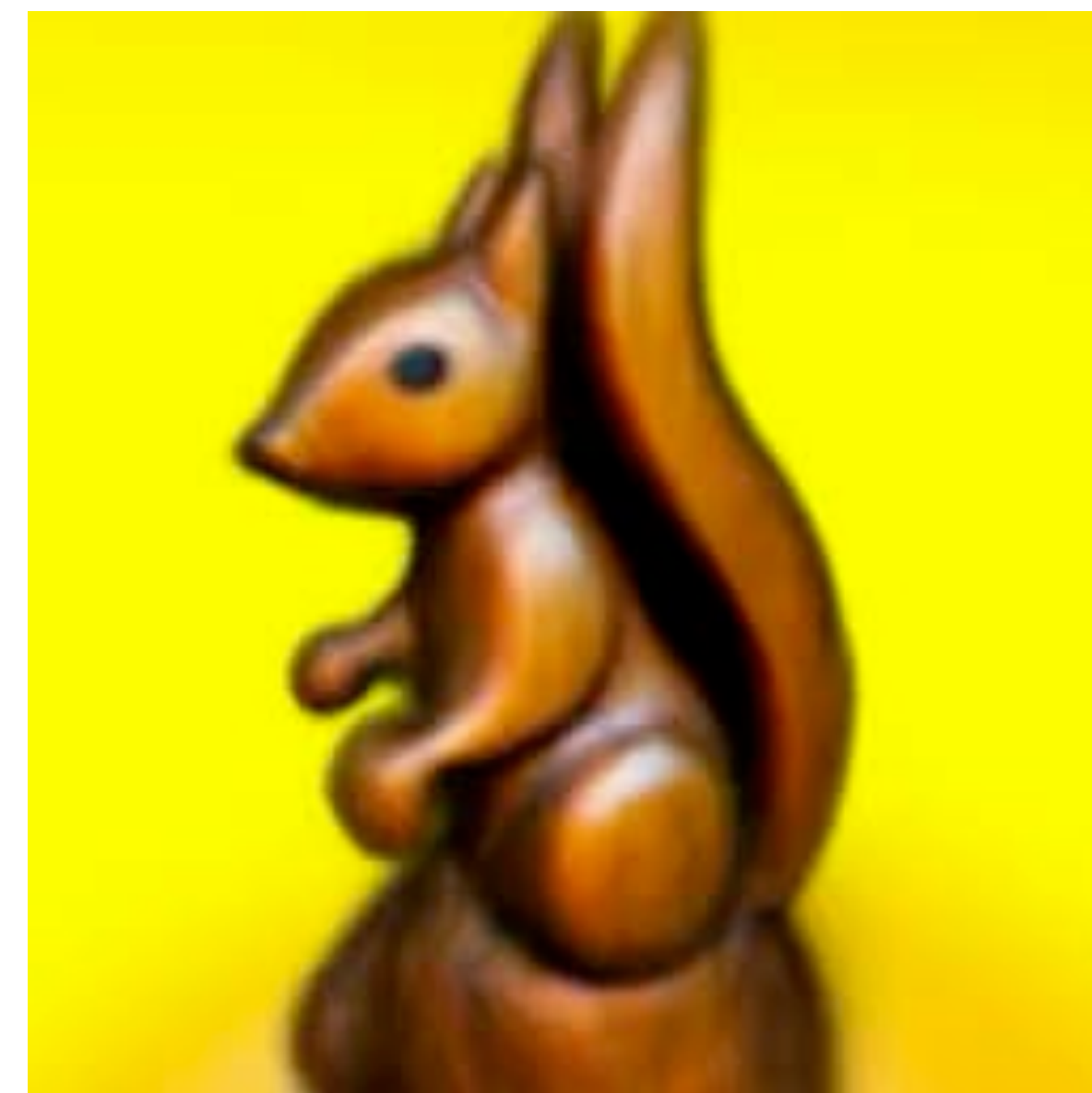
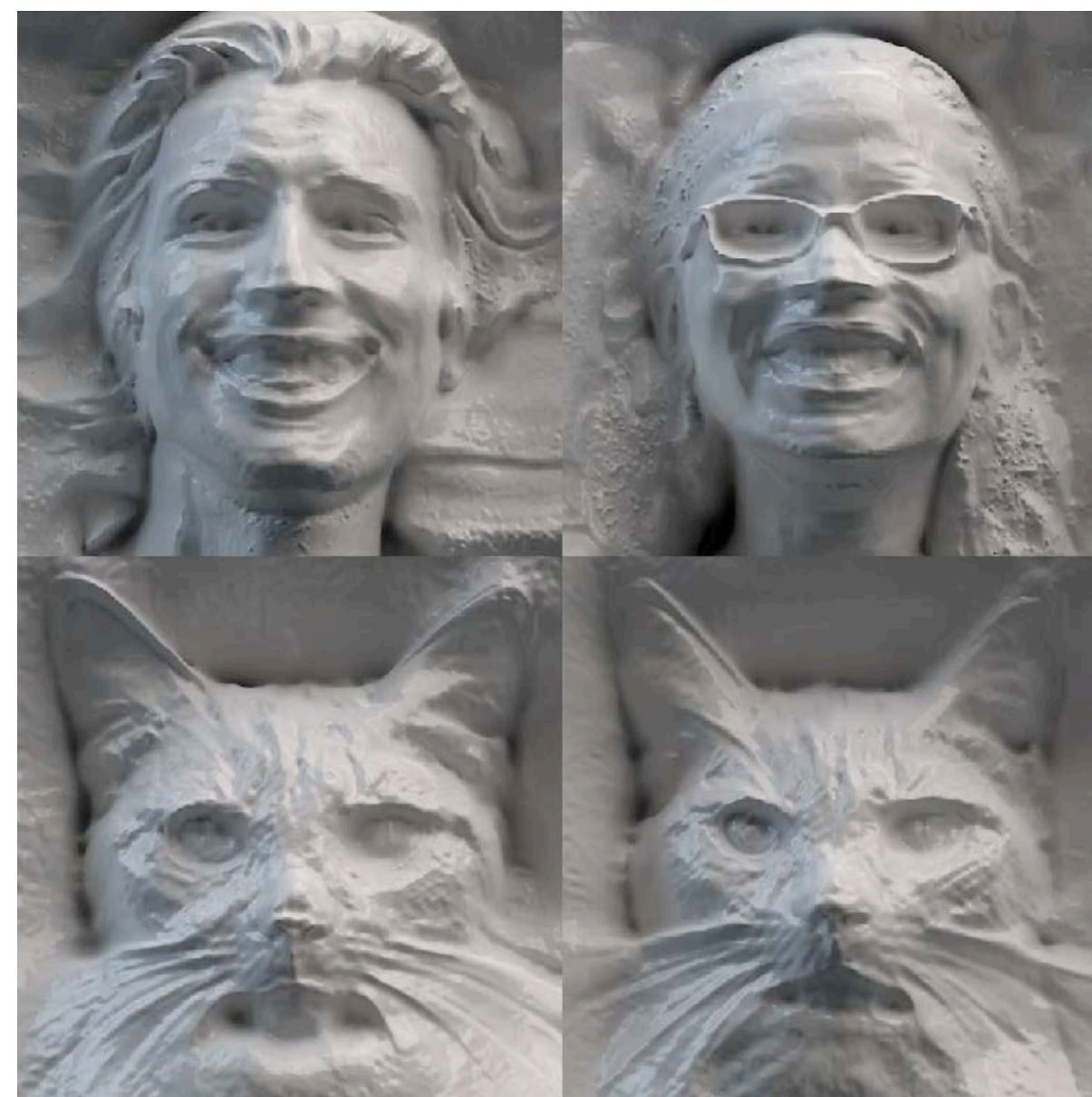
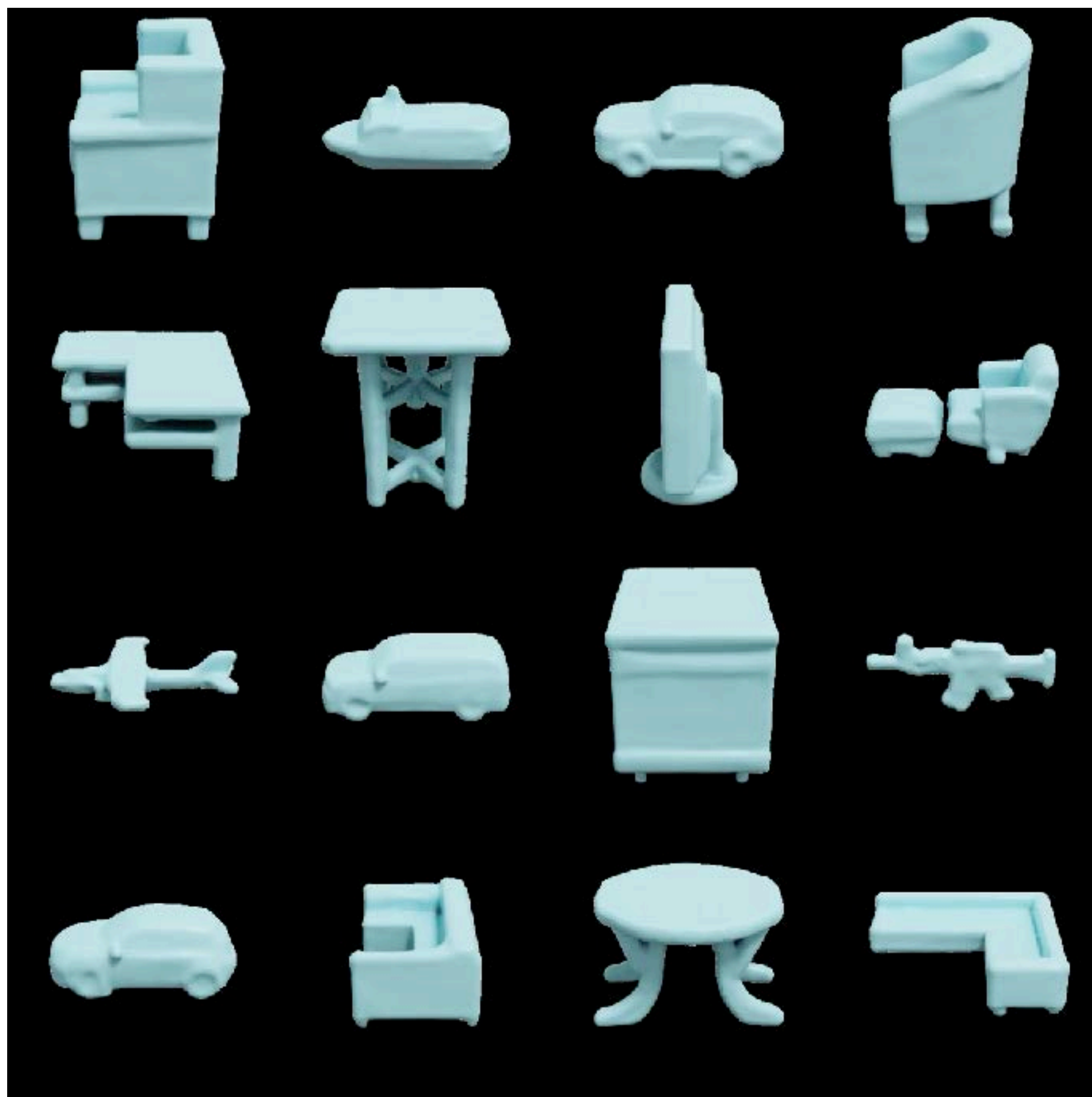


*Erode the part connection  
regions and re-generate  
the local geometry*

Yin et al., “**COALESCE: Component Assembly by Learning to Synthesize Connections**”, *3DV* 2020



# Summary





# Today's Focus





# Outline

- Voxel Networks
- Point Networks
  - PointNet
  - PointNet++
  - Sampling Issues



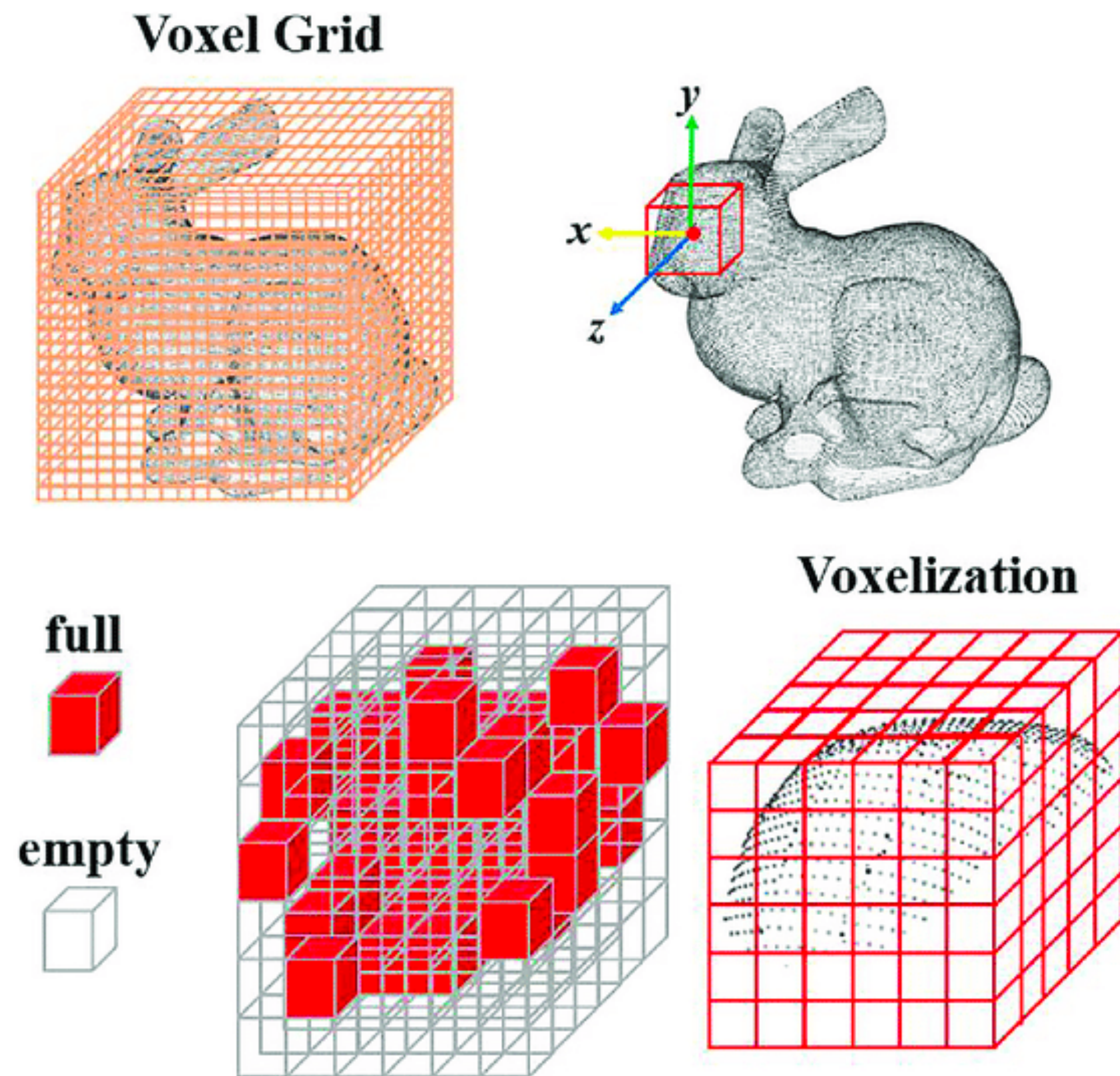
# Outline

- *Voxel Networks*
- Point Networks
  - PointNet
  - PointNet++
  - Sampling Issues



# Voxelization

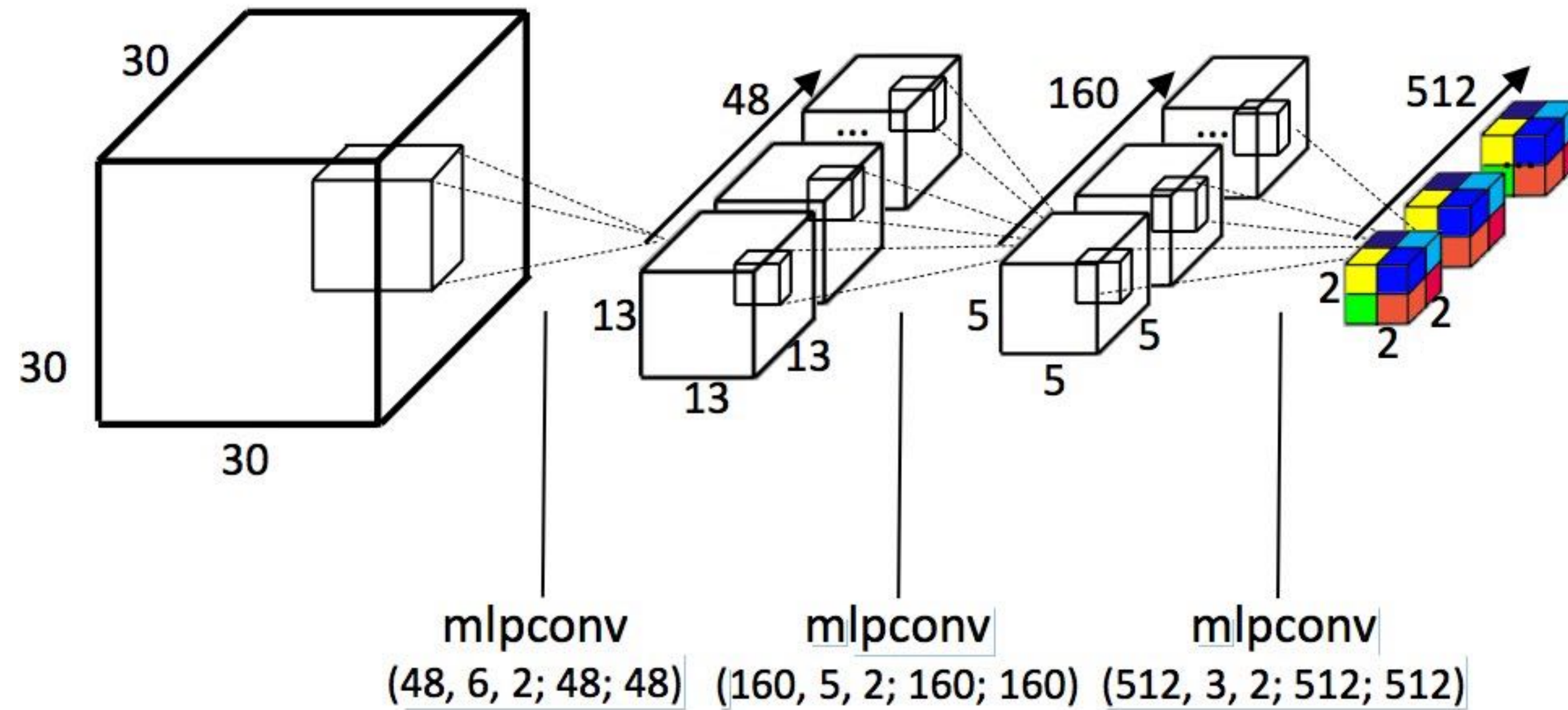
Represent the occupancy of regular 3D grids





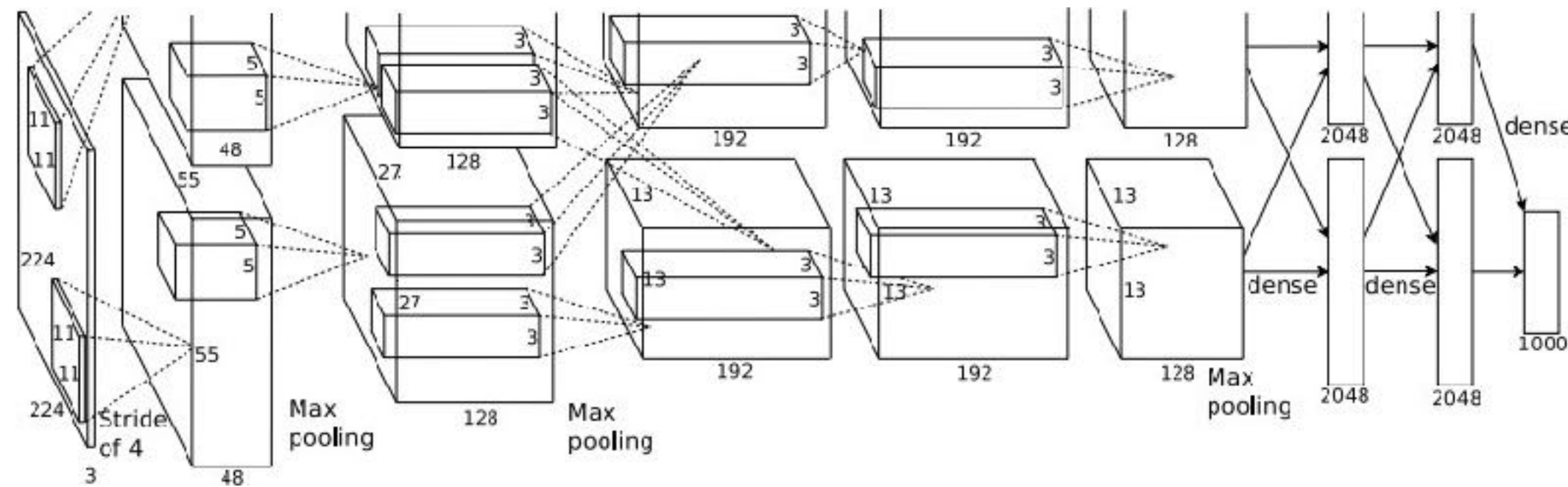
# 3D CNN on Volumetric Data

3D convolution uses 4D kernels





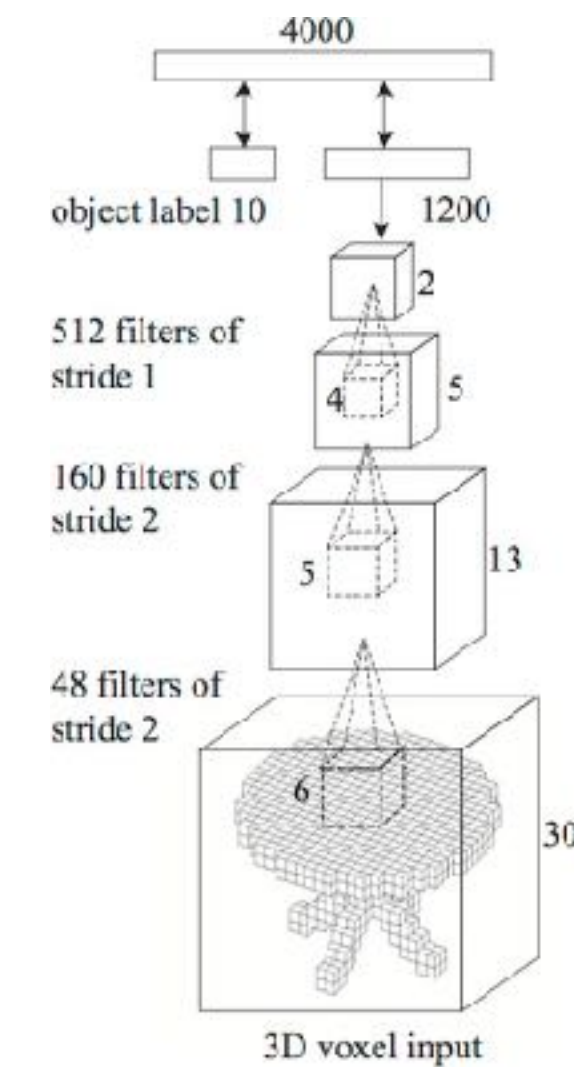
# Complexity Issue



## AlexNet, 2012

Input resolution: 224x224

$$224 \times 224 = 50176$$



## 3DShapeNets, 2015

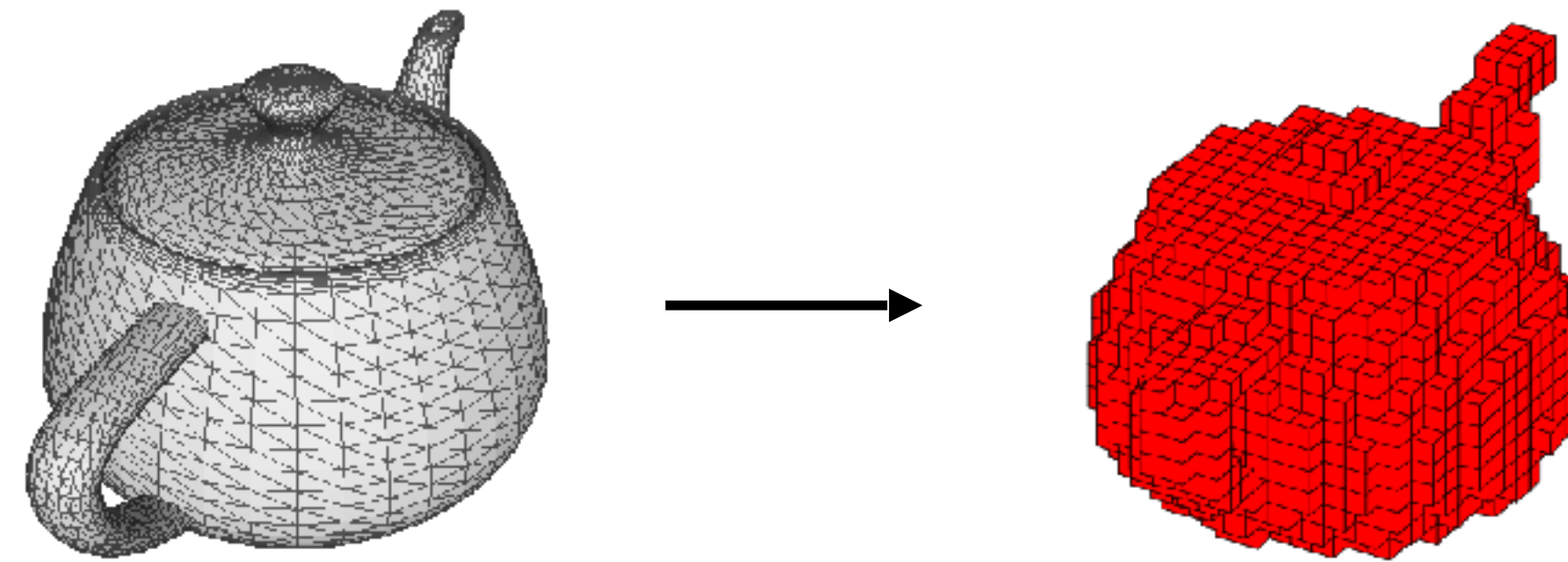
Input resolution:

$$30 \times 30 \times 30$$

$$27000$$



# Complexity Issue



Polygon  
Mesh

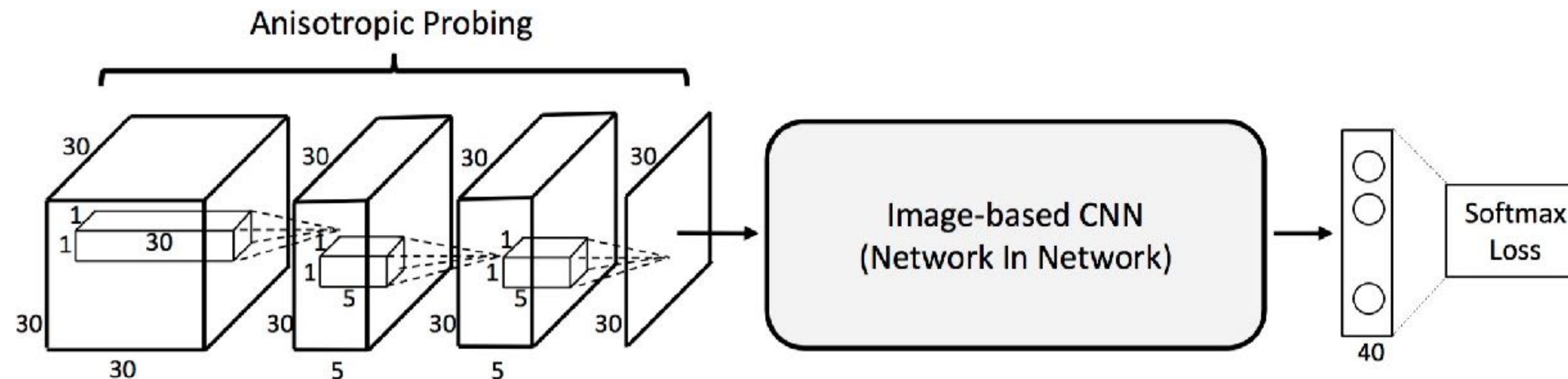
Occupancy Grid  
30x30x30

**Information loss in voxelization**



# Idea 1: Learn to Project

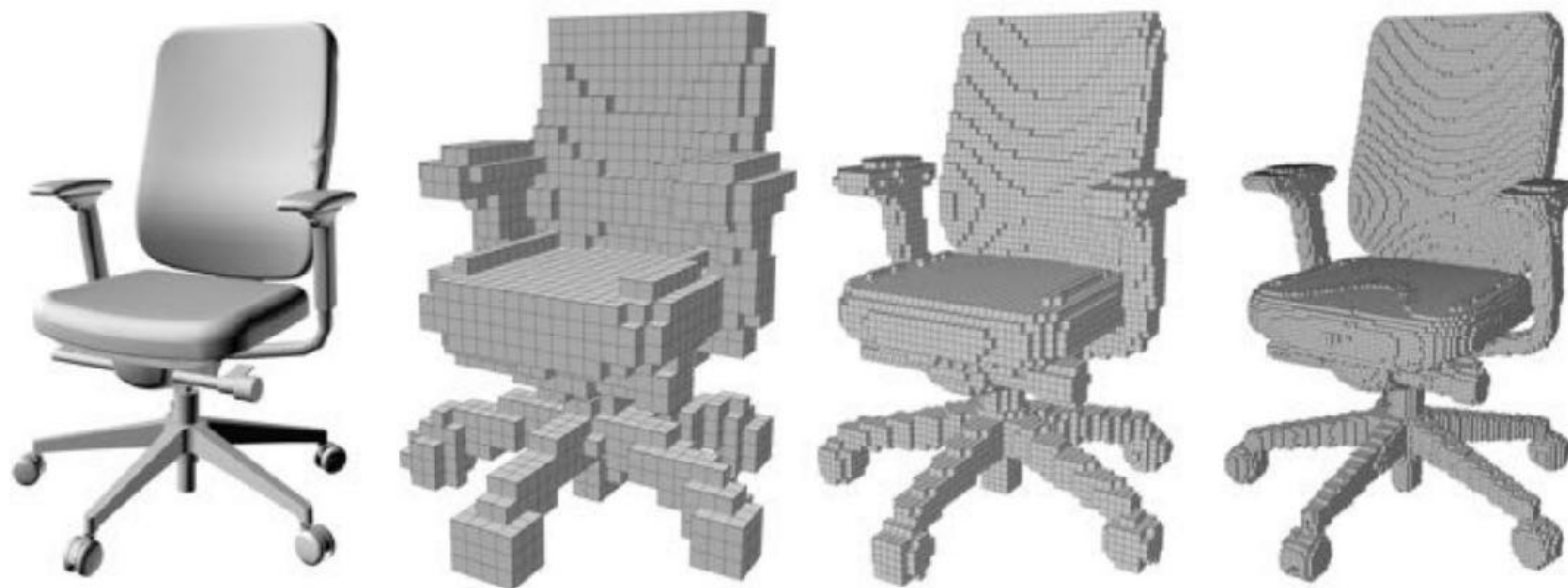
*Idea: “X-ray” rendering + Image (2D) CNNs  
very low #param, very low computation*



Su et al., “Volumetric and Multi-View CNNs for Object Classification on 3D Data”, *CVPR 2016*

Many other works in autonomous driving use **bird's eye view** for object detection

# More Principled: Sparsity of 3D Shapes



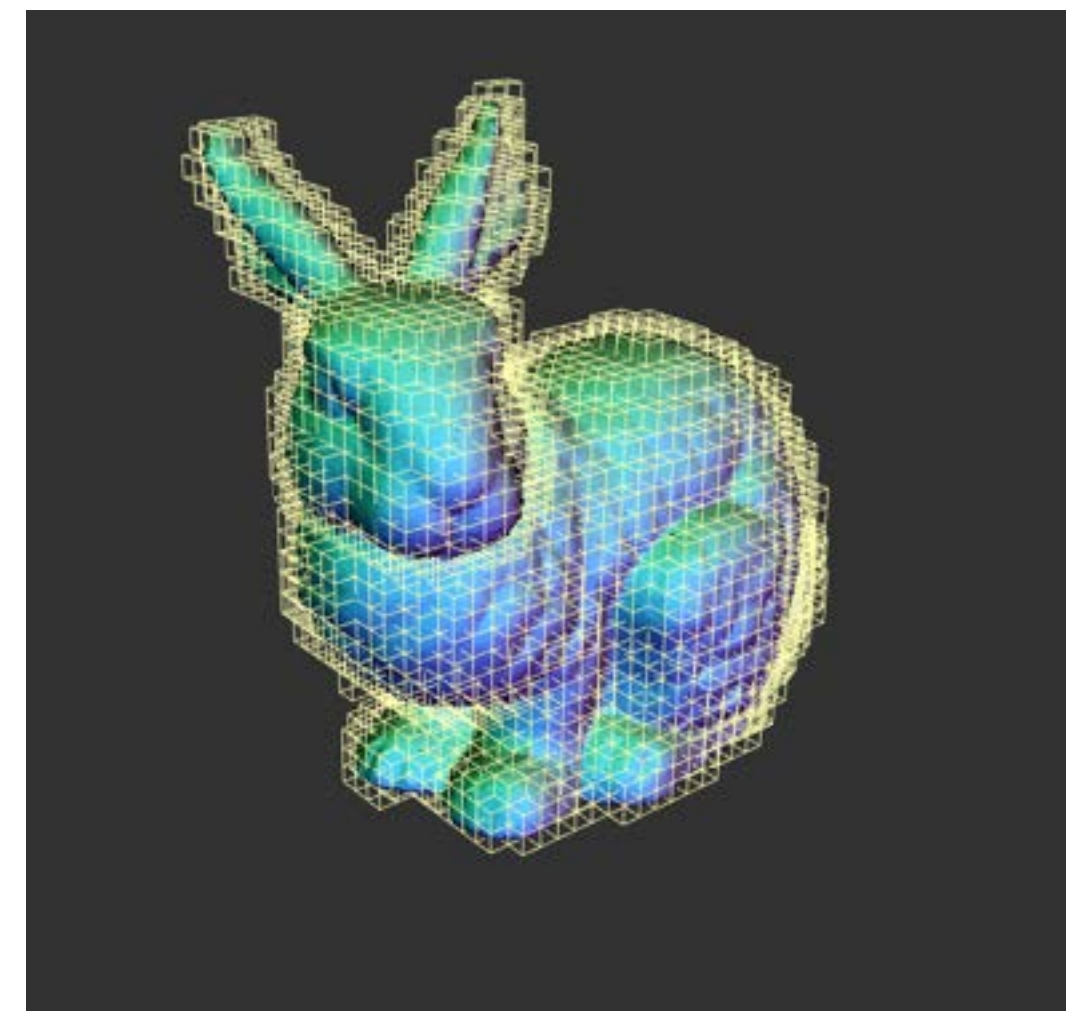
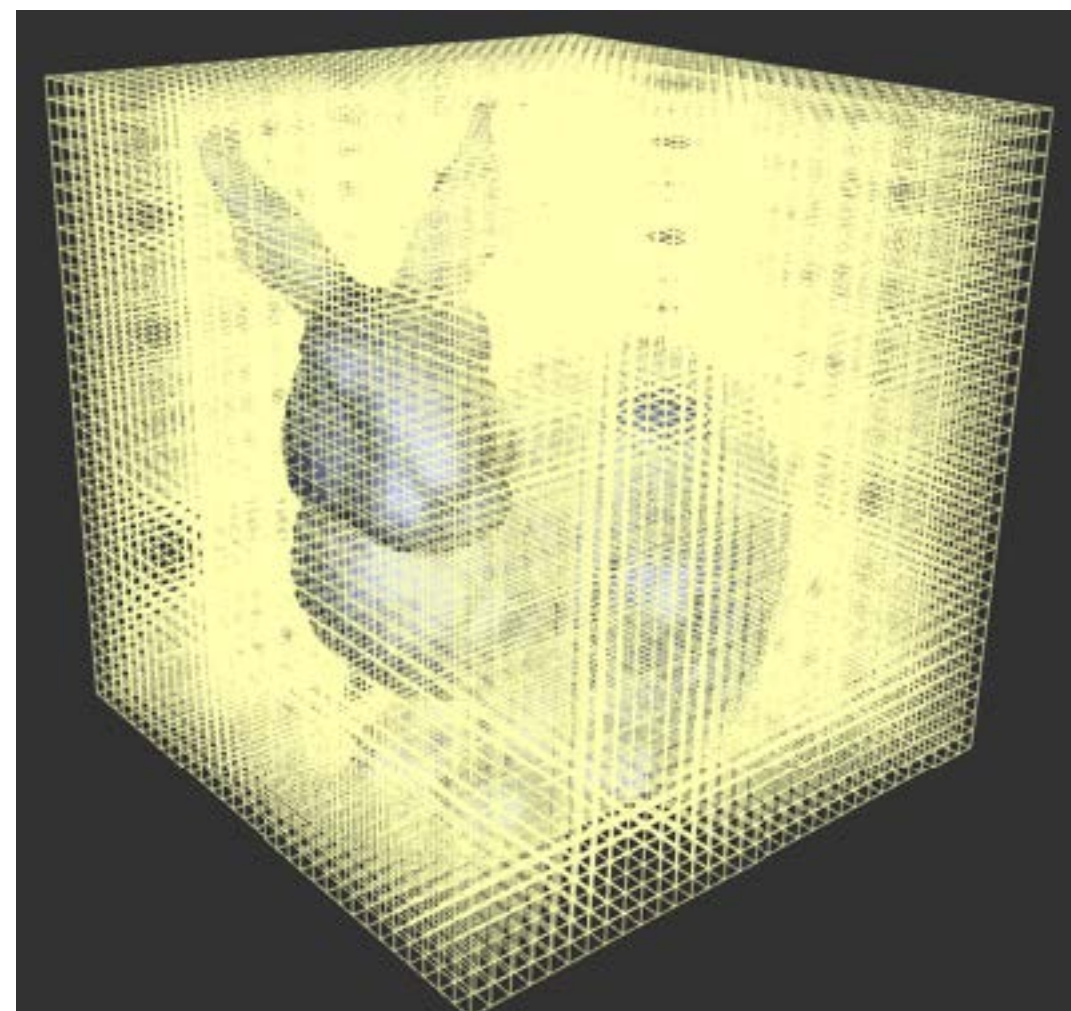
$\frac{\#occupied\ grid}{\#total\ grid}$

|             |        |       |       |
|-------------|--------|-------|-------|
| Occupancy:  | 10.41% | 5.09% | 2.41% |
| Resolution: | 32     | 64    | 128   |

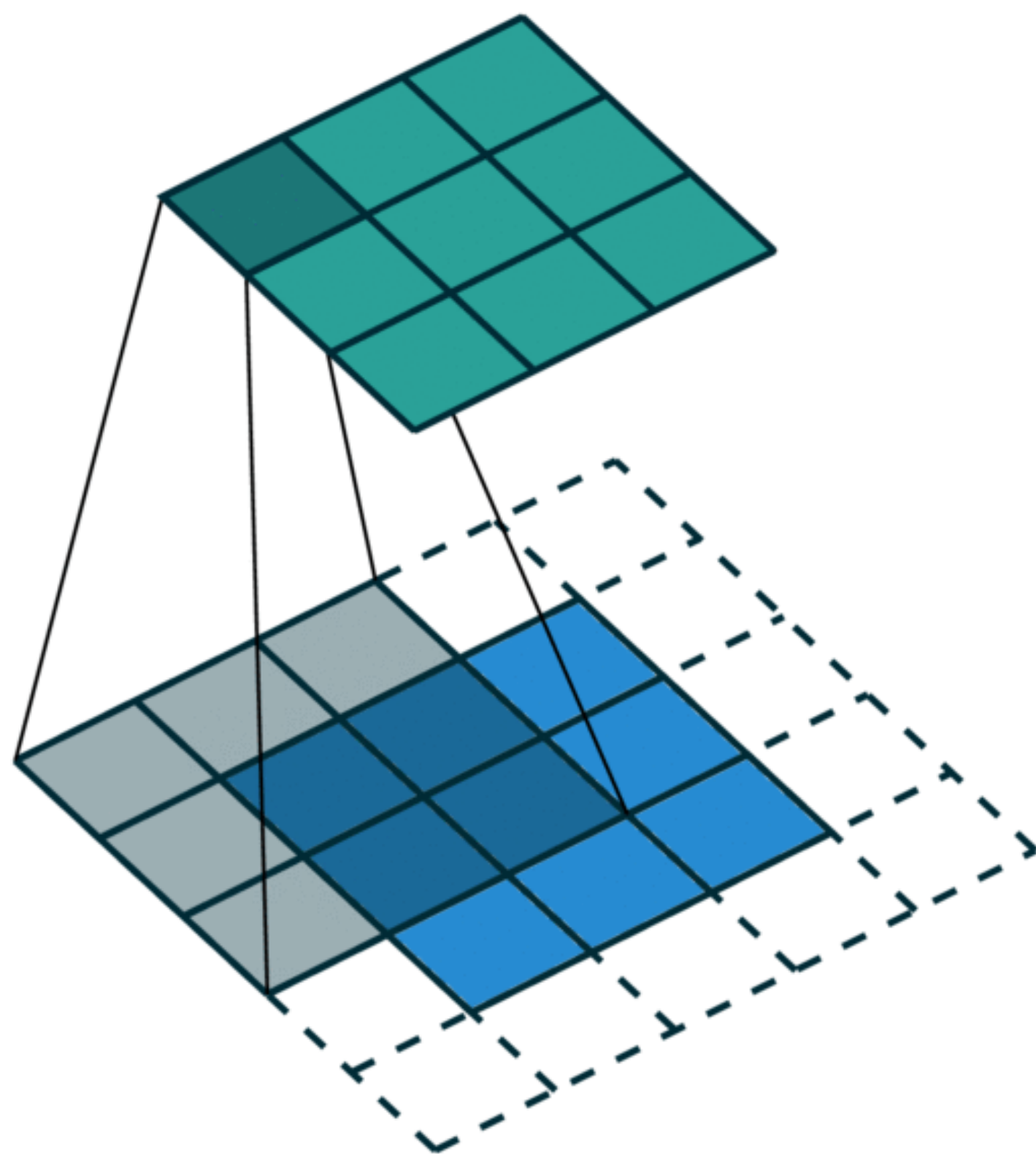


# Store only the Occupied Grids

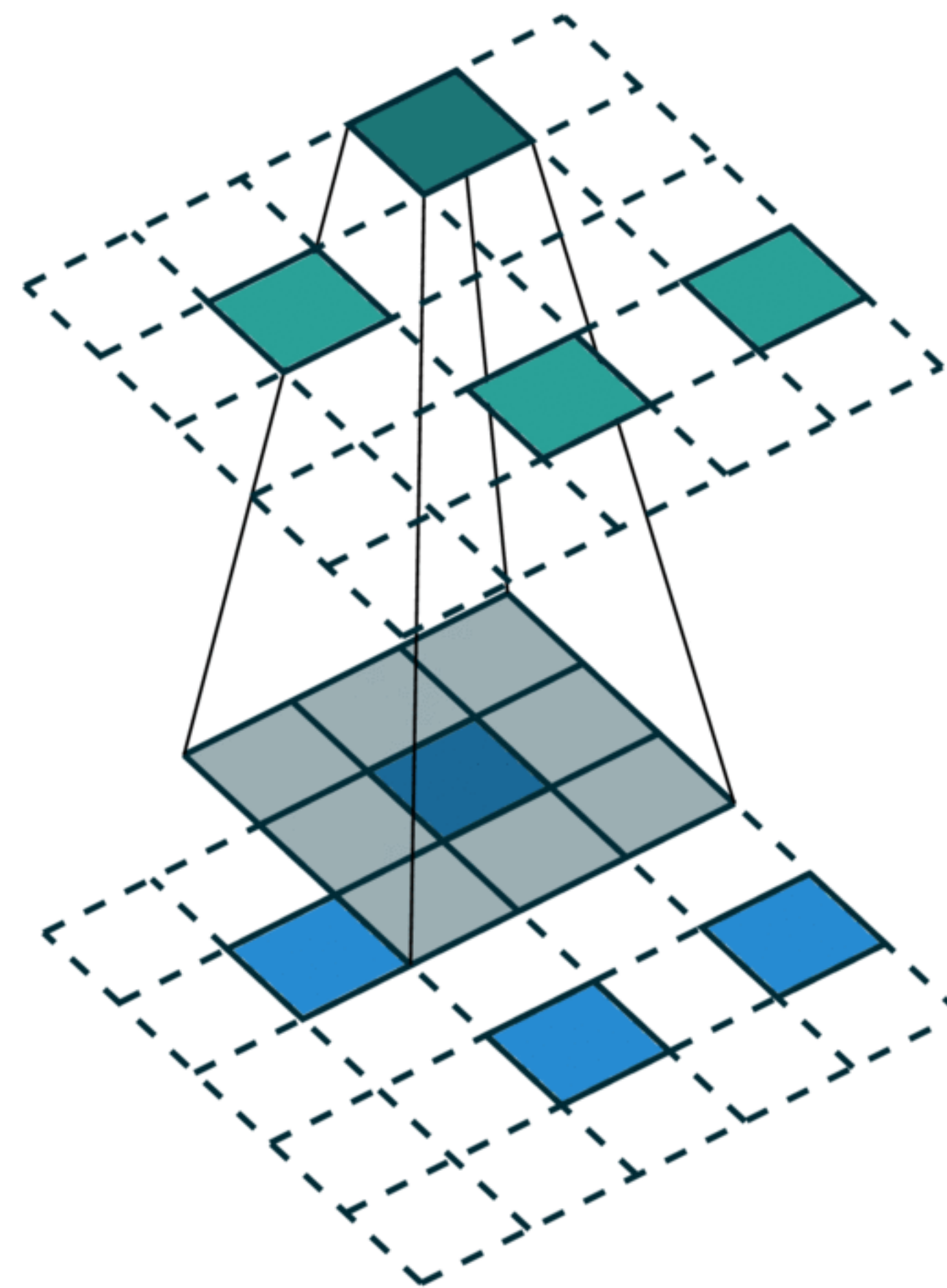
- Store the sparse surface signals
- Constrain the computation near the surface



# Sparse Convolution



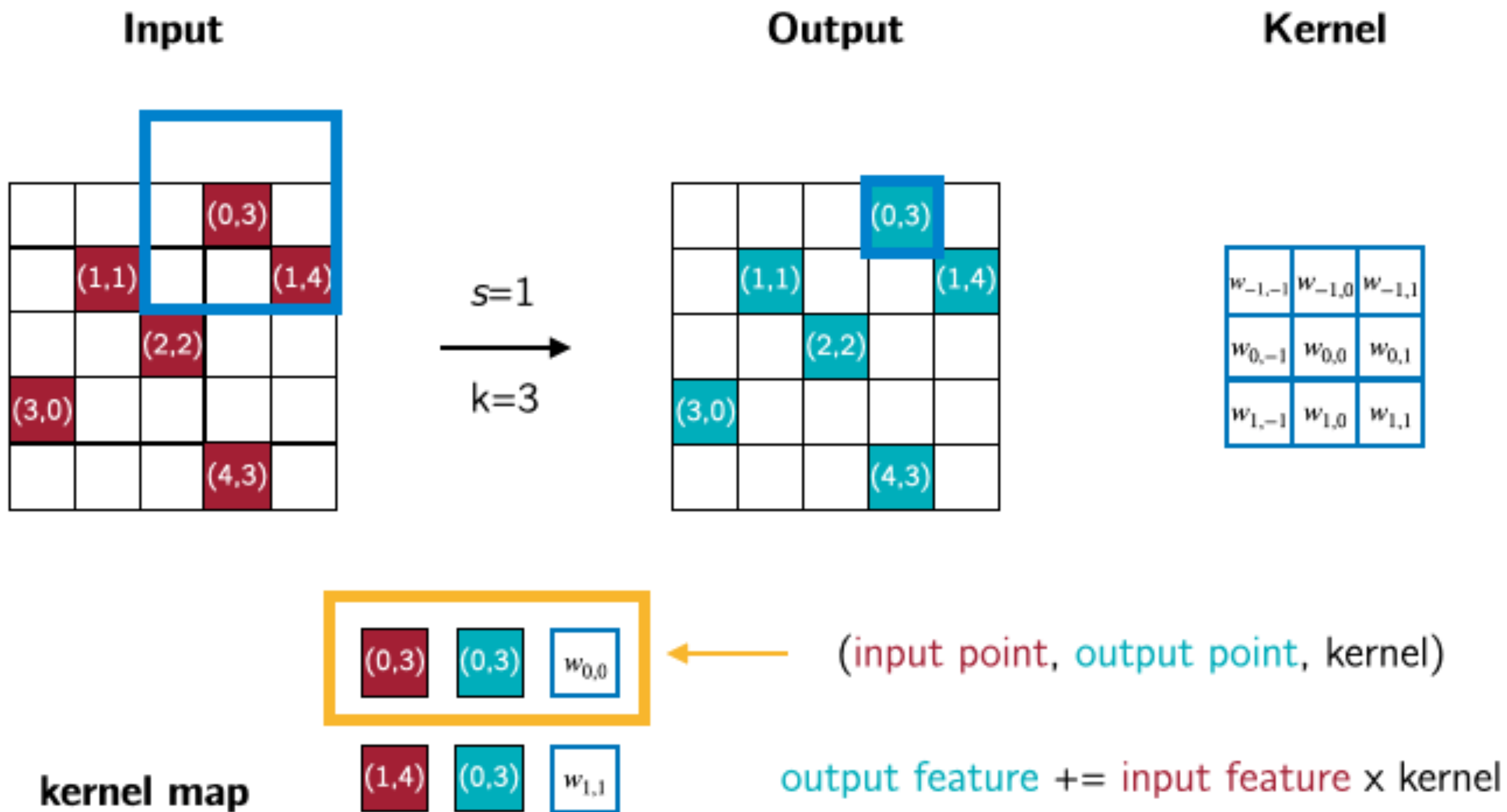
Dense Conv



Sparse Conv



# Sparse Convolution



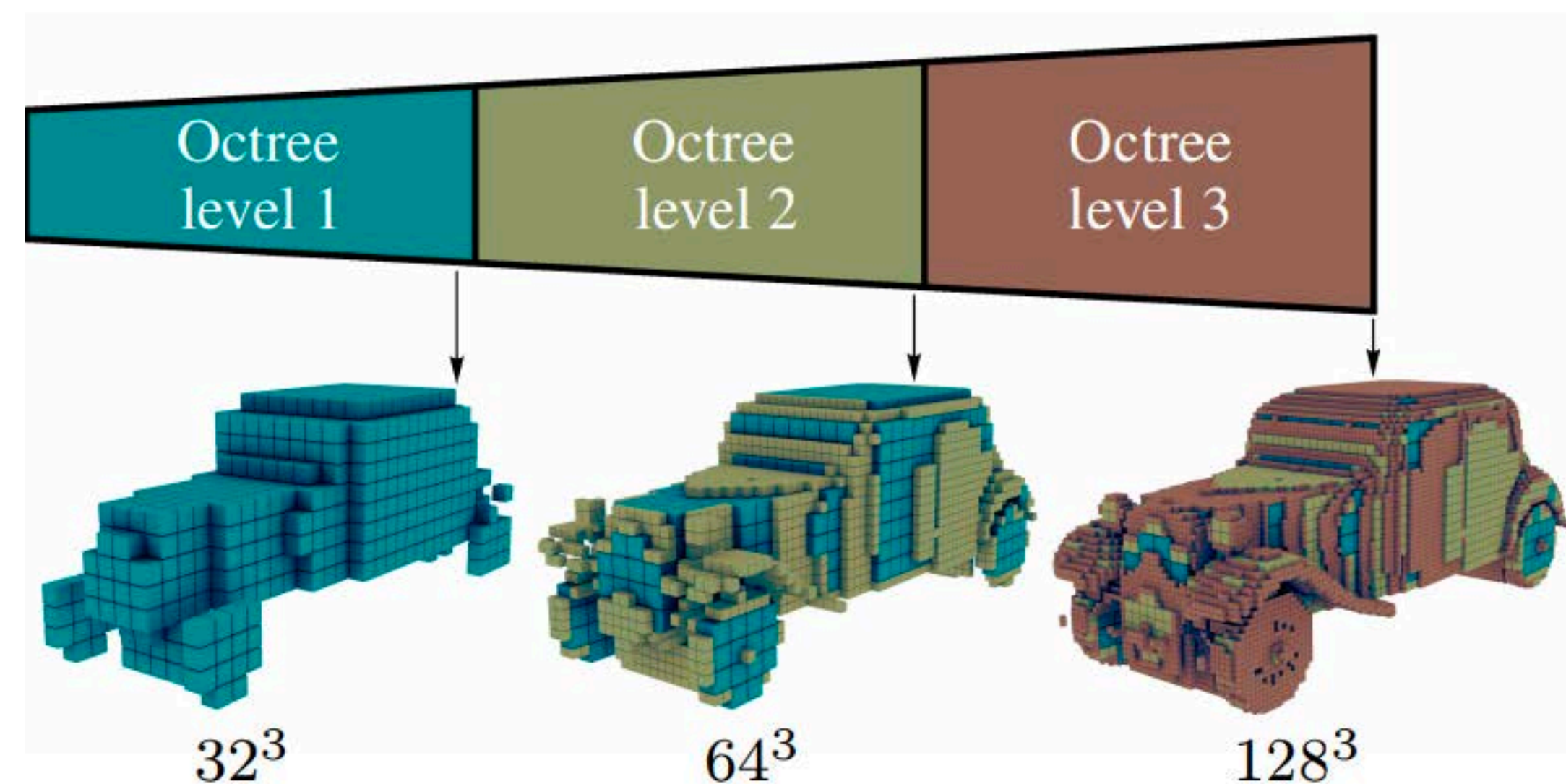
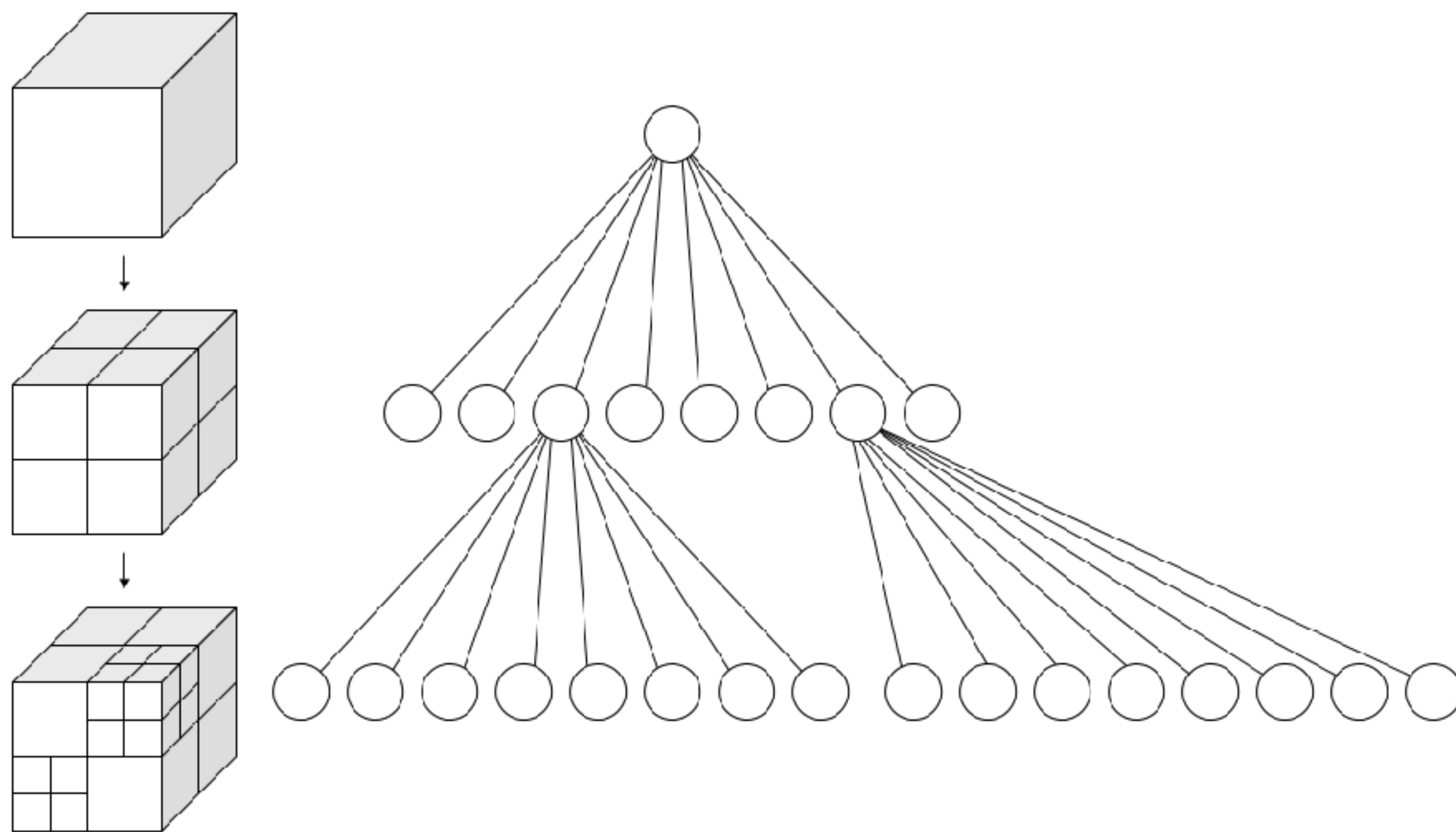
# Implementation

- SparseConvNet
  - [https://github.com/facebookresearch/ SparseConvNet](https://github.com/facebookresearch/SparseConvNet)
  - Uses ResNet architecture
  - Takes time to train
- MinkowskiEngine
  - <https://github.com/NVIDIA/MinkowskiEngine>
- TorchSparse
  - <https://github.com/mit-han-lab/torchsparse>
- Tensorflow3D
  - <https://github.com/google-research/google-research/tree/master/tf3d>

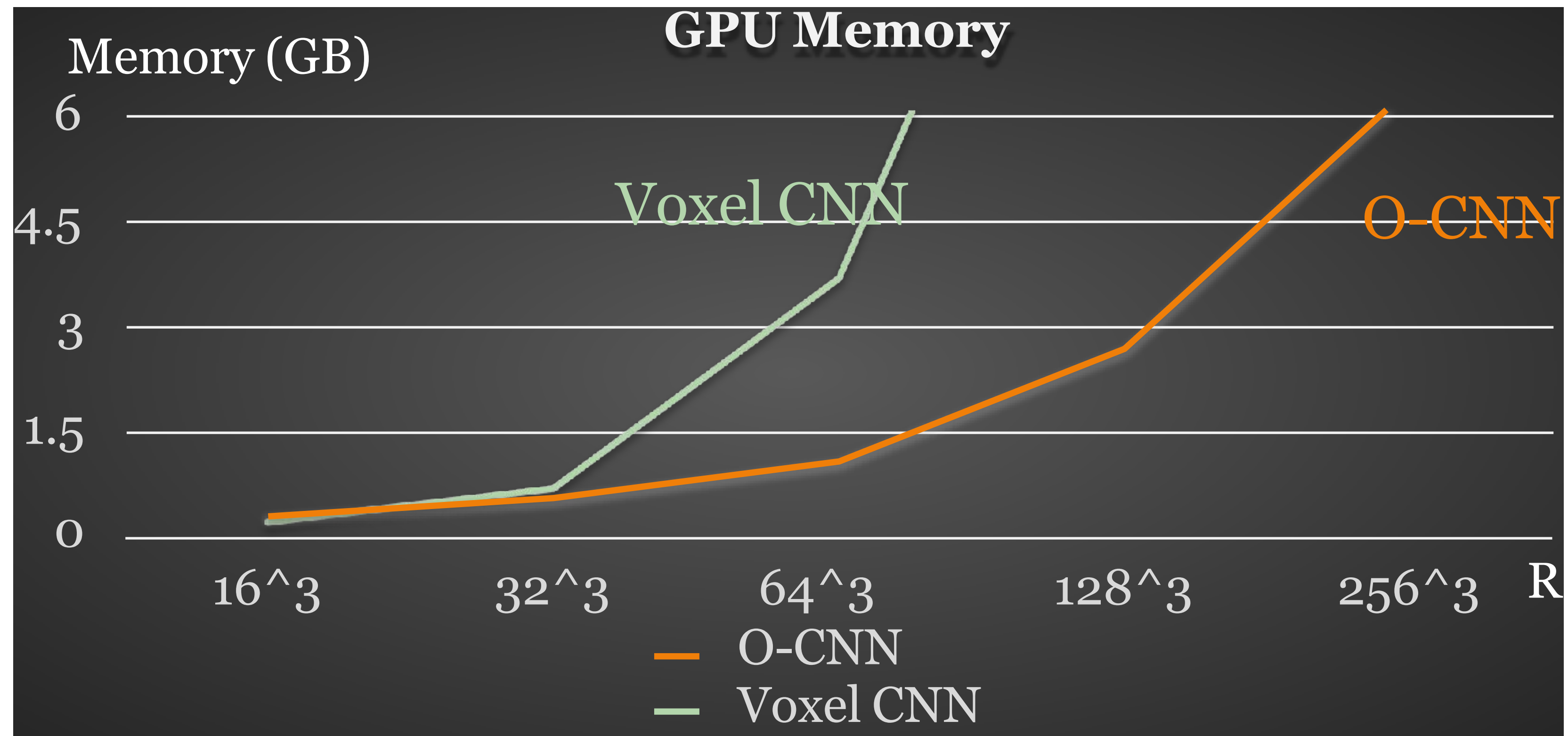


# Octree: Recursively Partition the Space

- Each internal node has exactly eight children
- Neighborhood searching: Hash table



# Memory Efficiency

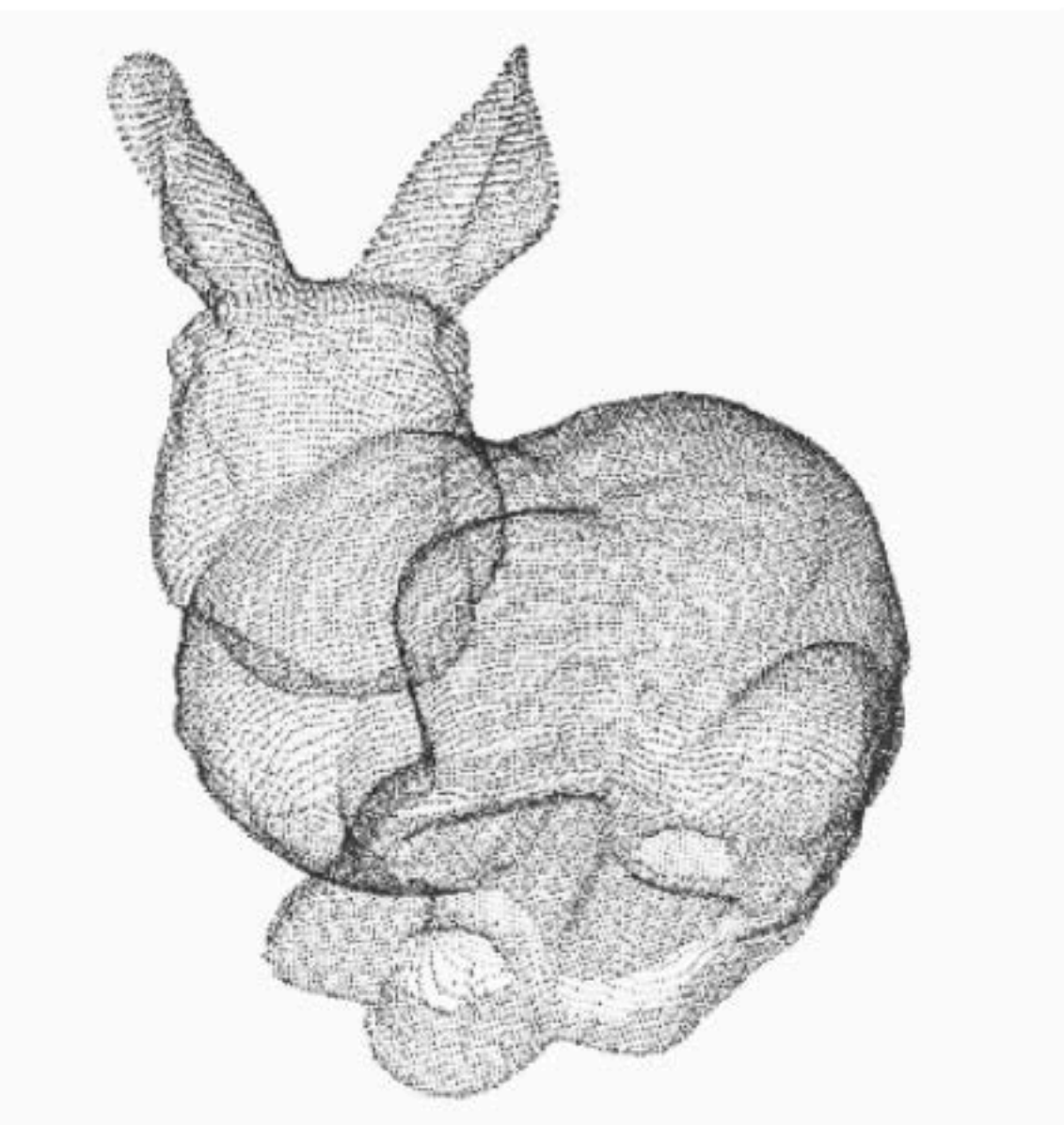




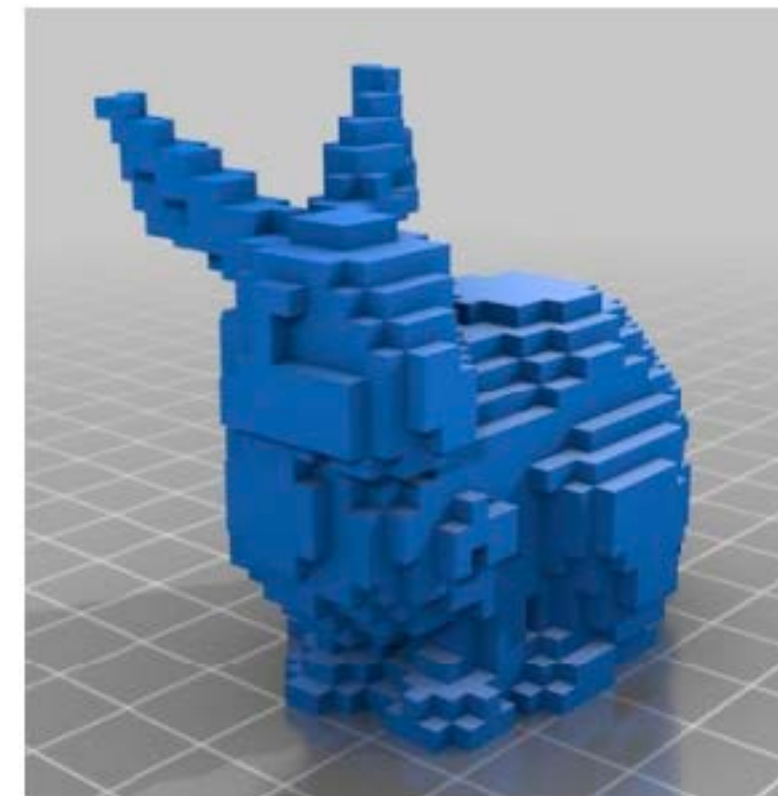
# Outline

- Voxel Networks
- Point Networks
  - *PointNet*
  - PointNet++
  - Sampling Issues

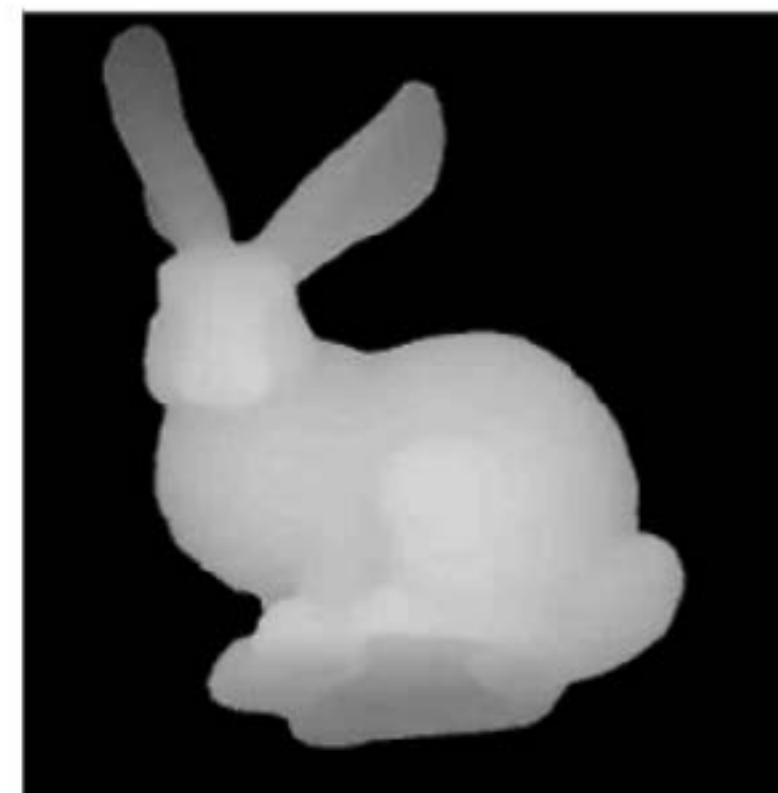
# Straightforward ways of processing point clouds



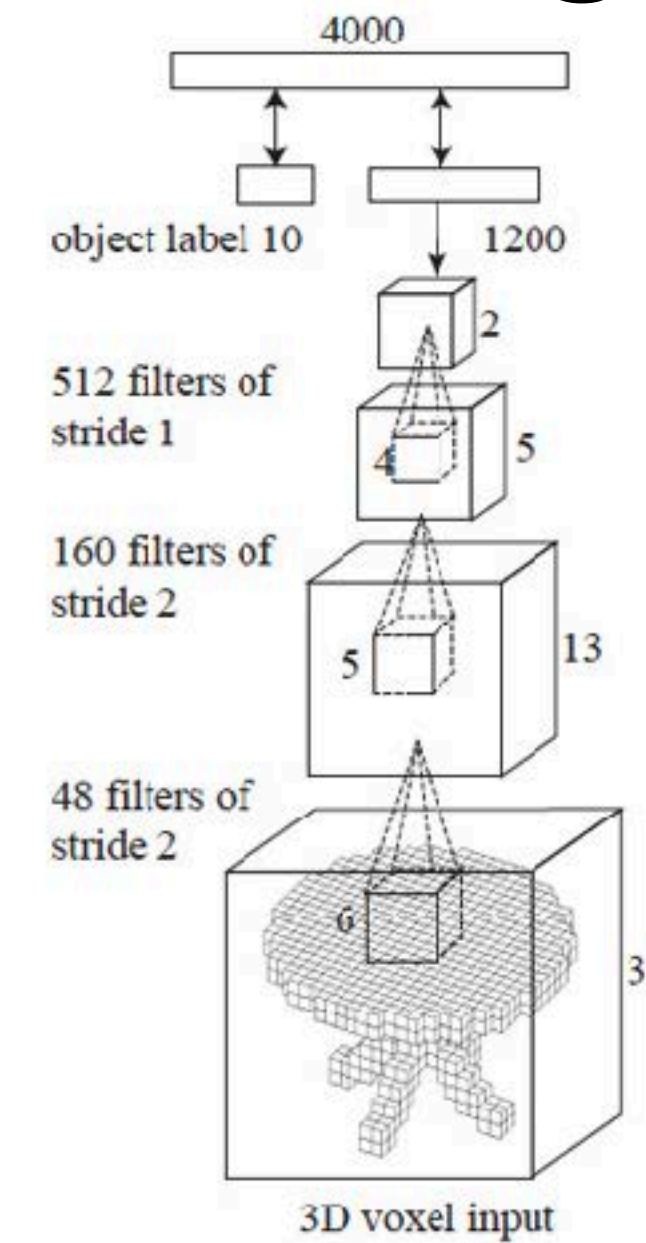
point cloud



voxel grids



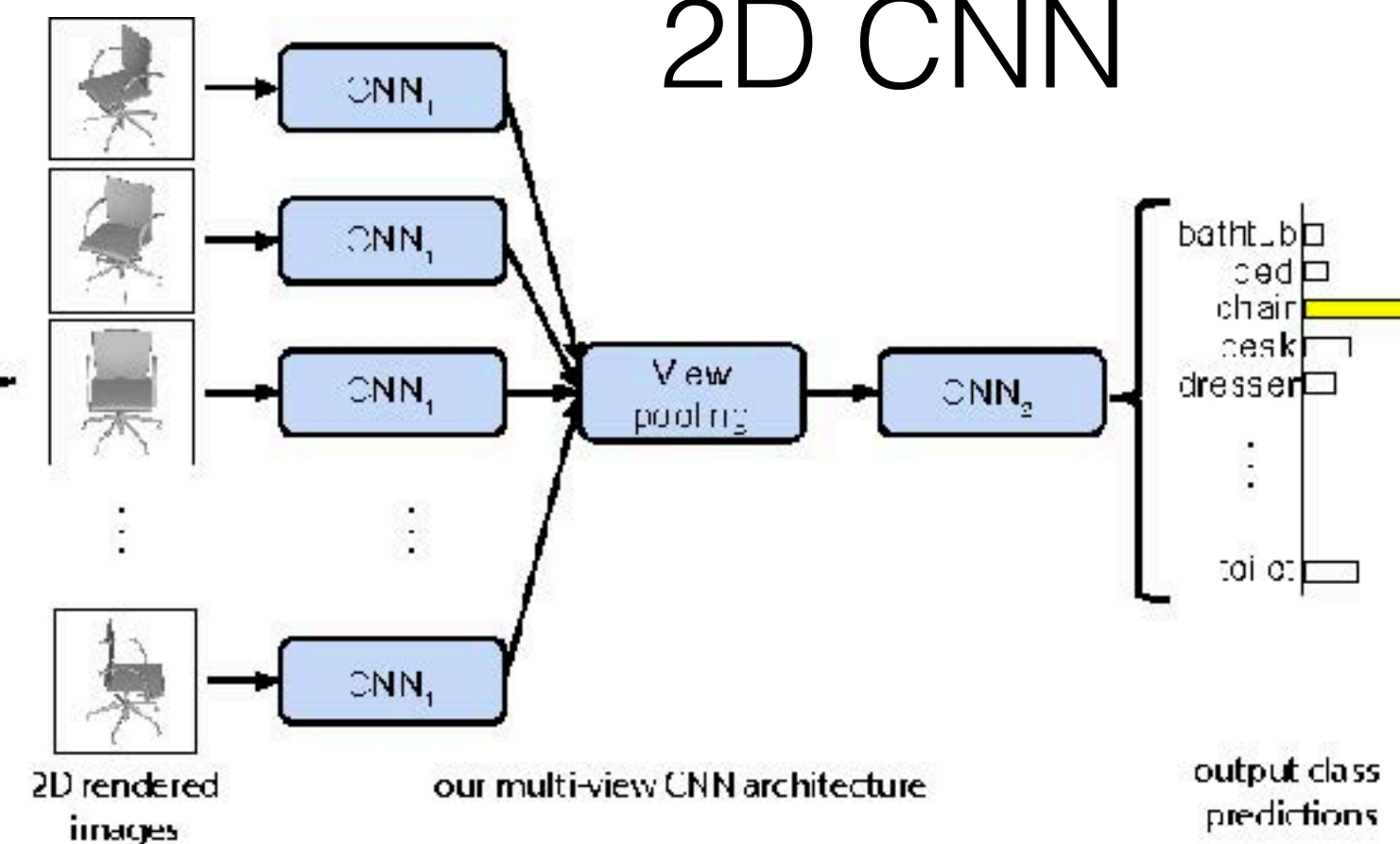
2D projection



3D CNN



3D shape model  
rendered with  
different virtual cameras



2D CNN

output class  
predictions



# Regular image grid v.s. irregular 3D point clouds

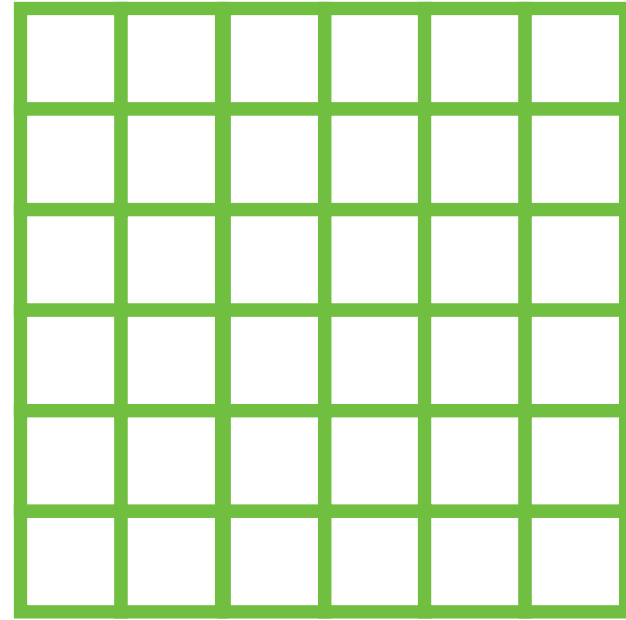
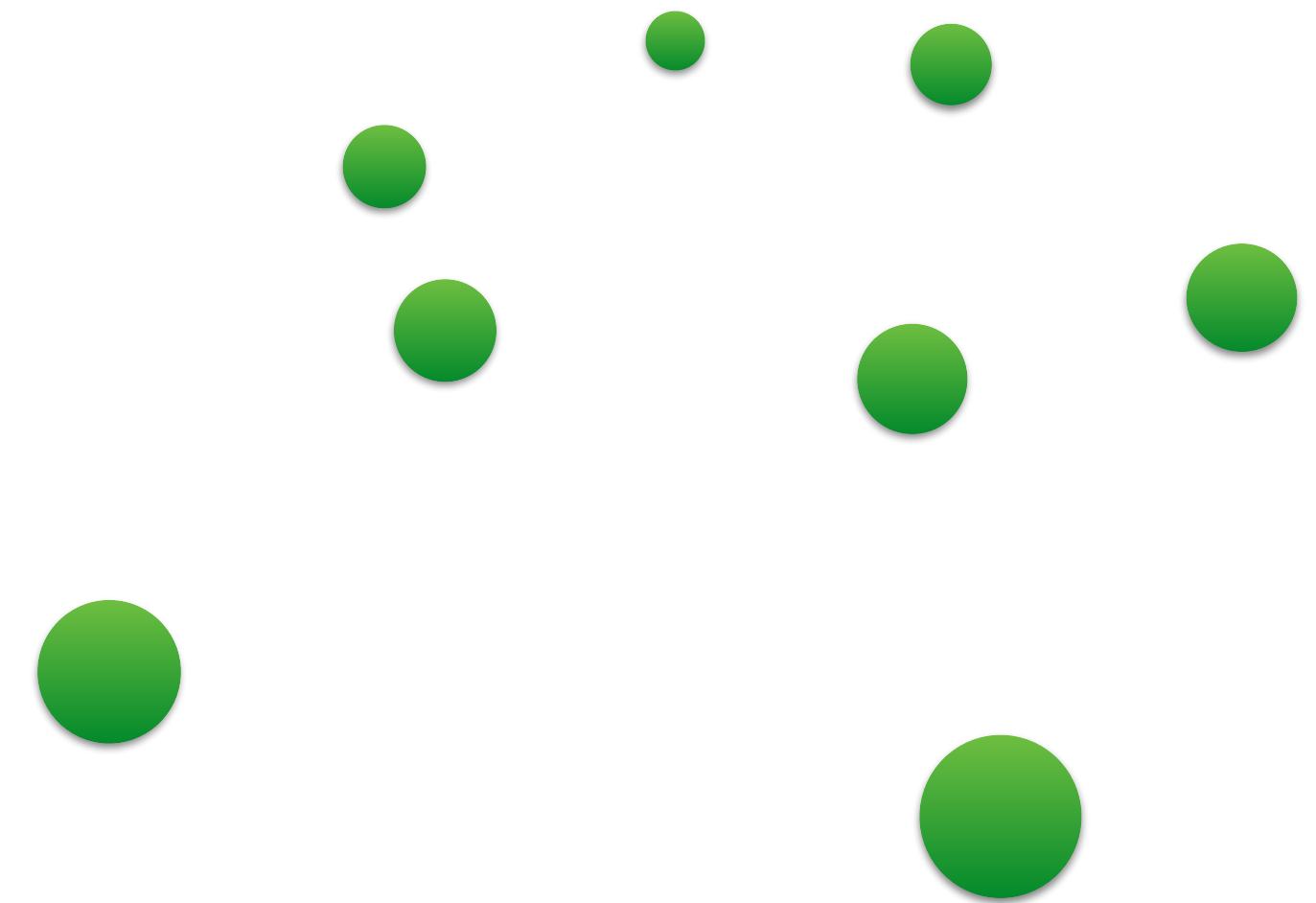


Image grid



Point cloud

# Regular image grid v.s. irregular 3D point clouds

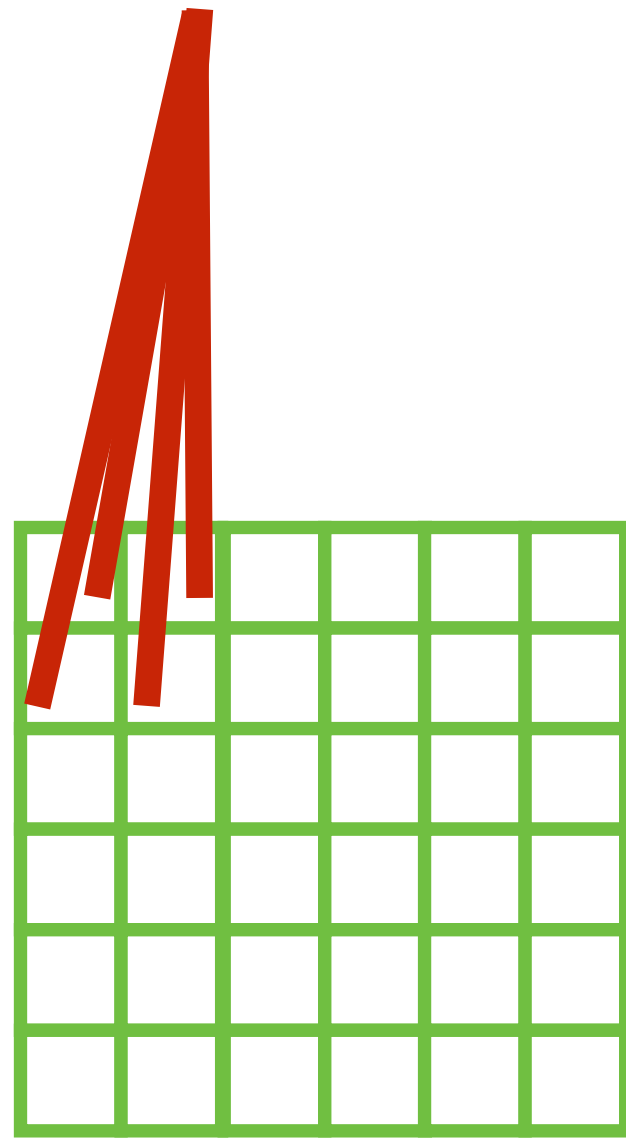
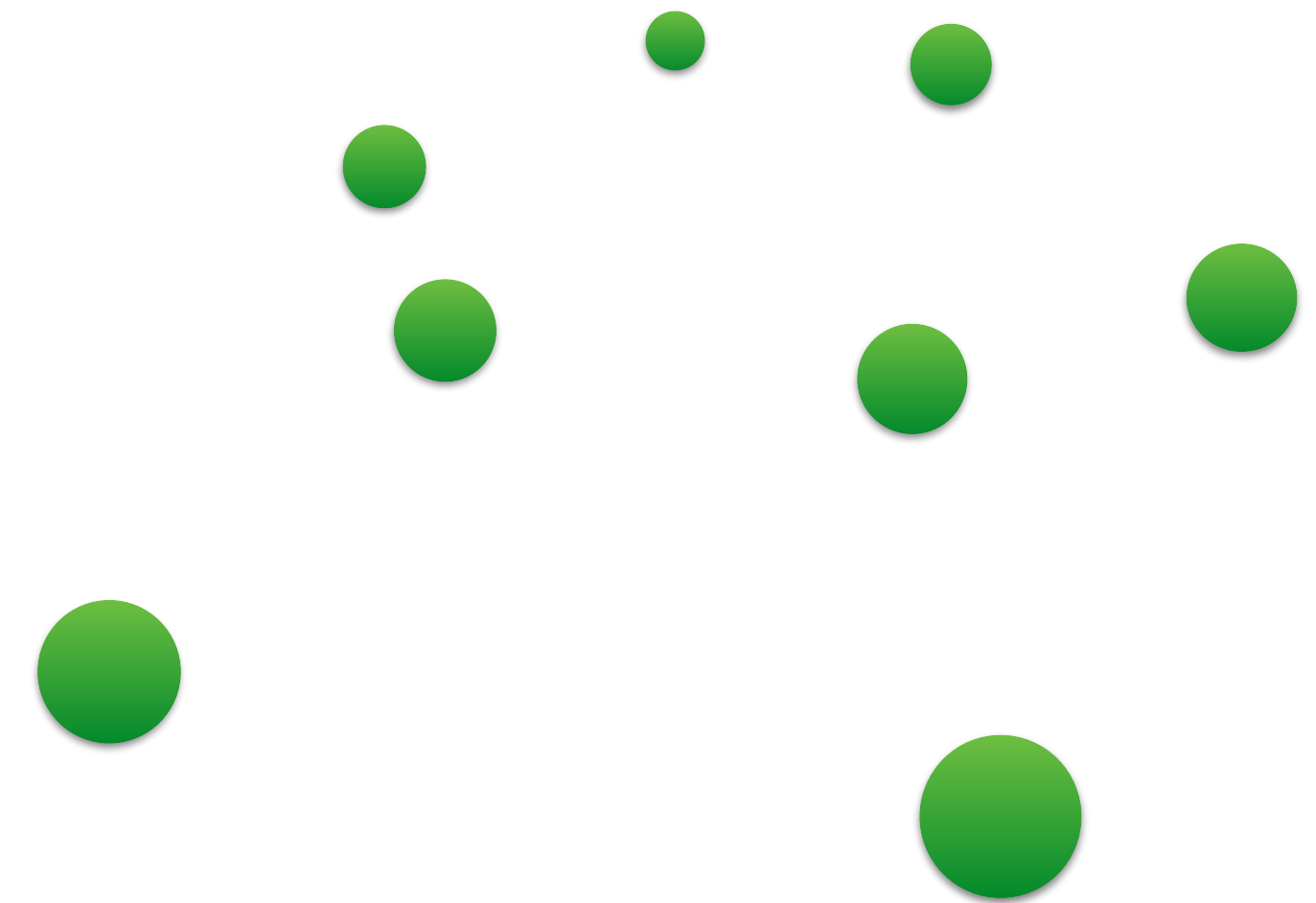


Image grid



Point cloud



# Regular image grid v.s. irregular 3D point clouds

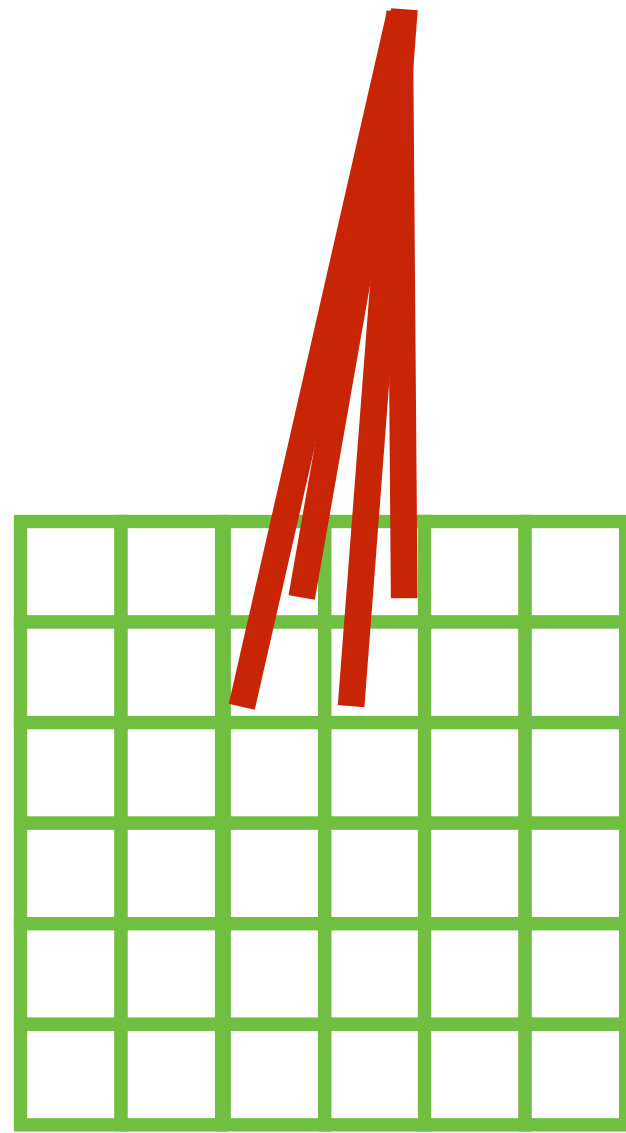
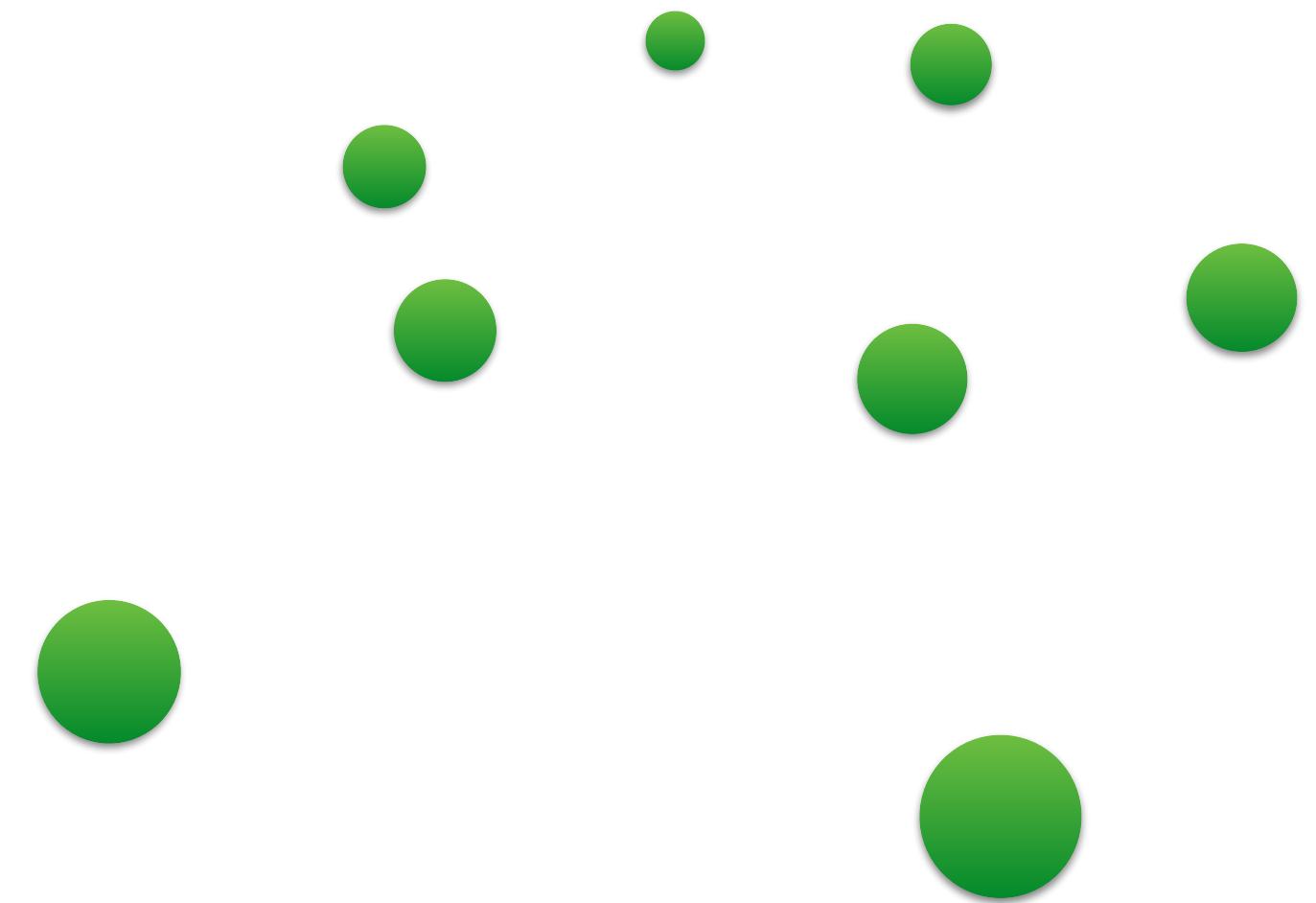


Image grid



Point cloud

# Regular image grid v.s. irregular 3D point clouds

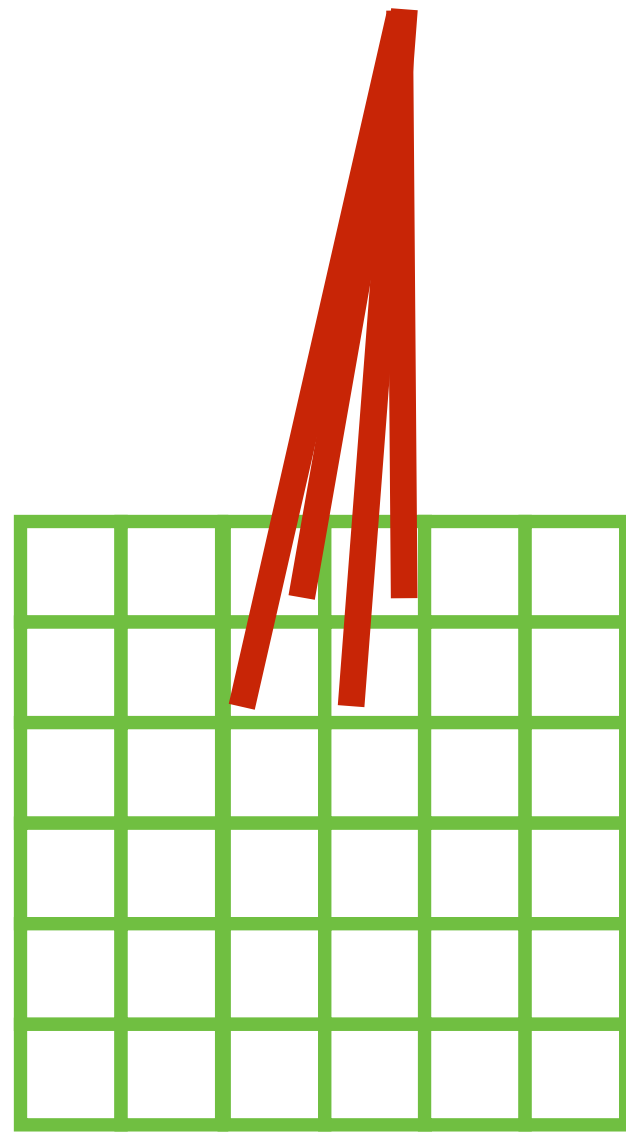
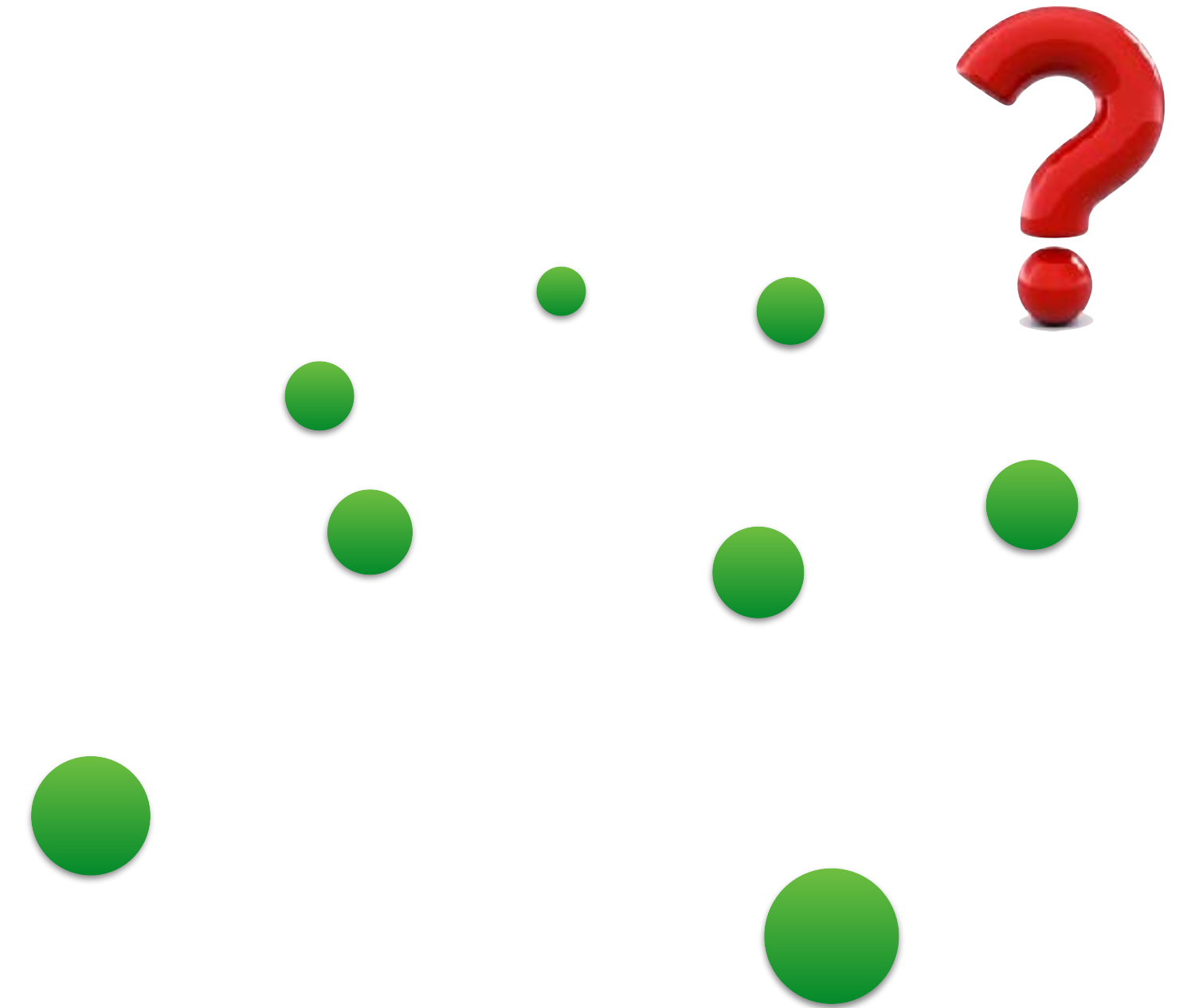


Image grid

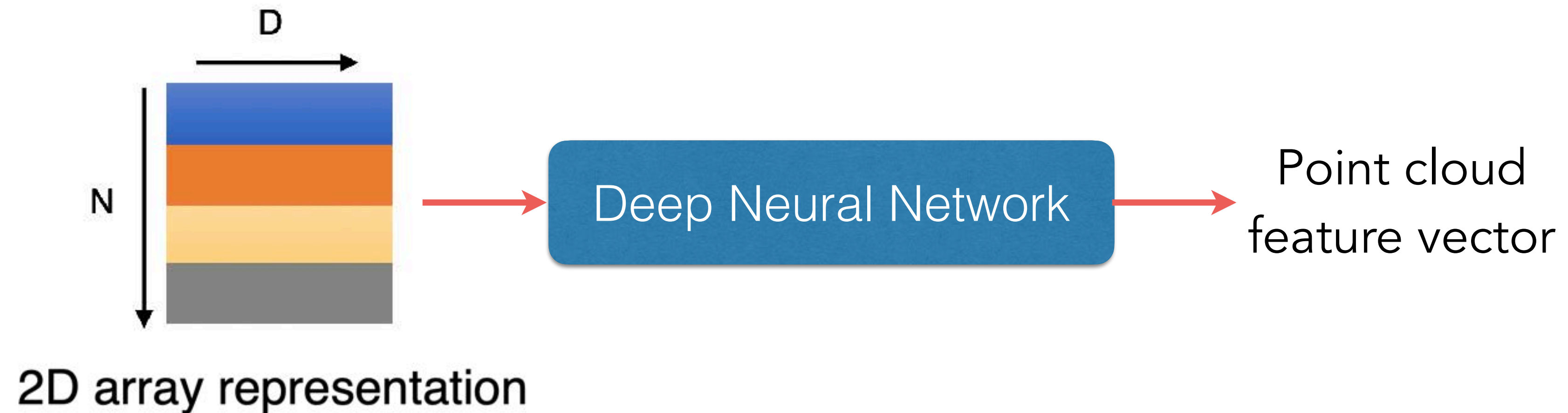


Point cloud



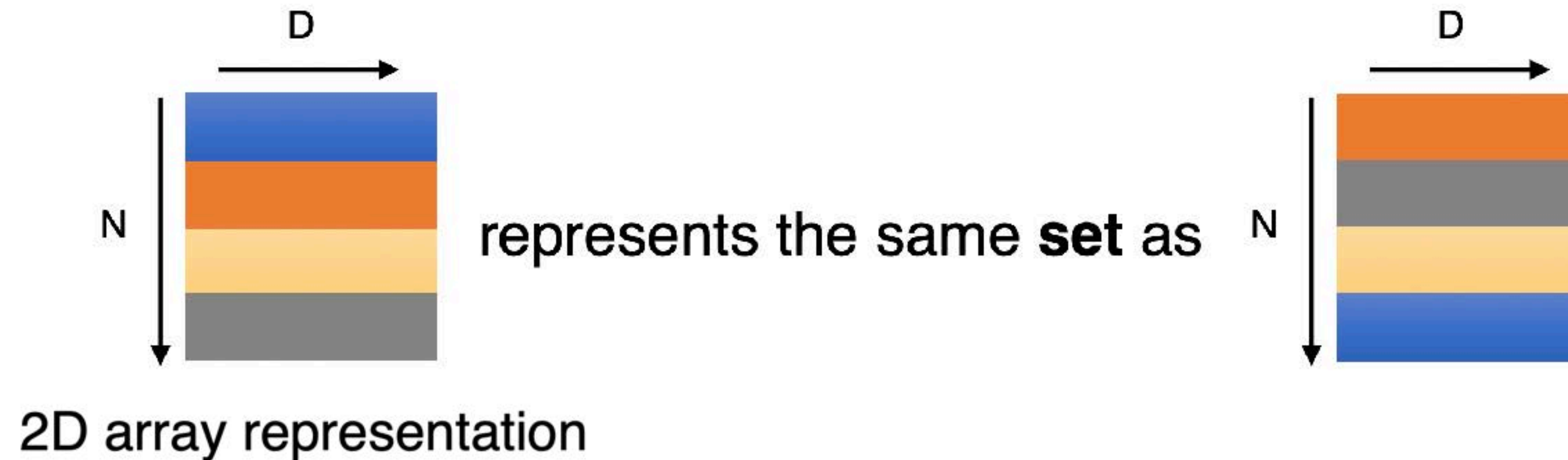
# Unordered input

- Point cloud:  $N$  orderless points, each represented by a  $D$  dim coordinate



# Unordered input

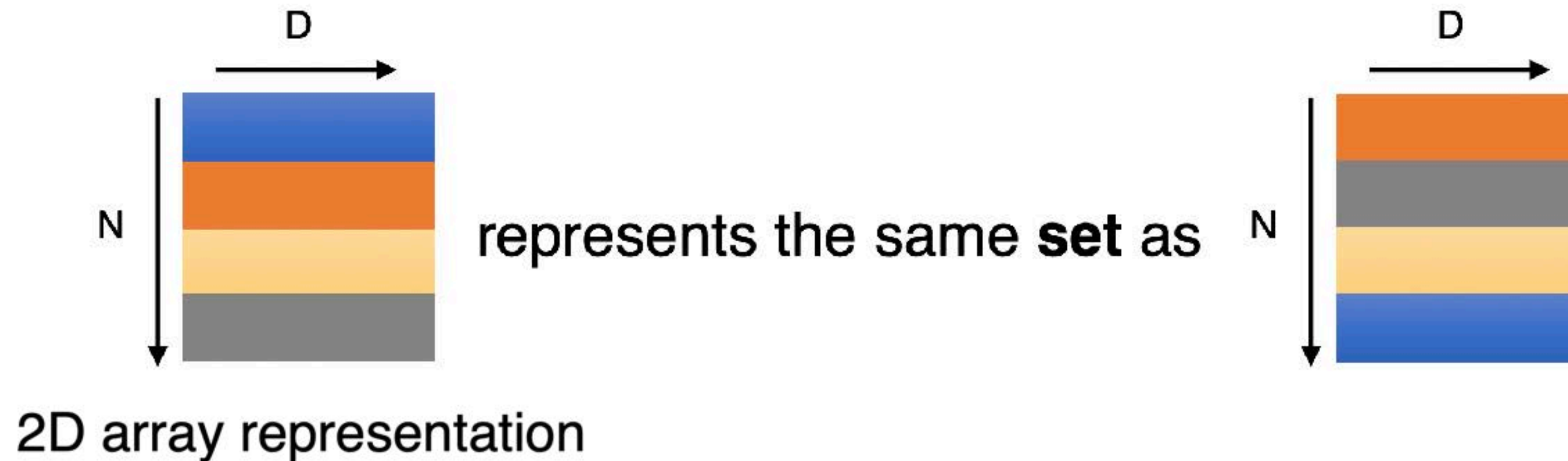
- Point cloud:  $N$  orderless points, each represented by a  $D$  dim coordinate





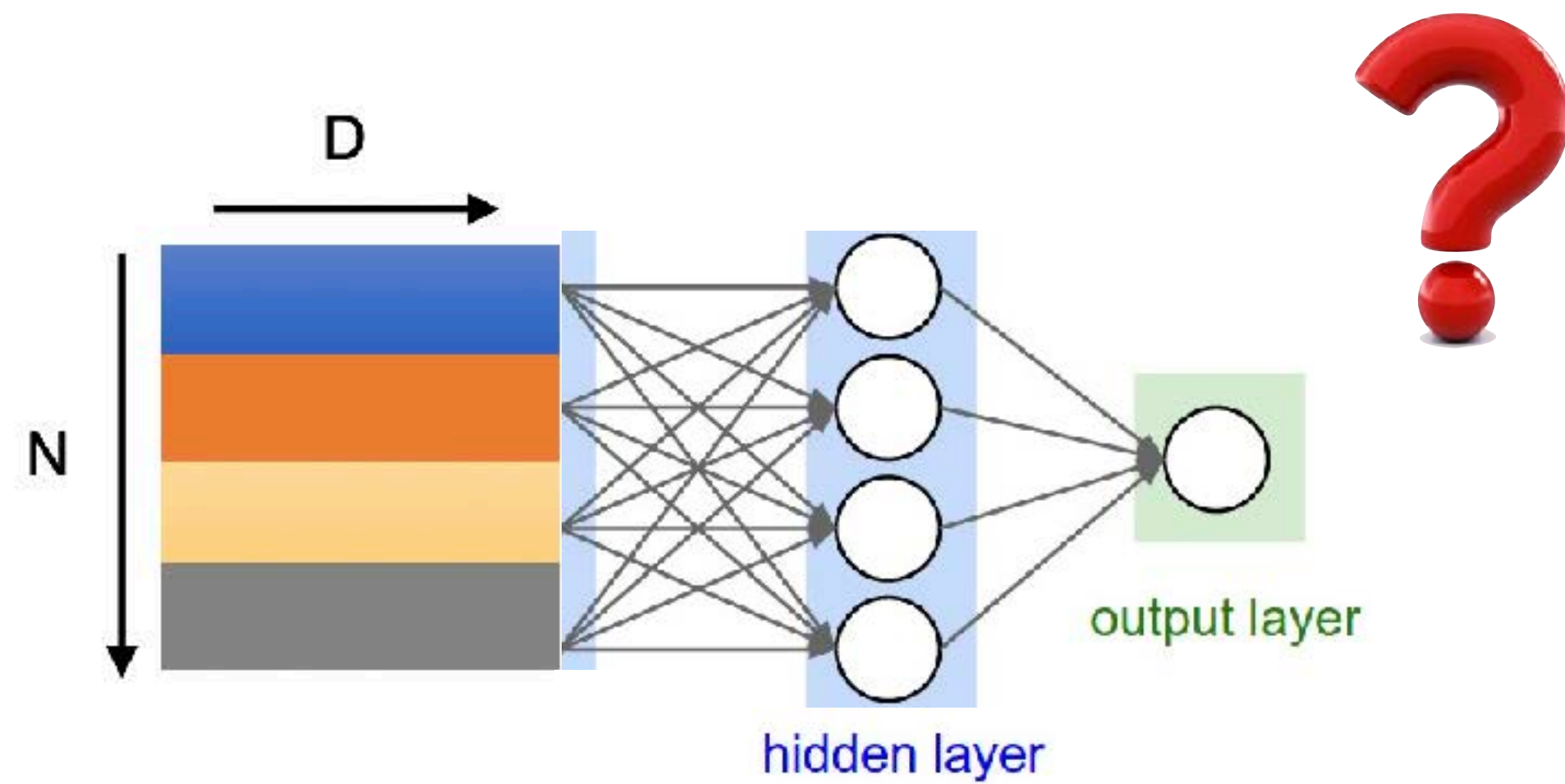
# Desired properties of a point network

- Point cloud:  $N$  orderless points, each represented by a  $D$  dim coordinate

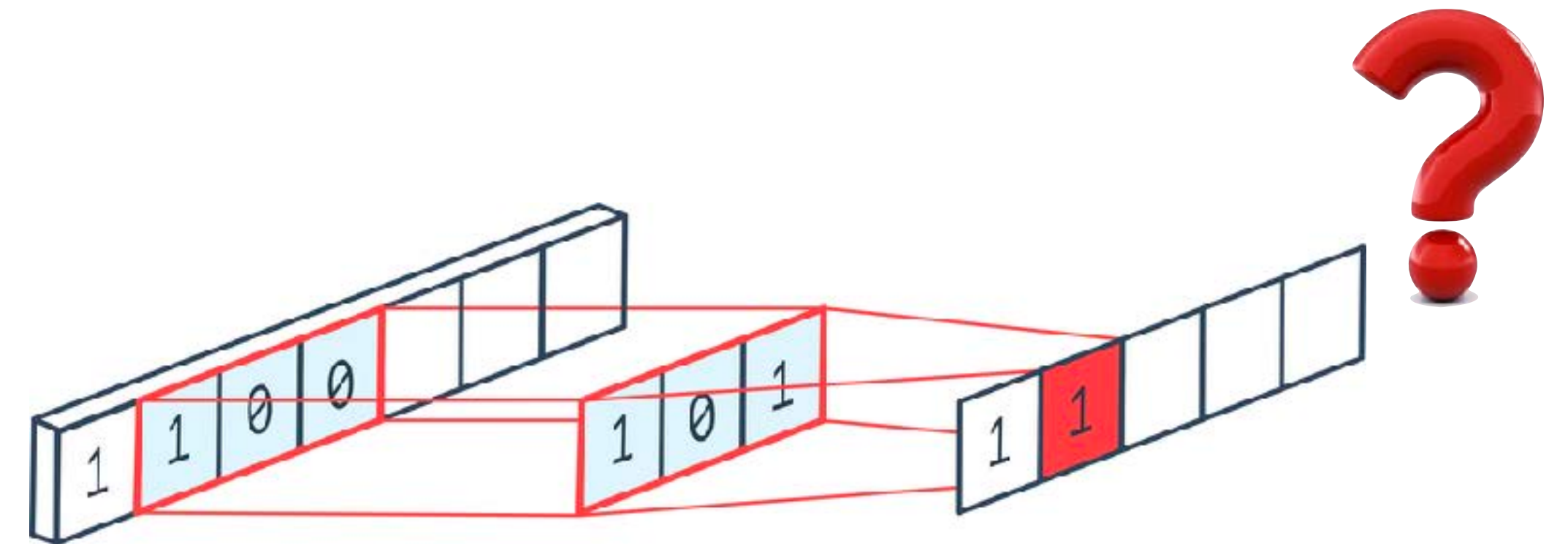


- Deep net needs to be invariant to  $N!$  permutations

# Permutation invariance



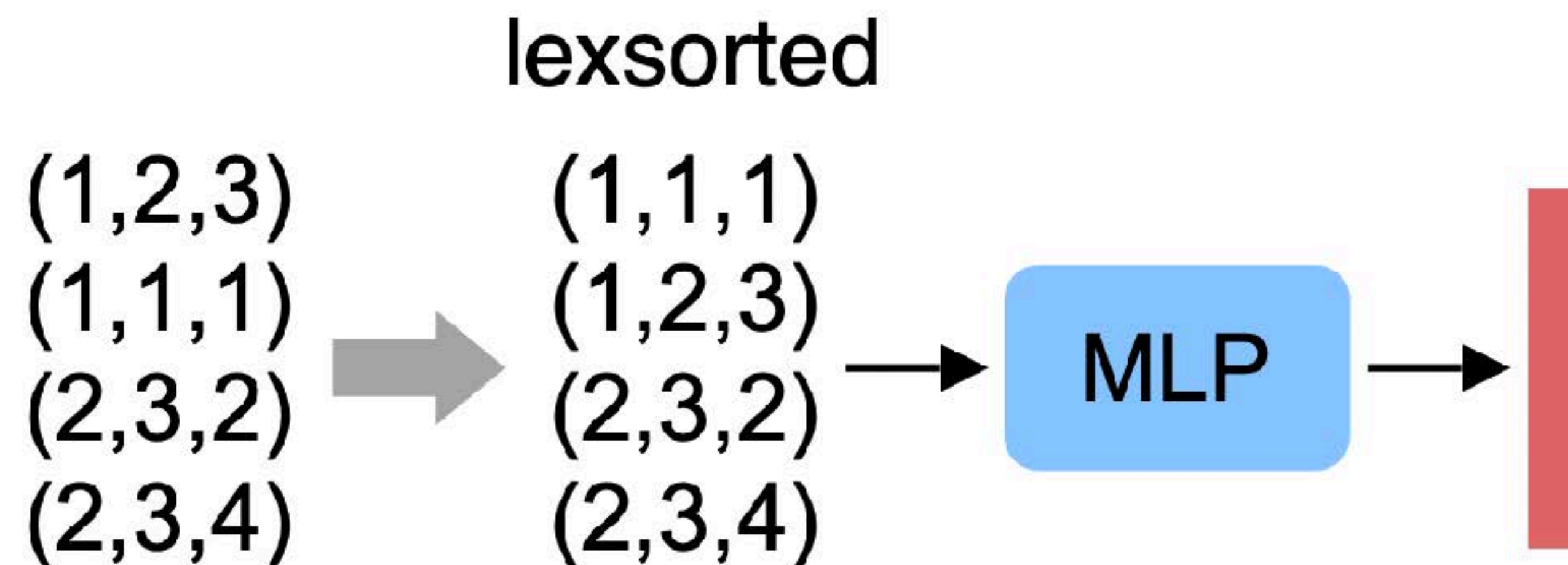
Fully connected network



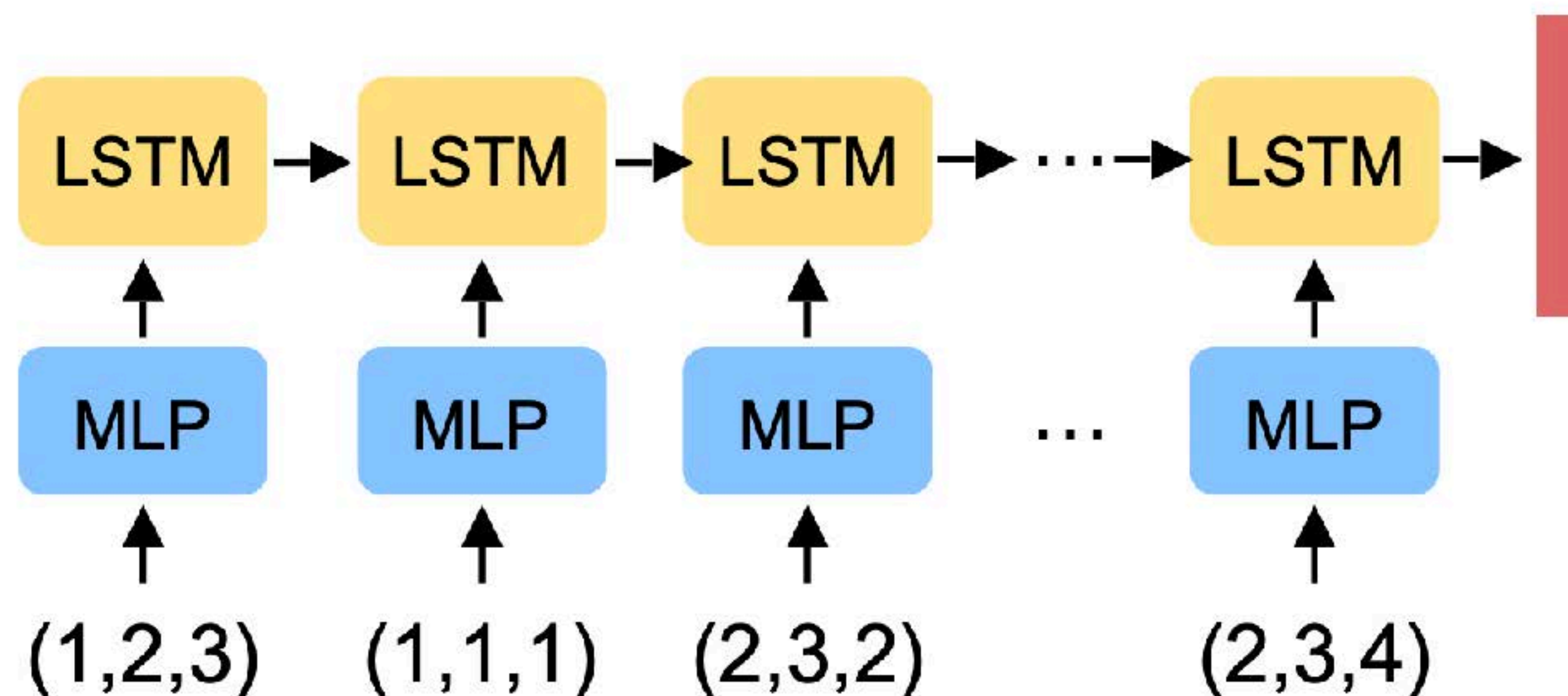
1D convolutional network



# Permutation invariance — sorting?



# Permutation invariance — RNN?





# Permutation invariance: Symmetric function

$$f(x_1, x_2, \dots, x_n) \equiv f(x_{\pi_1}, x_{\pi_2}, \dots, x_{\pi_n}), \quad x_i \in \mathbb{R}^D$$

# Permutation invariance: Symmetric function

$$f(x_1, x_2, \dots, x_n) \equiv f(x_{\pi_1}, x_{\pi_2}, \dots, x_{\pi_n}), \quad x_i \in \mathbb{R}^D$$

## Examples:

$$f(x_1, x_2, \dots, x_n) = \max\{x_1, x_2, \dots, x_n\}$$

$$f(x_1, x_2, \dots, x_n) = x_1 + x_2 + \dots + x_n$$

...



# Permutation invariance: Symmetric function

$$f(x_1, x_2, \dots, x_n) \equiv f(x_{\pi_1}, x_{\pi_2}, \dots, x_{\pi_n}), \quad x_i \in \mathbb{R}^D$$

## Examples:

$$f(x_1, x_2, \dots, x_n) = \max\{x_1, x_2, \dots, x_n\}$$

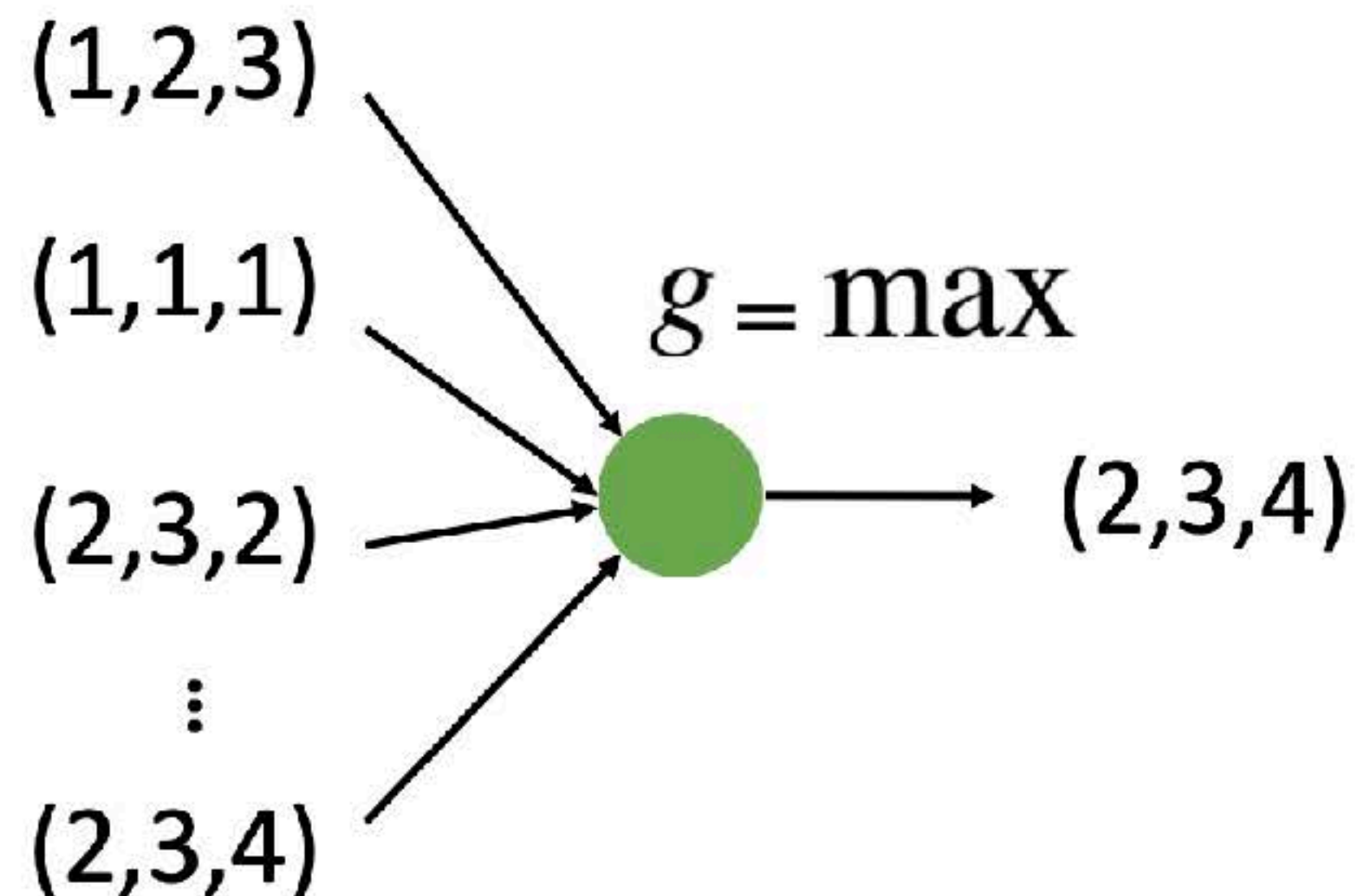
$$f(x_1, x_2, \dots, x_n) = x_1 + x_2 + \dots + x_n$$

...

**How can we construct a universal family of symmetric functions by neural networks?**

# Construct symmetric functions by neural networks

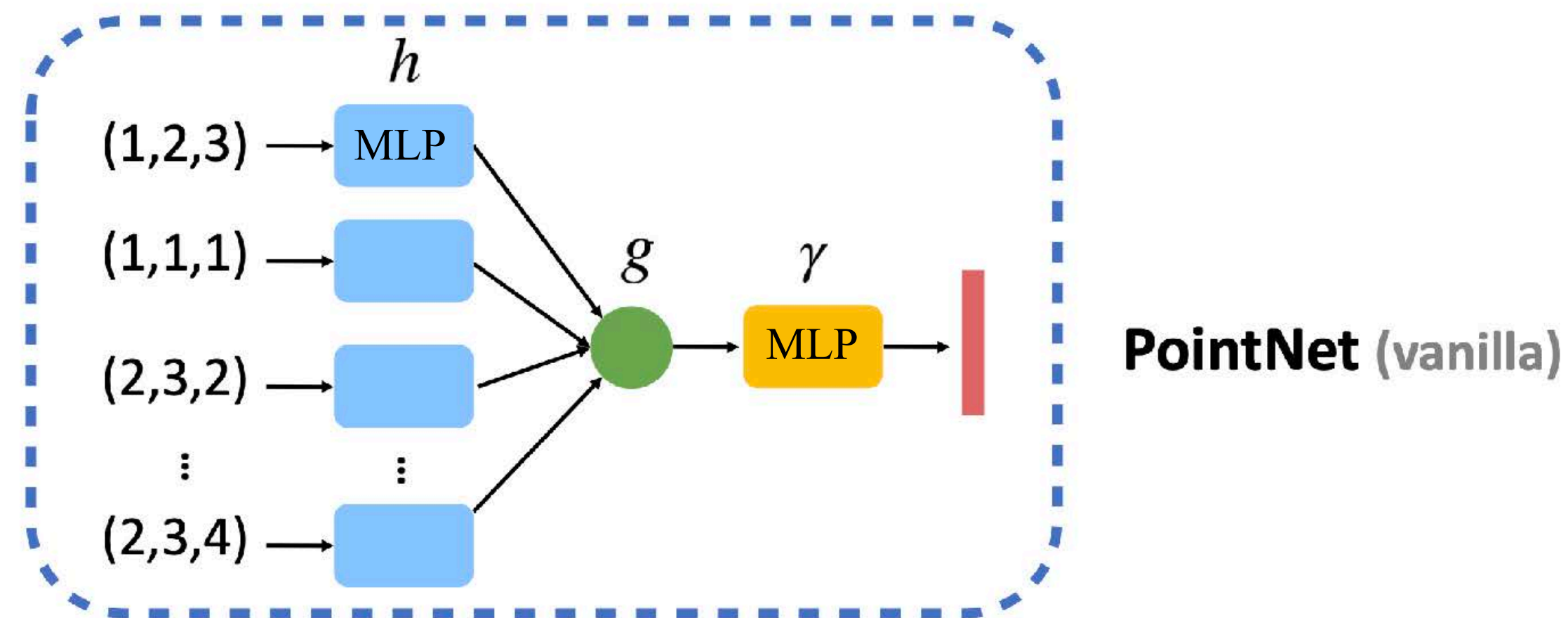
Simplest form: directly aggregate all points with a symmetric operator  $g$   
**Just discovers simple extreme/aggregate properties of the geometry.**





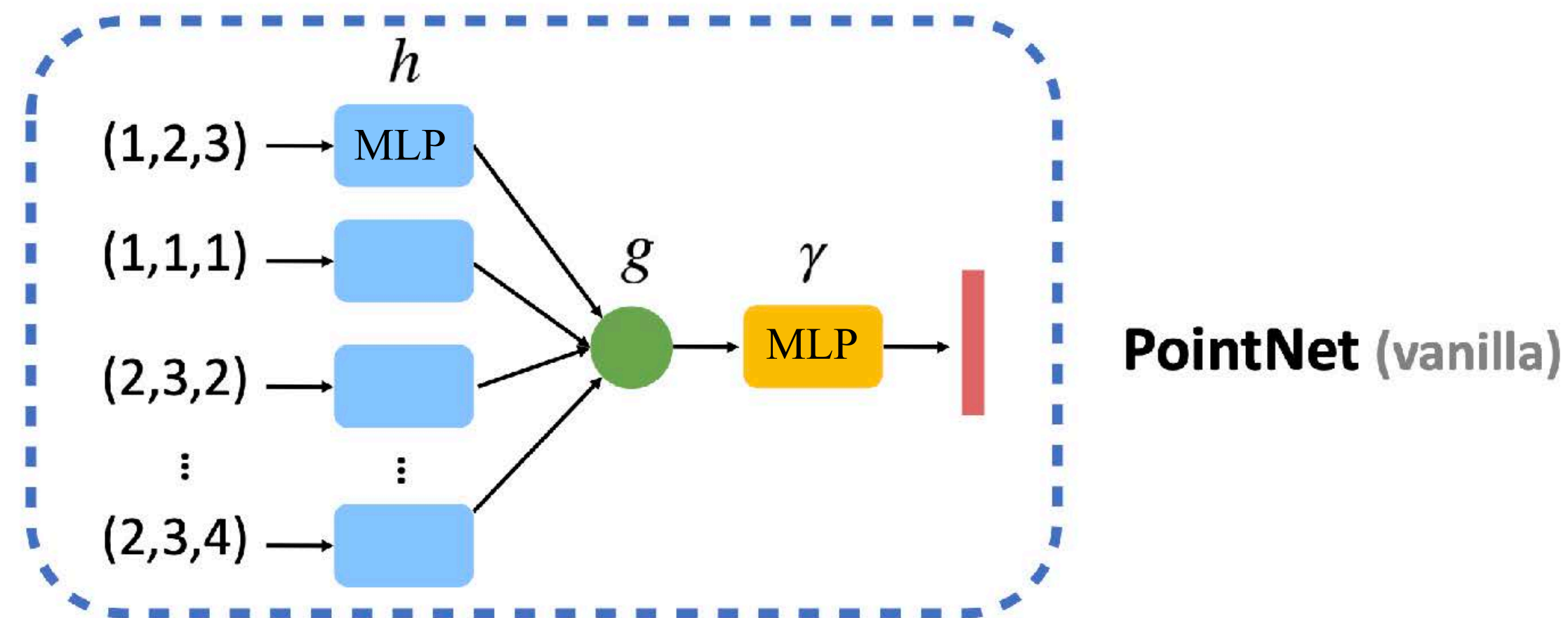
# Construct symmetric functions by neural networks

$f(x_1, x_2, \dots, x_n) = \gamma \circ g(h(x_1), \dots, h(x_n))$  is symmetric if  $g$  is symmetric



# Construct symmetric functions by neural networks

$f(x_1, x_2, \dots, x_n) = \gamma \circ g(h(x_1), \dots, h(x_n))$  is symmetric if  $g$  is symmetric

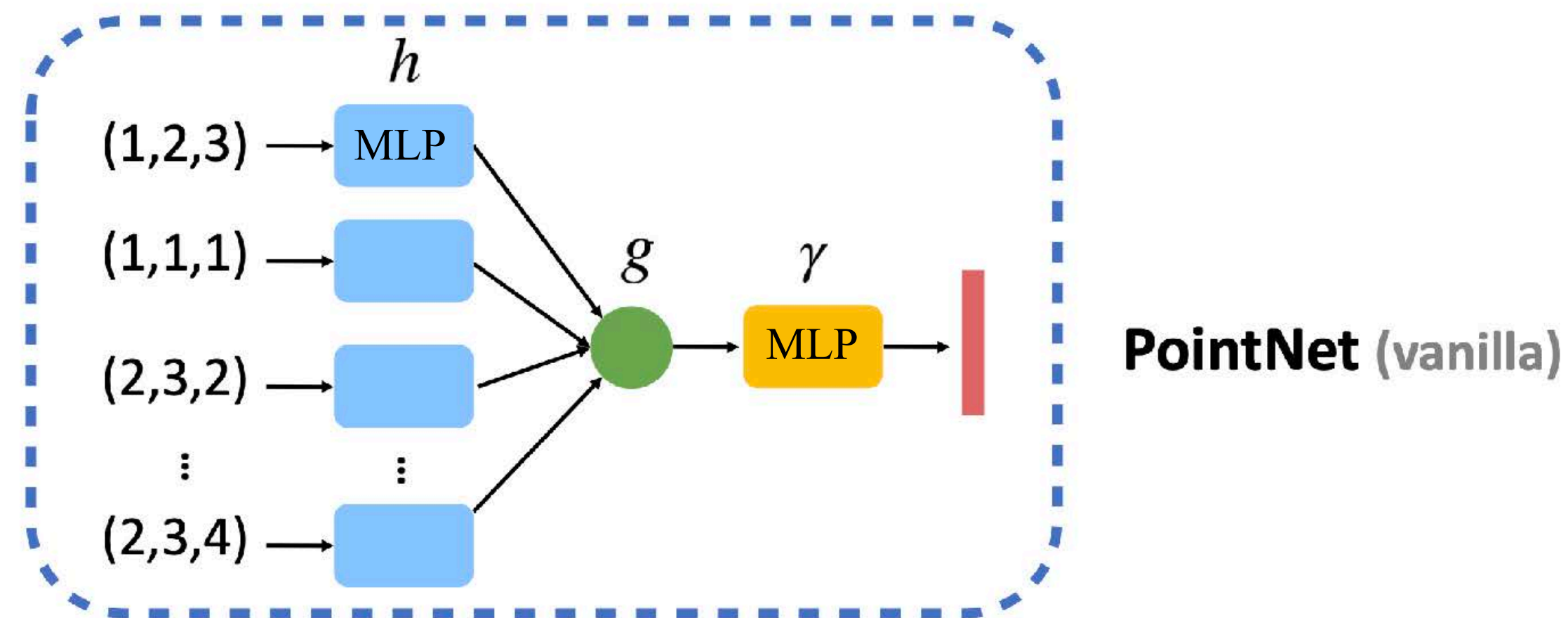


- Reflection: assuming  $g$  is a max operation, construct a function  $h$  where geometric details get kept after applying  $g$ .

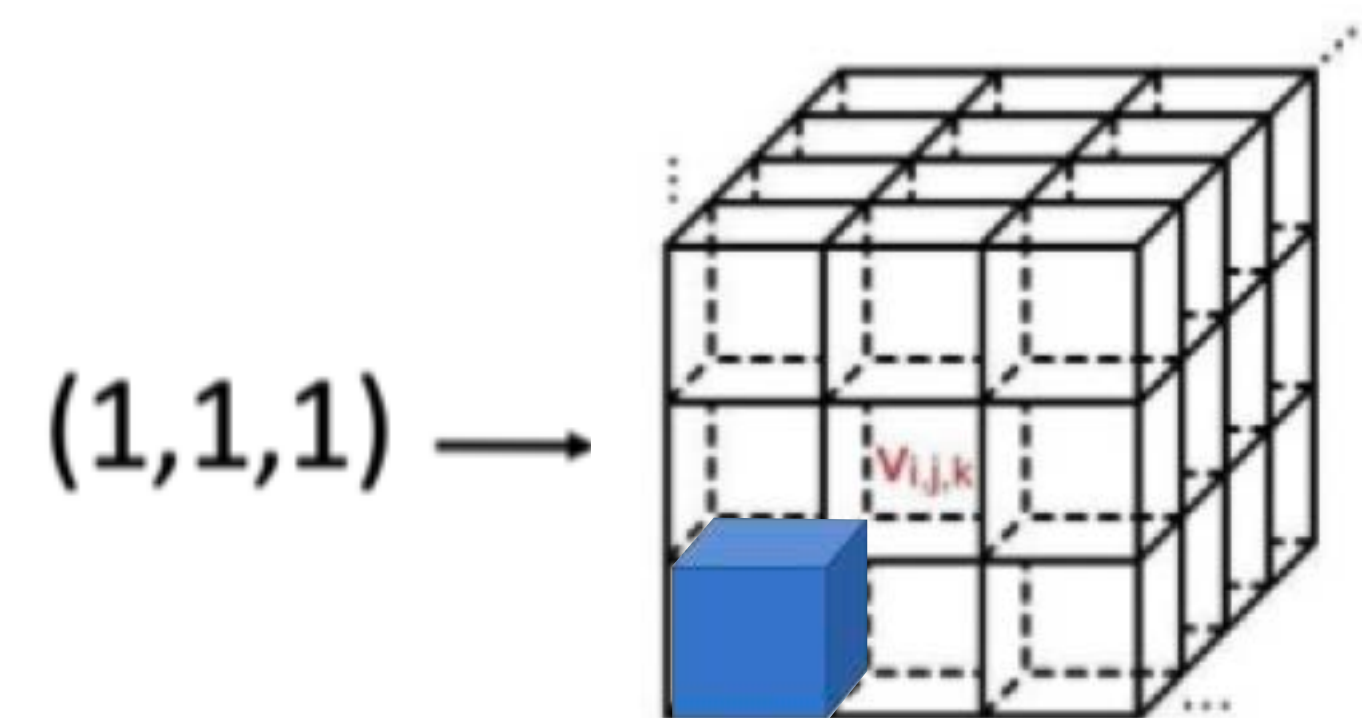


# Construct symmetric functions by neural networks

$f(x_1, x_2, \dots, x_n) = \gamma \circ g(h(x_1), \dots, h(x_n))$  is symmetric if  $g$  is symmetric



- Reflection: assuming  $g$  is a max operation, construct a function  $h$  where geometric details get kept after applying  $g$ .



Spatial Hashing Function

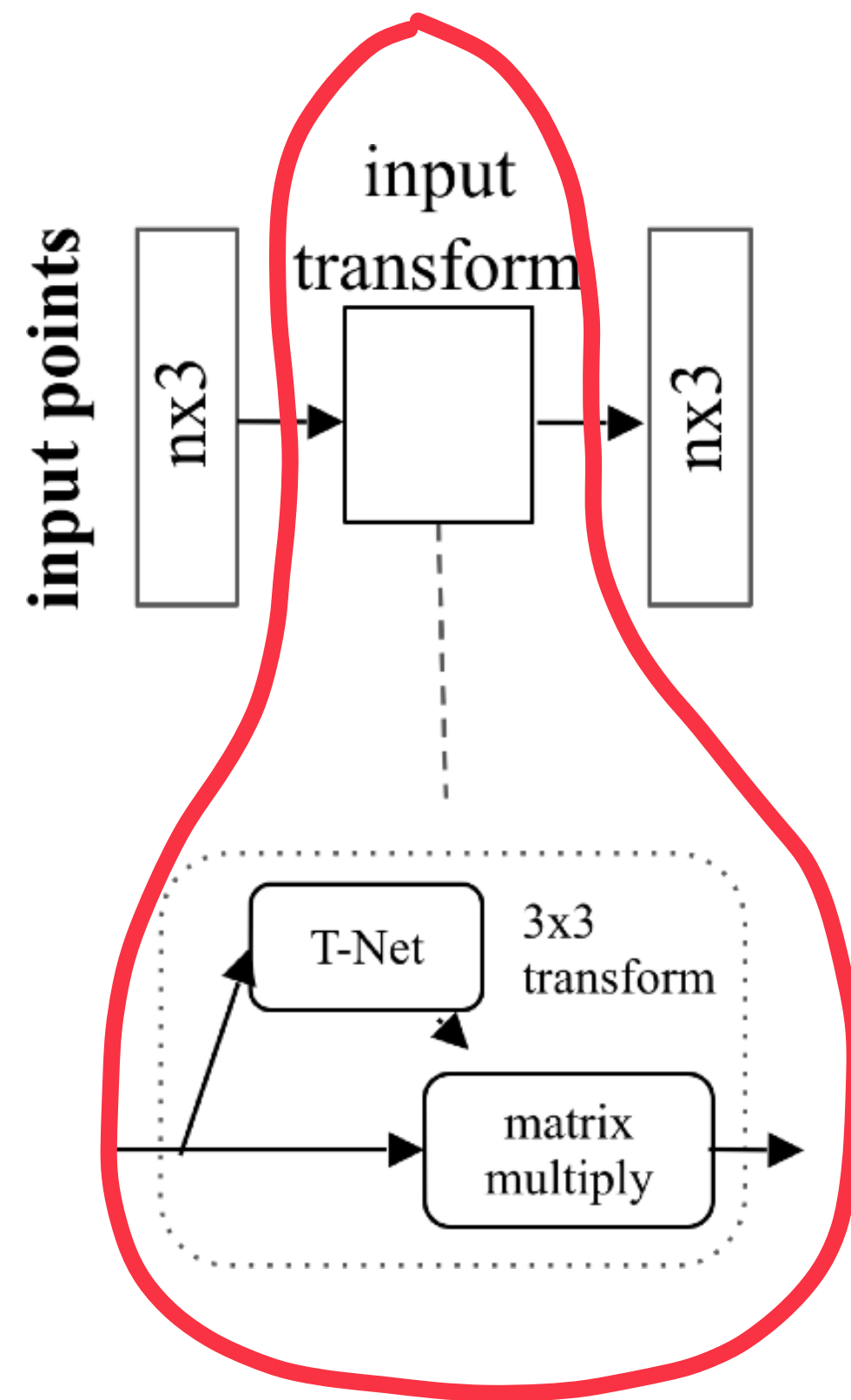
# A detailed implementation of PointNet

**input points**

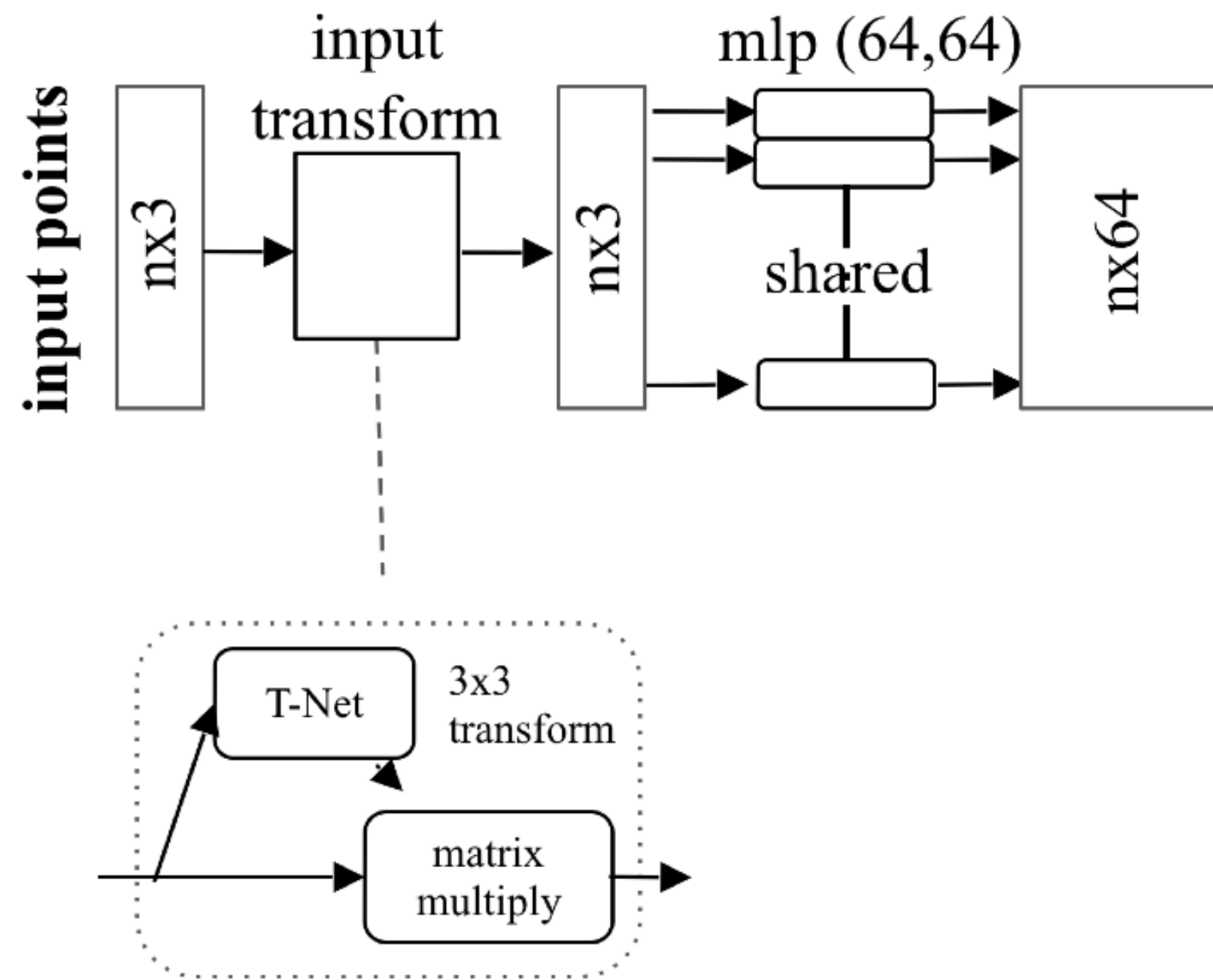
|              |
|--------------|
| $n \times 3$ |
|--------------|



# A detailed implementation of PointNet

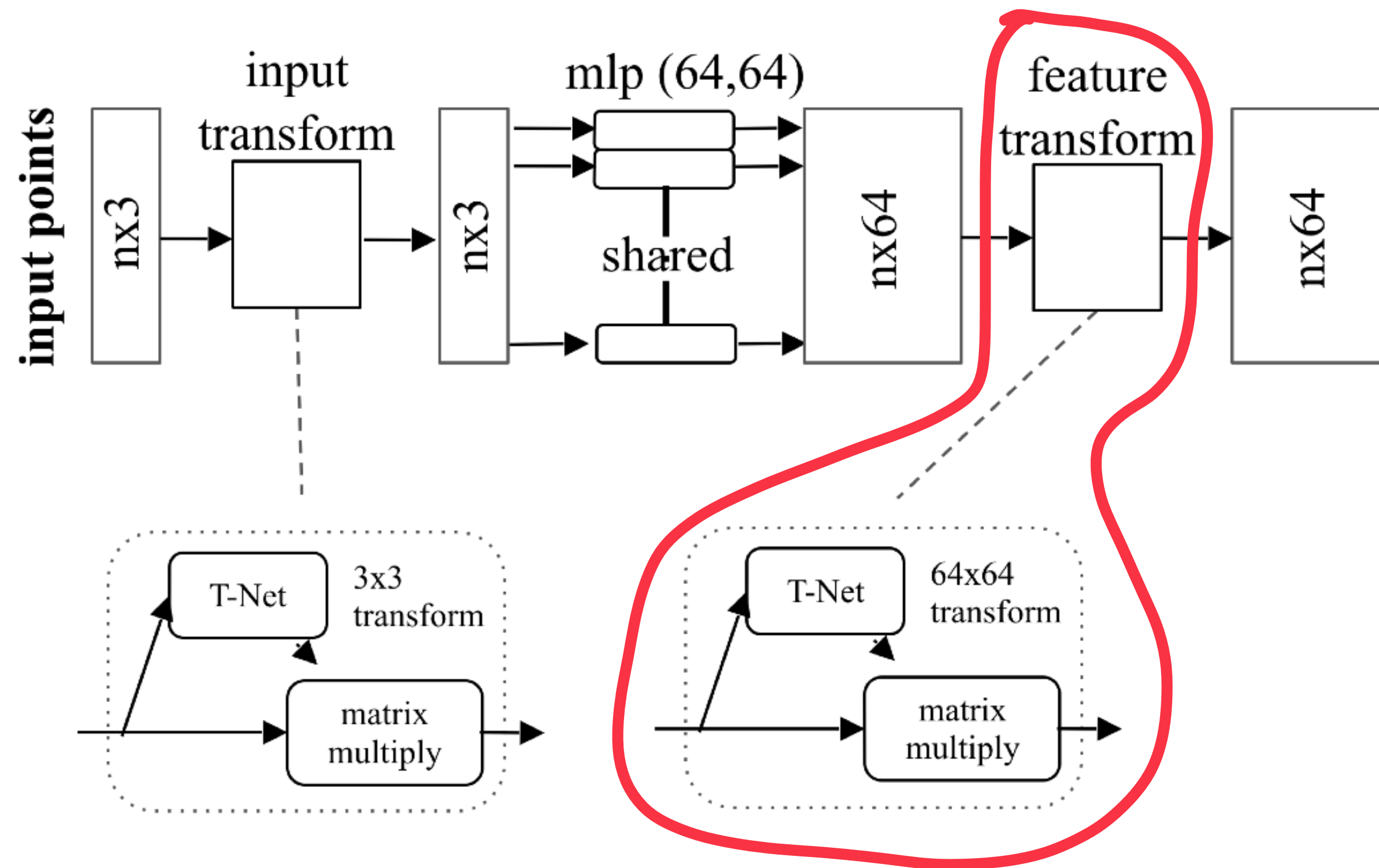


# A detailed implementation of PointNet

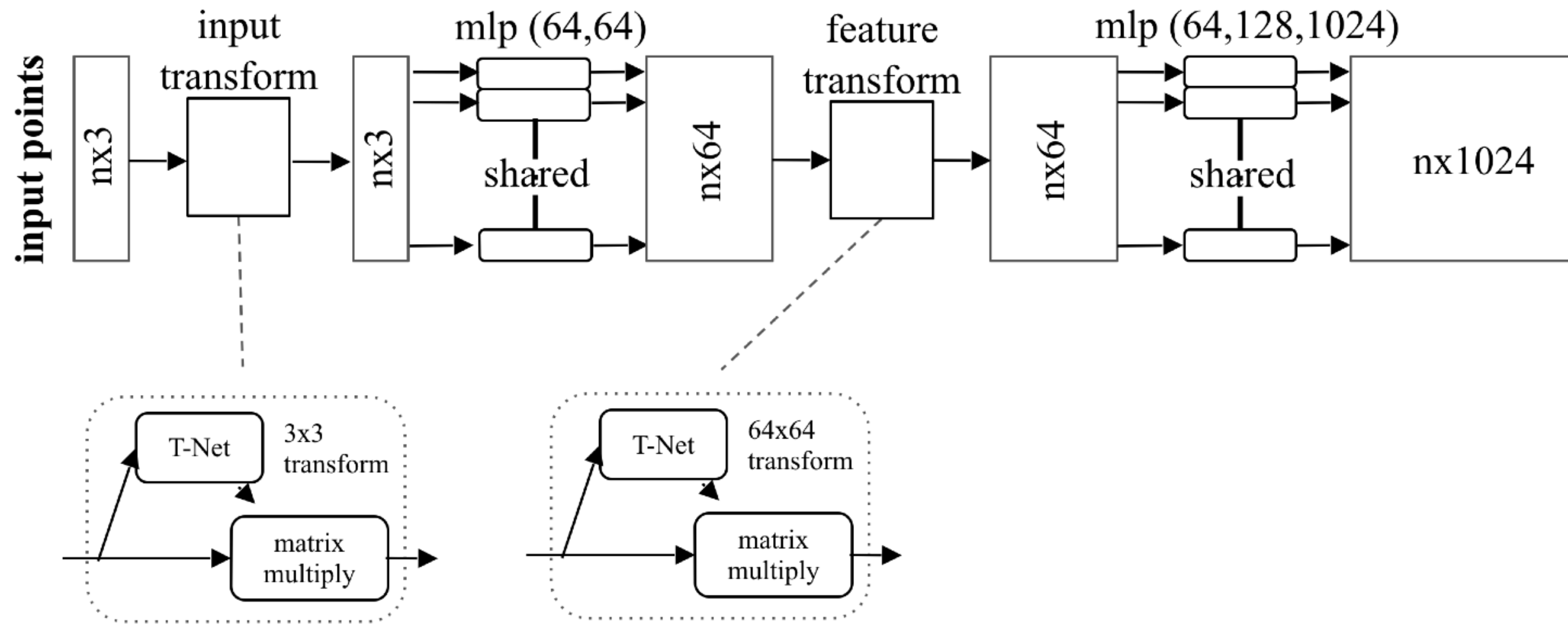




# A detailed implementation of PointNet

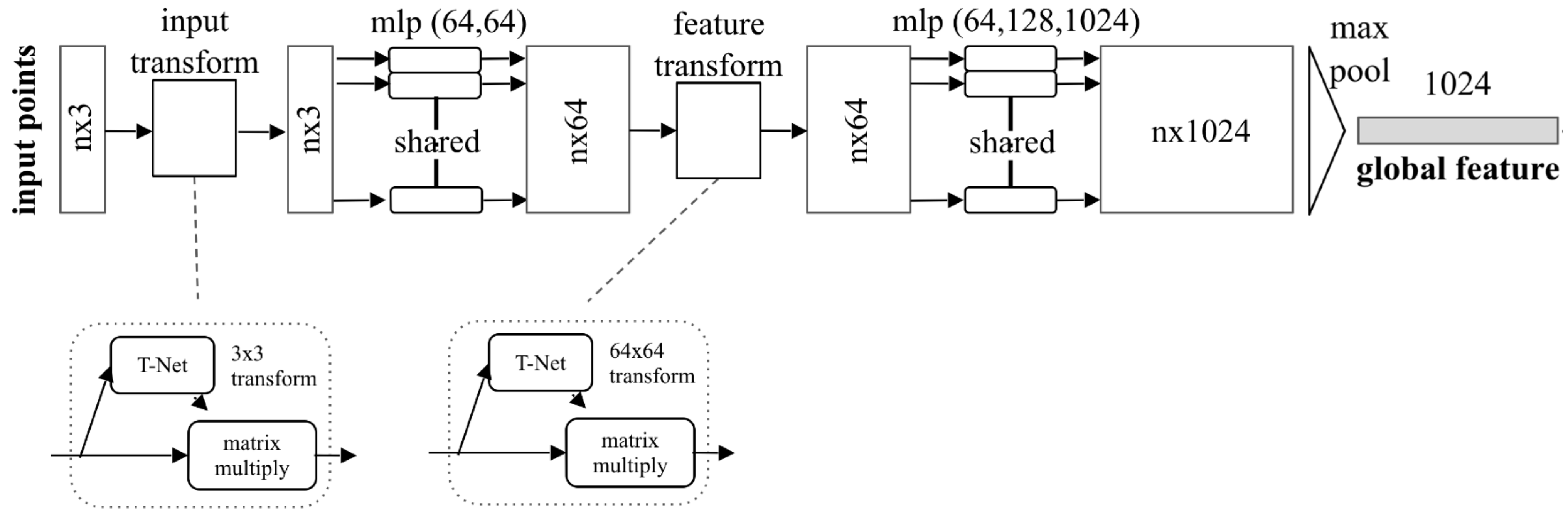


# A detailed implementation of PointNet

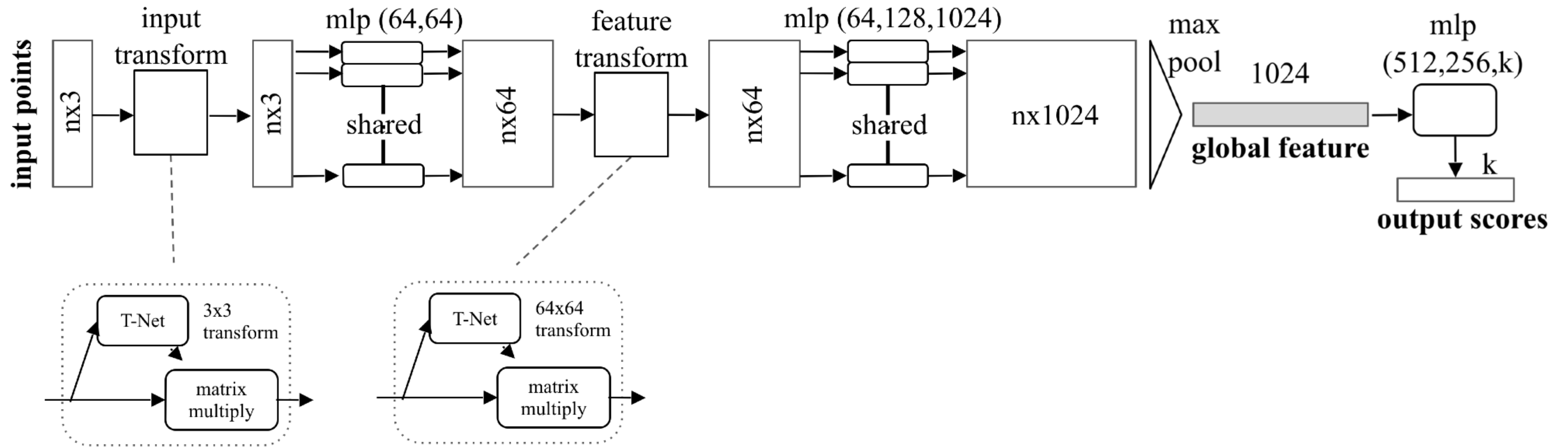




# A detailed implementation of PointNet

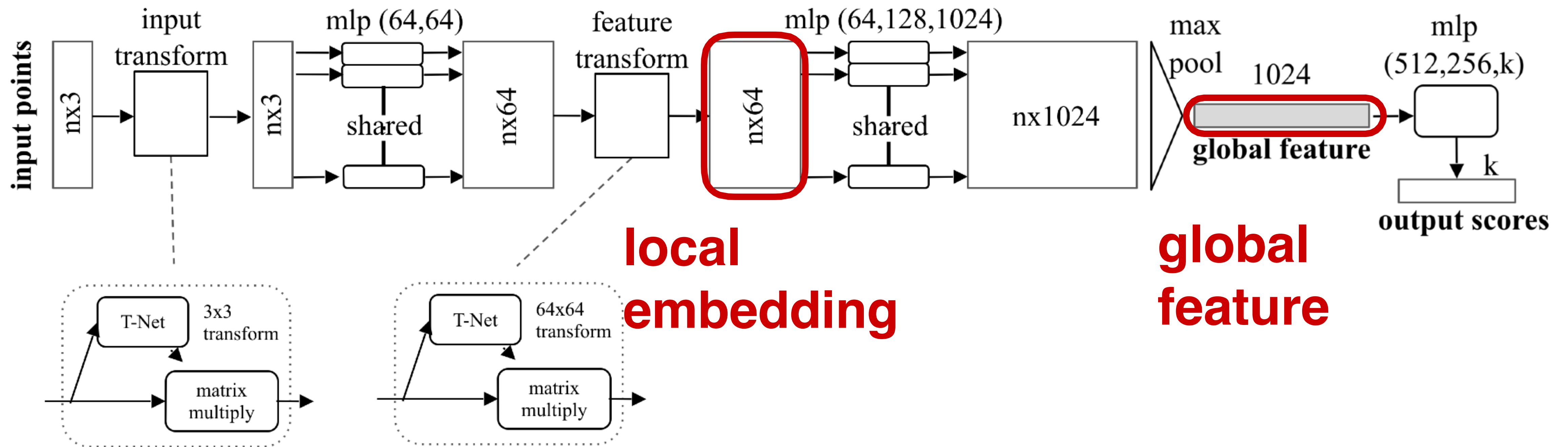


# A detailed implementation of PointNet

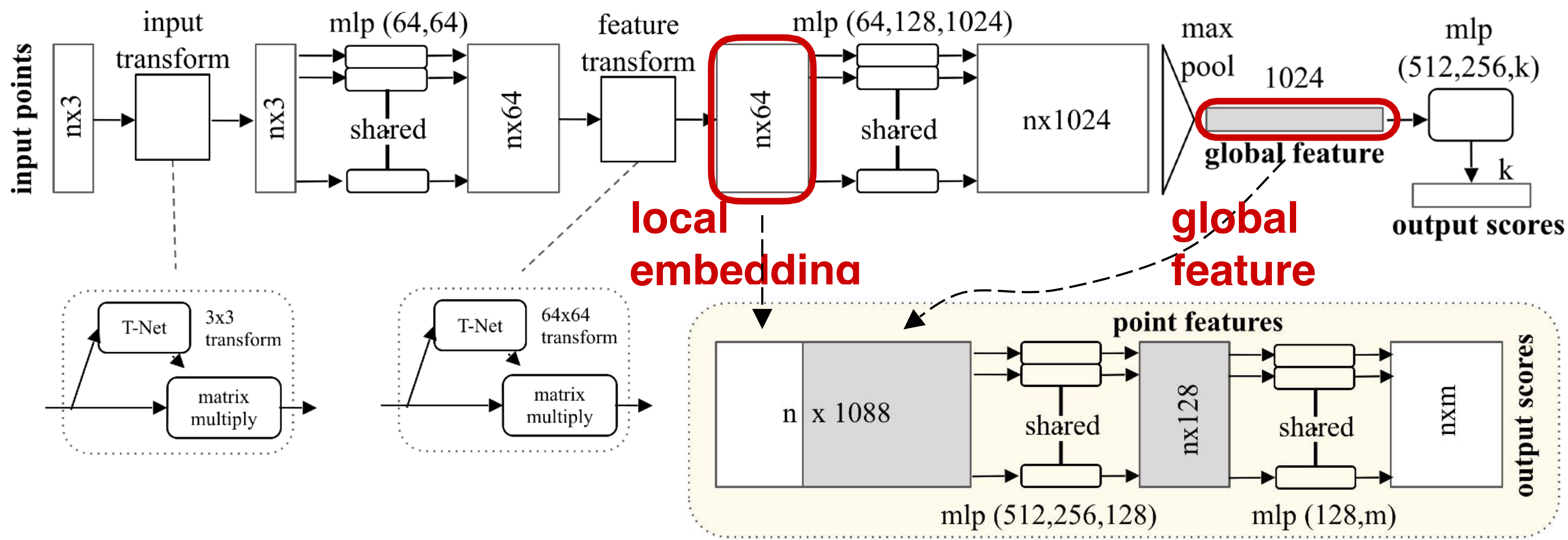




# Extension to segmentation network

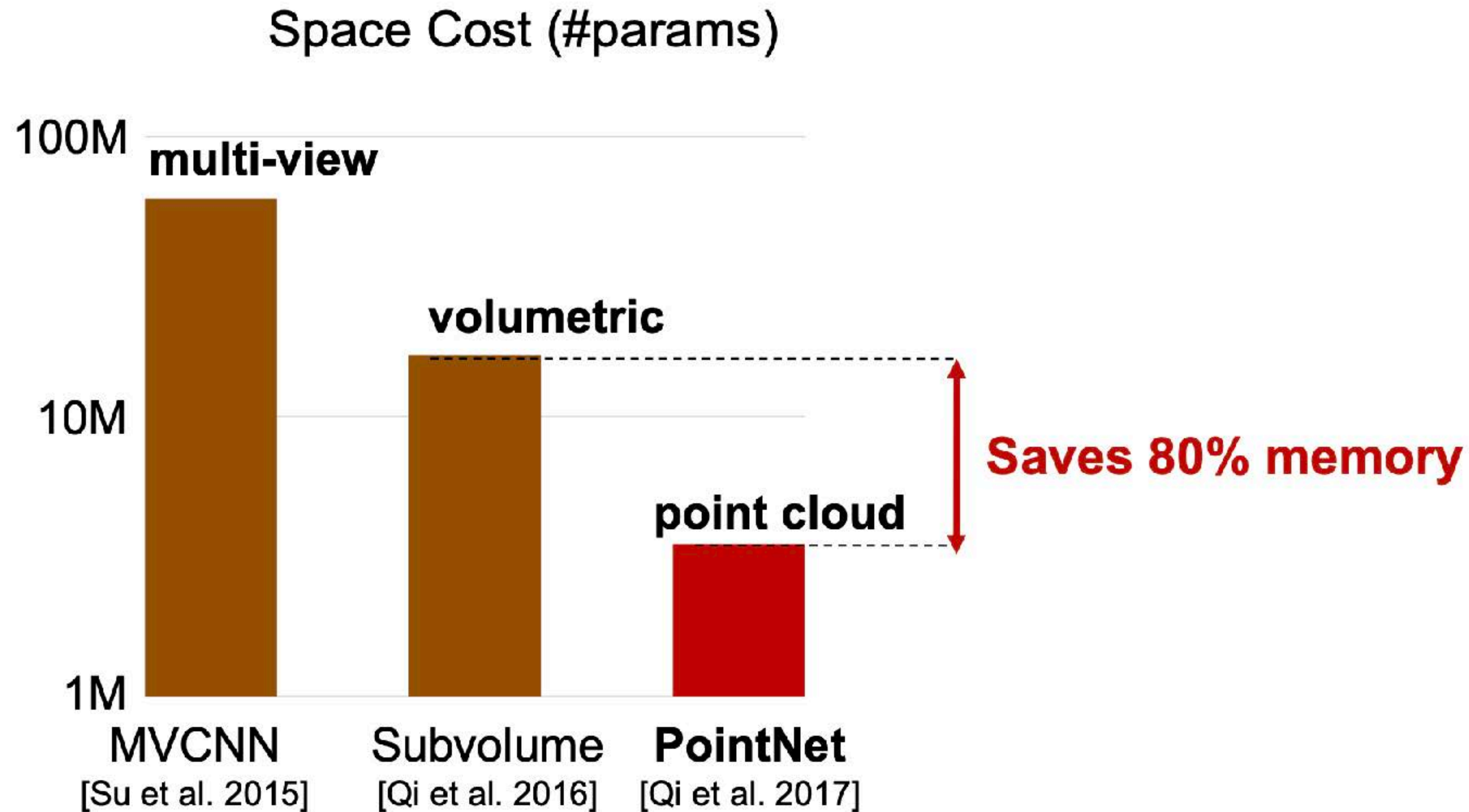


# Extension to segmentation network

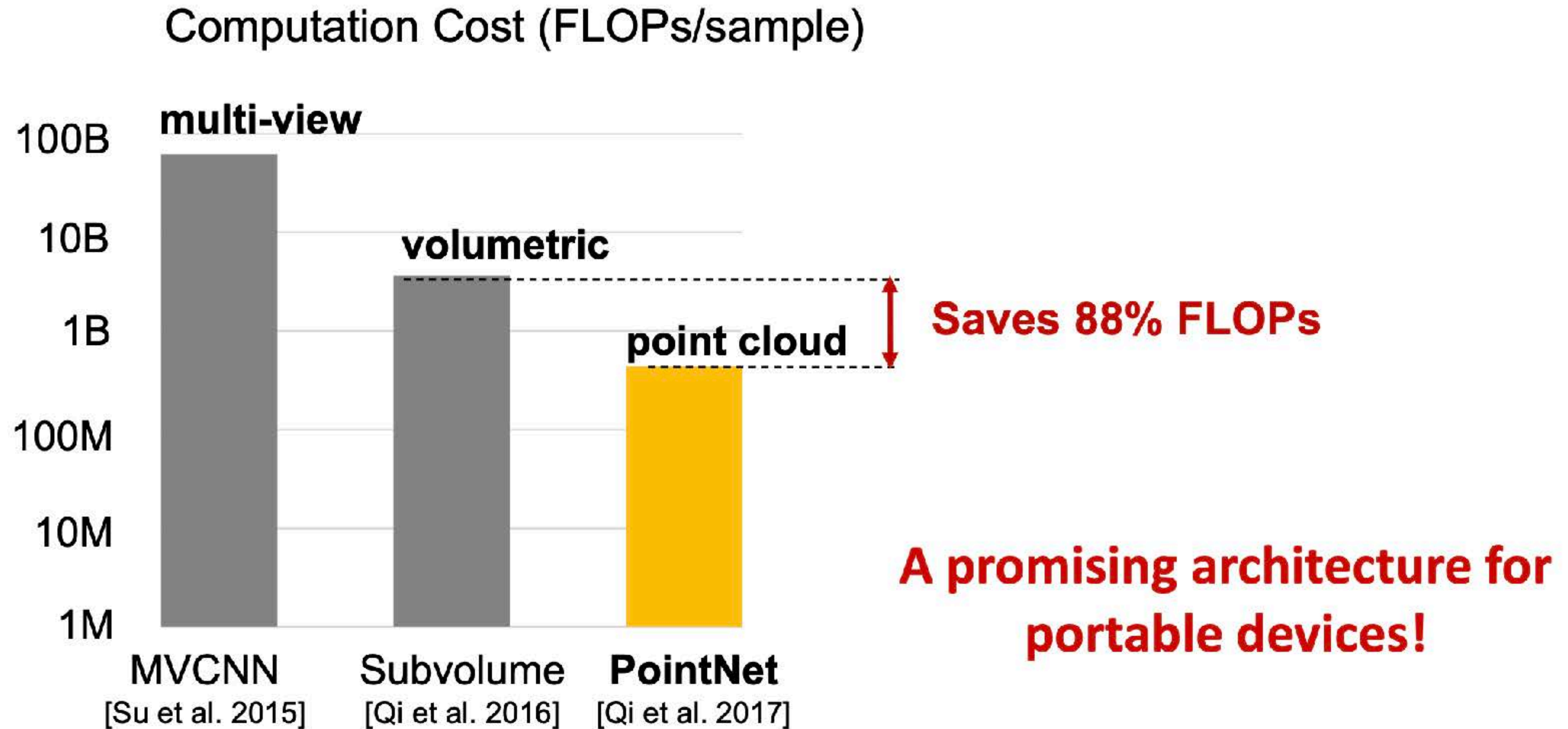




# PointNet is light-weight and fast



# PointNet is light-weight and fast



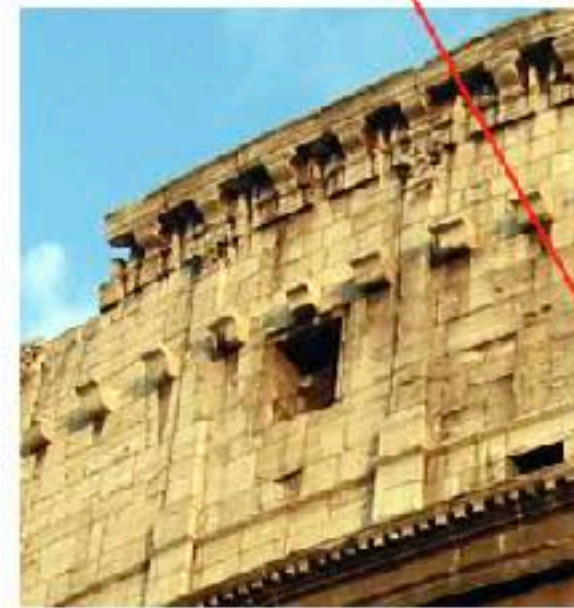


# Robustness to data corruption

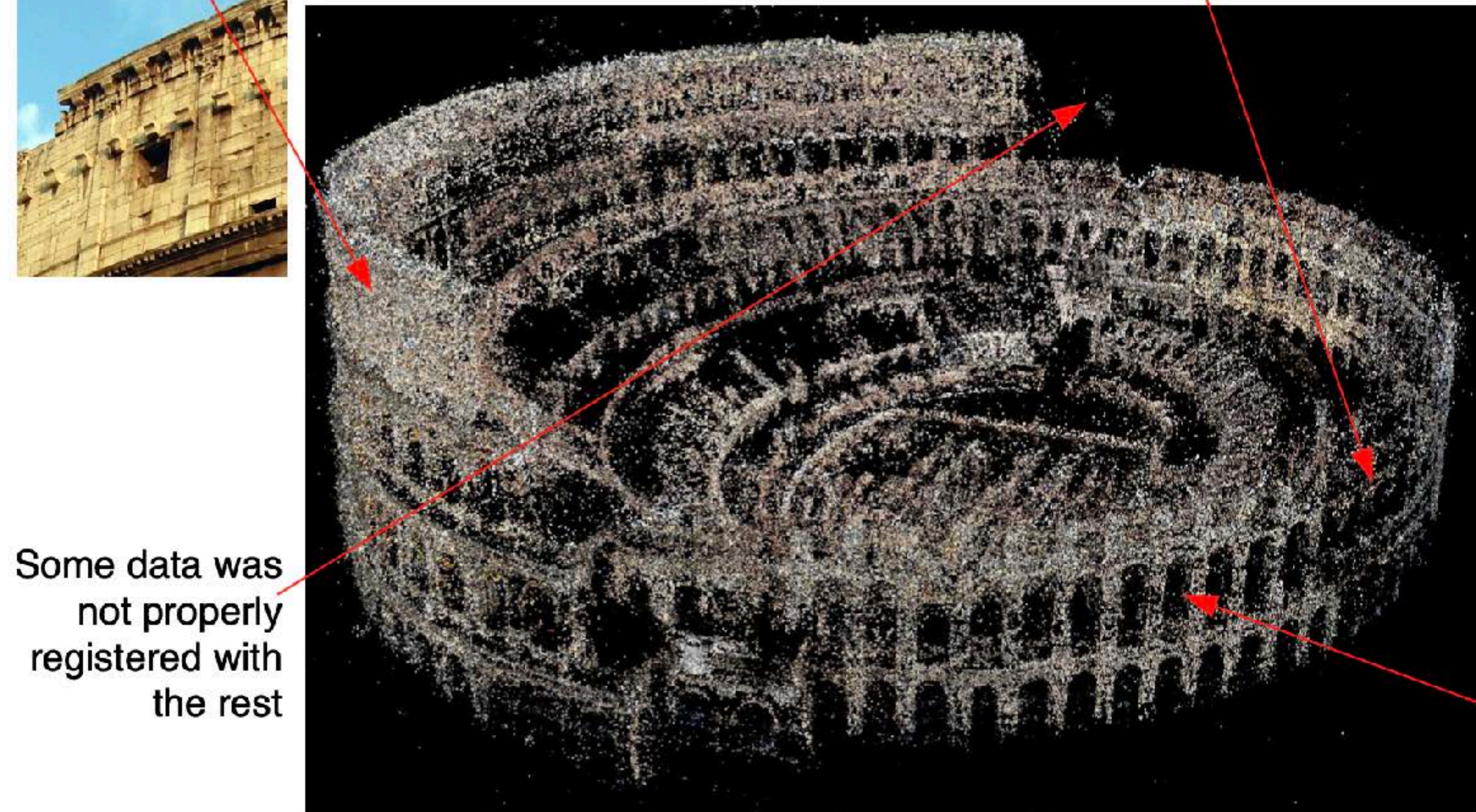
- Many challenges

- Resolution
- Occlusion
- Noise
- Registration

Noise → Poor detail reproduction



Low resolution further obscures detail



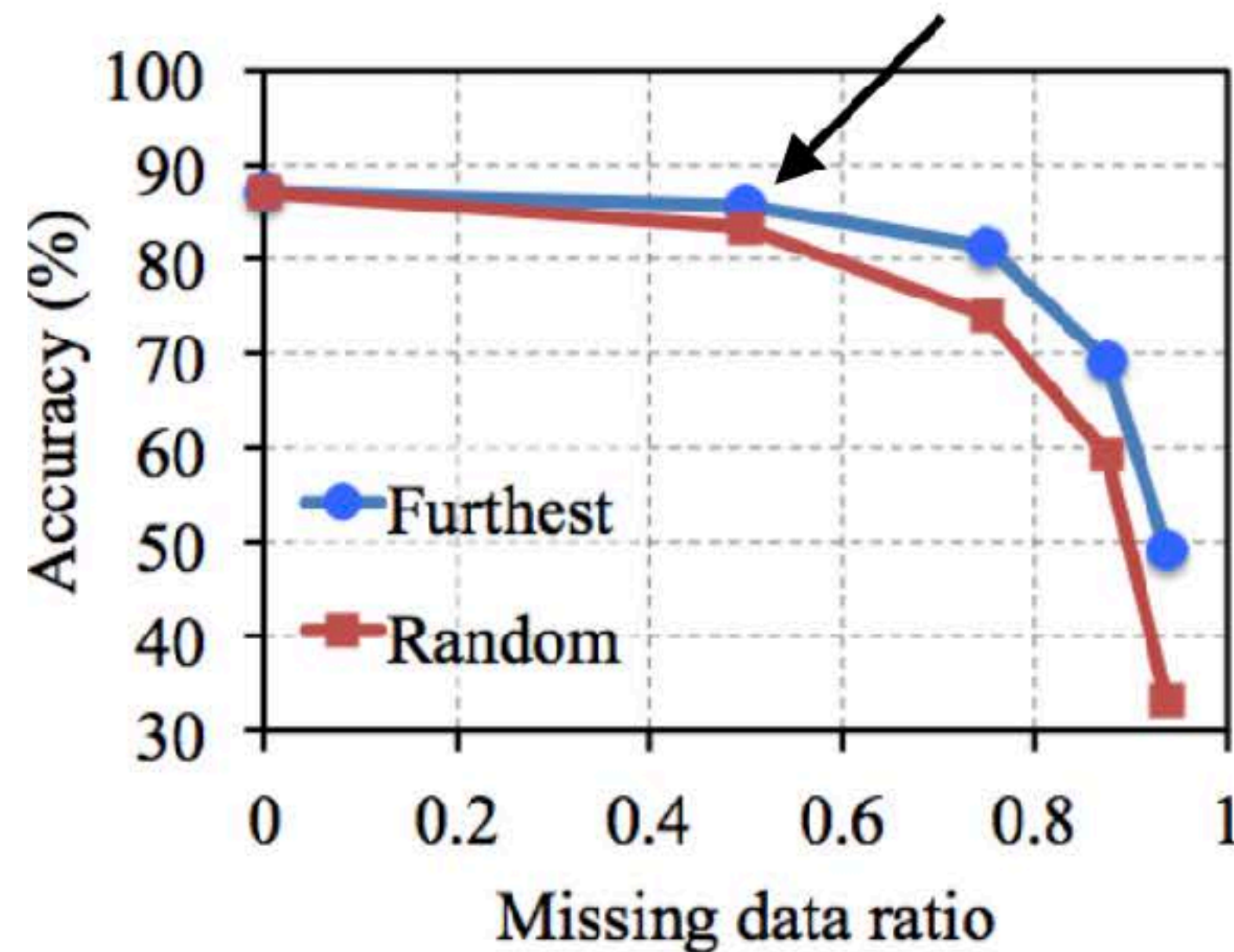
Some data was  
not properly  
registered with  
the rest

Occlusion →  
Interiors not  
captured



# Robustness to data corruption

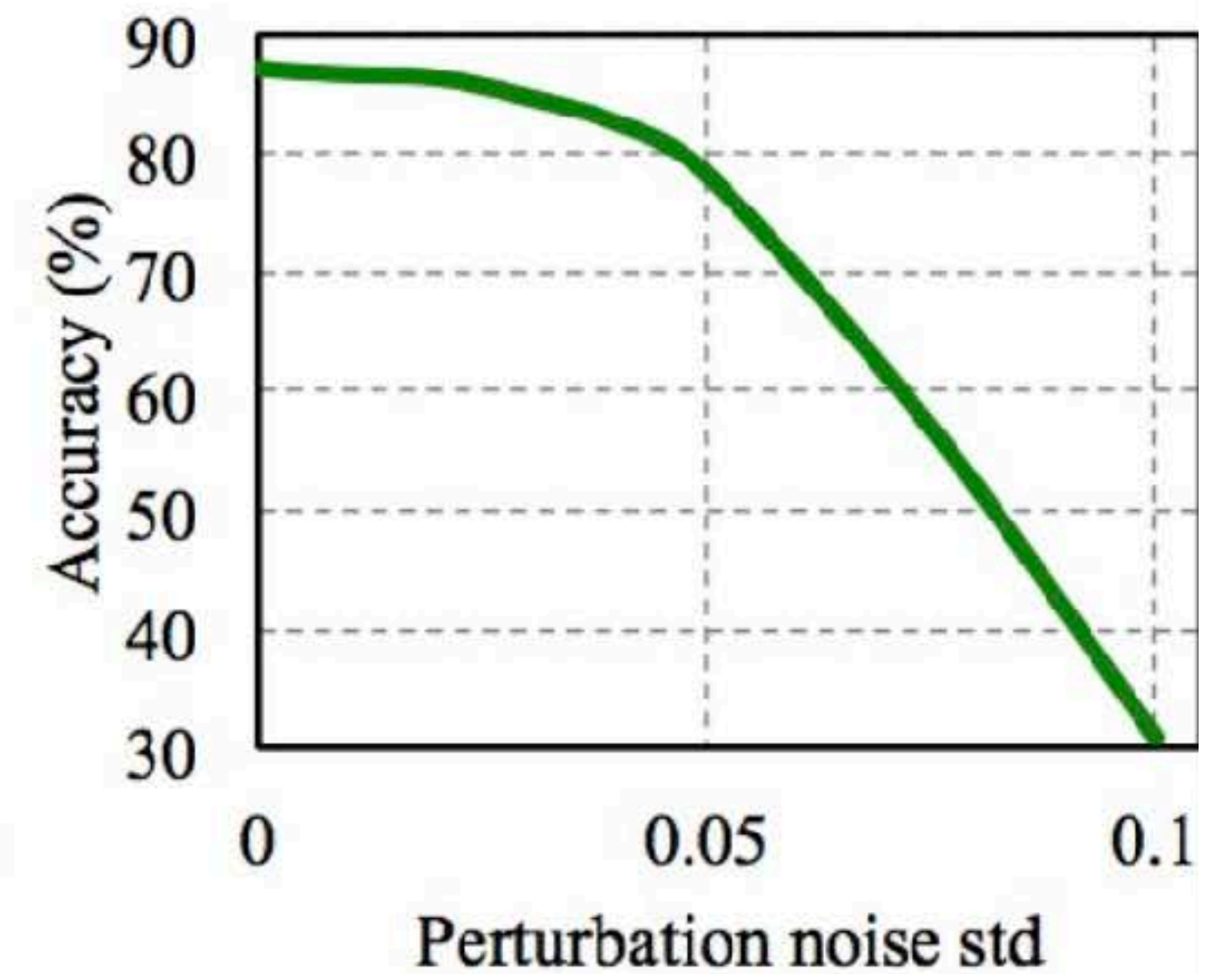
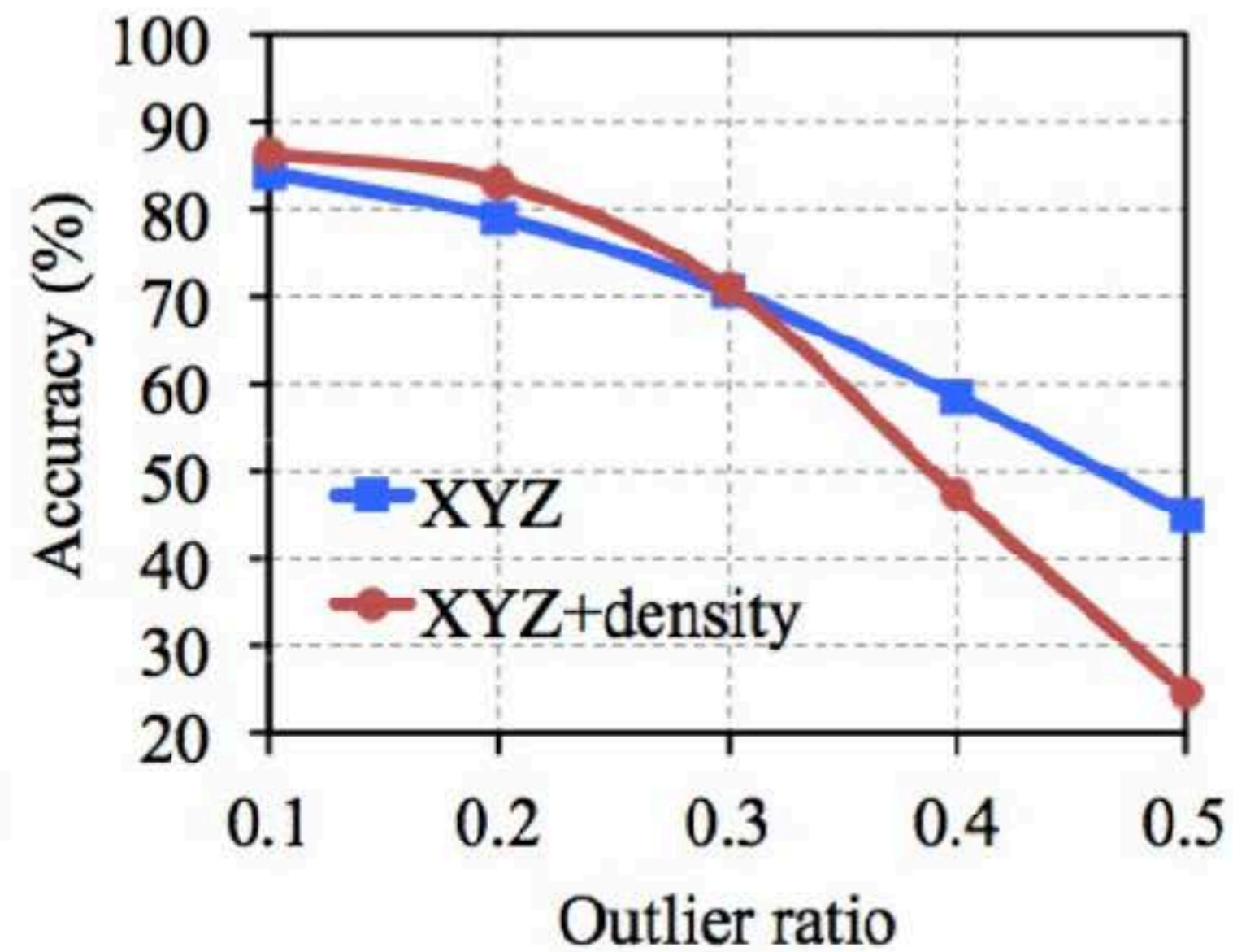
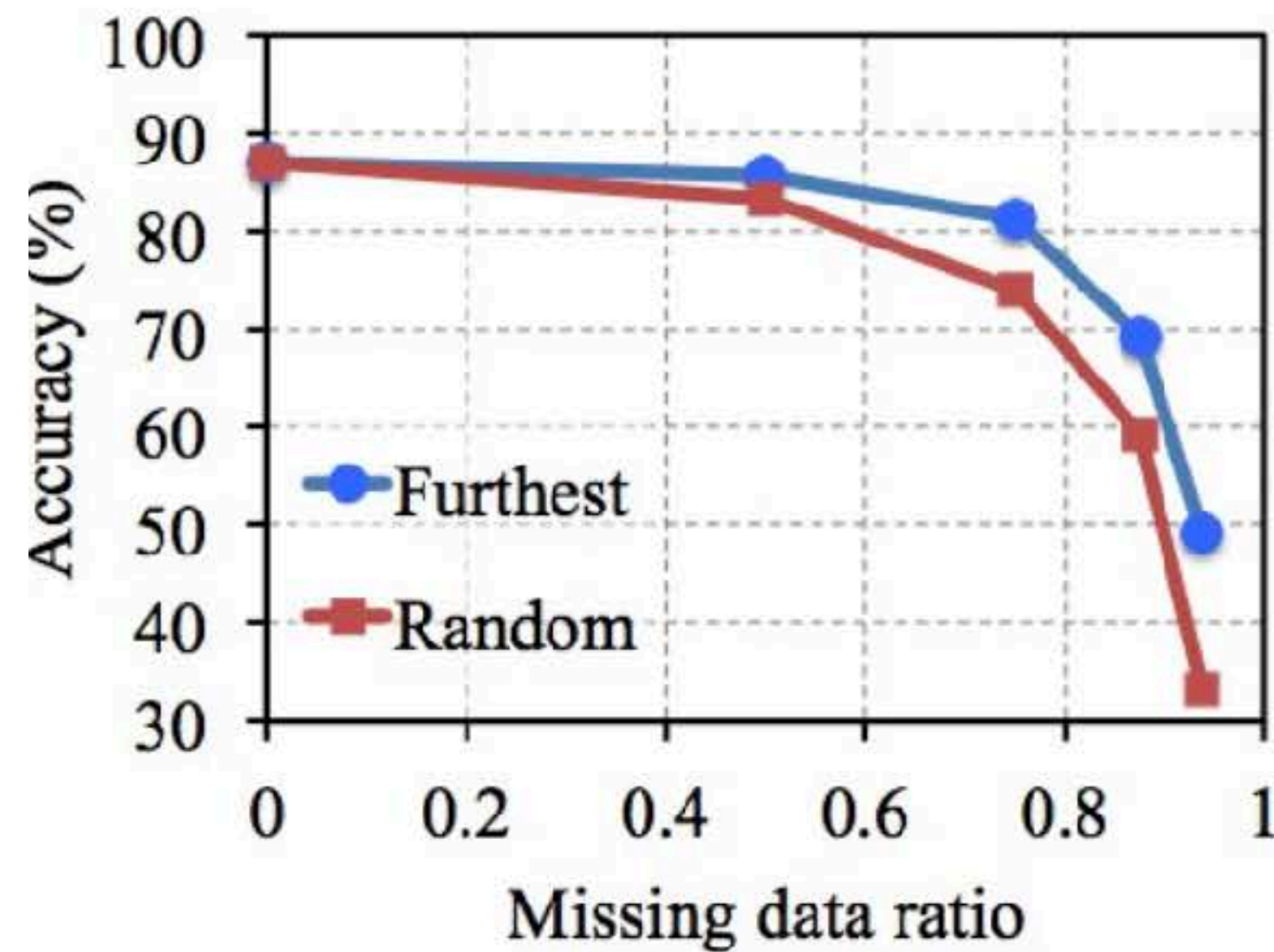
Less than 2% accuracy drop with 50% missing data



*dataset: ModelNet40; metric: 40-class  
classification accuracy (%)*

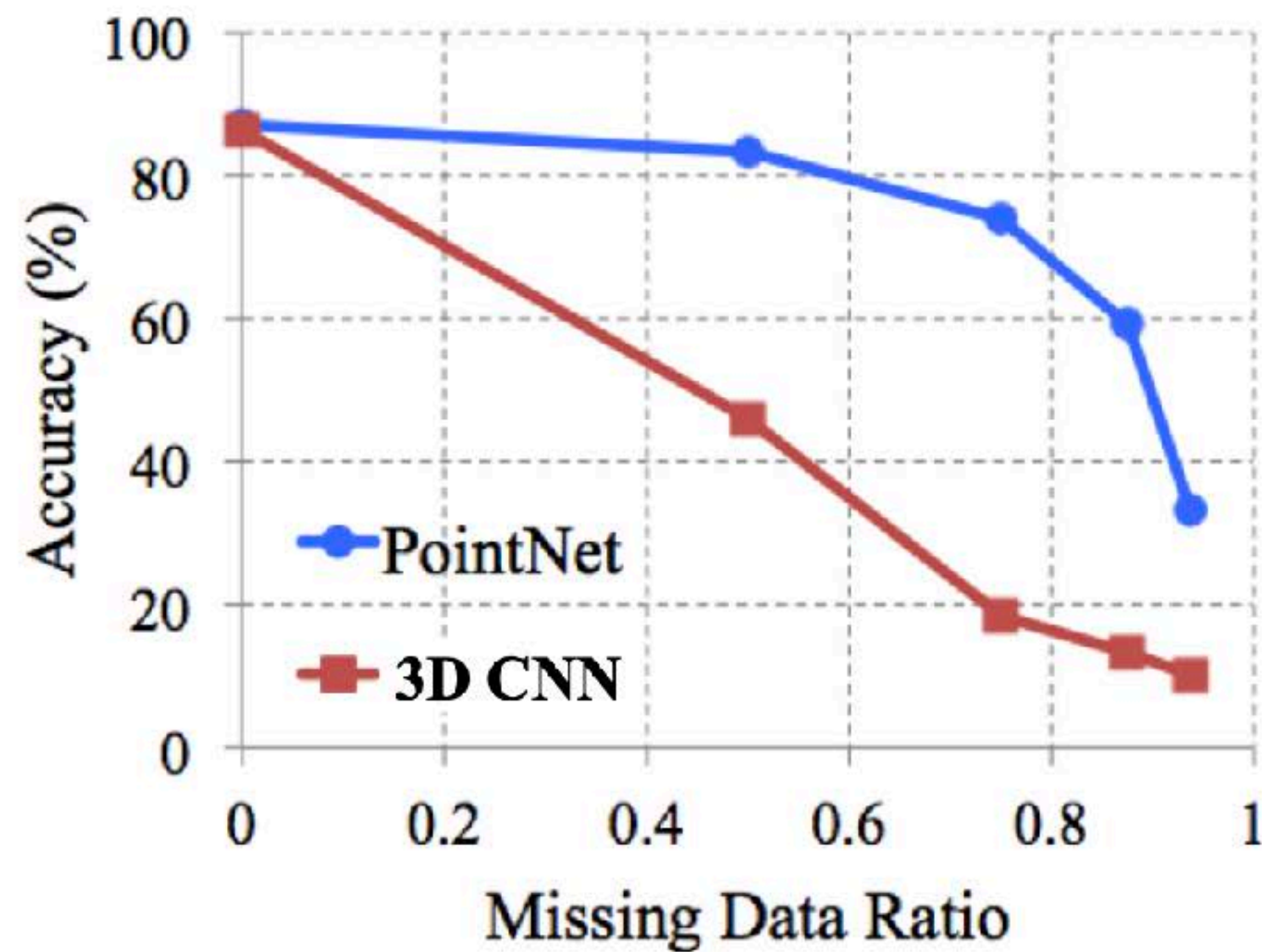


# Robustness to data corruption



*dataset: ModelNet40; metric: 40-class  
classification accuracy (%)*

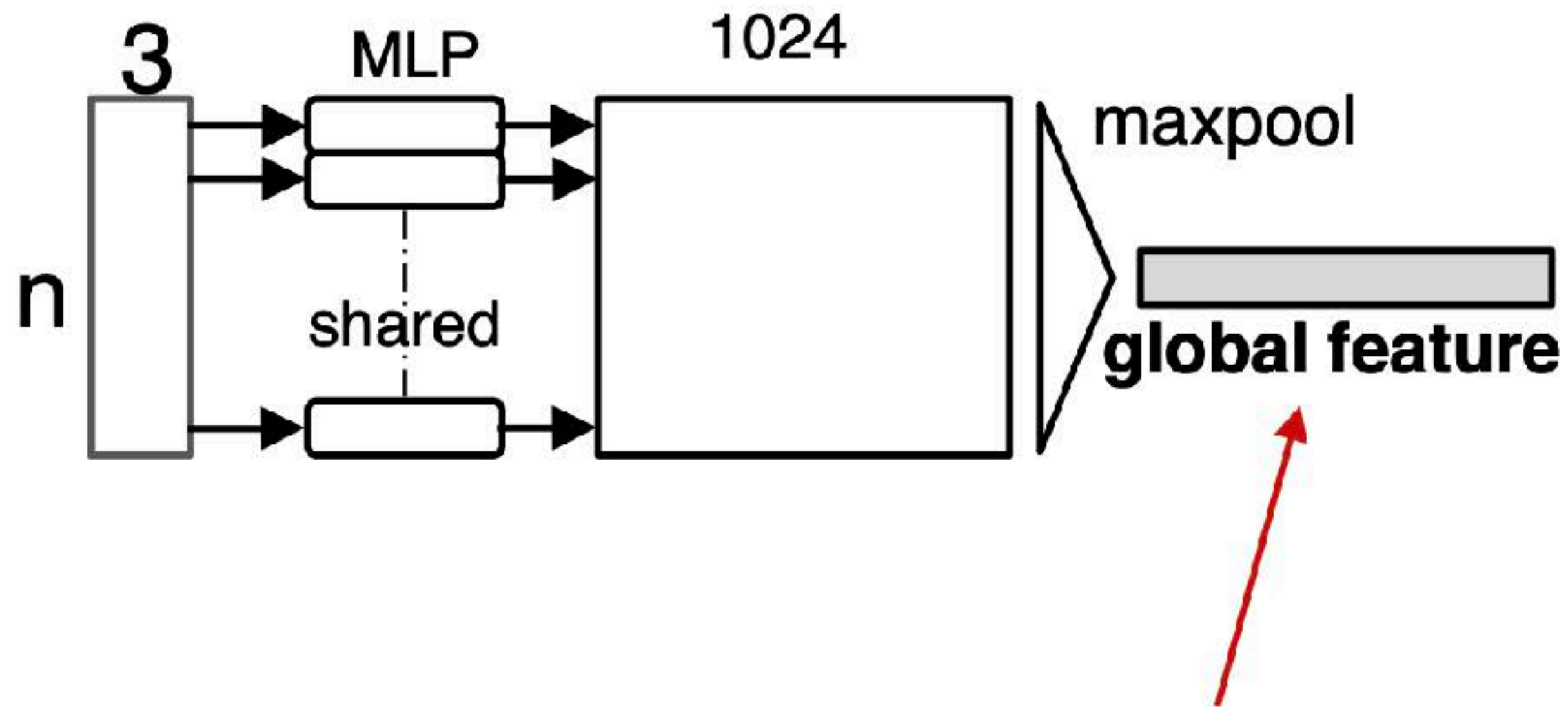
# Robustness to data corruption



Why is PointNet so robust to missing data?



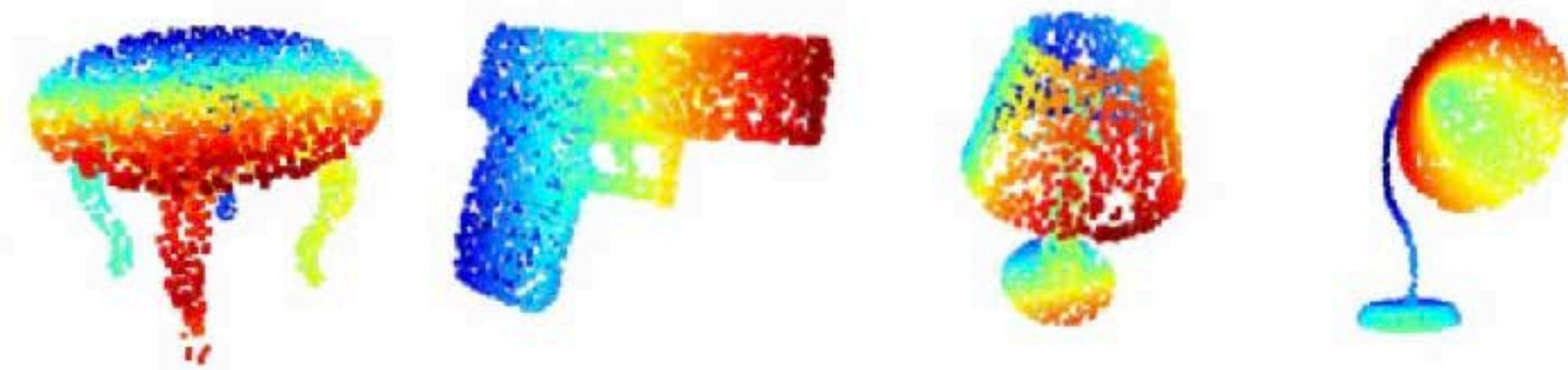
# Visualizing global point cloud features



Which input points are contributing to the global feature?  
(critical points)

# Visualizing global point cloud features

Original Shape:

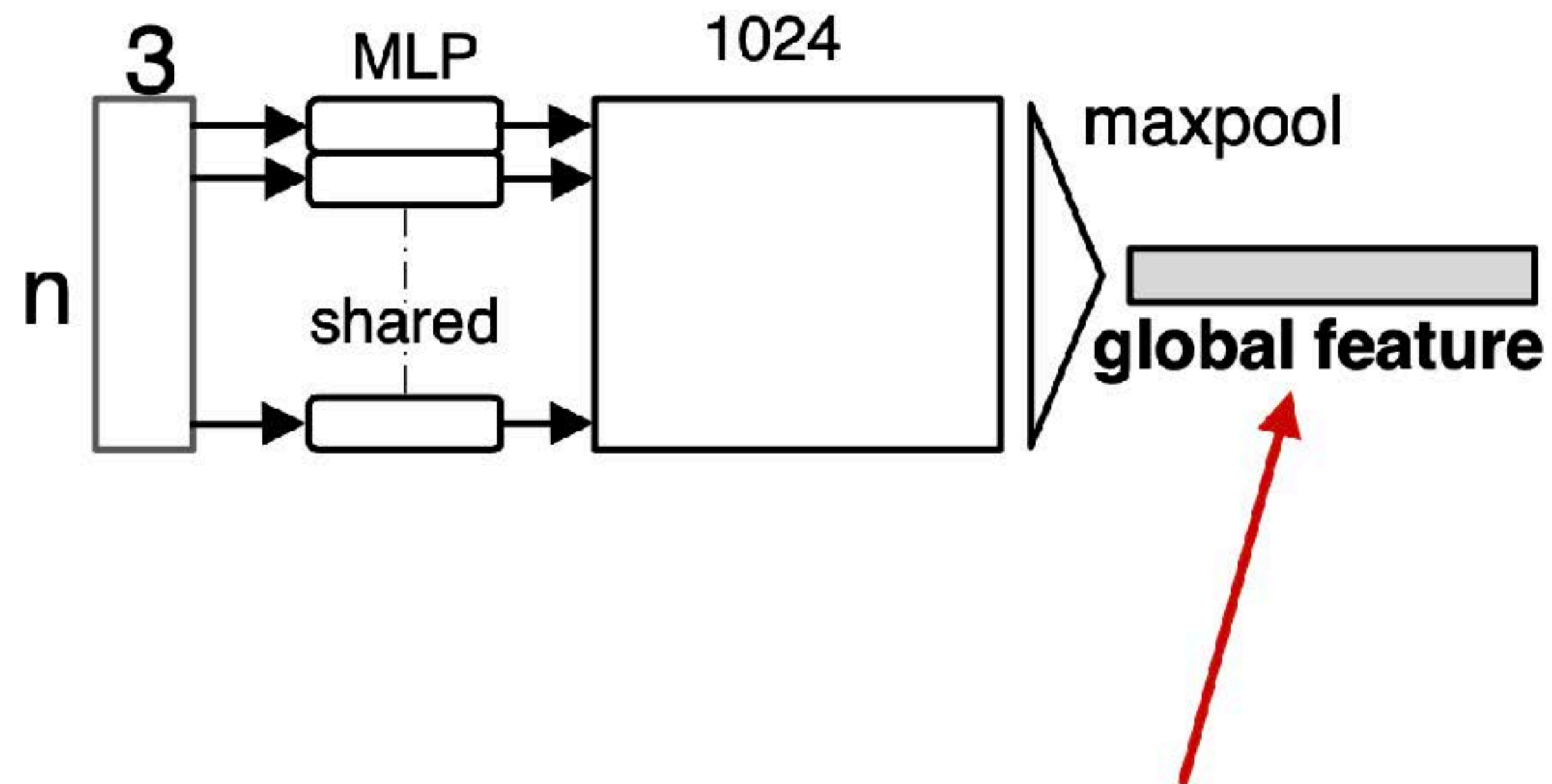


Critical Point Set:





# Visualizing global point cloud features



Which points won't affect the global feature?

# Visualizing global point cloud features

Original Shape:



Critical Point Set:



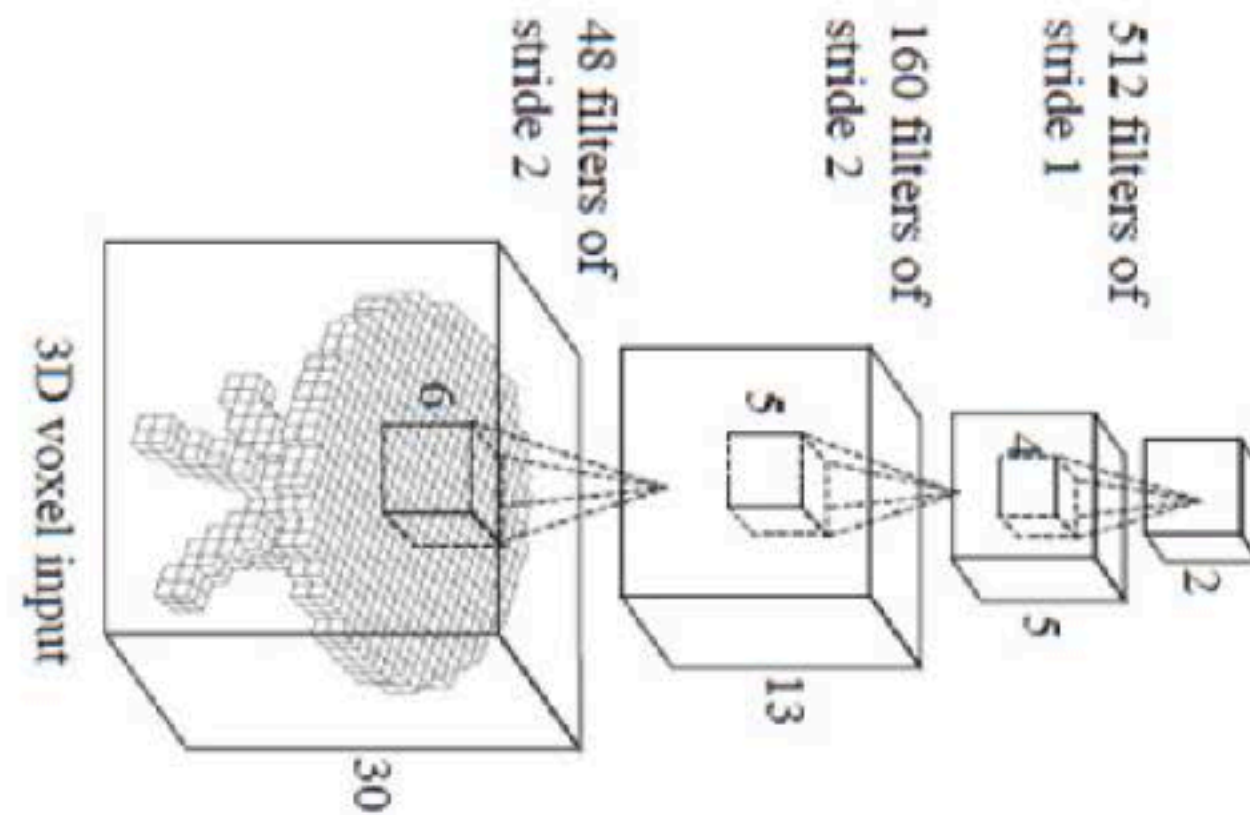
Upper bound set:





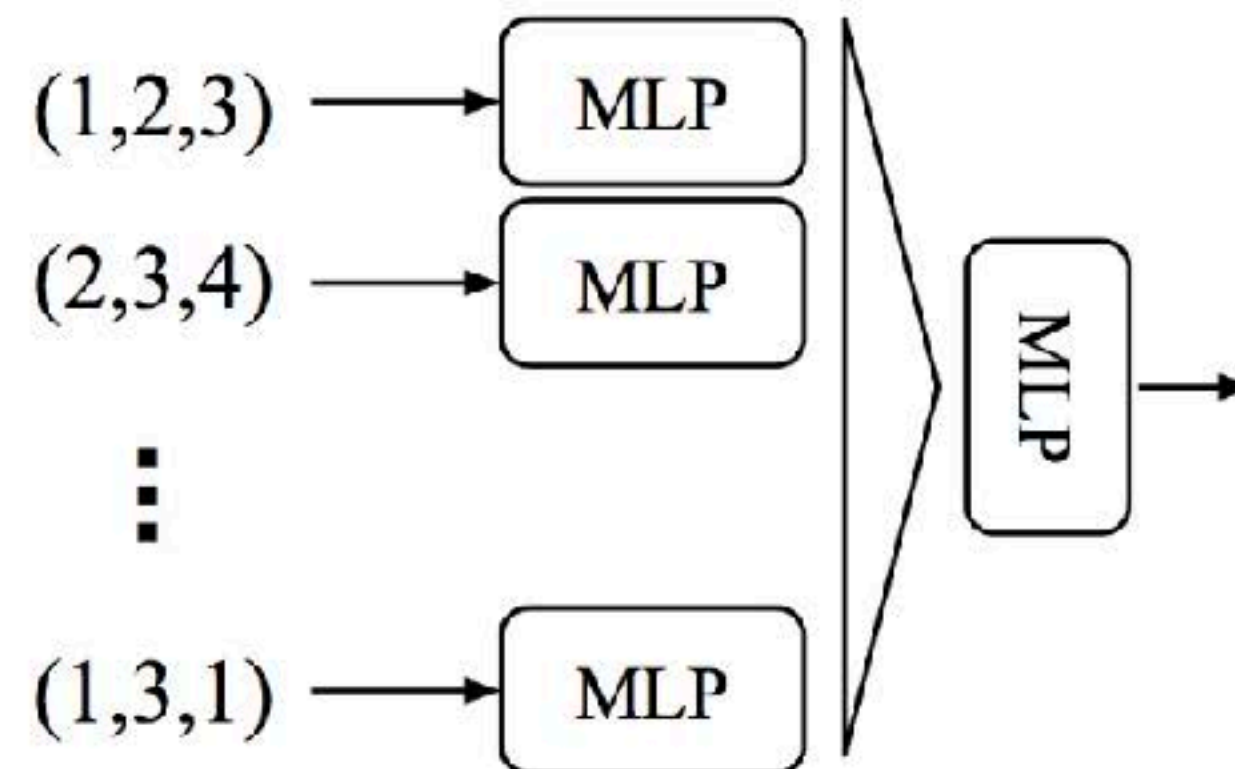
# Limitation of PointNet

Hierarchical feature learning  
Multiple levels of abstraction



**3D CNN (Wu et al.)**

Global feature learning  
Either one point or all points



**PointNet (vanilla) (Qi et al.)**

- No local context for each point
- Global feature depends on absolute coordinate. Hard to generalize to unseen scene configurations!

# Limitation of PointNet

Hierarchical feature learning  
Multiple levels of abstraction

Global feature learning  
Either one point or all points

Reflection:

What are the possible ways of improving PointNet?



**3D CNN (Wu et al.)**



**PointNet (vanilla) (Qi et al.)**

- No local context for each point
- Global feature depends on absolute coordinate. Hard to generalize to unseen scene configurations!



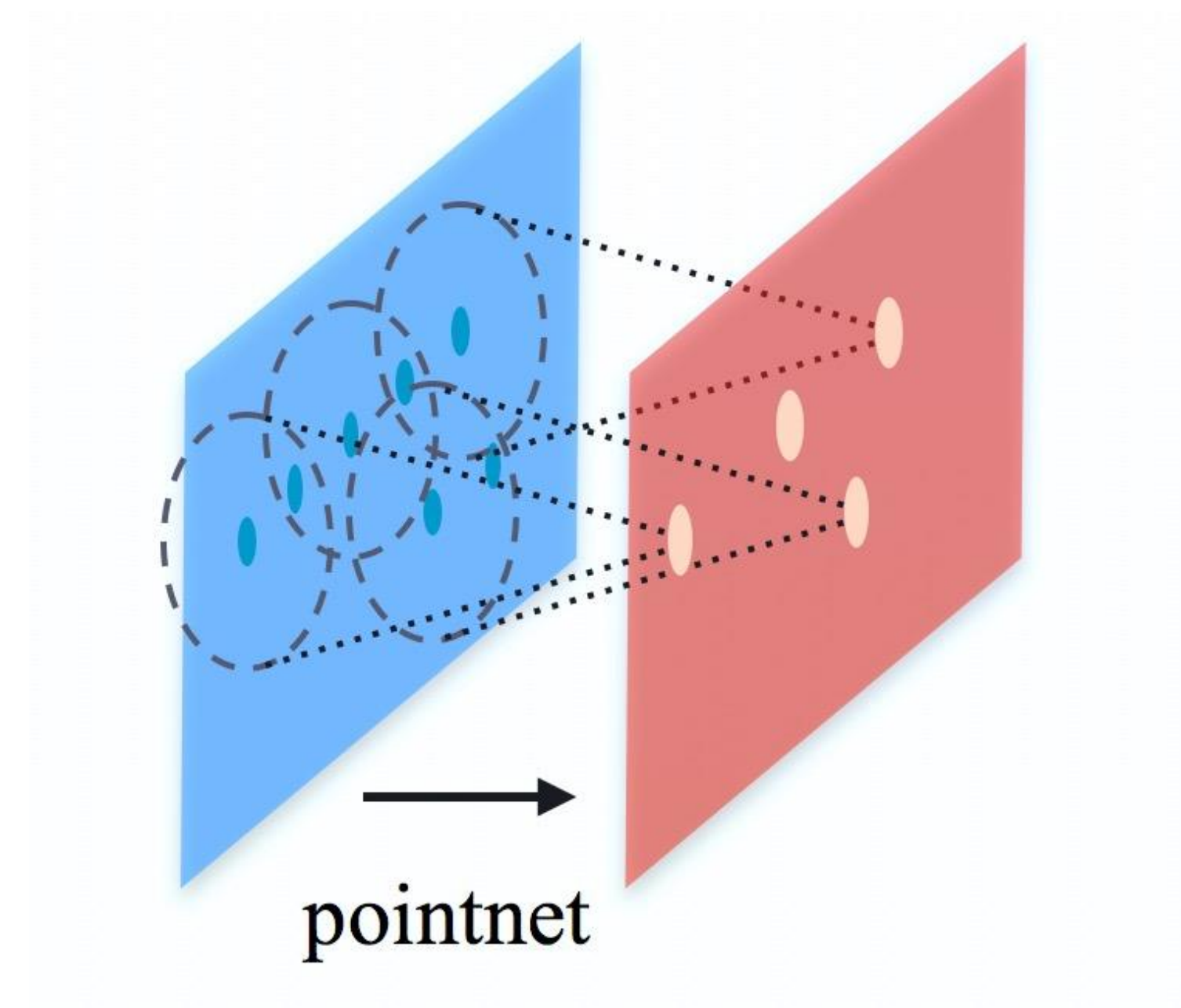
# Outline

- Voxel Networks
- Point Networks
  - PointNet
  - *PointNet++*
  - Sampling Issues

# PointNet++

Basic idea: Recursively apply pointnet at local regions.

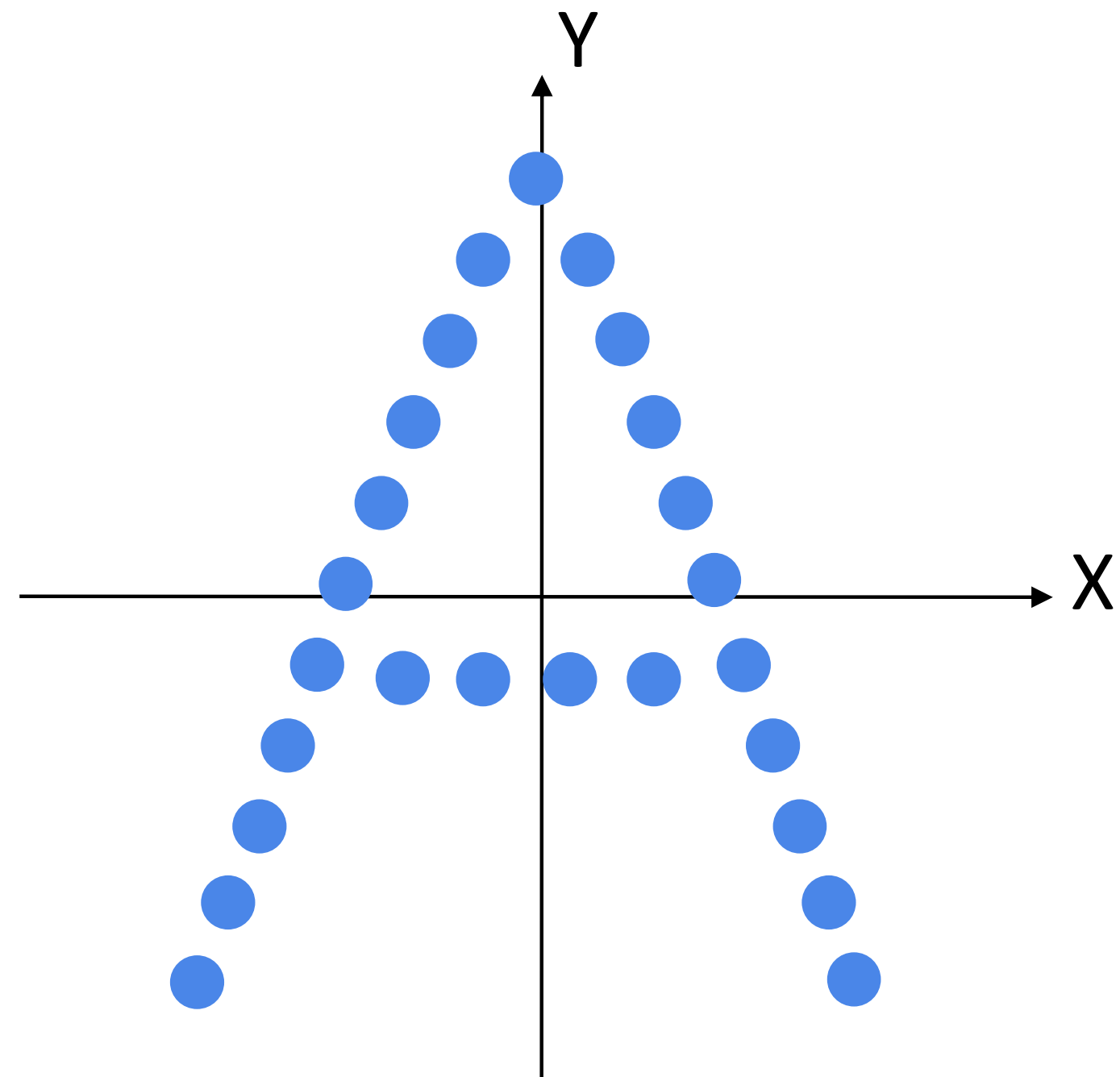
- ✓ Hierarchical feature learning
- ✓ Local translation invariance
- ✓ Permutation invariance



Charles R. Qi, Li Yi, Hao Su, Leonidas Guibas. PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space (NIPS'17)

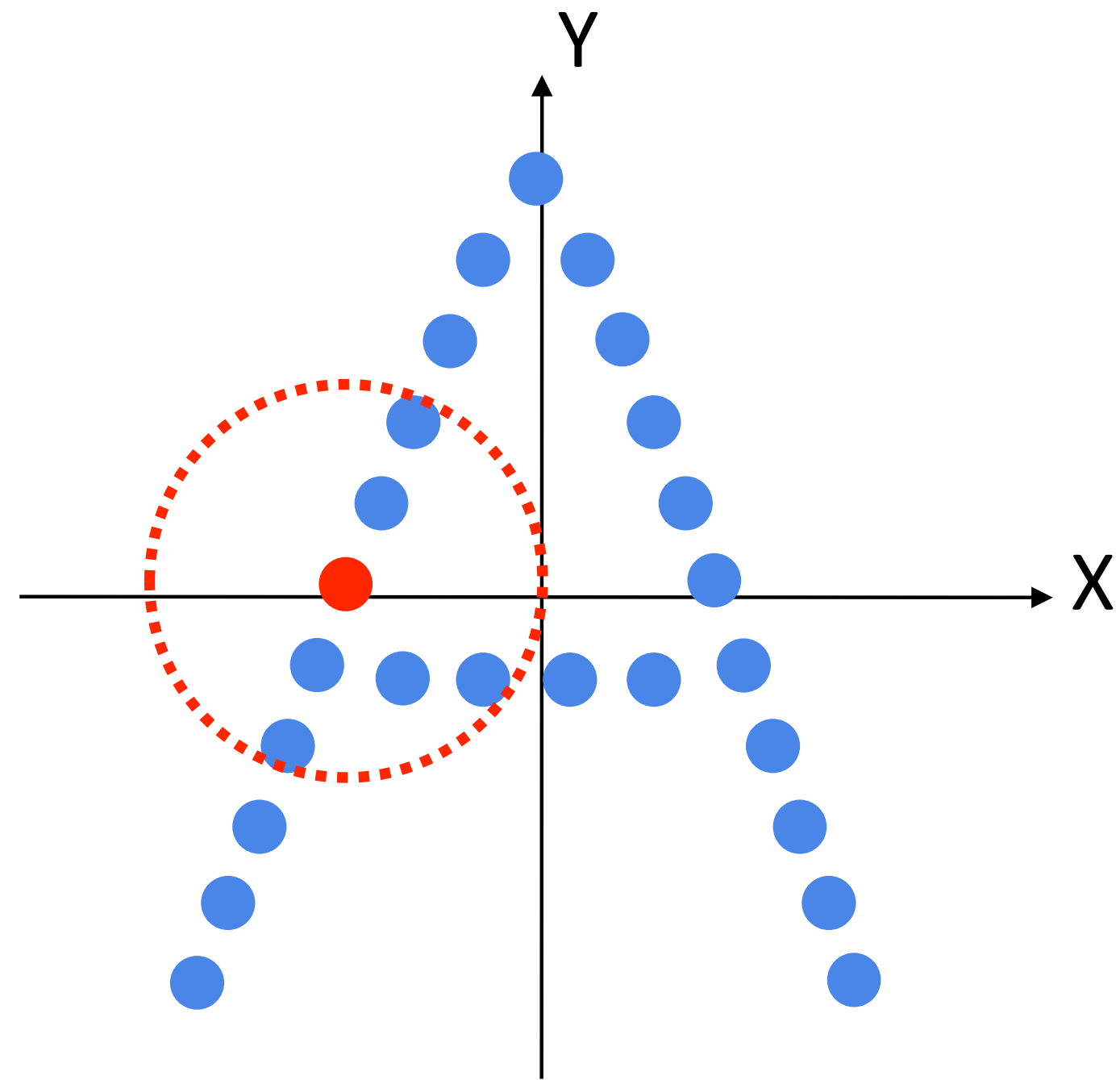


# Hierarchical point feature learning

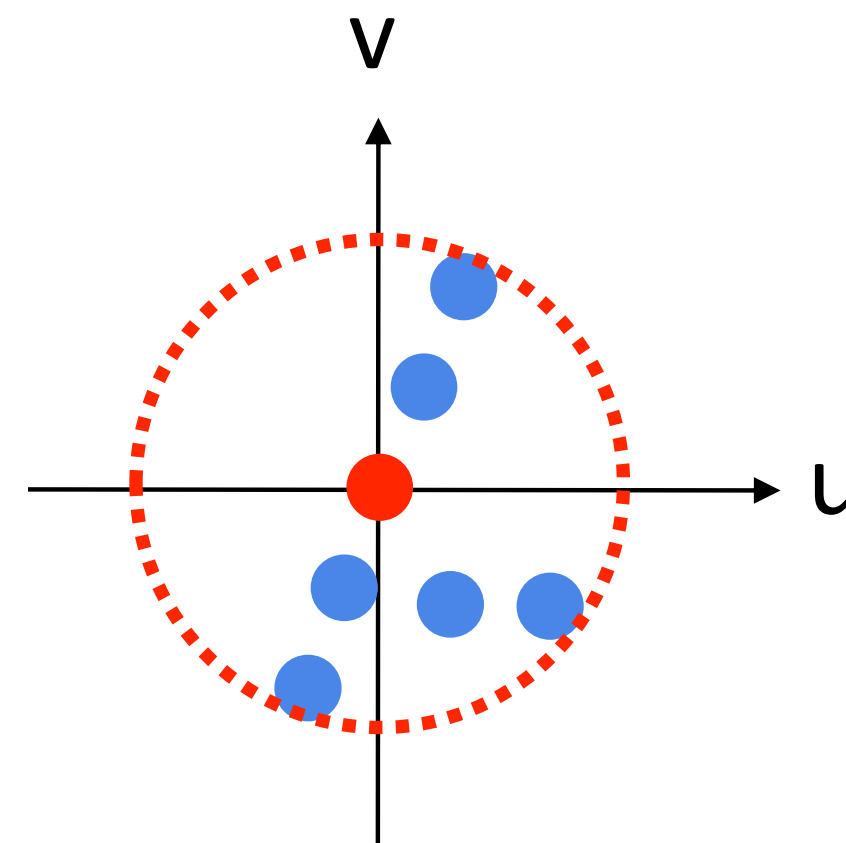


$N$  points in  $(X,Y)$

# Hierarchical point feature learning



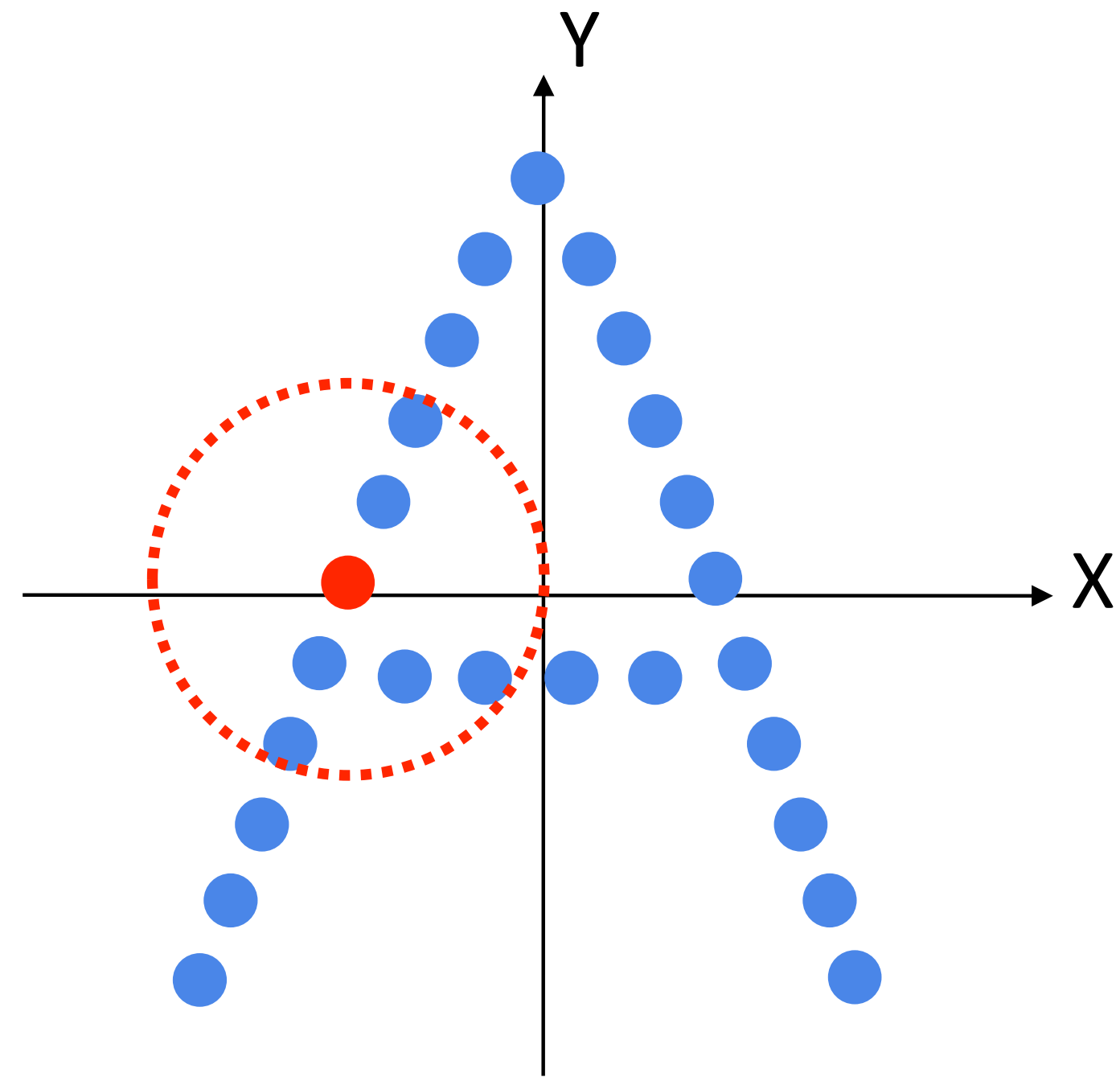
$N$  points in  $(X,Y)$



$k$  points in local  
coordinates  $(u,v)$

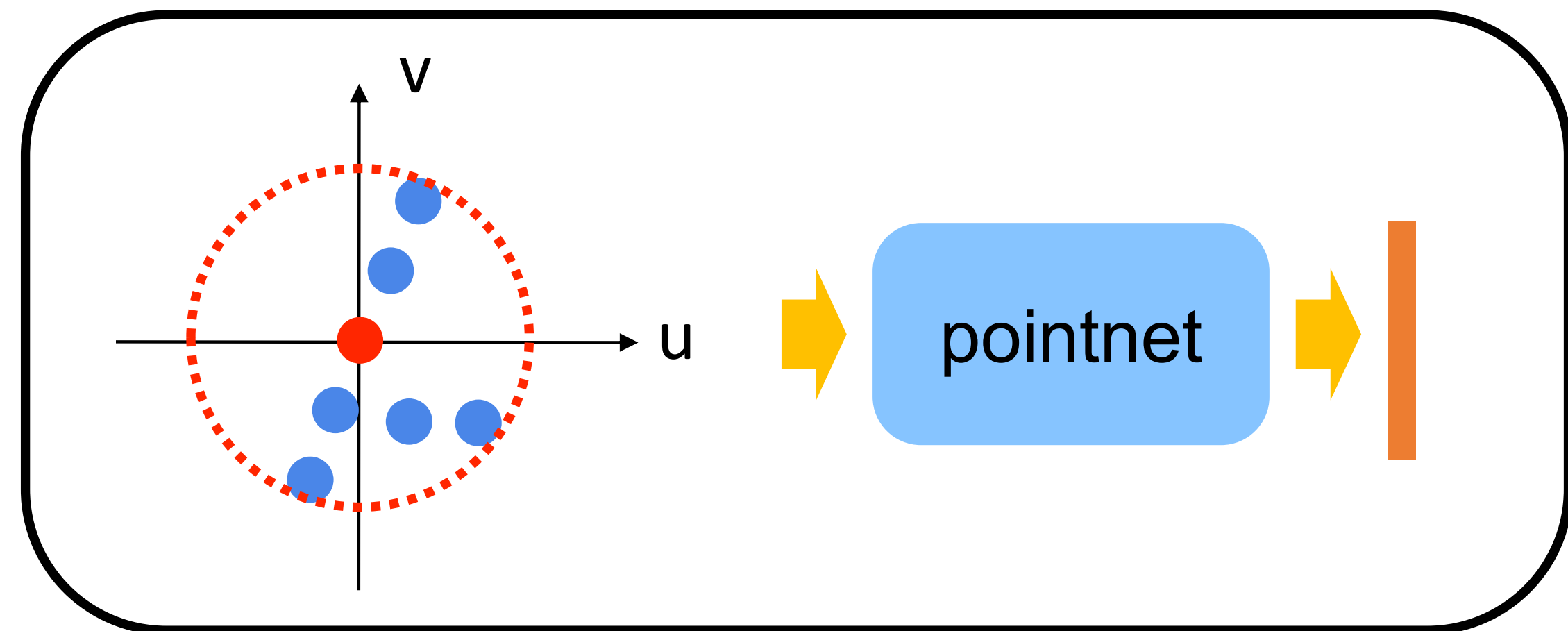


# Hierarchical point feature learning



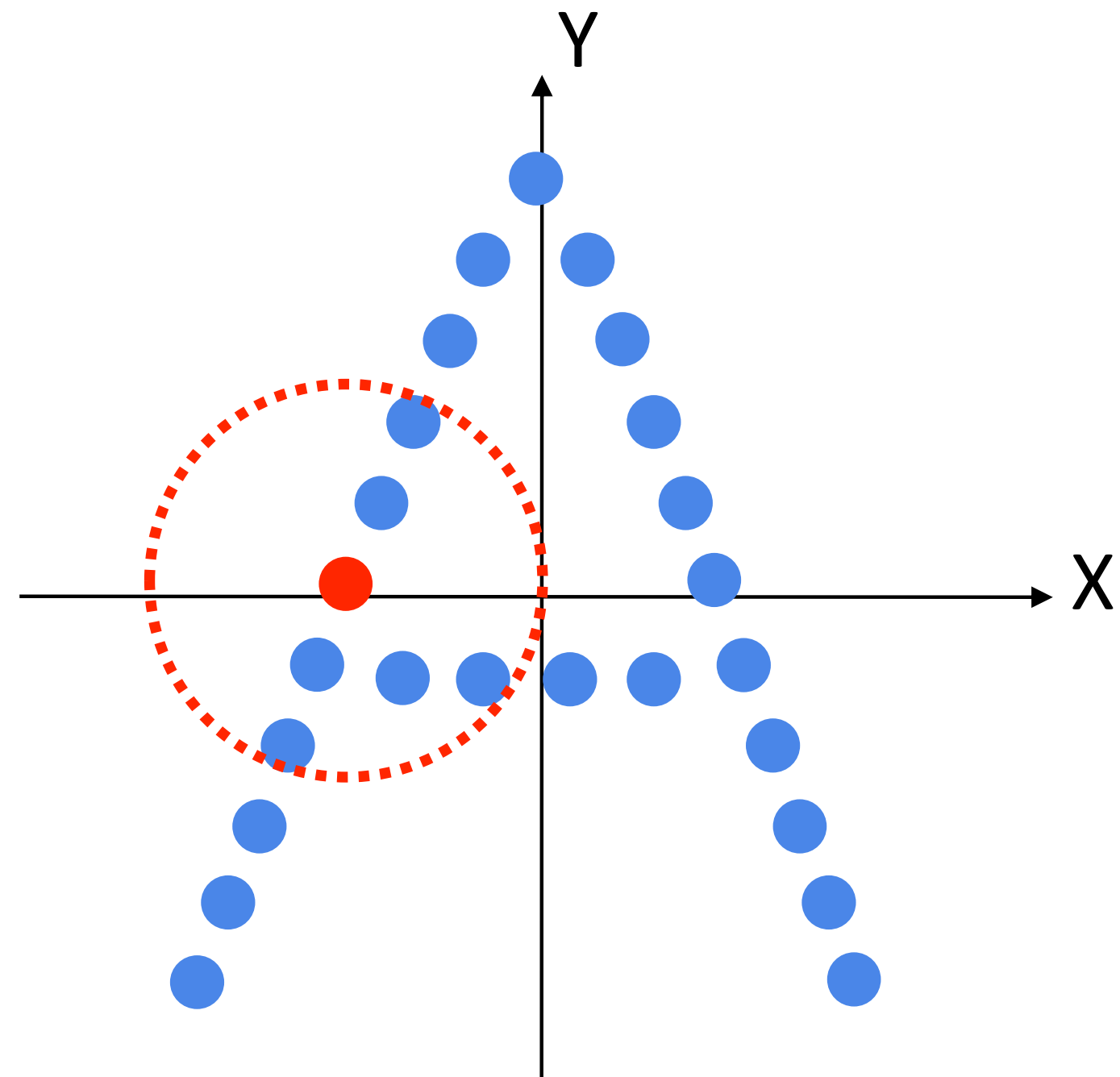
N points in (X,Y)

Apply pointnet at a local region

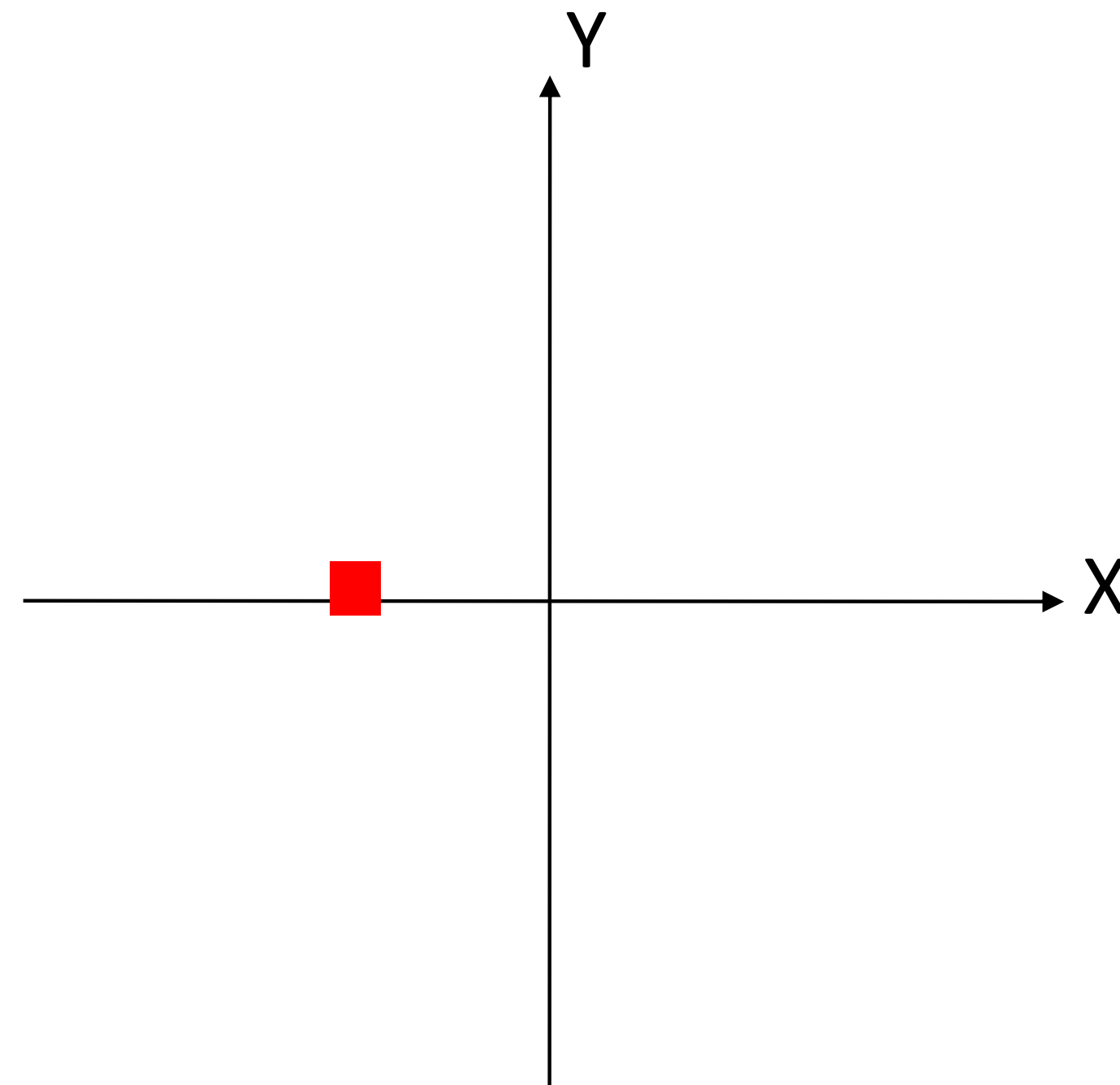


k points in local  
coordinates (u,v)

# Hierarchical point feature learning



$N$  points in  $(X,Y)$



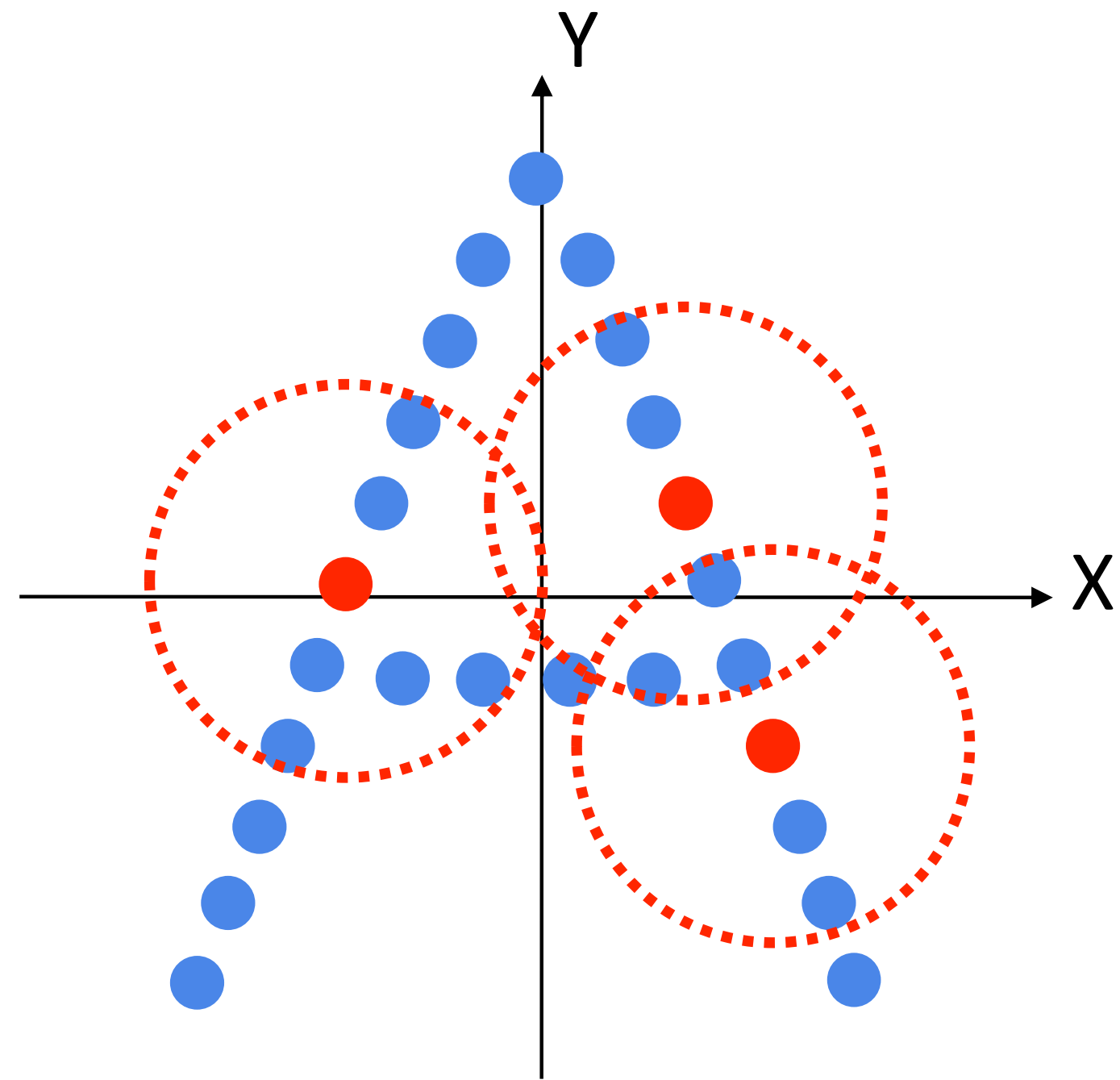
points in  $(X,Y, \mathbf{F})$

Euclidean space

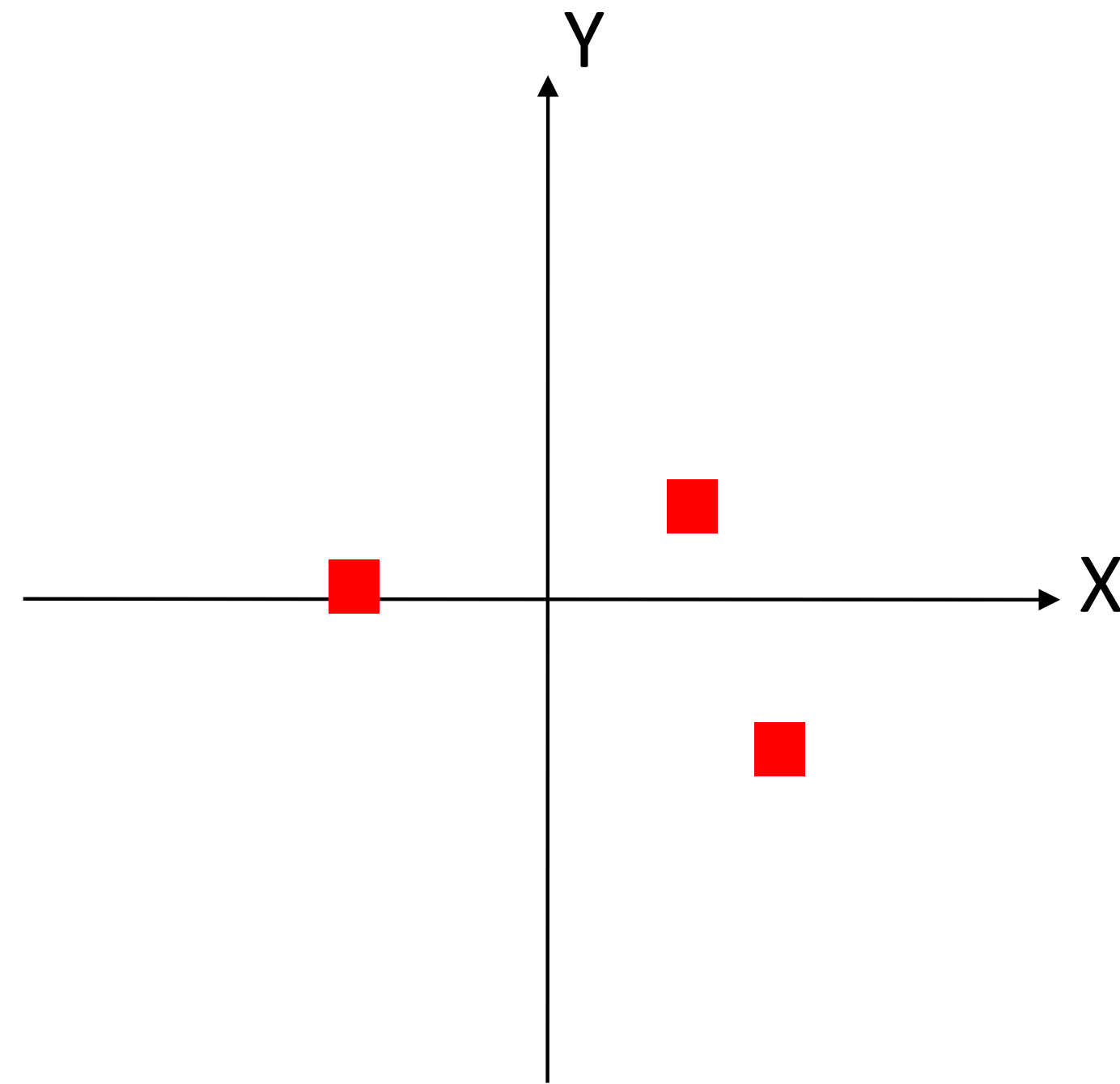
high-dim feature space



# Hierarchical point feature learning

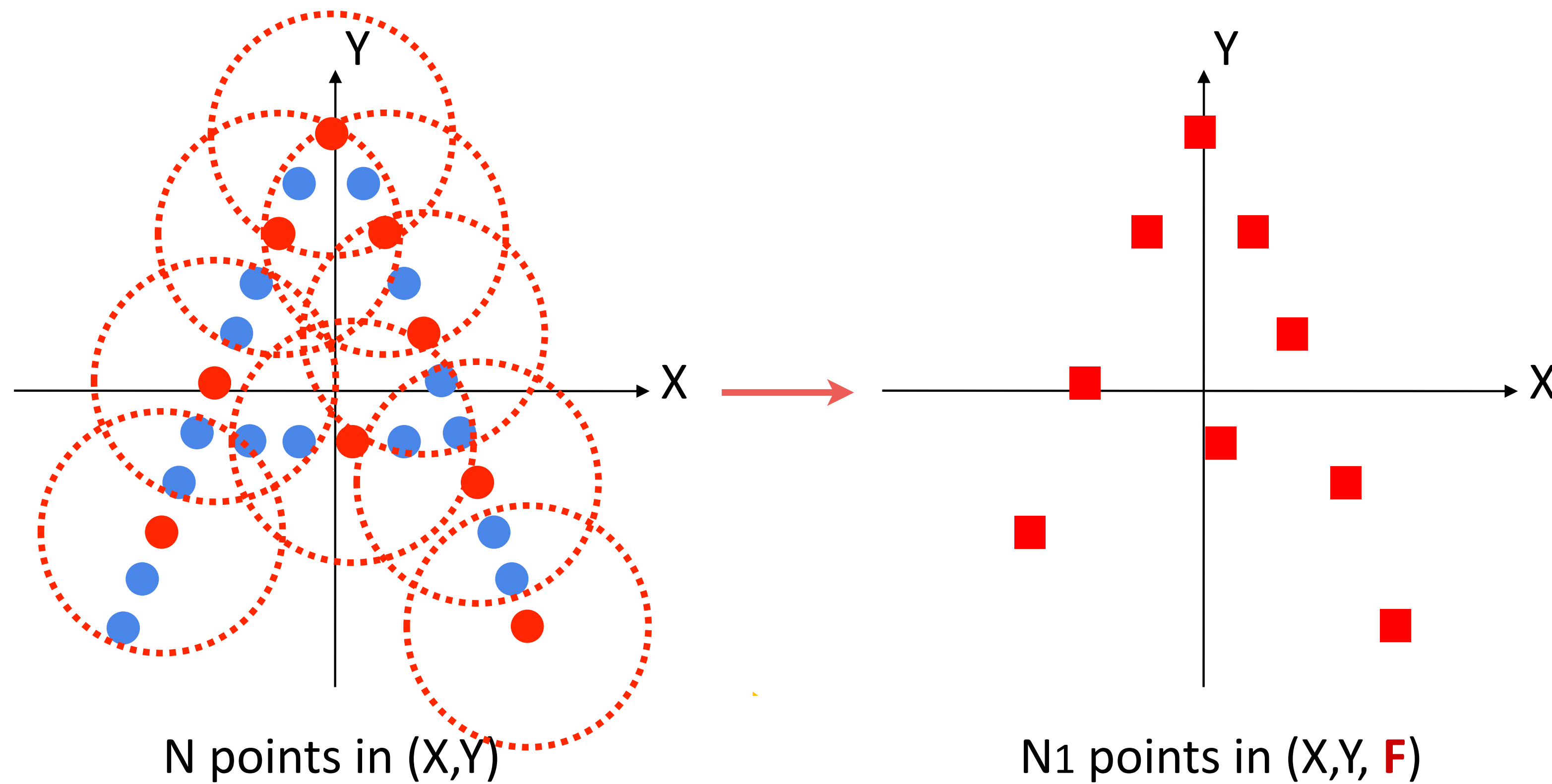


N points in (X,Y)



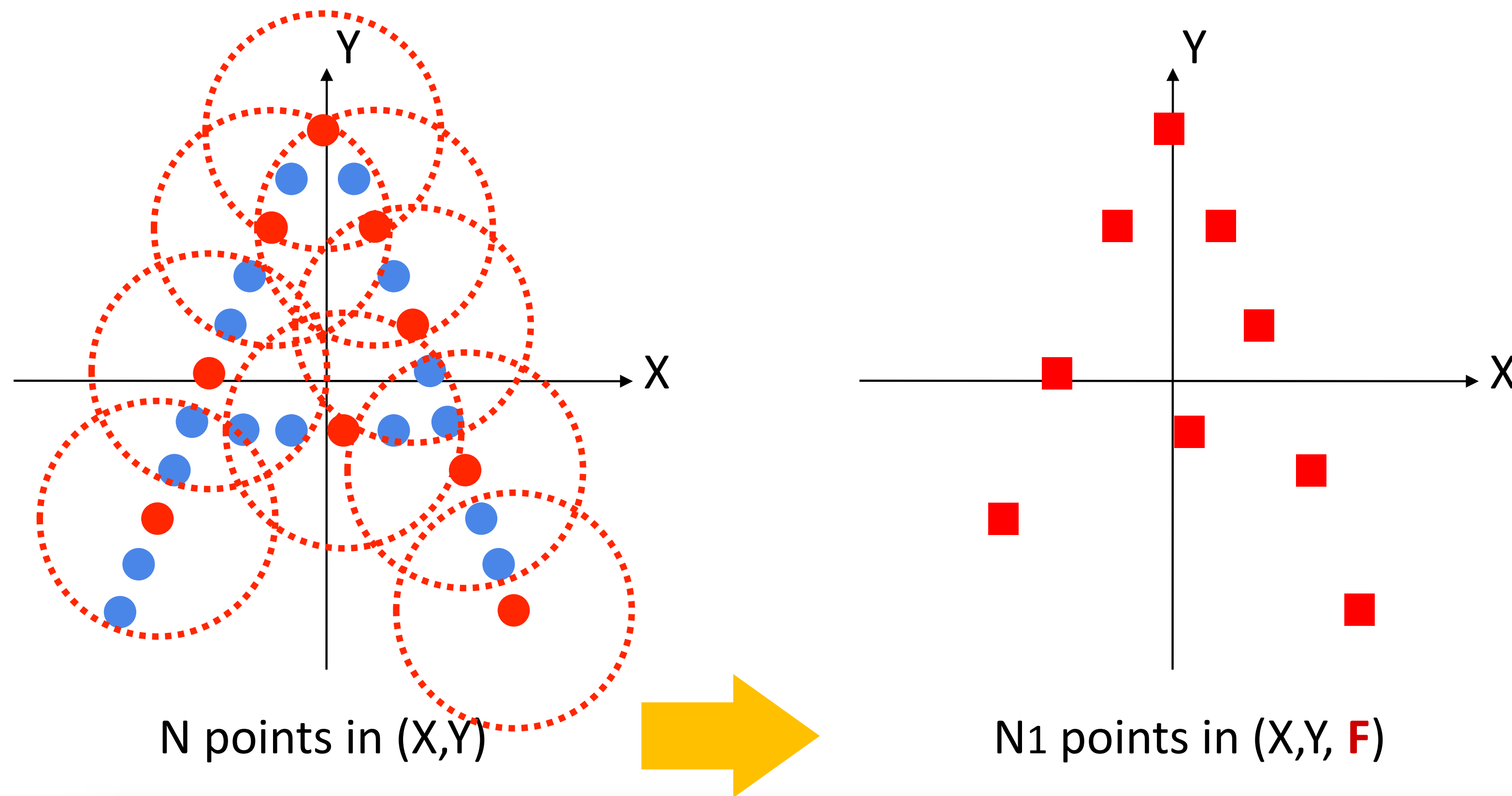
points in (X,Y, **F**)

# Hierarchical point feature learning





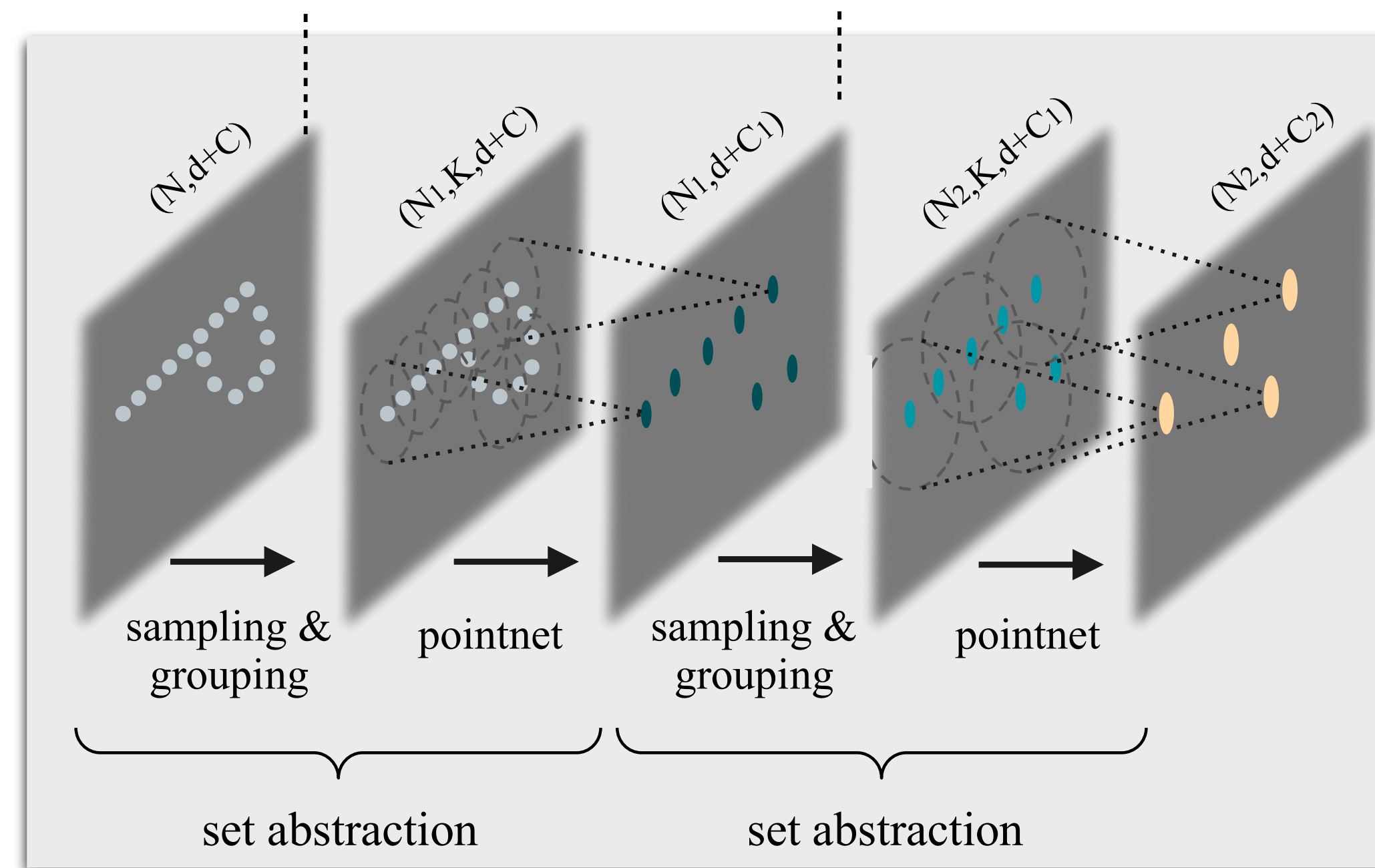
# Hierarchical point feature learning



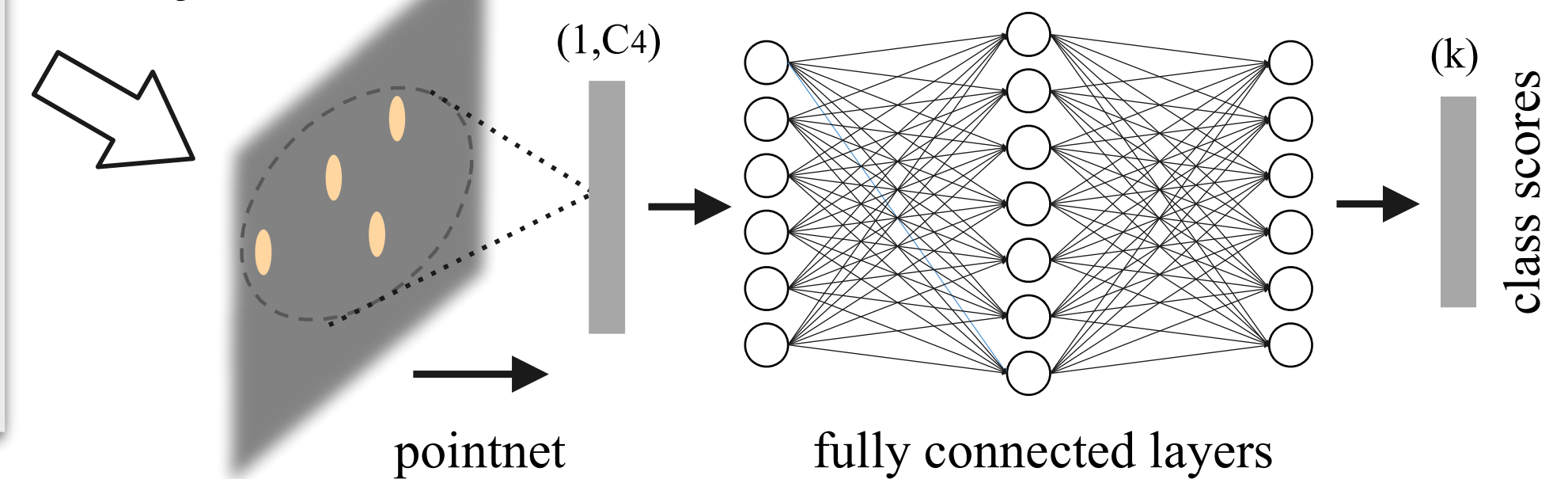
**Set Abstraction:** farthest point sampling + grouping + pointnet

# PointNet++ for classification

## *Hierarchical point set feature learning*

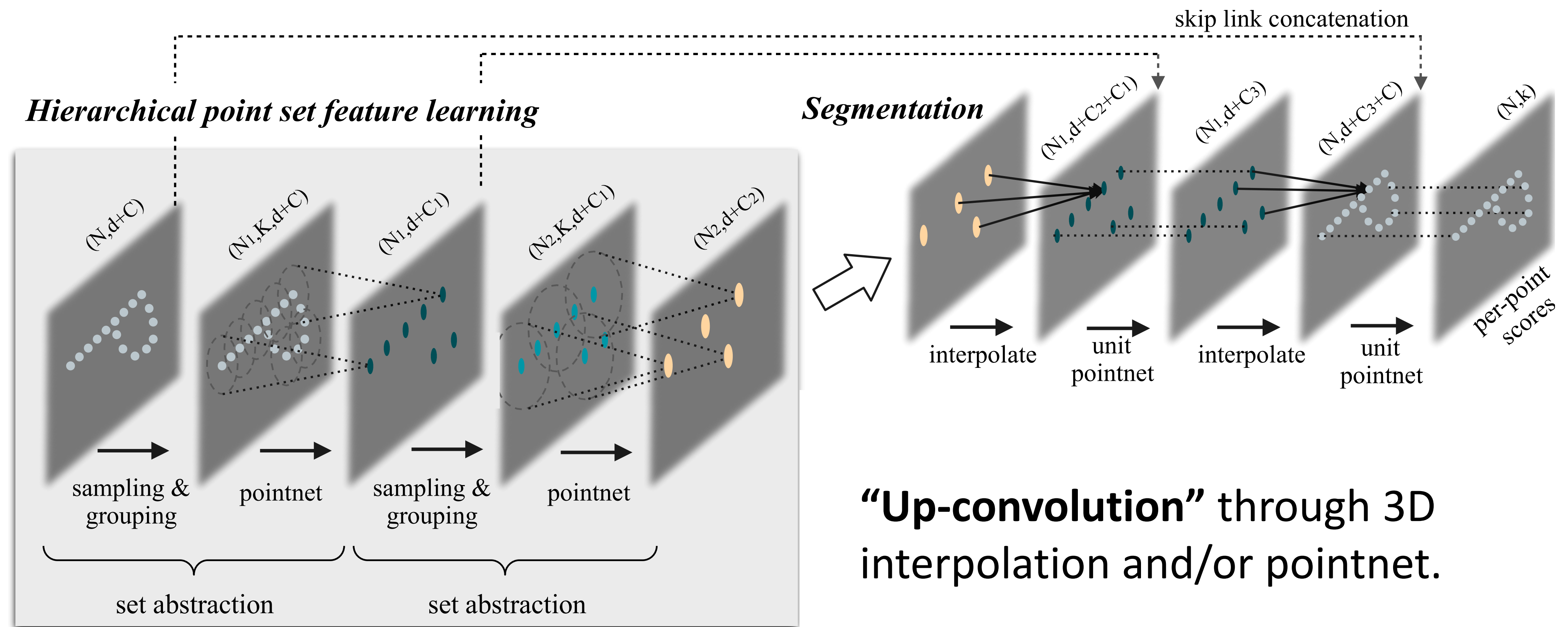


## *Classification*

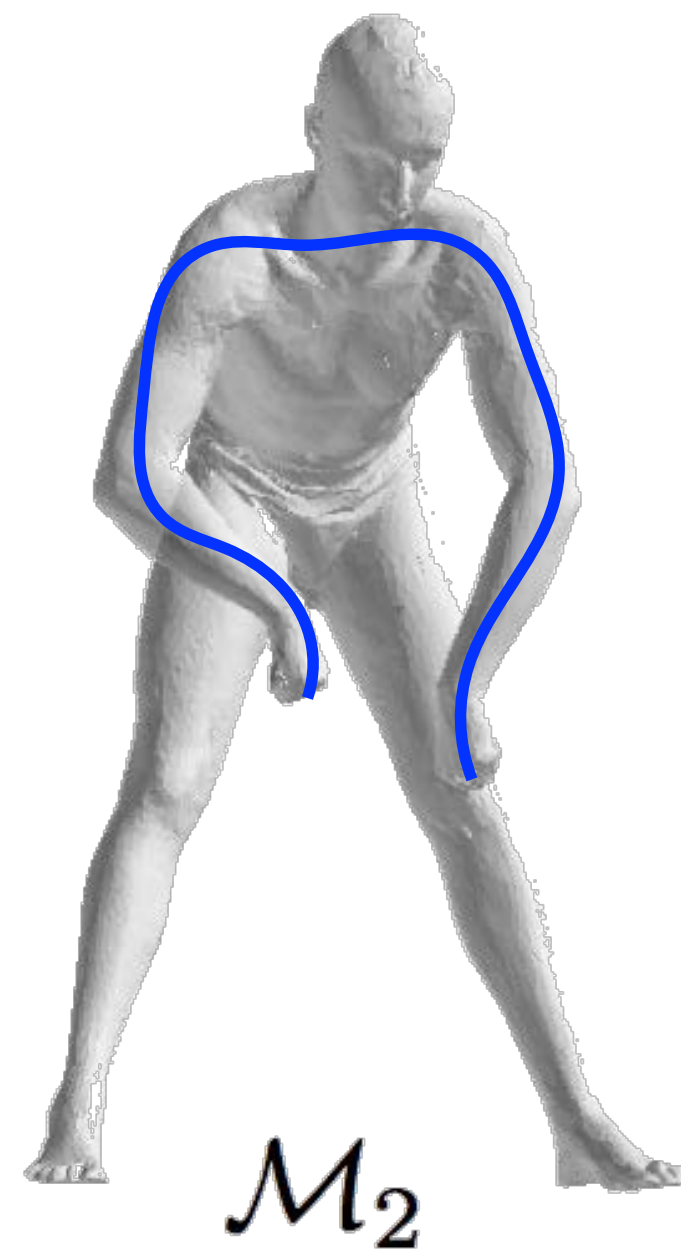
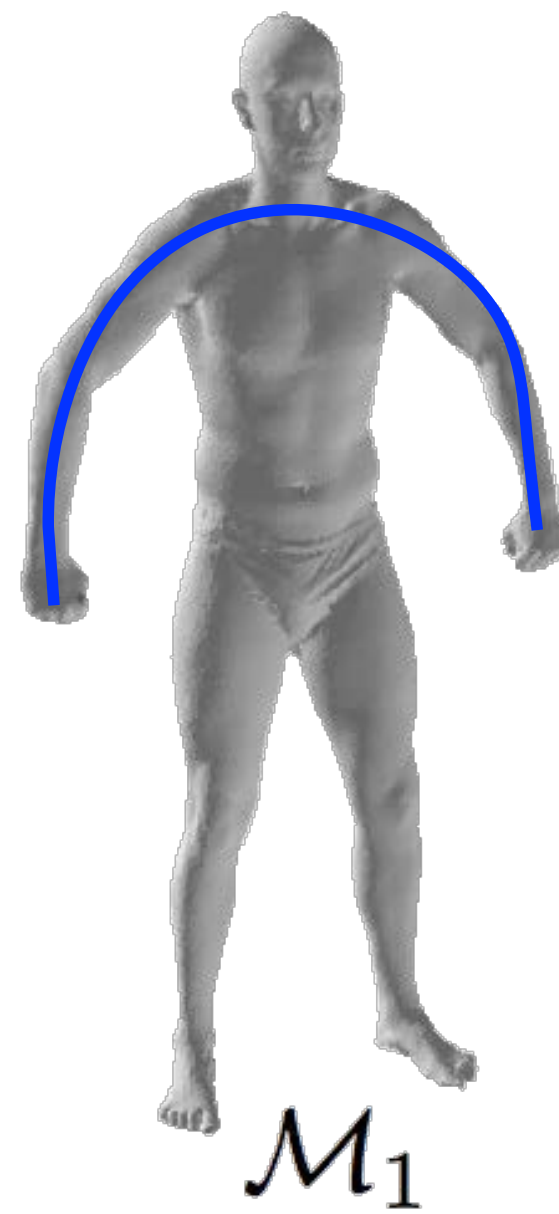




# PointNet++ for segmentation



# Recognition with isometric invariance



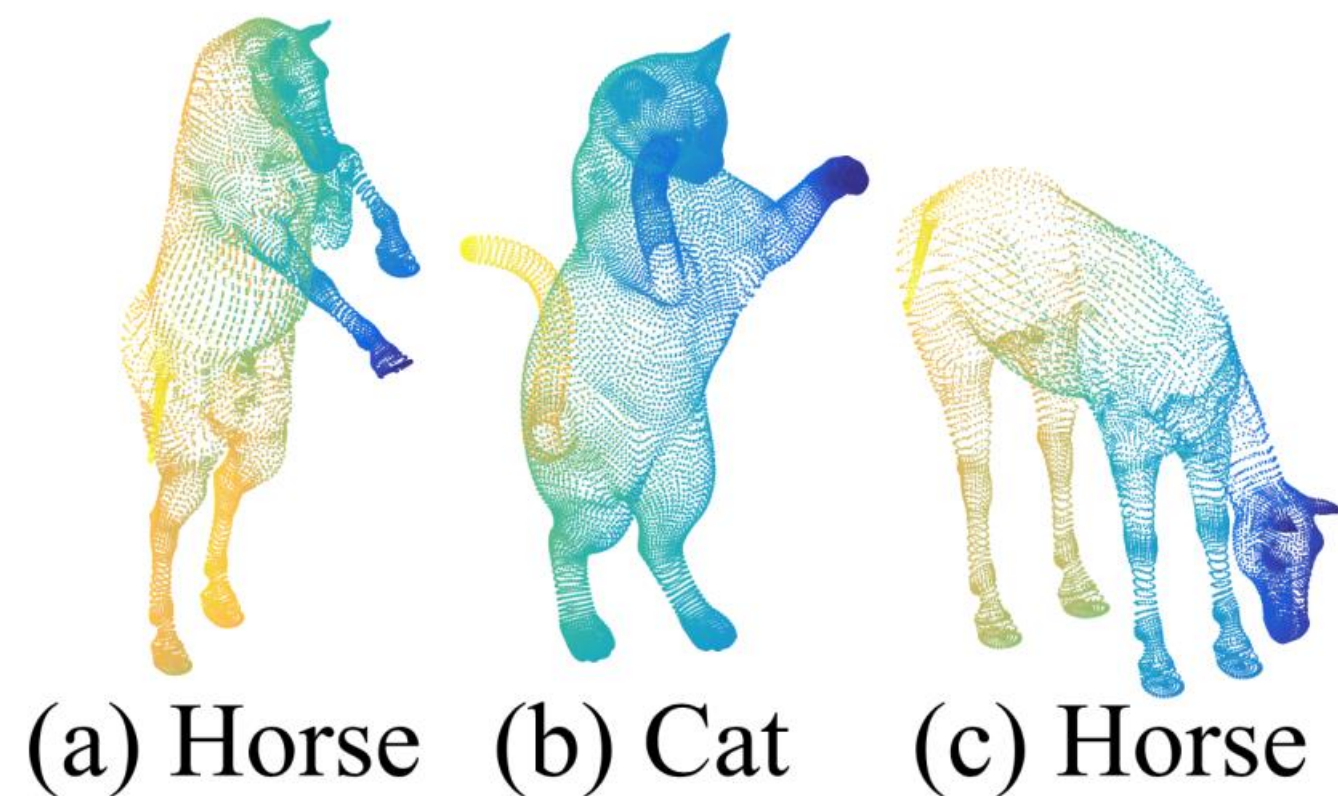
isometry = length-preserving transform



# PointNet++ in non-Euclidean space

**For animate shape recognition, PointNet++ can generalize to non-Euclidean spaces**

- ❖ intrinsic point features (HKS, WKS, Gaussian curvature)
- ❖ intrinsic distance metric (geodesic)



|             | Metric space  | Input feature      | Accuracy (%) |
|-------------|---------------|--------------------|--------------|
| DeepGM [13] | -             | Intrinsic features | 93.03        |
| Ours        | Euclidean     | XYZ                | 60.18        |
|             | Euclidean     | Intrinsic features | 94.49        |
|             | Non-Euclidean | Intrinsic features | <b>96.09</b> |

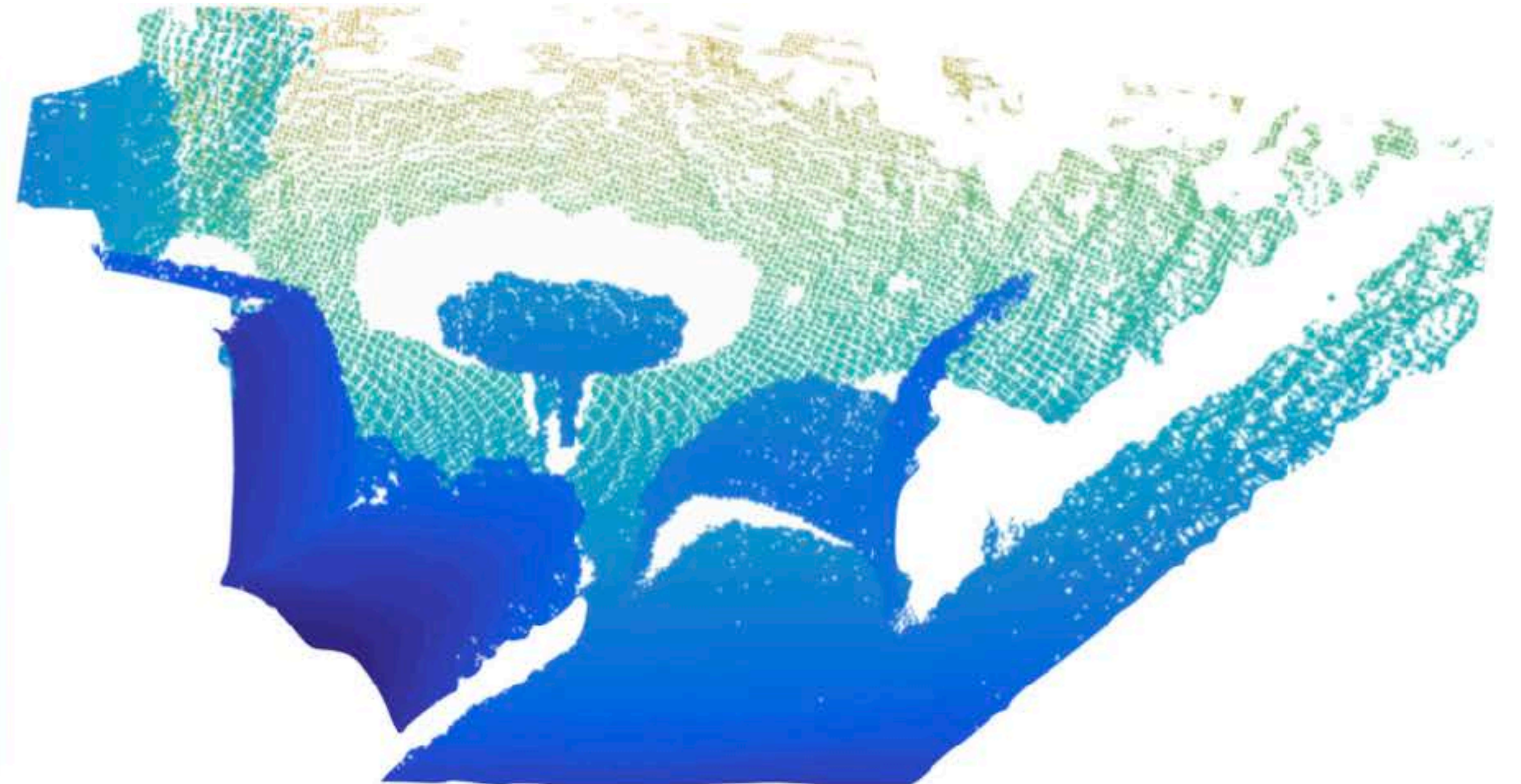
*dataset: SHREC15; metric: shape classification accuracy (%)*

# Outline

- Voxel Networks
- Point Networks
  - PointNet
  - PointNet++
  - *Sampling Issues*

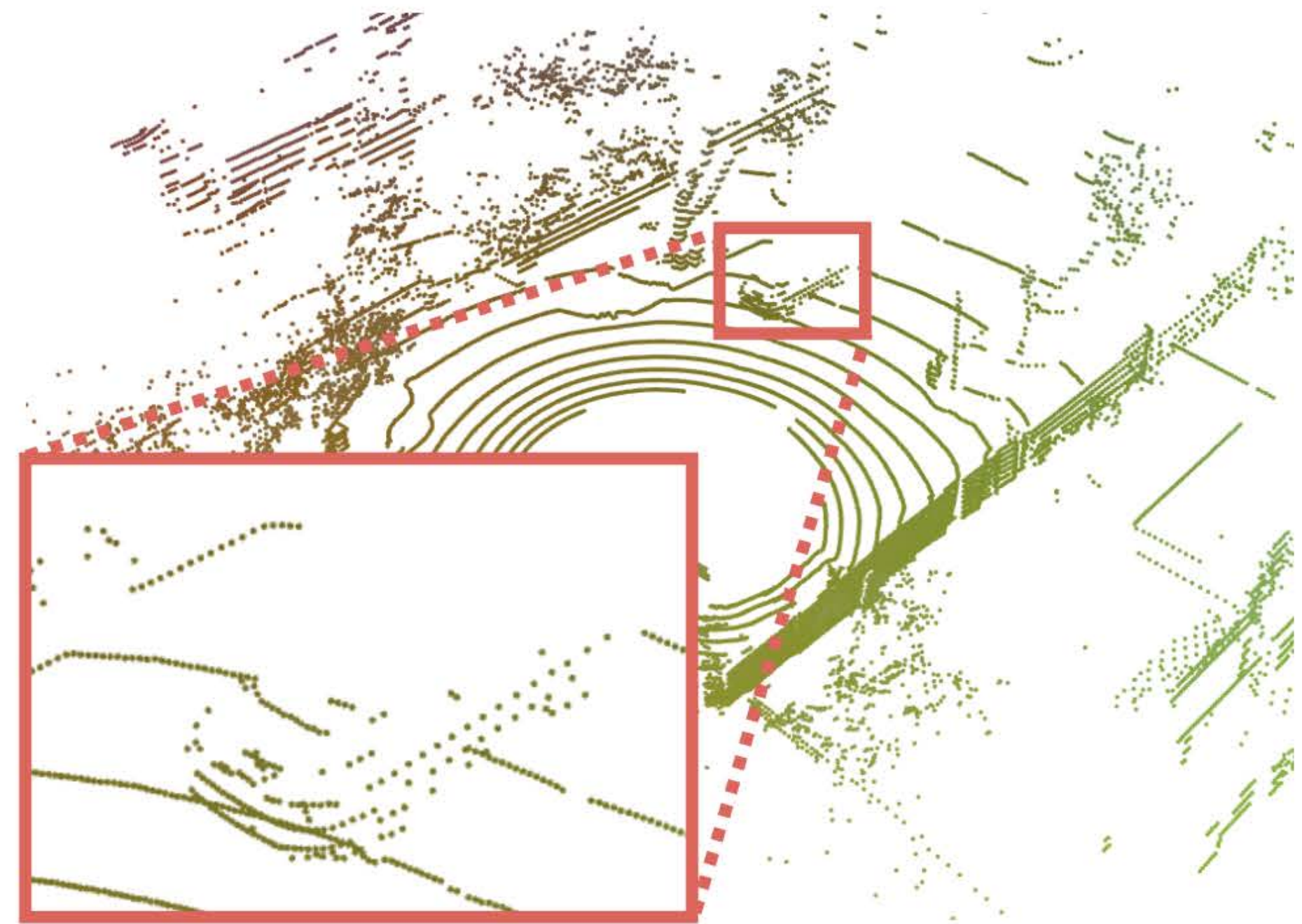
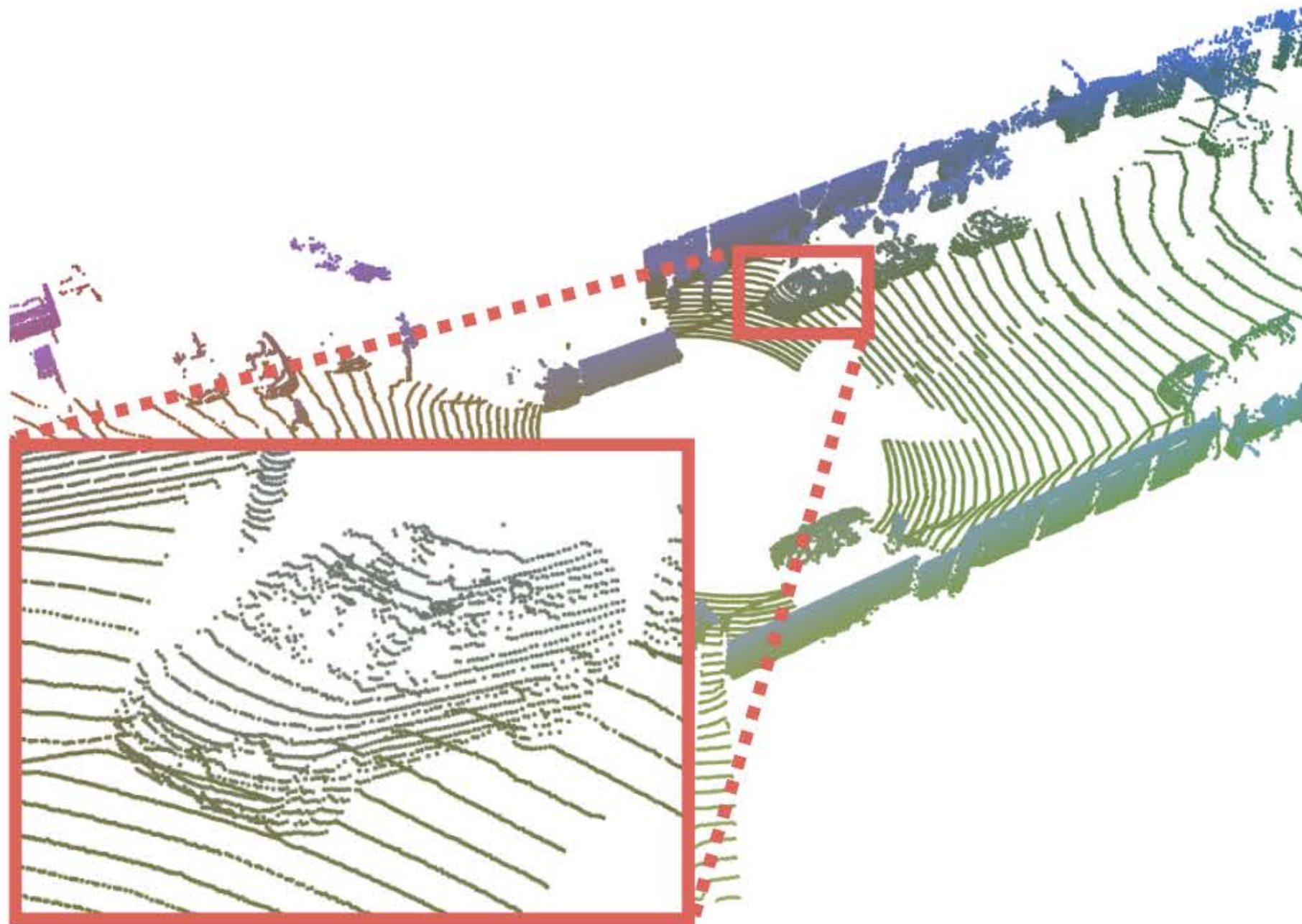


# Sampling issues in point clouds





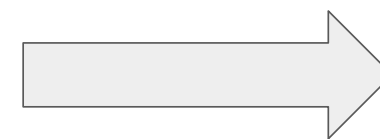
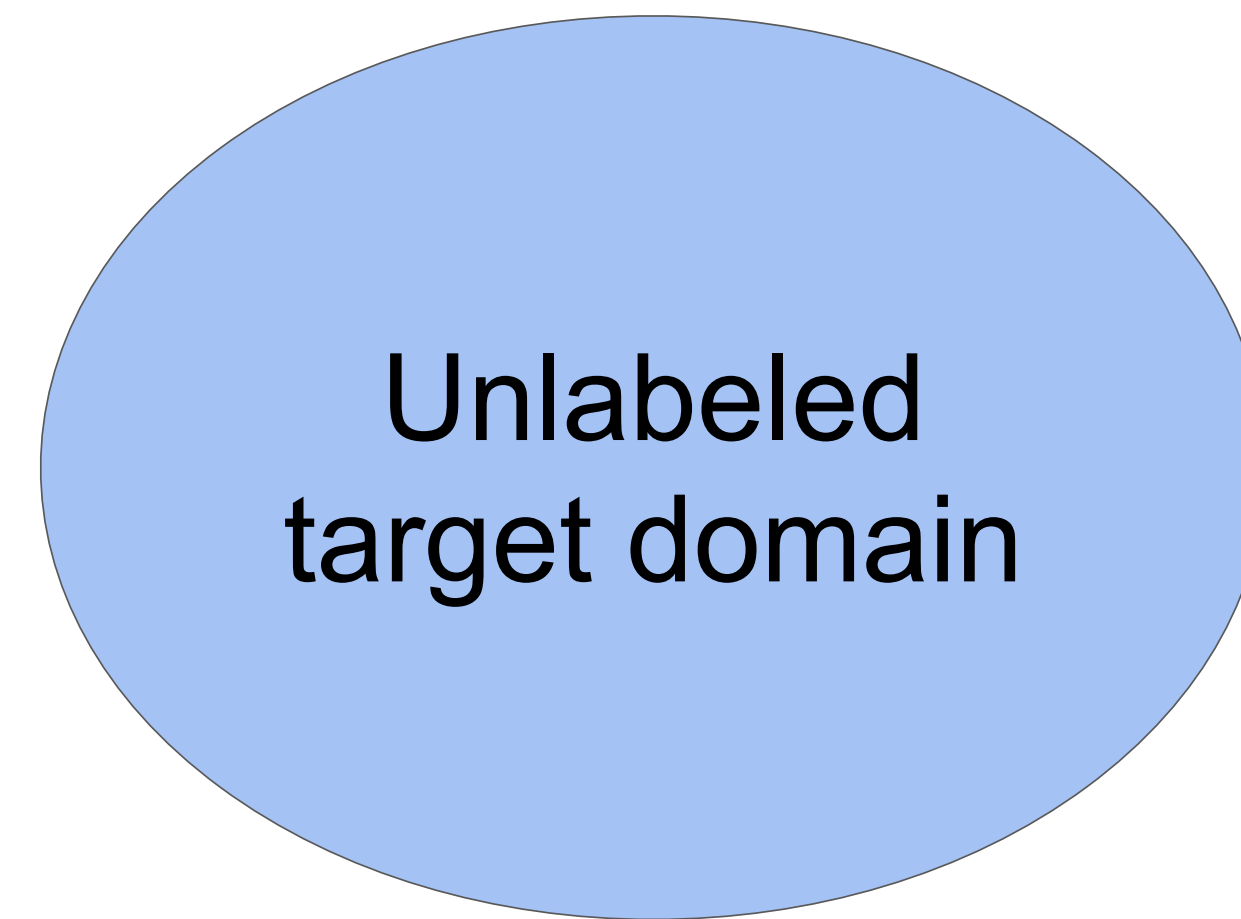
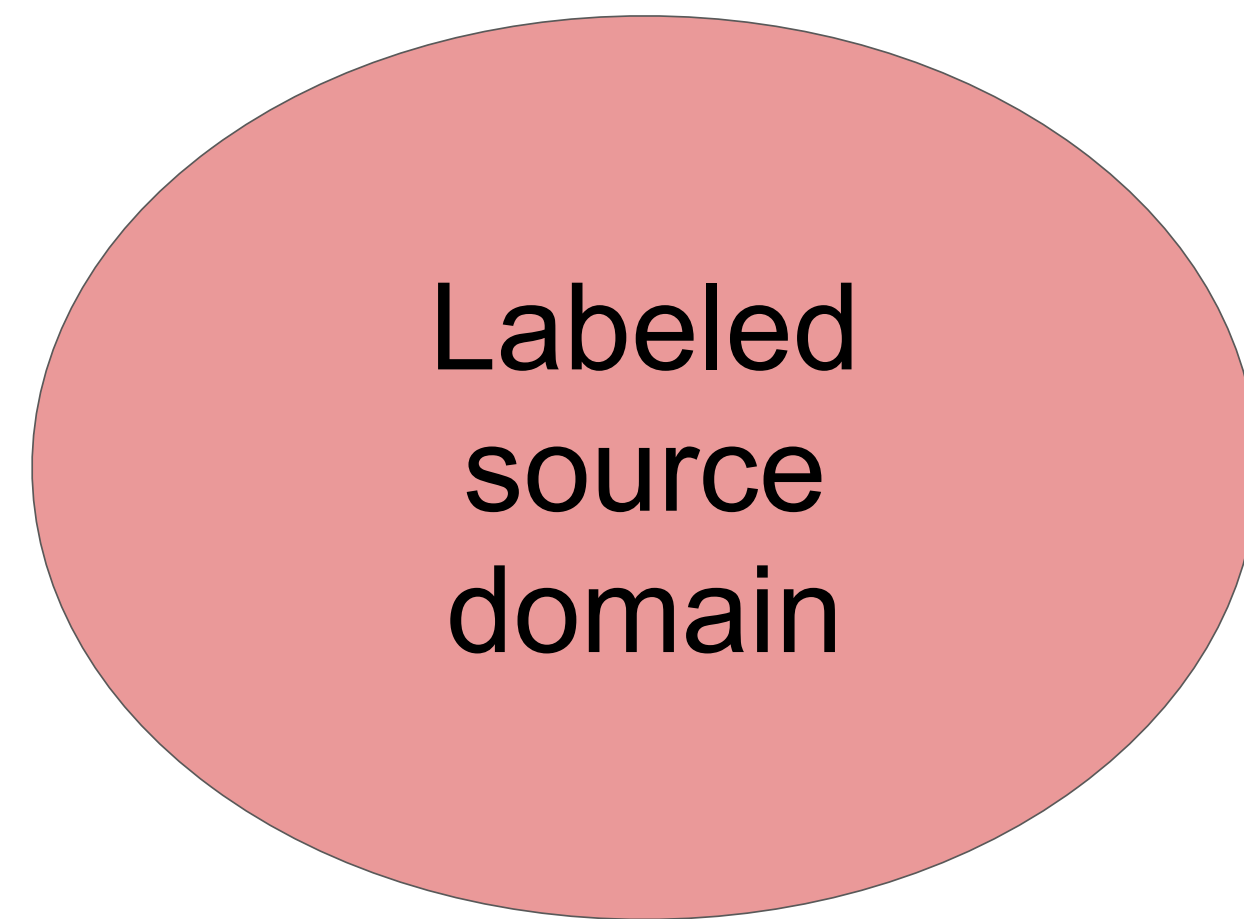
# Sampling issues in point clouds



(a) captured by a 64-beam LiDAR    (b) captured by a 32-beam LiDAR



# Sampling caused domain gaps



# Some directions to address the sampling issue

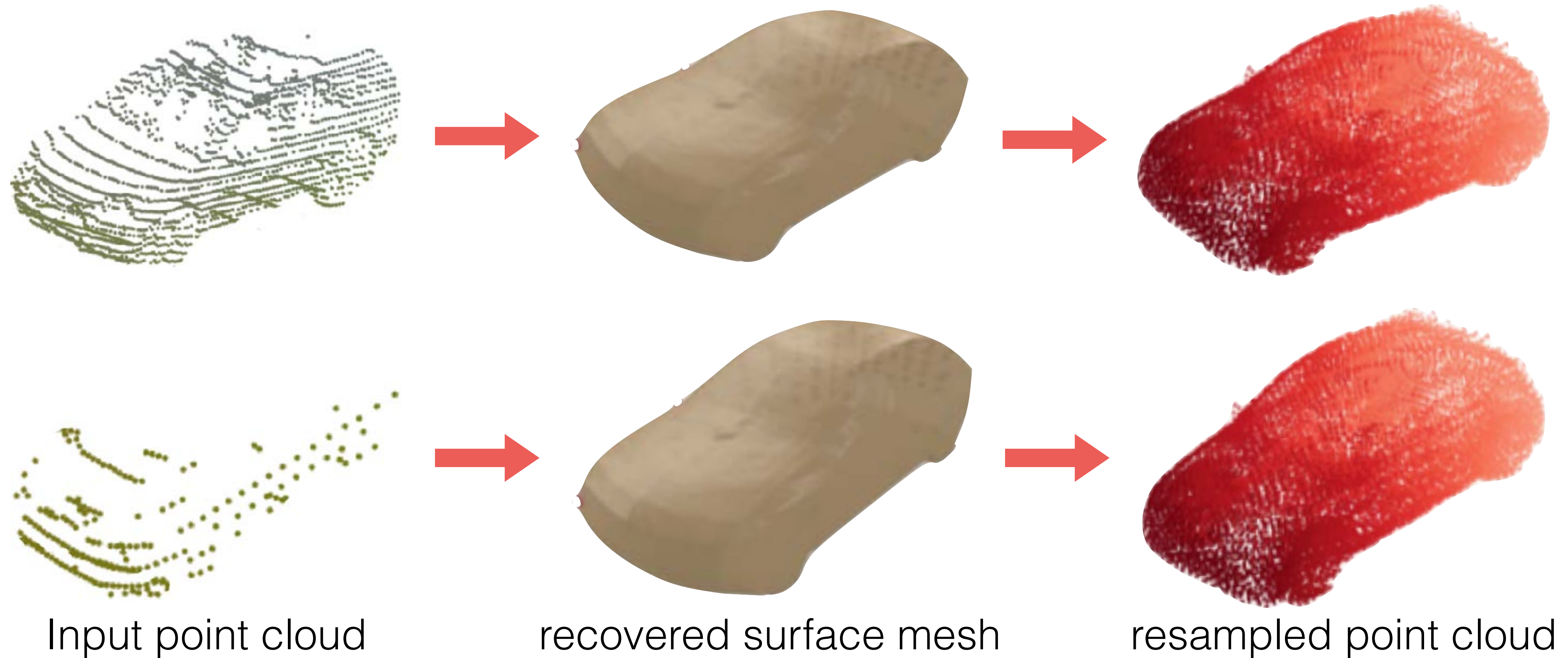
- Randomly throw away some points in the training data by a dropout layer (as in PointNet++)
- Learn to canonicalize the point cloud
- Use density-aware convolution



# Some directions to address the sampling issue

- Randomly throw away some points in the training data by a dropout layer (as in PointNet++)
- **Learn to canonicalize the point cloud**
- Use density-aware convolution

# Learning to canonicalize the point cloud

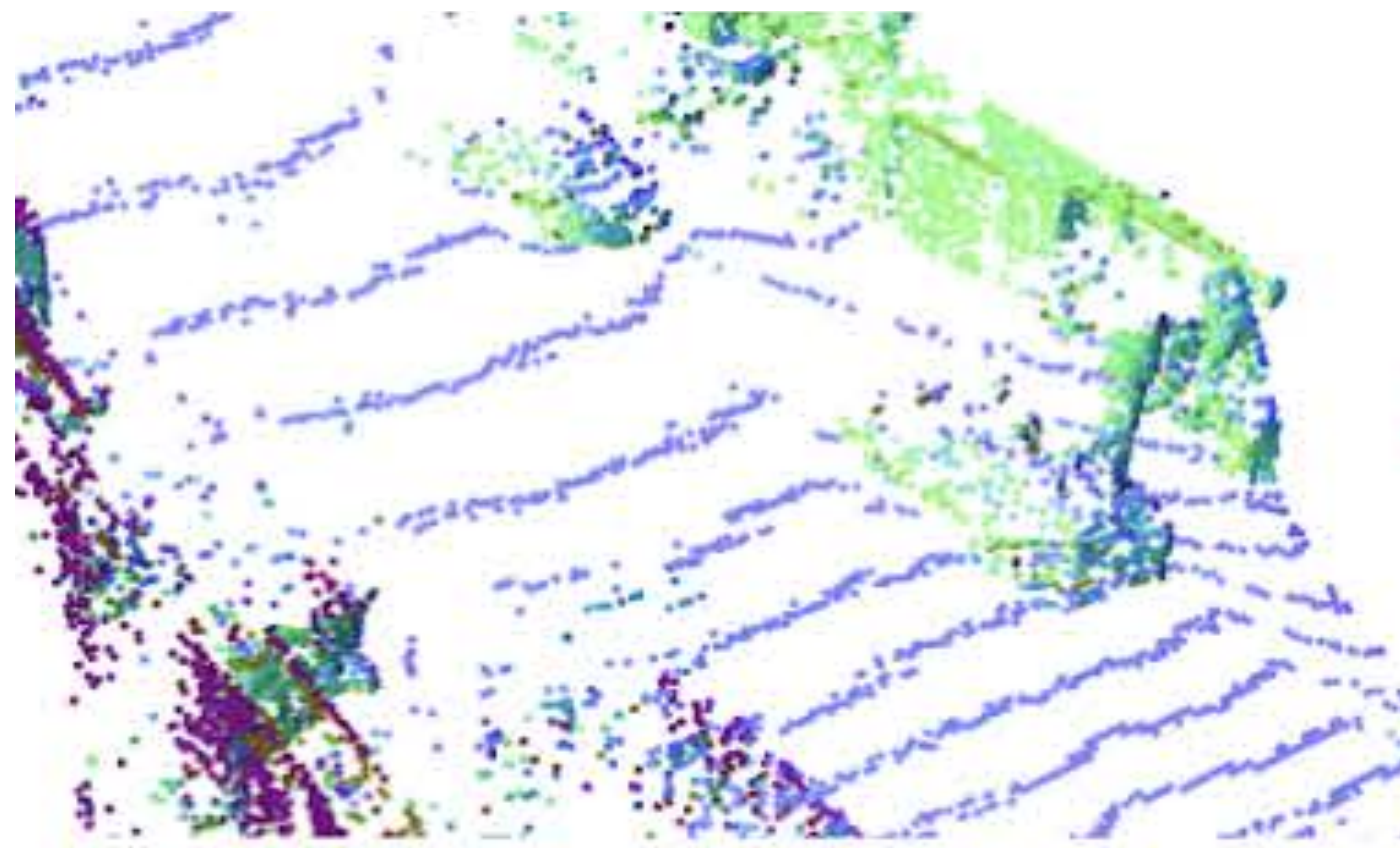


***Complete & Label: A Domain Adaptation Approach to  
Semantic Segmentation of LiDAR Point Clouds***

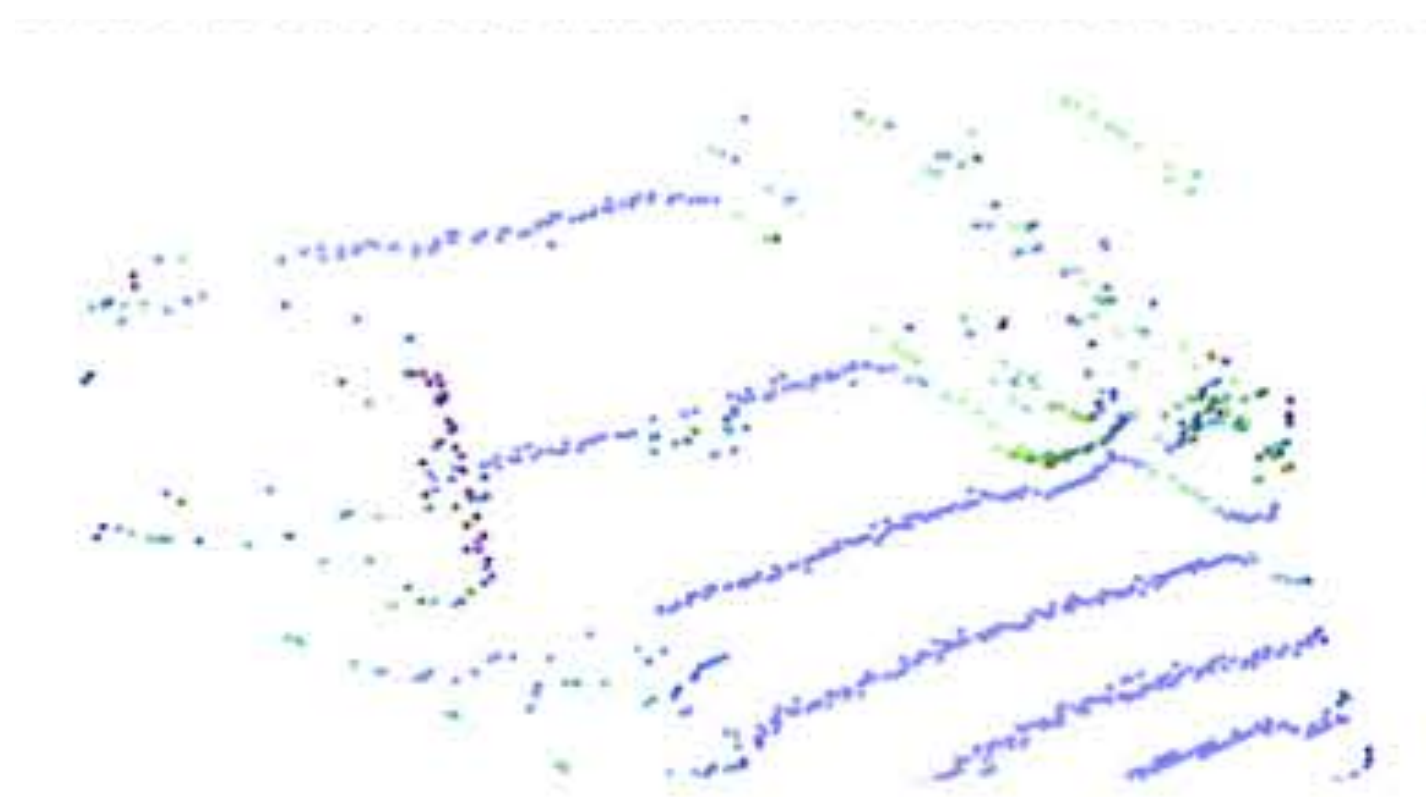
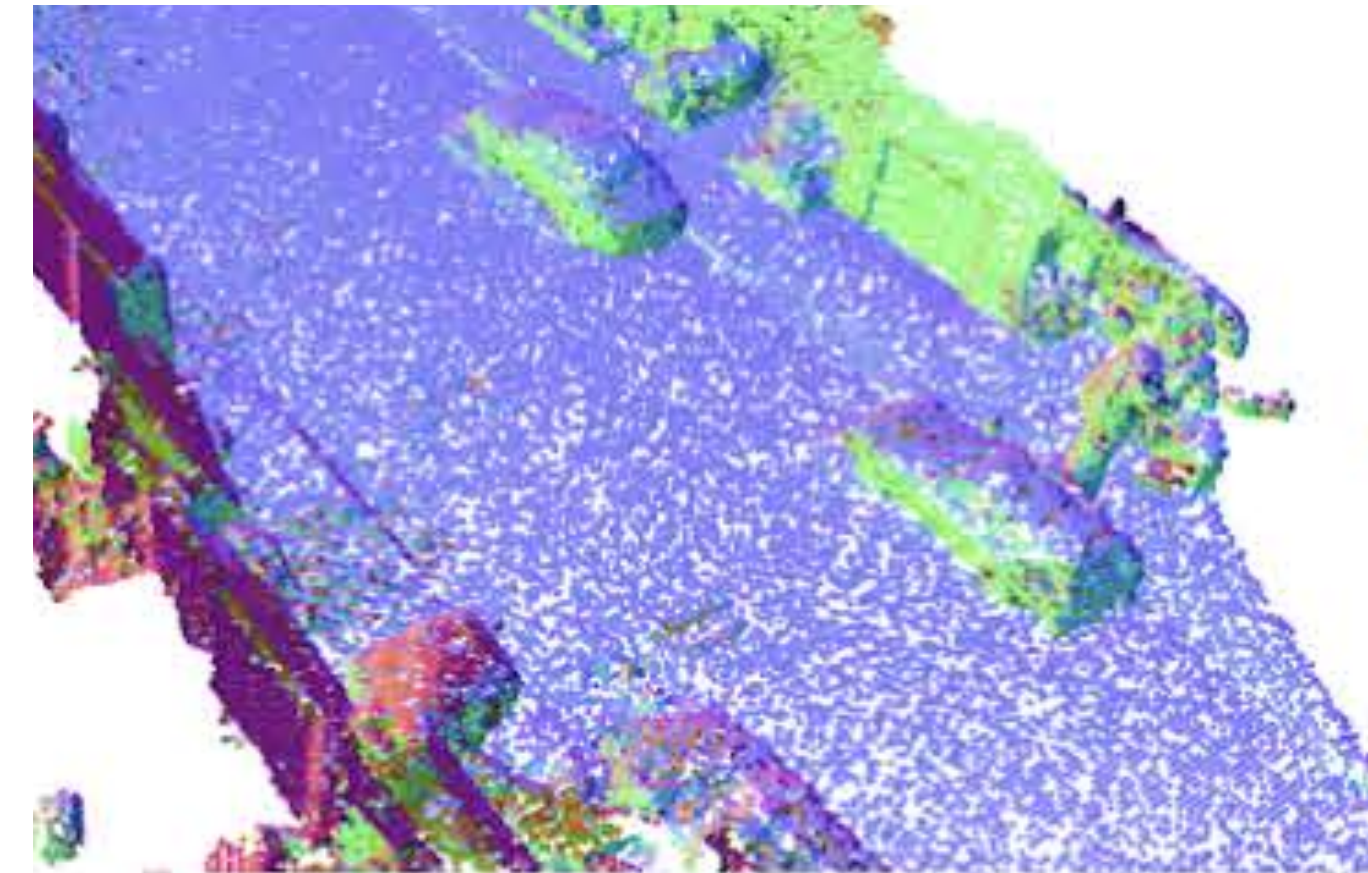
Li Yi, Boqing Gong, Thomas Funkhouser, CVPR 2021



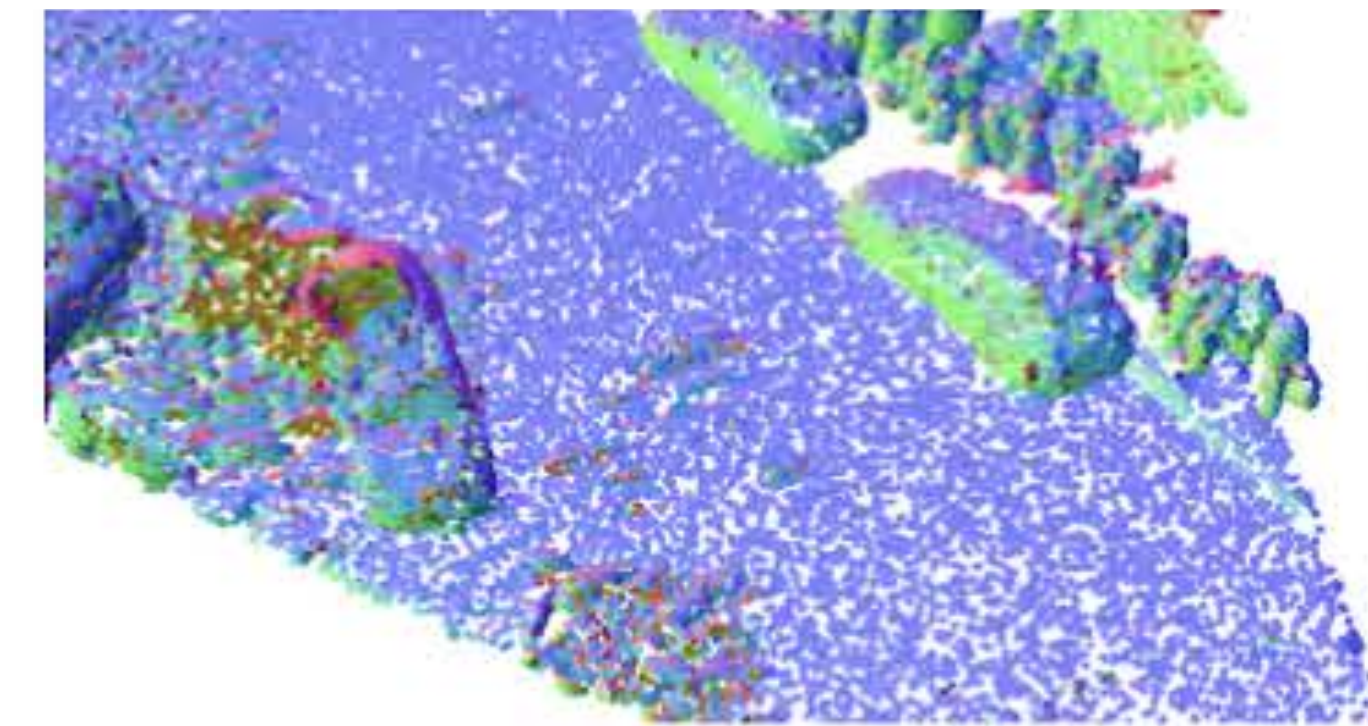
# Learning to canonicalize the point cloud



(a) captured by a 64-beam LiDAR

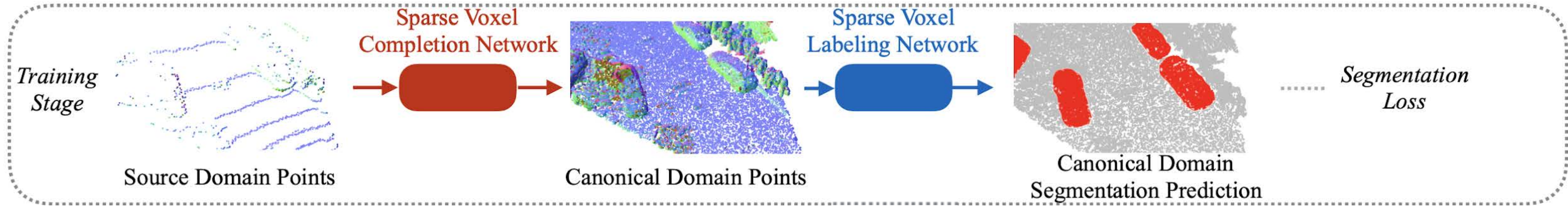


(b) captured by a 32-beam LiDAR



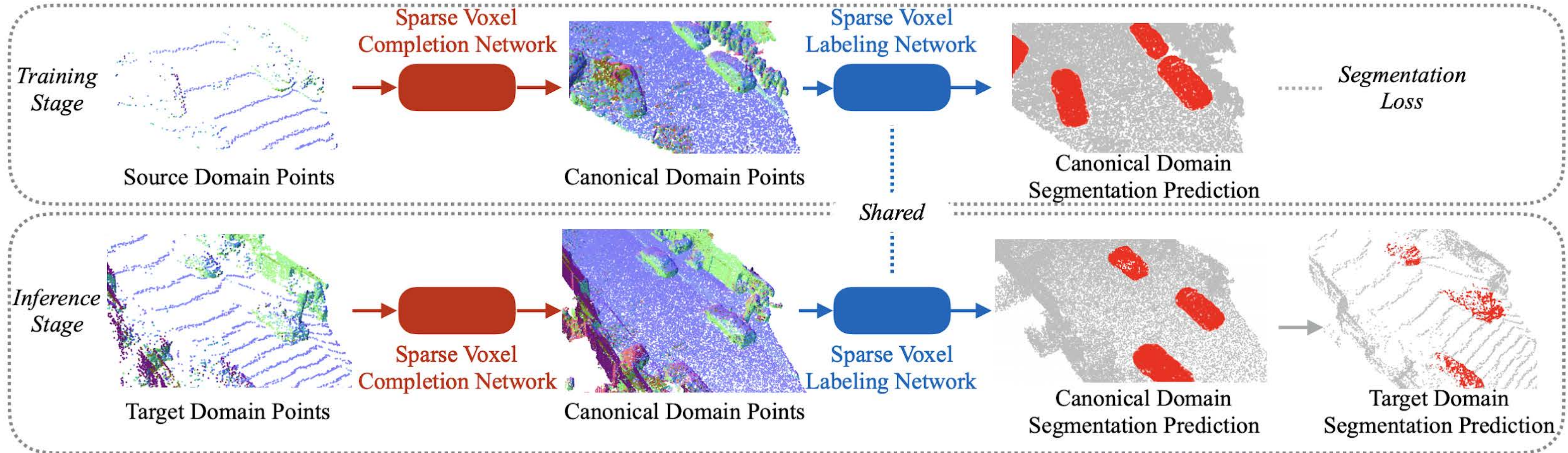


# Learning to canonicalize the point cloud



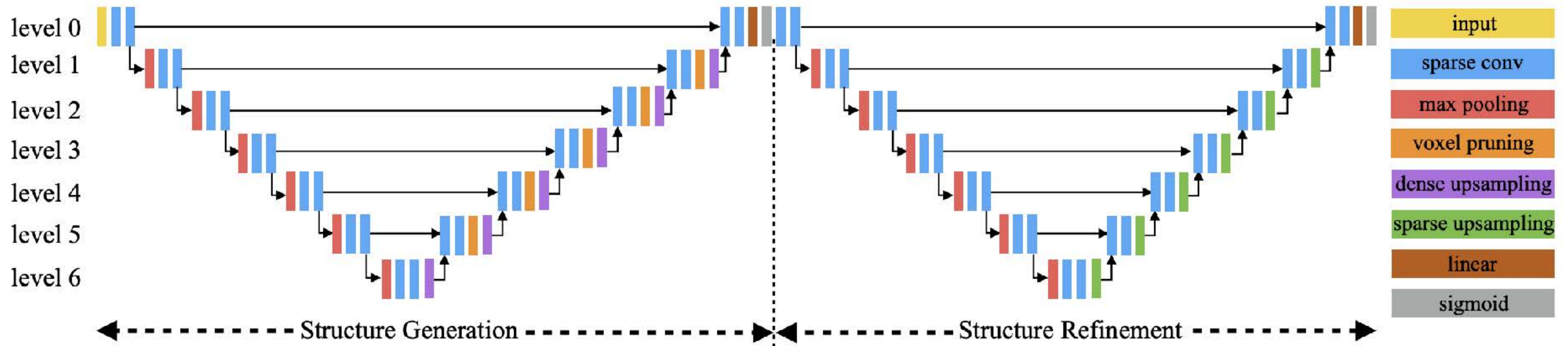


# Learning to canonicalize the point cloud



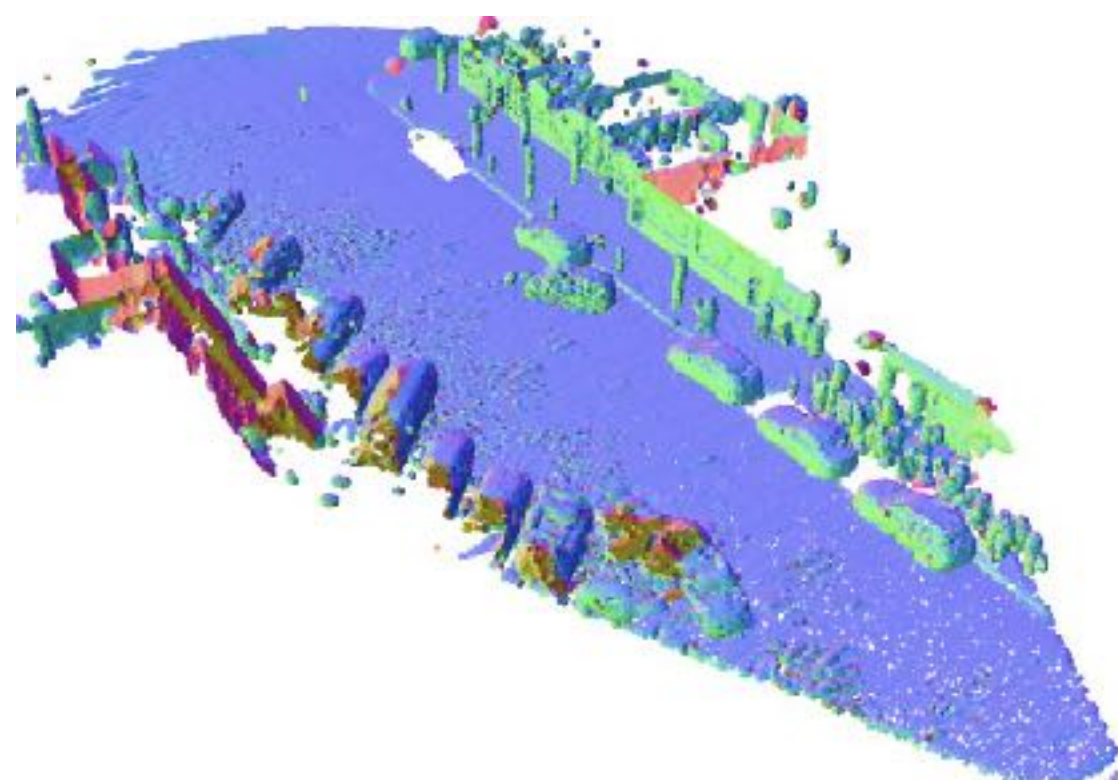


# Sparse Voxel Completion Network (SVCN)

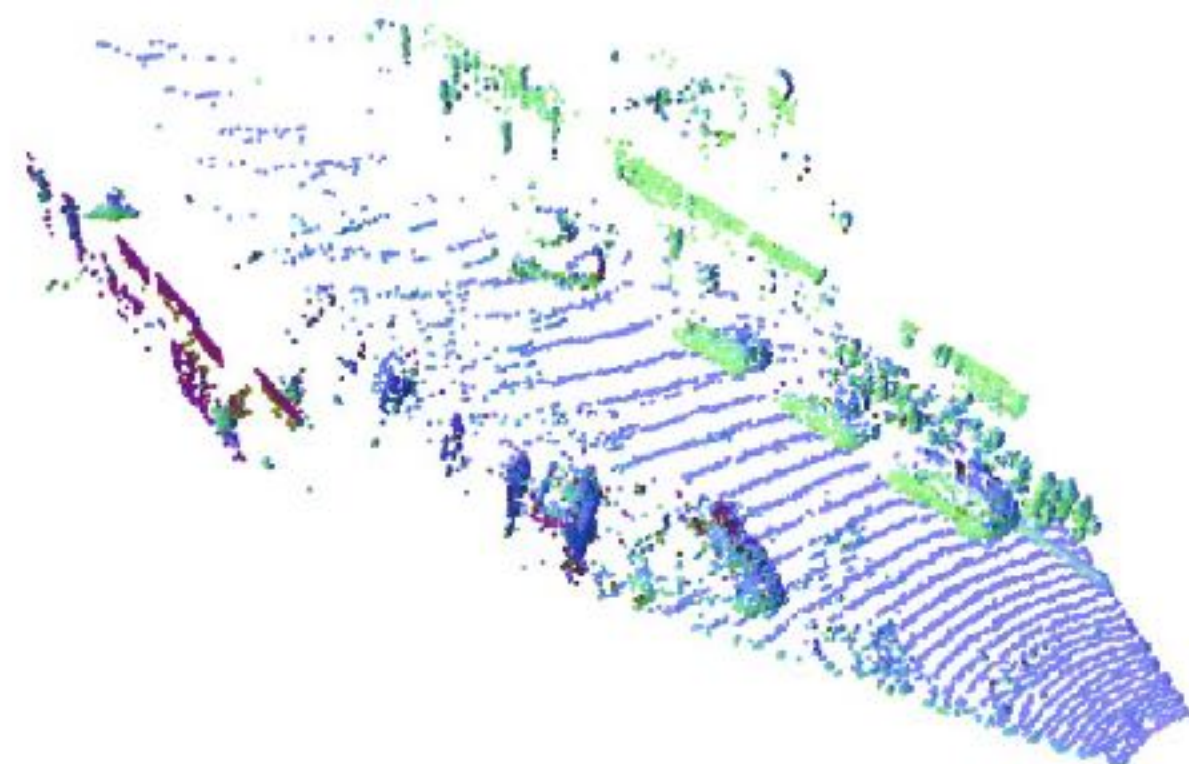




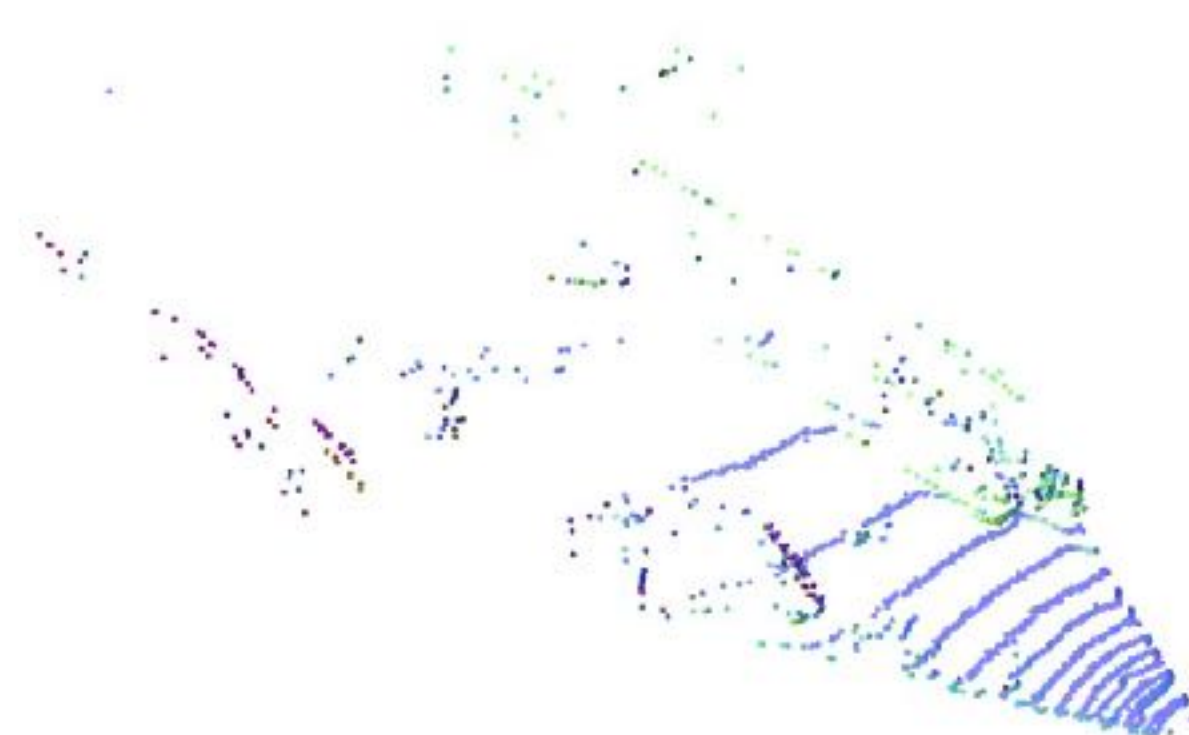
# Training Data Preparation



(a) complete scene point cloud

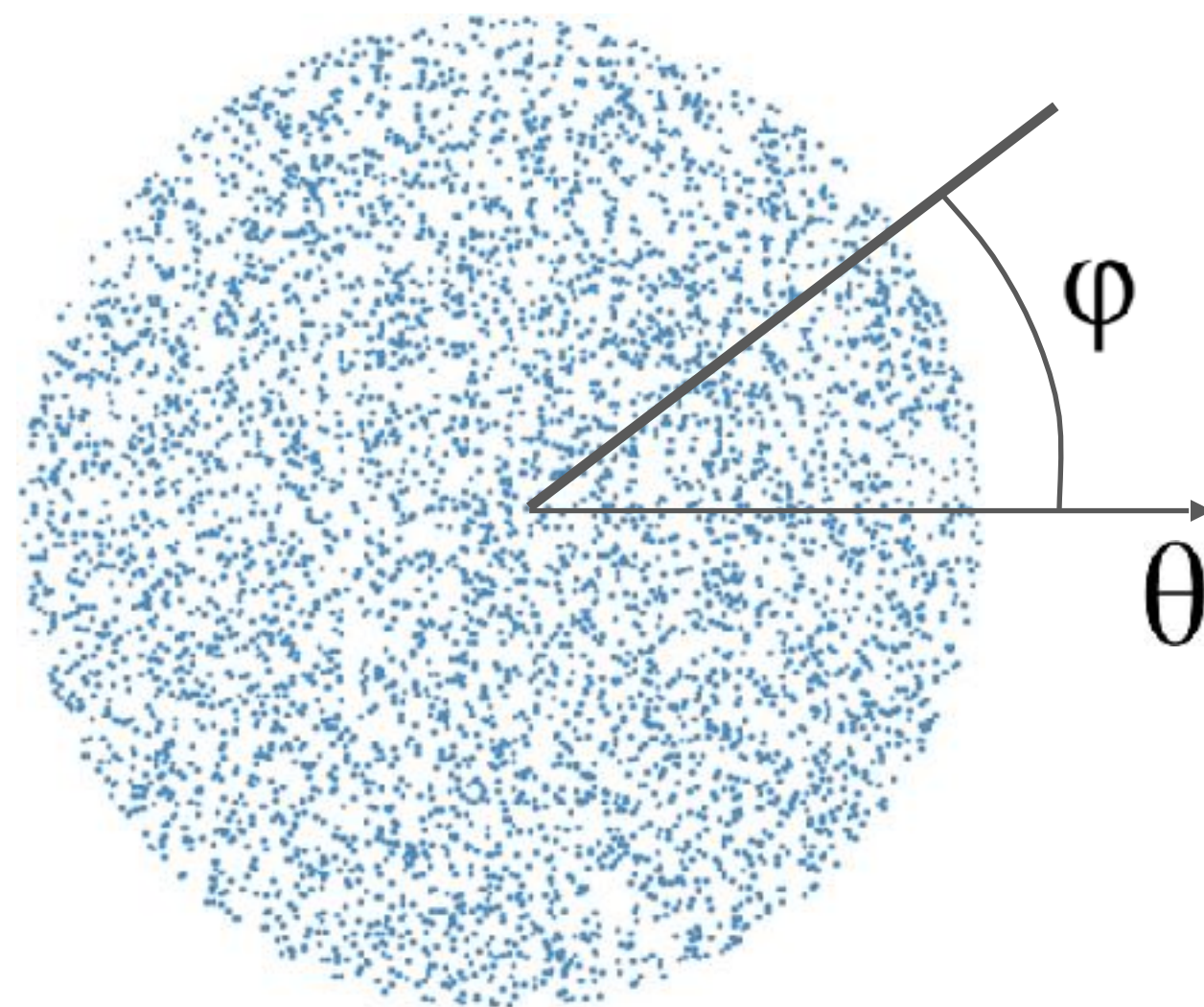
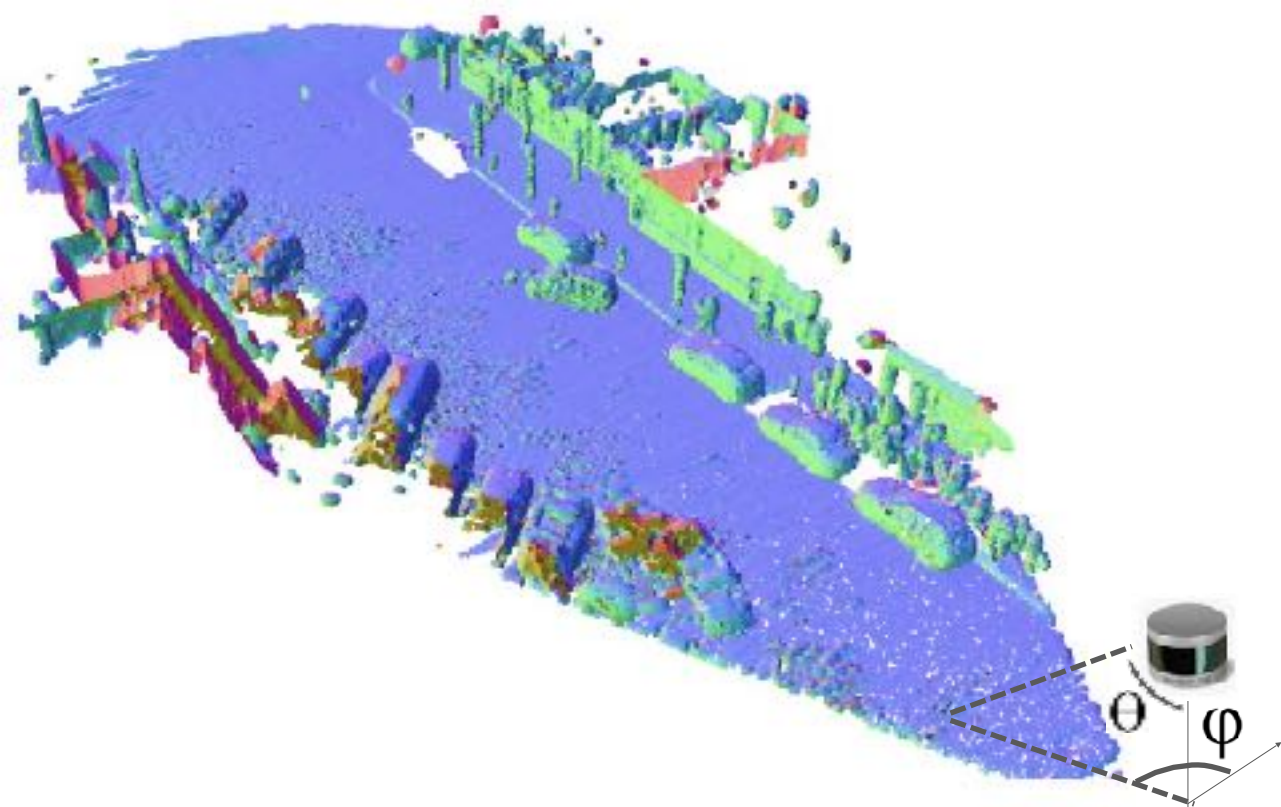


(b) simulated incomplete point cloud with sampling pattern transferred from Waymo

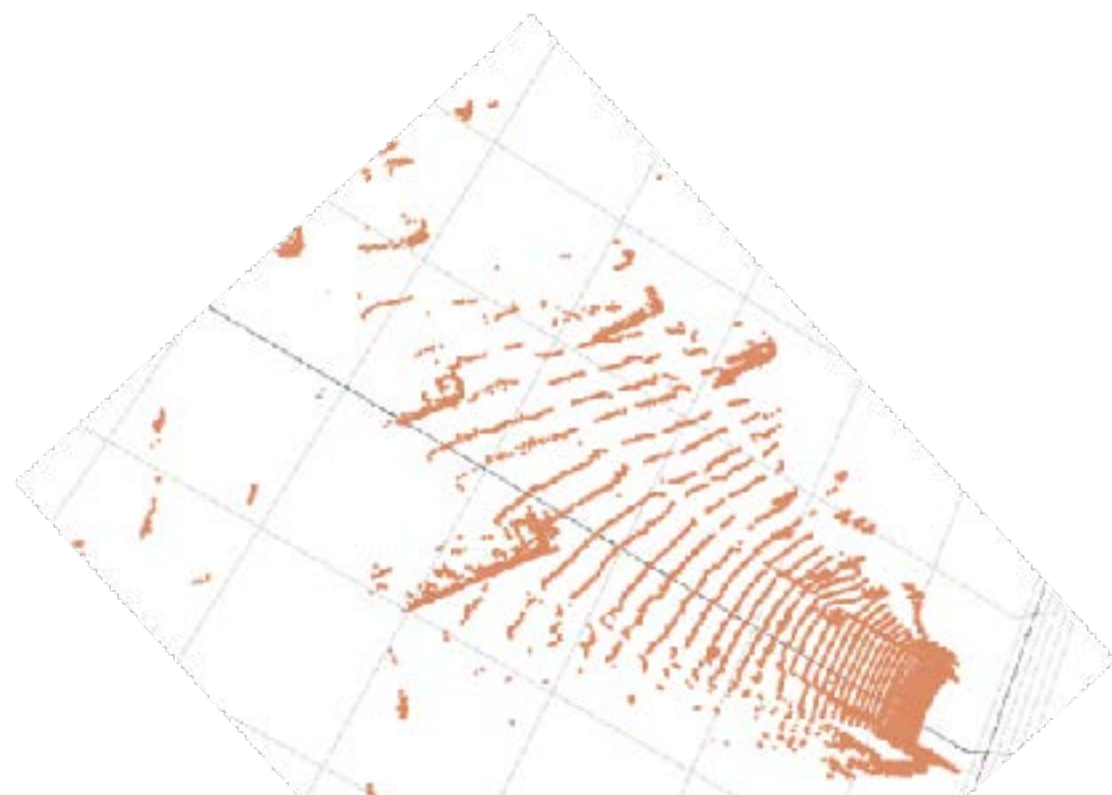
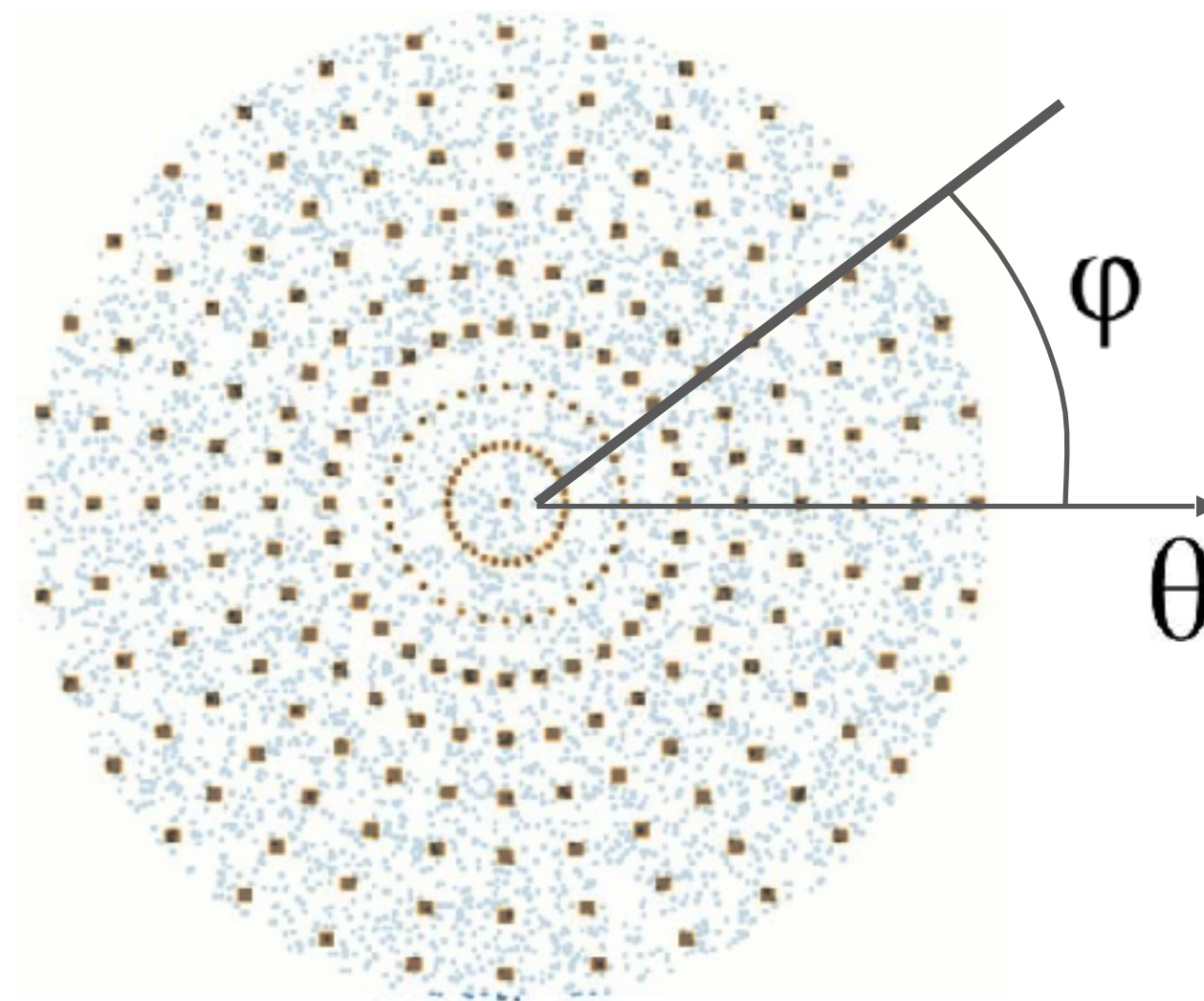
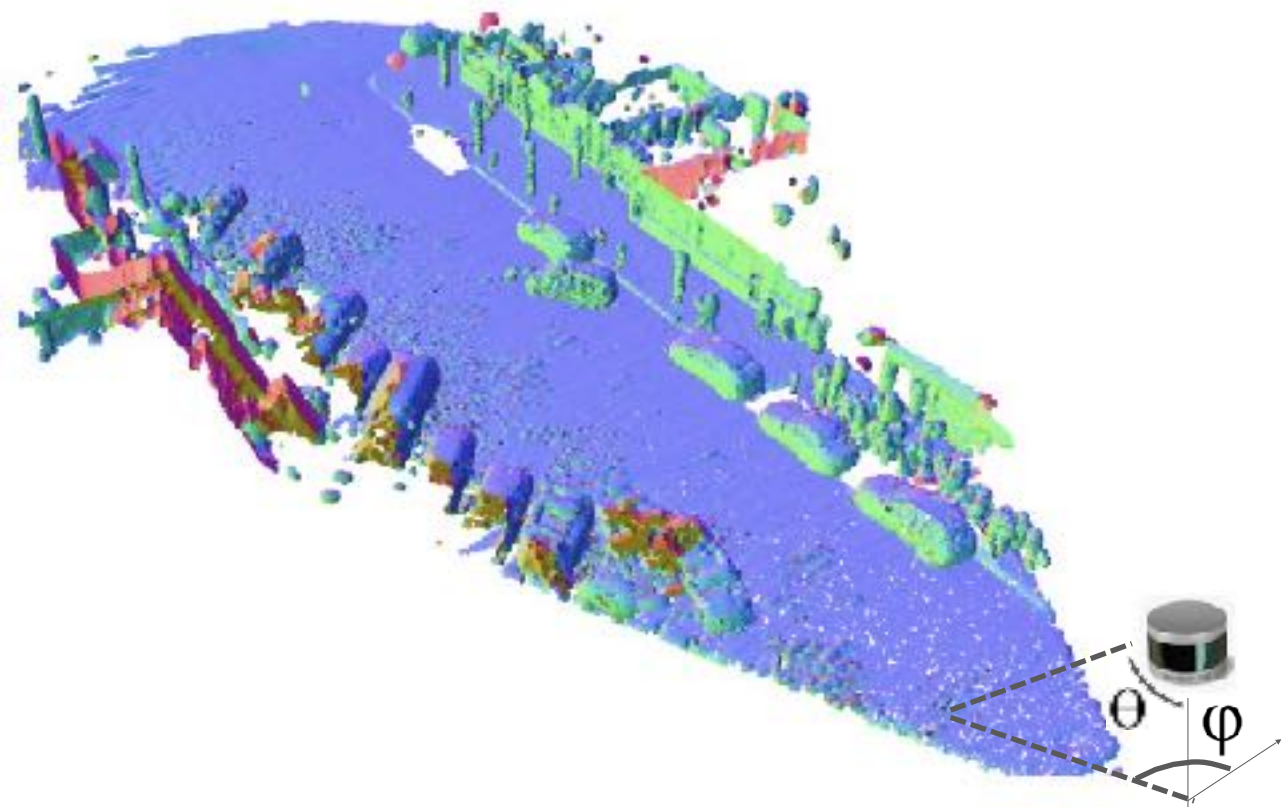


(c) simulated incomplete point cloud with sampling pattern transferred from nuScenes



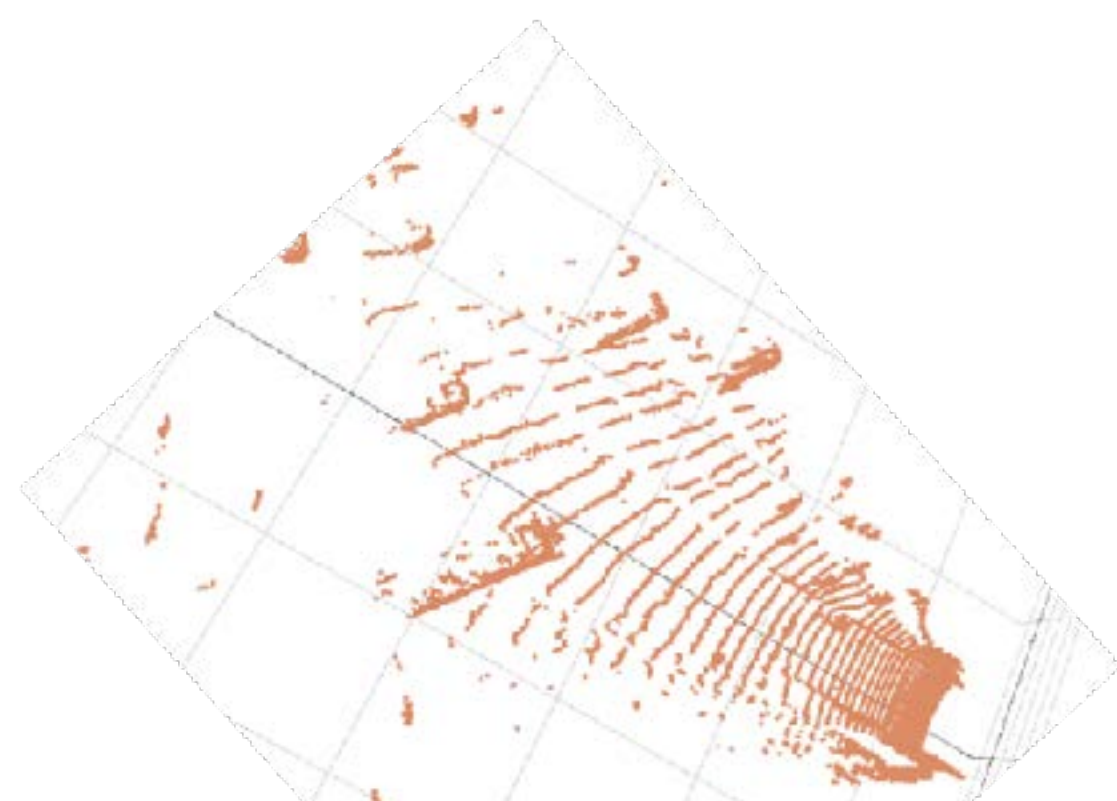
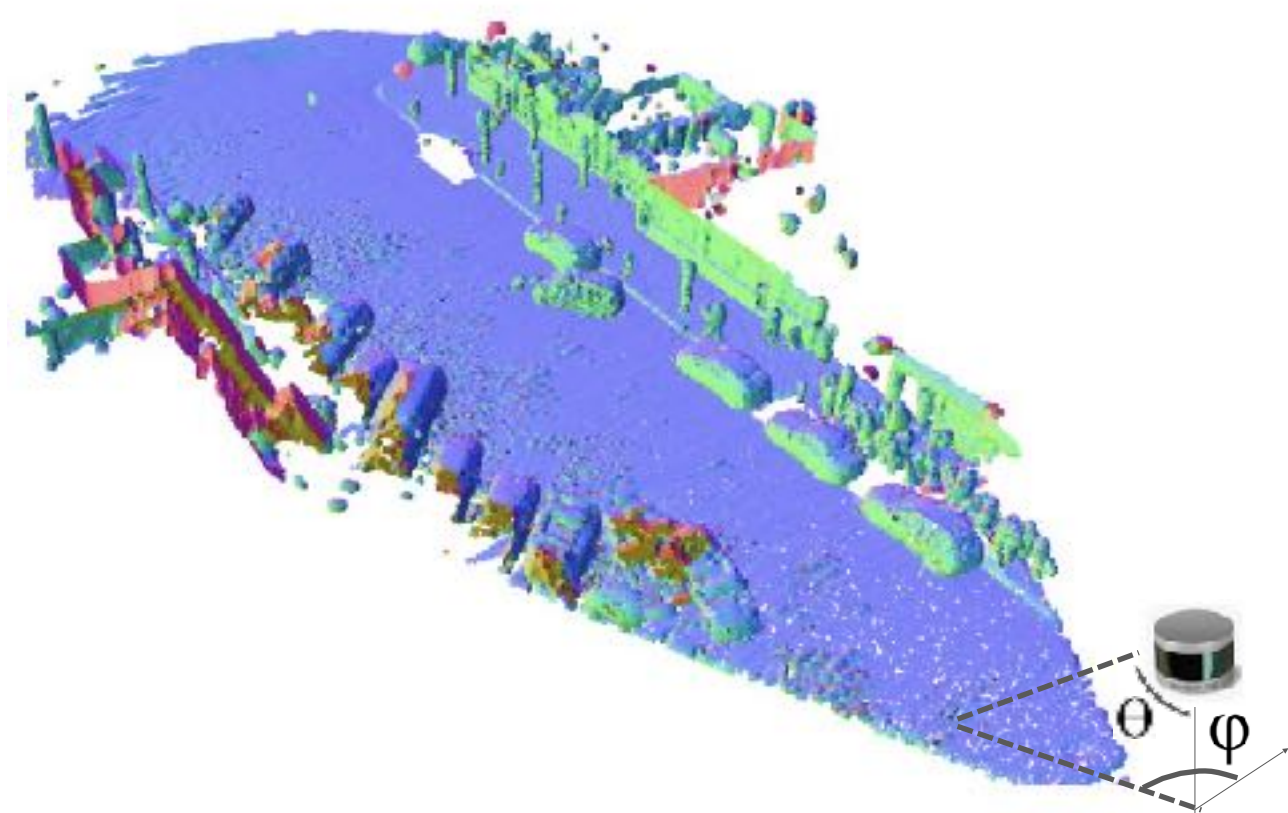




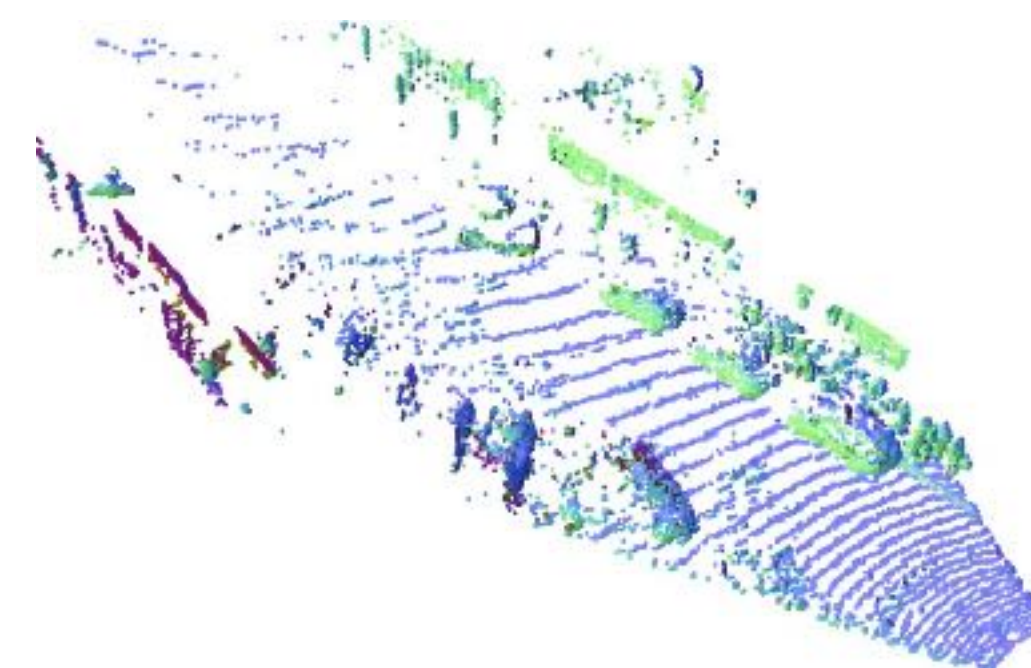
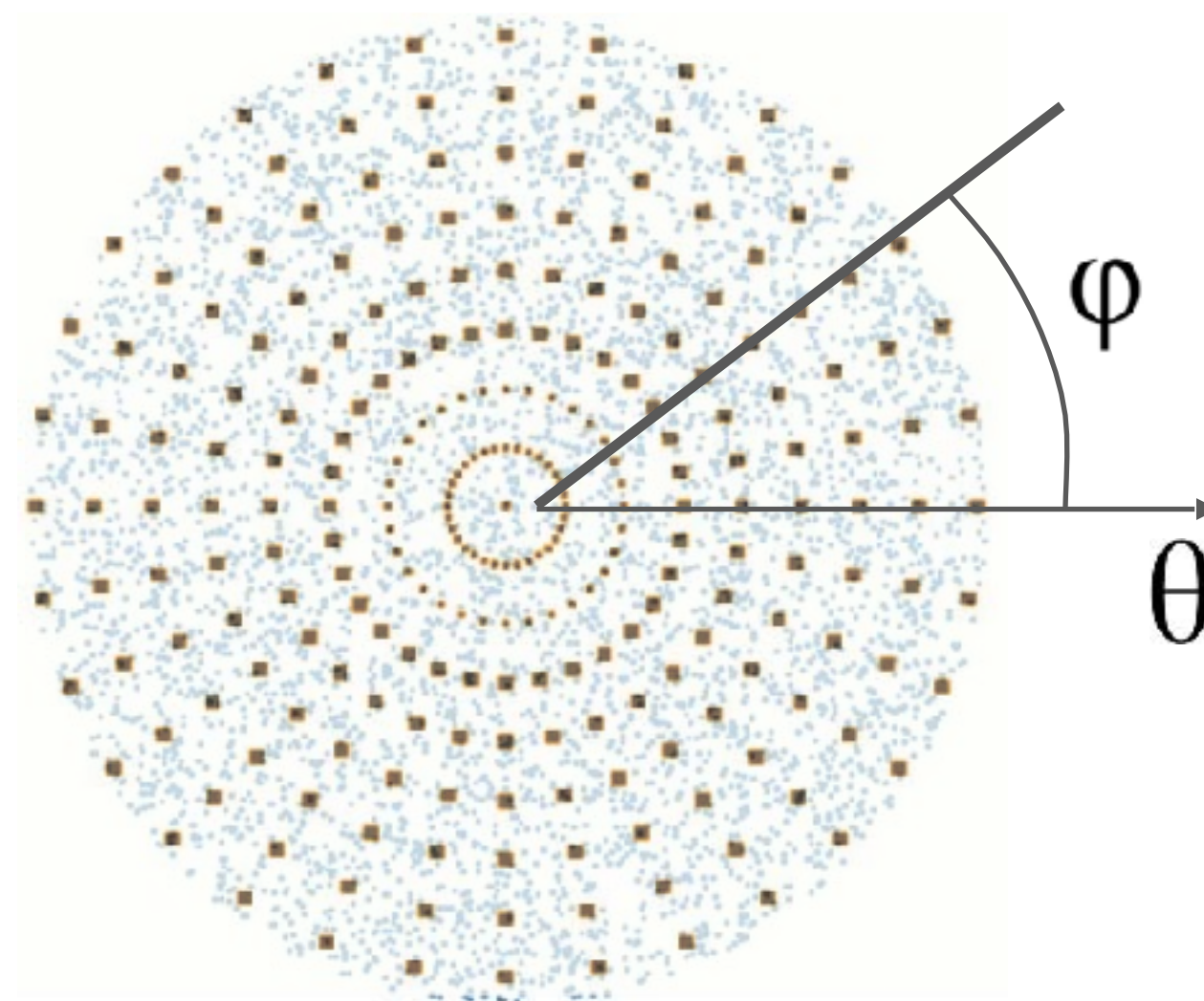


Reference  
frame





Reference  
frame



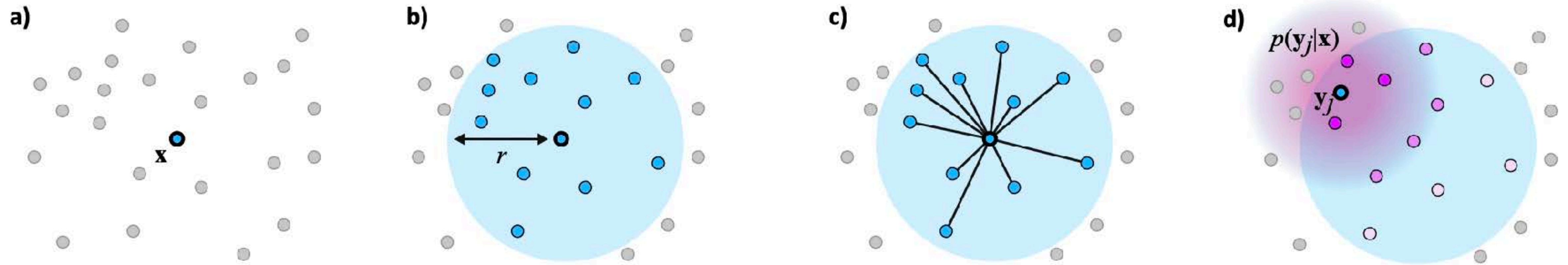
simulated  
samples



# Some directions to address the sampling issue

- Randomly throw away some points in the training data by a dropout layer (as in PointNet++)
- Learn to canonicalize the point cloud
- **Use density-aware convolution**

# Monte Carlo Convolution



HERMOSILLA et al. 2018



# Future directions

- **Scalability**

How to scale up from processing 100k points to 1M or even 10M points?

(1024 x 1024 image ~ 1M pixels)

Trade-offs in neighborhood sampling

More memory efficient operators





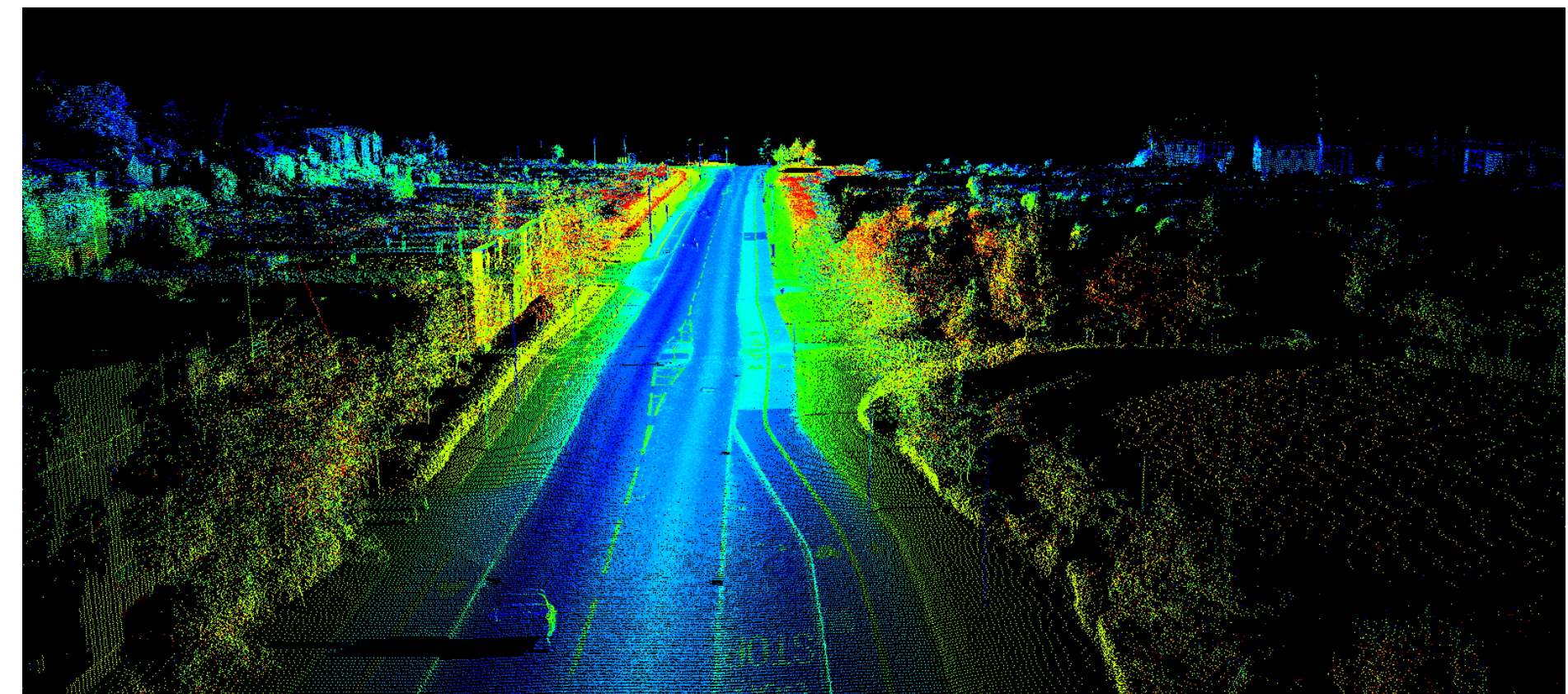
# Future directions

- Scalability
- **Multi-modality**



*RGB images*

*High resolution  
Rich textures*



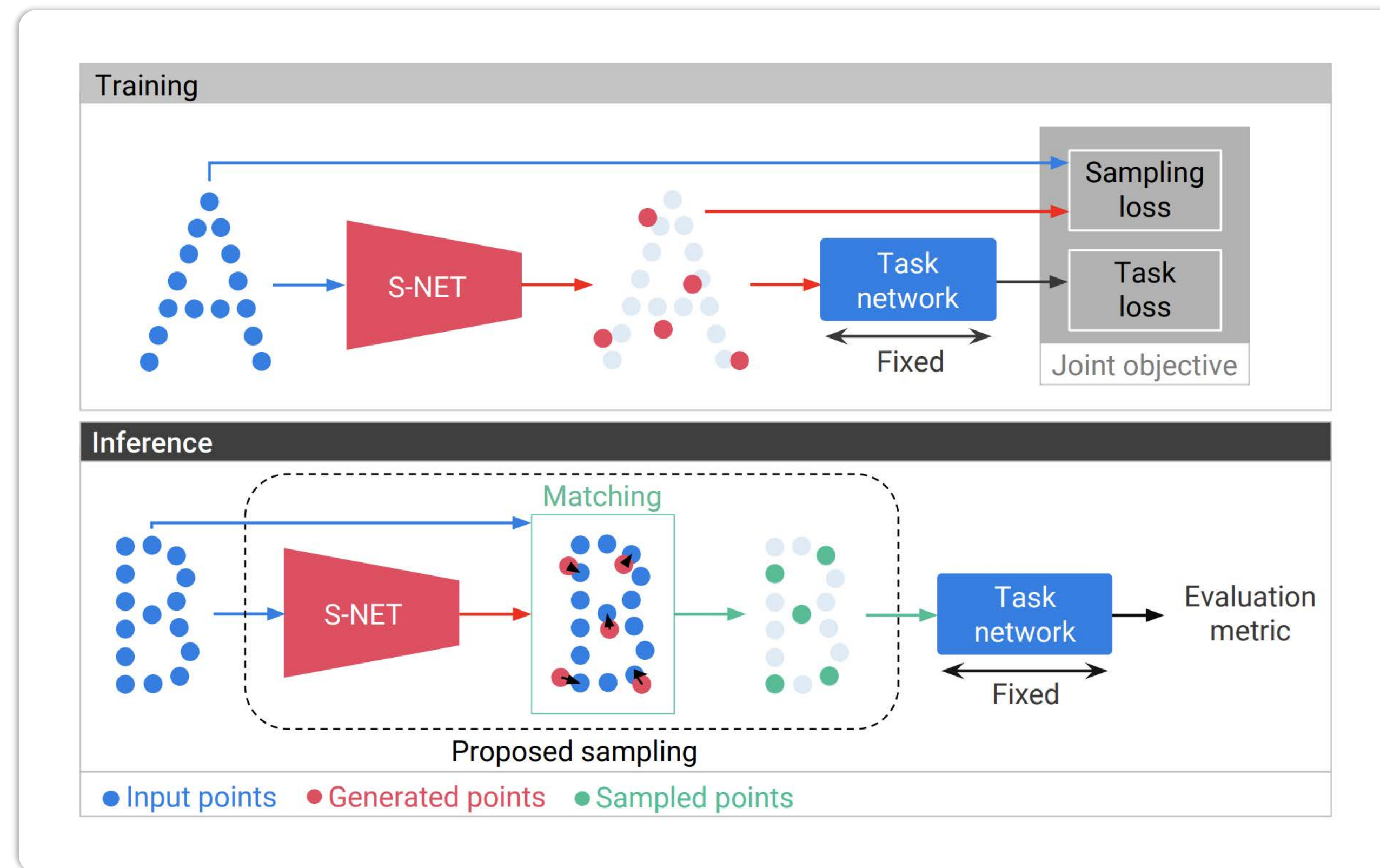
*Lidar point clouds*

*Accurate depth  
Accurate 3D geometry*



# Future directions

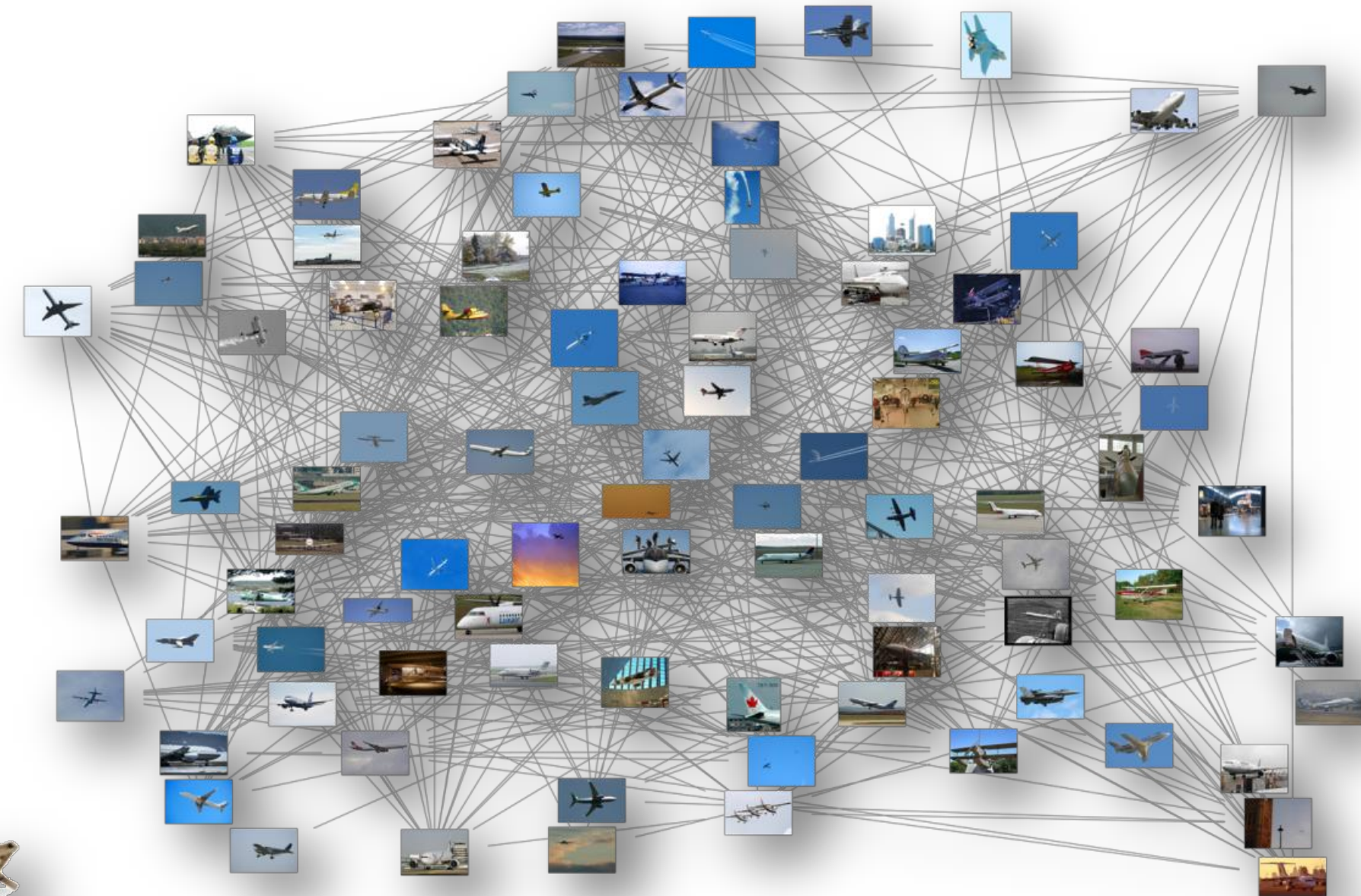
- Scalability
- Multi-modality
- **Sampling**



Learning to sample [Dovrat et al.]

# Future directions

- Scalability
- Multi-modality
- Sampling
- **Set processing**





# Next Time

- More about 3D backbone networks